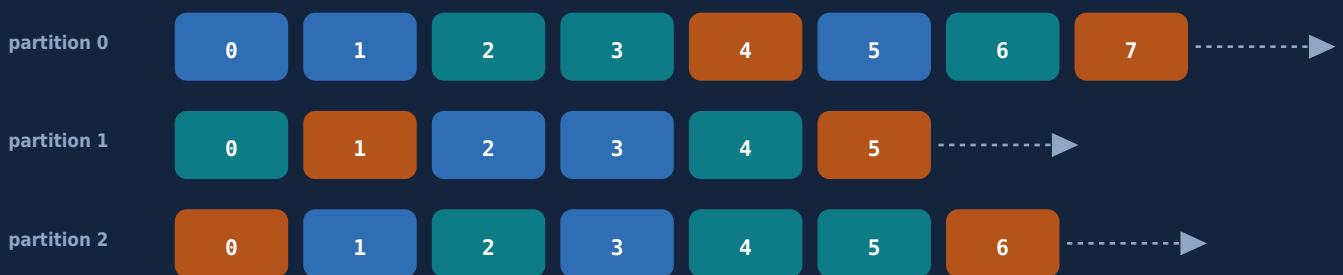


Apache Kafka

The Complete Guide

From first principles to production: how Kafka works inside, how to build with it in Java and Spring Boot, the patterns that matter, where it is used, and complete end-to-end projects.

Fundamentals · Internals · Producer & Consumer APIs · Spring Boot · Streams · Connect
Patterns · Use cases · Projects · Operations · Interview preparation



Version basis

Apache Kafka 4.x in KRaft mode (4.2 and 4.3 current at the time of writing, October 2026)

Spring Boot 4.1 with Spring for Apache Kafka 4.1 (Spring Framework 7, Java 17+; examples use Java 21)

Debezium 3.x for change data capture

How to use this guide

- **Read it in order the first time.** The opening parts build the mental model (what Kafka is and how it works inside), then come the plain Java clients, schemas and Spring Boot, followed by Kafka Streams, Kafka Connect and the design patterns. Use the contents page to jump to what you need. This edition does not yet include the hands-on installation labs, the consumer API chapters, the use-case catalogue, the end-to-end projects, the operations part or the interview kit.
- **Every chapter follows the same rhythm:** what it is, why it exists, how it works, how to implement it (with code), what goes wrong, and a short key-points box. Coloured boxes mean: NOTE (extra detail), TIP (practical advice), WATCH OUT and GOTCHA (mistakes that cost people days), REAL WORLD (how it appears in production), INTERVIEW ANGLE (what interviewers like to ask).
- **Labs are hands-on.** A dark box is a terminal session: lines beginning with `$` are what you type, the other lines are the output you should see. Output is shown as typical output; IDs, timestamps and offsets on your machine will differ.
- **About the code.** Every code listing was written against the official documentation for the versions named on the cover and checked for syntax. They were not executed against a live broker while this guide was produced, so treat the first run on your machine as part of the lab: if a command or listing behaves differently, the official docs for your exact version are the final word.
- **Running example.** Most examples use one fictional online shop, *ShopStream*, with orders, payments, inventory and notifications. Seeing the same domain across chapters lets you focus on Kafka, not on the business problem.
- **Conventions.** `monospace` is code, a command or a configuration name. Configuration defaults quoted in this guide are those of Kafka 4.2 documentation unless stated. Topic names use lowercase with dots or hyphens.

Contents

Part 1: Kafka Fundamentals	5
1.1 The Problem Kafka Was Built to Solve	5
1.2 What Kafka Is (and What It Is Not)	7
1.3 The Building Blocks at a Glance	9
1.4 Kafka Compared with Other Tools	13
1.5 What Kafka Guarantees (and What It Does Not)	15
1.6 A Tour of the Kafka Ecosystem	16
1.7 A Short History and the Kafka 4.x Era	18
1.8 How to Study This Book	19
Part 2: Inside Kafka: Architecture and Internals	22
2.1 Topics, Partitions and the Commit Log	22
2.2 Records, Batches and the Wire Format	27
2.3 Brokers, the Controller and KRaft	30
2.4 Replication: Leaders, Followers and the ISR	34
Part 3: The Java Producer API	40
3.1 The Producer at a Glance	40
3.2 What Happens Inside <code>send()</code>	43
3.3 Sending Records: Patterns and the <code>ProducerRecord</code>	46
3.4 Reliability: Acknowledgements, Retries and Idempotence	49
3.5 Throughput and Latency Tuning	53

3.6 Partitioning in Practice	56
Part 4: Serialization, Schema Registry and Data Contracts	60
4.1 Serialization Basics	60
4.2 Why Schemas Matter	64
4.3 Schema Registry	66
4.4 Avro in Practice	71
4.5 Protobuf and JSON Schema	74
4.6 Evolving Schemas Safely	77
Part 5: Spring Boot and Spring for Apache Kafka	81
5.1 Spring Boot and Kafka: Project Setup	81
5.2 Producing with KafkaTemplate	84
Part 6: Spring Kafka in Production: Errors, Retries, Transactions, Testing and Observability	89
6.1 Error Handling in Spring Kafka Listeners	89
6.2 Non-Blocking Retries with @RetryableTopic	97
6.3 Idempotent Consumers and the Inbox Pattern	103
6.4 Transactions in Spring Kafka	106
Part 7: Kafka Streams and ksqlDB	111
7.1 Stream Processing Concepts	111
7.2 Your First Kafka Streams Application	113
7.3 Stateless Operations: Filter, Map, Branch and Merge	117
7.4 Aggregations and State Stores	122
7.5 Windows and Time	128
Part 8: Kafka Connect and Change Data Capture	132
8.1 Why Kafka Connect	132
8.2 Running Connect: Workers and the REST API	134
8.3 Connectors in Practice	140
8.4 Converters and Single Message Transforms	143
Part 9: Design Patterns with Kafka	147
9.1 Designing Events and Topics	147
9.2 Event Notification and Event-Carried State Transfer	151
9.3 The Dual-Write Problem and the Transactional Outbox	154
9.4 Idempotent Receivers: Choosing a De-duplication Strategy	157
9.5 Retry and Dead-Letter Patterns	158
9.6 Sagas: Choreography and Orchestration	161
9.7 Event Sourcing	164
9.8 CQRS and Materialised Views	166

9.9 Compacted Topics as Tables	167
--	-----

PART 1

Kafka Fundamentals

What Kafka is, why it exists, how its pieces fit together, and when to choose it and when not to.

In this part

- 1.1 The Problem Kafka Was Built to Solve
- 1.2 What Kafka Is (and What It Is Not)
- 1.3 The Building Blocks at a Glance
- 1.4 Kafka Compared with Other Tools
- 1.5 What Kafka Guarantees (and What It Does Not)
- 1.6 A Tour of the Kafka Ecosystem
- 1.7 A Short History and the Kafka 4.x Era
- 1.8 How to Study This Book

1.1 The Problem Kafka Was Built to Solve

Imagine you are building ShopStream, an online shop. When a customer clicks **Place order**, a surprising amount of work has to happen. The card must be charged. Stock must be reserved. A confirmation email should go out. The warehouse needs a shipment to pack. The analytics team wants the sale on their dashboard, and the search page should learn which products are selling fast. ShopStream is the running example for this whole book, so it is worth spending a few minutes on how this work should be organised. That question is the reason Kafka exists.

1.1.1 The direct-call design and its tangle

The design most teams reach for first is the direct one. `order-service` saves the order and then calls every other system in turn, usually over HTTP.

OrderService.java (the direct-call design, simplified)

JAVA

```
public void placeOrder(Order order) {
    orderRepository.save(order);
    paymentClient.charge(order);           // HTTP call: can be slow, can fail
    inventoryClient.reserve(order);        // another HTTP call
    emailClient.sendConfirmation(order);    // and another
    analyticsClient.recordSale(order);
    searchClient.updatePopularity(order);
}
```

This works on day one and hurts on day ninety. Look at what `order-service` has to know, and what it has to put up with.

- **It knows every consumer of its data.** Each call hard-codes an address, an API and a payload format. A new interested system, say a fraud checker, means changing and redeploying `order-service`.
- **Everyone must be awake right now.** If the email service is down, either the order fails or you write retry and fallback code for every single call.
- **The slowest callee sets the pace.** The customer waits while the analytics call finishes, although the customer does not care about analytics.
- **The problem multiplies.** `order-service` is only one producer of facts. `payment-service` has facts that others need (a payment was authorised), and so do inventory, shipping and the web front end.

If N systems produce data and M systems need it, point-to-point wiring needs up to $N \times M$ integrations. Each one has its own protocol, data format, error handling, monitoring and owner. Ten producers and fifteen consumers can mean as many as 150 connections to build and keep alive (an illustrative number, but real systems get there faster than anyone plans).

BEFORE: point-to-point. Each X is a separate integration to build, secure, monitor and fix.

	payment	inventory	shipping	email	analytics	search
order-service	X	X	X	X	X	X
payment-service	.	X	X	X	X	.
inventory-service	.	.	X	.	X	X
web-frontend	X	X	.	.	X	X

4 producers x 6 consumers: 17 connections here, up to 24 in the worst case

1.1.2 Batch versus real time

The other classic answer is the **batch job**. Once a night a script extracts yesterday's data from the order database, transforms it, and loads it into a warehouse, a search index and a reporting database. This pattern is called ETL (extract, transform, load). Batch processing is simple, robust and very efficient for large volumes, and it is still the right tool for many jobs, such as monthly revenue reports.

Its weakness is **staleness**: the data is exactly as old as the last run. Some questions can wait until tomorrow. Others cannot. Is this card payment fraudulent? Is the last pair of size-42 red shoes about to sell out? Which products should the home page promote right now? The value of this kind of information decays quickly with age.

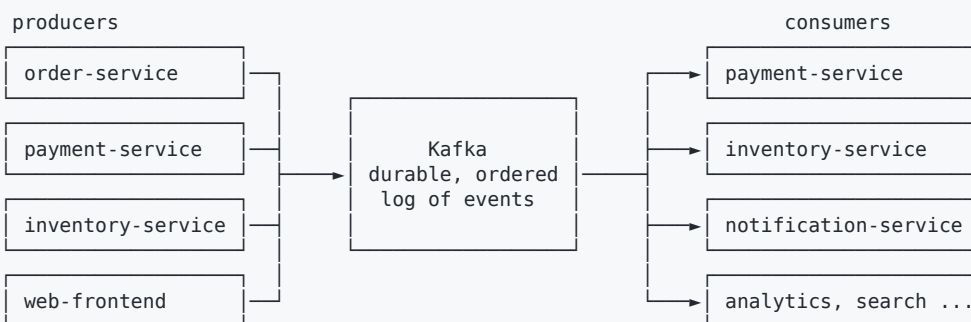
Batch extraction also tends to hit the application database directly, which adds load to a system that is busy serving customers, and it couples every job to the internal table layout of the source system.

"Real time" in this book does not mean microseconds. It means reacting within seconds, or less, **as events happen**, instead of hours later. What we want is a way to move facts between systems continuously, reliably and without every pair of systems having to know about each other.

1.1.3 The shared log in the middle

The idea behind Kafka is to put one durable, ordered, append-only **log** in the middle. A producer does not call anybody. It appends an event describing what happened to the log. Consumers read the log at their own pace, and each consumer keeps its own bookmark that says how far it has read.

AFTER: everyone talks to the log, never to each other.



4 producers + 6 consumers = 10 connections, and all of them look the same.

Think of a shared notebook in the office. Anyone can write a new line at the bottom, nobody erases lines, and every reader keeps a bookmark at the last line they read. A reader on holiday does not stop anybody else from writing; when they return they simply continue from their bookmark. That picture is surprisingly close to how Kafka behaves, and the "bookmark" will come back as the **offset** in the next chapters.

Here is what this buys ShopStream:

- **Decoupling in space.** `order-service` publishes `OrderCreated` once. It does not know, and does not need to know, who reads it.
- **Decoupling in time.** If `notification-service` is down for an hour, events wait in the log. When the service comes back, it continues from its bookmark. Nothing is lost and nobody else noticed.
- **Decoupling in speed.** A slow consumer slows only itself. The producer and the other consumers carry on.
- **History and replay.** A new consumer can start from the beginning of the log. If you find a bug in your analytics code, you fix it, move the bookmark back, and process the history again.

- **One agreed order.** All consumers see the same events in the same order, so their copies of the truth do not drift apart for ordering reasons.

Six months later the data science team wants fraud scoring. In the direct design they must negotiate with the **order-service** team, wait for a new release, and accept that the new HTTP call slows every checkout. In the log design they write a consumer, start it at the oldest event and let it catch up. The **order-service** code does not change at all.

Why a *log* and not just a clever queue? Appending to the end of a file and reading it sequentially is what disks, operating systems and CPU caches do best. An append-only structure is also easy to replicate: followers copy the leader's log in the same order. The architecture part of this book shows how much of Kafka's performance and reliability comes from this one design choice.

WATCH OUT

A log in the middle is not free. You now operate (or pay for) a distributed system. Consumers see data slightly later than the producer wrote it, so the system is **eventually consistent**: the confirmation email may go out a second after the order. Events become a public contract, and changing their shape can break readers you have never met. Debugging a flow means following an event across several services instead of reading one stack trace. And delivery is normally *at least once*, so consumers must cope with seeing the same event twice. Later parts show how to handle each of these.

INTERVIEW ANGLE

Why put a broker such as Kafka between services instead of calling them directly? Because it decouples producers from consumers in space (they do not know each other), in time (a consumer can be down and catch up later) and in speed (a slow consumer does not slow the producer). It also turns $N \times M$ point-to-point integrations into $N + M$, and new consumers can be added years later without touching the producer. Direct calls remain the right tool when the caller needs an answer right now, for example to validate a card before confirming an order.

KEY POINTS

What to remember from this chapter

- Point-to-point integration needs up to $N \times M$ connections and couples systems in code, in time and in speed.
- Batch jobs are simple and efficient, but the data they deliver is as old as the last run.
- Kafka puts a durable, ordered, append-only log in the middle: producers append, consumers read at their own pace and keep their own bookmark.
- The log gives decoupling, replay and one agreed order, at the price of running a distributed system and of eventual consistency.

1.2 What Kafka Is (and What It Is Not)

Apache Kafka is an open-source, distributed **event streaming platform**. That phrase is dense, so this chapter unpacks it: what an event is, what "streaming" adds, and what you get when you run a Kafka cluster. The official introduction calls event streaming "the digital equivalent of the human body's central nervous system". Things happen at the edges of your business (a click, a payment, a sensor reading), signals travel along the nerves, and the right parts of the organisation react immediately.

1.2.1 Events

An **event** records the fact that something happened: a customer placed order **o-1001**, a payment was authorised, a visitor viewed a page. An event usually carries a description of what happened (the *value*), often something that says what it is about (the *key*, such as the order id), and a *timestamp*. Kafka's documentation uses the words event, record and message interchangeably. This book says **event** when it talks about the business meaning, and **record** when it talks about what Kafka stores and about the Java API (`ProducerRecord`, `ConsumerRecord`).

Two properties of events matter. First, they are *facts about the past*, so they are not edited. If an order is cancelled, you append an `OrderCancelled` event; you do not rewrite `OrderCreated`. Second, their *order* is information: it matters that the payment was authorised after the order was created. A structure that keeps events in order and never changes them is a log, and the log is the data structure Kafka is built around.

1.2.2 The three capabilities

Kafka's documentation names three key capabilities of an event streaming platform. They also make a handy map of this book.

1. **Publish and subscribe.** Write streams of events into Kafka and read them out again, including the continuous import and export of data from and to other systems.
2. **Store.** Keep streams of events durably and reliably for as long as you want. Records are written to disk and copied to several servers. "As long as you want" is a policy that you set per topic: a number of days, a size, or forever.
3. **Process.** Work on streams of events as they occur, or retrospectively by reading history again.

Capability	Main building blocks	Where this book covers it
Publish and subscribe	Producer API, Consumer API, Kafka Connect	the parts on the Java producer, the Java consumer and Kafka Connect
Store	Brokers, partitions, replication, retention	the part on architecture and internals
Process	Kafka Streams, ksqldb, plain consumers	the part on Kafka Streams and ksqldb

The three capabilities and the parts of Kafka that deliver them.

1.2.3 Servers and clients

Kafka is a distributed system made of **servers** and **clients** that talk to each other over a binary protocol on top of TCP. The servers run as a cluster of one or more machines, and the documentation notes that a cluster can span several data centres or cloud regions. Some servers form the storage layer and are called **brokers**. Others run Kafka Connect to move data in and out of external systems. Your own applications are the clients: they use a client library to write events to the brokers (producers), read them (consumers), or process them.

The Apache Kafka project ships these Java APIs: the Producer API, the Consumer API, the Admin API (to manage and inspect topics, brokers and other objects), the Kafka Streams API for stream processing, and the Kafka Connect API for reusable import and export connectors. Client libraries for many other languages exist as well, and because the wire protocol is documented, anybody can write one.

1.2.4 A durable, replicated log

This is the mental model to carry through the book. A topic is a named stream of records. Inside Kafka, the records of a topic are written to **append-only logs** on disk, and every log is replicated to other servers. Four rules follow.

- Records are only ever *appended* at the end, and an existing record is never modified.
- Every record gets a position in its log, the **offset**, which only grows.
- **Reading does not delete.** A consumer remembers a position and moves it forward. Many consumers can read the same record independently, and a new one can start from the beginning.
- Records leave the log only by policy: **retention** removes records that are older than a configured age (seven days by default) or when the log gets too large, and **compaction** can keep just the latest record per key.

Everything else in this book, from consumer groups to transactions, is built on these four rules.

1.2.5 What Kafka is not

Being clear about the limits saves projects. Kafka is not the right tool for the following jobs, although you can find people who have tried all of them.

- **Not a replacement for your database.** Kafka stores history, but you cannot ask it "give me all customers in Pune with tier GOLD". There are no ad hoc queries and no secondary indexes. Kafka tells you what *happened*; a database tells you what is *true now* and lets you query it. Compacted topics and Kafka Streams state stores give you key lookups on the latest value, which is useful, but it is not a general query engine.
- **Not an RPC layer.** When a caller needs an answer right now (is this card valid?), use HTTP or gRPC. Request and reply over Kafka is possible (a pattern covered later) but adds latency and complexity.
- **Not a task scheduler.** Classic Kafka consumer groups have no delayed delivery, no priorities, no per-message timeouts. The newer share groups add queue-like per-record acknowledgement and redelivery, but not priorities or scheduled delivery.

- **Not a file or blob store.** The default maximum record size is about 1 MiB. Put large files in object storage and send a reference through Kafka (the claim-check pattern, covered in the design patterns part).
- **Not magic.** Kafka does not give you exactly-once effects in external systems, schema discipline, or free operations. It gives you a very good log; the rest is engineering.

NOTE

Database engineers know this structure as a *commit log* or write-ahead log: every change is written to the end of a log first, and tables are just a convenient view of the log. Kafka makes that internal database structure a shared, replicated service for the whole organisation. That is why the Kafka documentation lists "Commit Log" among its use cases.

INTERVIEW ANGLE

What is Apache Kafka? A distributed, fault-tolerant event streaming platform. Producers append records to topics, which are split into partitions stored as replicated, append-only logs on a cluster of brokers; consumers read the records independently at their own pace. Records are retained by policy rather than deleted on read, so data can be replayed and processed as it arrives or later.

KEY POINTS

What to remember from this chapter

- An event is an immutable fact about something that happened; Kafka stores events as records in ordered, replicated, append-only logs.
- The three capabilities are publish and subscribe, durable storage, and stream processing.
- Reading never deletes. Consumers keep their own position, and retention or compaction decides what leaves the log.
- Kafka complements databases, RPC and job schedulers. It does not replace them.

1.3 The Building Blocks at a Glance

Kafka has a small vocabulary, and every later chapter leans on it. Instead of defining ten terms in a row, we first follow one order through the system and then collect the vocabulary in a table you can come back to.

1.3.1 One order, end to end

A customer, *c-77*, buys two pairs of shoes on ShopStream. Each Kafka term appears in bold the first time it is used.

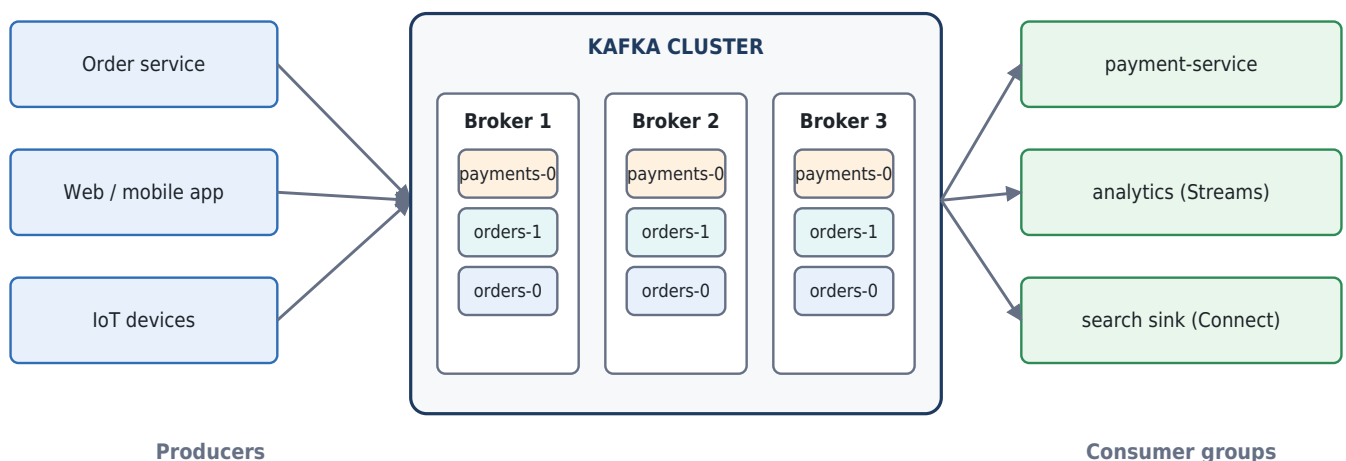


Figure 1.1. The big picture. Producers write to topics stored on a cluster of brokers, consumer groups read them independently, and Kafka Connect and Kafka Streams are simply more clients of the same cluster.

1. *order-service* saves the order in its own database and builds an *OrderCreated* event. It hands the event to a **producer**, the Kafka client library running inside the service.
2. The event is addressed to a **topic** named *orders*. The producer sets the order id *o-1001* as the record's **key**.
3. The producer knows the addresses of a few **brokers** (the *bootstrap.servers* setting). It asks one of them for the cluster's metadata: which **partitions** does *orders* have, and which broker is the **leader** of each? It hashes the key to choose a partition, say partition 2, and sends the **record** to the broker that leads it.

- The leader appends the record to the end of that partition's log. The record receives the next **offset**, say 41. Two **follower** replicas on other brokers copy it. With `acks=all` the leader reports success to the producer only after every in-sync replica has the record.
- `payment-service` runs several instances that all use the group id `payment-service`. Together they form one **consumer group**, and Kafka gives each partition of `orders` to exactly one instance of the group. The instance that owns partition 2 **polls**, receives offset 41, charges the card, and **commits** offset 42 ("the next record I want"). The commit is stored in Kafka itself.
- `inventory-service`, `notification-service` and `analytics-app` each have their own group. They read the same record independently, each at its own pace and with its own committed offsets. Nobody removes the record; it stays for the retention period (seven days by default).

What to notice: the producer talks directly to the broker that leads the partition, the key keeps all events of one order in one partition and therefore in order, and the services that read `orders` know nothing about each other.

1.3.2 Topics and partitions

A **topic** is a named stream of records: the thing you publish to and subscribe to. ShopStream uses topics such as `orders`, `payments` and `page-views`. Kafka's documentation compares a topic to a folder in a filesystem, with the events as the files in that folder. A topic is a logical name, not one file. Kafka splits every topic into one or more **partitions**, and each partition is an ordered, append-only log that is stored on one broker and copied to others.

Why split? One log on one machine would limit a topic to what that machine can store and serve, and only one consumer could read it at a time. Spreading partitions over many brokers lets a topic grow beyond a single machine, and lets many consumers read different partitions in parallel.

Which partition does a record go to? If the record has a key, the producer hashes the key, so records with the same key always land in the same partition. If there is no key, the producer spreads records across partitions. Kafka guarantees order only **within a partition**, which is why `orders` uses `orderId` as its key: every event of order `o-1001` stays in the order in which it was written.

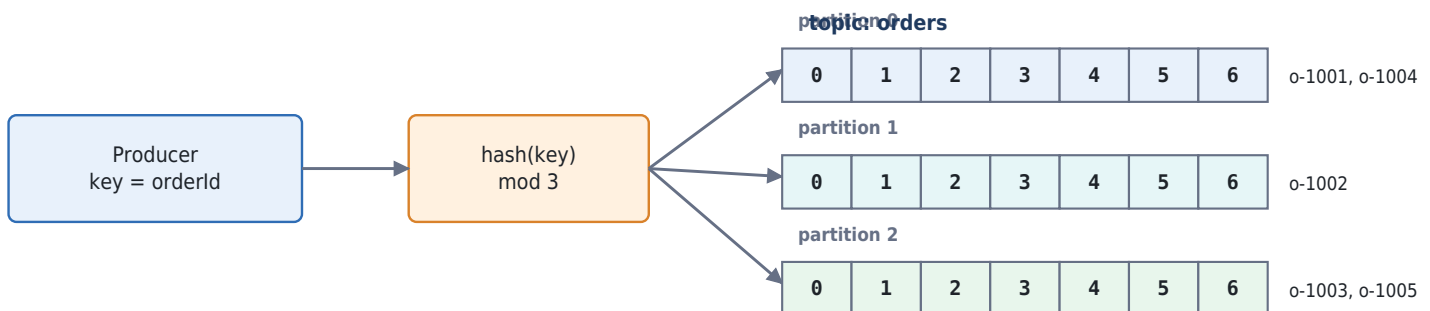


Figure 1.2. The topic `orders` with three partitions. The producer hashes the key, so every event of the same order lands in the same partition and keeps its order, while different orders spread across the partitions.

1.3.3 Offsets: a bookmark in the log

Inside a partition every record receives an **offset**, a sequential number that starts at 0 and only grows. The offset is the record's address and the consumer's bookmark. A consumer that has processed up to offset 41 will ask for offset 42 next; a new consumer may start at offset 0 and read the whole history; a consumer that crashed can resume where it left off.

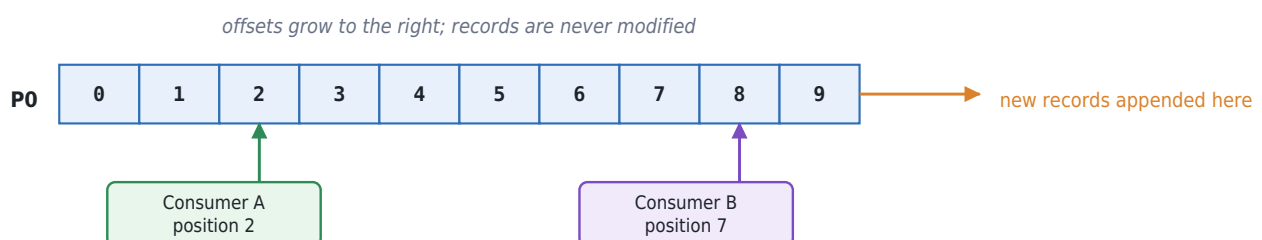


Figure 1.3. One partition as an append-only log. The producer adds records at the end; two consumers read the same partition at different offsets without affecting each other.

The triple (topic, partition, offset) identifies one record uniquely. Offsets belong to a partition, and they are never reused, so "offset 41" means nothing until you say in which partition. A consumer keeps two related numbers for each partition: its **position**, the offset of the next record it will receive, and its **committed offset**, the last position it saved in Kafka so that it can resume after a restart. The chapters on the Java consumer explain the difference in detail.

1.3.4 Consumer groups

A **consumer group** is a set of consumers that share one `group.id` and cooperate to read a topic. Kafka assigns each partition to exactly one consumer in the group at a time. If `orders` has three partitions and `payment-service` runs three instances, each instance reads one partition. If one instance crashes, Kafka moves its partition to a survivor, which is called a **rebalance**.

Two consequences are worth remembering now. A group can use at most as many consumers as the topic has partitions; extra consumers sit idle as standbys. And different groups are completely independent: each group gets every record and keeps its own offsets. This single design gives Kafka both classic messaging models, as the next chapter shows.

1.3.5 Brokers, clusters and replication

A **broker** is a Kafka server process that stores partitions on disk and serves producers and consumers. Several brokers form a **cluster**. Every partition has a configurable number of copies, the **replication factor**. One copy is the **leader**, which handles the writes, and the others are **followers** that continuously fetch from the leader and append the same records in the same order. Followers that keep up form the **in-sync replica set** (ISR). If the broker holding a leader fails, the cluster promotes an in-sync follower to be the new leader and clients carry on. A replication factor of 3, meaning three copies of your data, is the usual production choice.

Someone also has to keep track of cluster metadata: which topics exist, where the leaders are, what the settings are. Since Kafka 4.0 Kafka does this itself with a protocol called **KRaft**. A small group of **controller** nodes, often three, keeps the metadata in a replicated log, and brokers read from it. Older Kafka versions needed a separate ZooKeeper cluster for this. The architecture part of this book explains KRaft and replication in depth.

1.3.6 The vocabulary at a glance

Term	Meaning	ShopStream example
Record (event, message)	One entry in a log: an optional key, a value, a timestamp and optional headers	<code>OrderCreated</code> for order <code>o-1001</code>
Topic	A named stream of records; what you publish to and subscribe to	<code>orders</code> , <code>payments</code> , <code>page-views</code>
Partition	One ordered, append-only log holding a slice of a topic; the unit of ordering and parallelism	<code>orders</code> has partitions 0, 1 and 2
Offset	The position of a record inside one partition, starting at 0	partition 2, offset 41
Key	Optional record field; records with the same key go to the same partition	the order id <code>o-1001</code>
Broker	A Kafka server that stores partitions and serves clients	brokers 1, 2 and 3
Cluster	The brokers (and controllers) that work together	the three-node lab cluster
Controller	A node that manages cluster metadata through KRaft	three controllers in production
Producer	A client that appends records to topics	<code>order-service</code>
Consumer	A client that reads records from topics	one instance of <code>payment-service</code>
Consumer group	Consumers sharing a <code>group.id</code> ; each partition is read by one member at a time	<code>payment-service</code>
Committed offset	A group's saved bookmark for a partition: the next record to read	group <code>payment-service</code> , partition 2: 42
Replication factor	The number of copies of each partition	3
Leader and follower	The copy that takes the writes, and the copies that fetch from it	broker 2 leads partition 2
ISR	The replicas that are fully caught up with the leader	brokers 2, 1 and 3
Retention	The policy that deletes old records by age or size	7 days (the default)
Bootstrap servers	Starting addresses a client uses to discover the whole cluster	<code>localhost:9092</code>

The core vocabulary of Kafka, with ShopStream examples.

GOTCHA

Offsets are per partition, not per topic. There is no global sequence number across the partitions of a topic. Offset 41 exists once in every partition. This is also why Kafka cannot tell you which of two records in different partitions was "first" except by comparing timestamps, and why ordering requirements must be translated into key choices.

INTERVIEW ANGLE

How do consumer groups give Kafka both queue and publish-subscribe behaviour? Within one group each partition is read by exactly one consumer, so the group's members share the work like competing consumers on a queue. Each group, however, receives every record independently, like a separate subscriber in publish-subscribe. One topic can therefore feed a scalable worker pool and several unrelated applications at once.

KEY POINTS

What to remember from this chapter

- A producer appends records to a topic. The topic is split into partitions, and each partition is an ordered log in which every record has an offset.
- The key decides the partition, so records with the same key stay together and in order. Order exists only within a partition.
- A consumer group shares a topic's partitions between its members; different groups read independently and keep their own committed offsets.
- Each partition is replicated on several brokers: one leader takes the writes, followers copy it, and a follower takes over if the leader fails.

1.4 Kafka Compared with Other Tools

No tool is best at everything. This chapter places Kafka among the tools it is usually compared with, in design discussions and in interviews. The comparisons are conceptual and avoid performance numbers on purpose: results depend on workload, configuration and hardware, and products change quickly. Always check a product's current documentation and licence before you decide.

1.4.1 Two classic models and Kafka's hybrid

Messaging systems traditionally offer one of two models. In a **queue**, many consumers compete for messages and each message is processed by exactly one of them, which is ideal for sharing work but means a second application cannot see the same message. In **publish-subscribe**, every subscriber receives every message, which is ideal for broadcasting but gives you no way to share the work inside one subscriber. Kafka's consumer groups combine both: competing consumers inside a group, broadcast between groups.

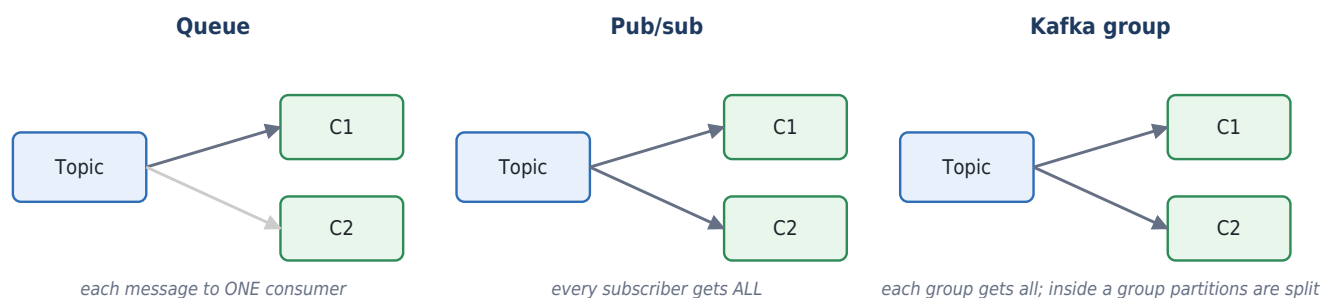


Figure 1.4. A queue delivers each message to one of several competing consumers, publish-subscribe delivers every message to every subscriber, and a Kafka topic with consumer groups does both: work is shared inside a group and every group sees everything.

1.4.2 Traditional message brokers

Classic brokers such as RabbitMQ and Apache ActiveMQ are built around queues. A producer sends a message, the broker routes it into one or more queues, delivers it to a consumer, waits for an acknowledgement and then deletes the message. The broker tracks the state of every message. Kafka inverts this: the broker stores an ordered log and does not care who has read it, while each consumer group tracks its own position and pulls the records it wants.

Aspect	Traditional broker (queue)	Kafka (partitioned log)
After a message is read	Deleted once acknowledged	Kept until retention expires
Delivery	Broker pushes, limited by a prefetch count	Consumer pulls with <code>poll()</code>
Who tracks progress	The broker, per message	The consumer group, one offset per partition
Replay of history	Normally not possible	Move the offset back and read again
Several independent readers	One queue (a copy) per subscriber	Any number of groups share one log
Ordering	Per queue; competing consumers can reorder	Per partition
Rich per-message features	Routing rules, priorities, TTL, delayed delivery	Keys and headers only; no priorities or delays
Scaling out	Mostly more consumers per queue	More partitions and more consumers

A fair comparison of the two designs. Both columns describe the typical case.

Be fair to both sides. Brokers excel at flexible routing, per-message TTLs, priorities and request-reply at moderate volume. RabbitMQ has also added a stream type that is a replayable append-only log, which shows how useful the log idea is. Kafka, on the other hand, now offers share groups, which give queue-style consumption of a topic with per-record acknowledgement. The two worlds are moving closer, but their defaults and strengths still differ.

1.4.3 Kafka and databases

A database stores the **current state** of your data and lets you query it in flexible ways. Kafka stores the **history of changes** and lets you read it in order. The two fit together more often than they compete.

- You can rebuild state by replaying a log of changes, and a table is just the latest value for each key in a stream of changes. This stream-table duality is the foundation of Kafka Streams and of compacted topics.
- A database answers "which customers in Pune are GOLD tier?" with an index. Kafka can only read by topic, partition and offset (or find an offset for a timestamp). Anything richer needs a consumer that builds a view of the data.
- The common production pattern is a database as the system of record and Kafka as the way changes leave it, using change data capture or the outbox pattern, both covered later. Some teams go further and treat the log as the system of record (event sourcing), accepting that every query then needs a projection.

1.4.4 Managed cloud messaging services

Every large cloud offers managed messaging. In outline: Amazon SQS is a managed queue, Amazon SNS fans notifications out to subscribers, Amazon Kinesis Data Streams is a managed partitioned stream, Google Cloud Pub/Sub is a managed publish-subscribe service with per-subscription acknowledgements, and Azure Event Hubs is a managed partitioned log. Microsoft's own documentation calls Kafka and Event Hubs "very similar", since both are partitioned logs where the client chooses which part of the retained log to read, and Event Hubs exposes a Kafka endpoint so that existing Kafka clients can connect after a configuration change. The same page lists limitations, so test your own workload.

The trade-off is usually the same. A managed service removes cluster operations and fits neatly into one cloud, but it ties you to that provider's API, limits and pricing. Kafka is an open protocol with many implementations and vendors, runs on your laptop as easily as in any cloud, and has the broader ecosystem of connectors and stream processing. For a small team that lives in one cloud and has modest needs, the native service can be the better choice.

1.4.5 Pulsar and Redpanda

Apache Pulsar is a separate Apache project with a different architecture. Its brokers are stateless, and messages are stored by Apache BookKeeper, a distributed log storage system, so serving and storage can scale separately. Geo-replication and tenant namespaces are part of its design, and it offers several subscription types, including queue-like shared subscriptions. The price is more moving parts to run (brokers, BookKeeper nodes and a metadata store) and a smaller surrounding ecosystem than Kafka's.

Redpanda is a Kafka-API-compatible streaming platform implemented in C++, so it runs without a JVM. Existing Kafka clients and many tools can talk to it. Compatibility is a strength, but it is never a guarantee of identical behaviour in every corner (transactions, administration, newer protocol features), so test with your own clients and check the licence terms. The same caution applies to every other system that speaks the Kafka protocol.

1.4.6 When to use Kafka, and when not to

Kafka is a strong choice when:

- Many systems need the same stream of events, and new consumers will appear over time.
- You need to replay history, or let consumers fall behind and catch up later.
- Volume is high or growing, and ordering per key matters (partitions scale, keys preserve order).
- You build event-driven microservices, change data capture pipelines, clickstream, log or metrics pipelines, or stream processing on live data.
- You want a durable buffer between systems that run at different speeds.

Think twice, or choose something else, when:

- The caller needs an immediate answer. Use HTTP or gRPC, not a log.
- You have a small job queue that needs delays, priorities or per-message timeouts. A broker, a cloud queue or a database-backed queue fits better (Kafka share groups help with some queue needs, not with these).
- You need ad hoc queries and multi-row transactions. That is a database's job.
- Messages are huge blobs. Store them in object storage and send a reference.
- You need one strict global order of all events. A single partition caps throughput, so rethink the requirement.
- The volume is tiny, the team is small and nobody can operate a cluster. A managed service or a simpler tool may save you real money and sleep.

REAL WORLD

Kafka's own "use cases" page lists messaging, website activity tracking, metrics, log aggregation, stream processing, event sourcing and commit log. Its "Powered By" page mentions, for example, LinkedIn using Kafka for activity stream data and operational metrics, Netflix running a real-time monitoring and event-processing pipeline on it, and Uber treating it as a central part of its infrastructure. Treat such lists as inspiration, not as proof that Kafka suits your problem.

INTERVIEW ANGLE

When would you not choose Kafka? When the interaction is request-response, when you need per-message features such as priorities or delays, when you need rich queries or transactions across records, when payloads are very large, or when the volume does not justify operating a cluster. In those cases an RPC call, a message broker or cloud queue, a database or object storage is usually the better fit.

KEY POINTS

What to remember from this chapter

- Queues share work between competing consumers; publish-subscribe broadcasts. Kafka's consumer groups do both.
- Brokers delete messages after acknowledgement and push them; Kafka keeps records, lets consumers pull, and lets them replay.
- Databases hold current state and answer queries; Kafka holds the history of changes. Use them together.
- Managed services trade portability for less operations work. Pulsar and Redpanda are real alternatives with different designs, and Kafka-protocol compatibility does not mean identical behaviour.

1.5 What Kafka Guarantees (and What It Does Not)

Before you trust a system with orders and payments, you need to know what it promises and under which conditions. This chapter states Kafka's guarantees in plain words. Each one comes with fine print, and later parts of the book explain the mechanics behind it.

1.5.1 The guarantees

Property	What Kafka promises	The fine print
Ordering	Records in one partition are stored and read in the order in which they were appended	Only within a partition. The default idempotent producer keeps the order even when it retries
Durability	A record is <i>committed</i> once every in-sync replica has it, and a committed record is not lost while at least one of those replicas survives	Needs <code>acks=all</code> . With replication factor 3 and <code>min.insync.replicas=2</code> , an acknowledged write sits on at least two brokers
Availability	If a leader fails, an in-sync follower is elected automatically and clients continue	Needs enough healthy brokers, and a controller quorum with a majority alive
Delivery	At least once when the consumer commits after processing; a crash can cause duplicates, but not a silent loss	At-most-once and exactly-once need different (explicit) designs
Replay	Records stay readable until retention deletes them, however often they were read	Seven days by default. A compacted topic keeps only the latest record per key
Scale	More partitions allow more parallel producers and consumers; more brokers hold more partitions	A group cannot use more consumers than partitions, and changing the partition count changes where keys go

What you can rely on, and the fine print that goes with it.

Two of these deserve a closer look because they are easy to misread.

Durability is about replication, not about forcing every write to disk. By default Kafka does not flush each write to disk before it acknowledges it. It acknowledges once the in-sync replicas have the record (in the operating system's page cache) and relies on the other replicas surviving the failure of a single machine. The architecture part of the book discusses this choice. For you it means that the replication factor and `min.insync.replicas` are your durability settings, not an afterthought.

Delivery is at least once unless you design otherwise. Suppose `payment-service` reads `OrderCreated` at offset 41 of partition 2, charges the customer's card, and crashes before it commits offset 42. After the restart the same record is delivered again, and without protection the customer is charged twice. Kafka did nothing wrong: it preferred

duplicates to loss. The fix is to make the consumer **idempotent**, for example by storing the payment id or event id and skipping events that were already handled. Kafka can also give exactly-once results for a read-process-write cycle that stays inside Kafka (transactions, covered later), but never for side effects in the outside world, such as a card charge or an email.

1.5.2 What is not guaranteed

- **Global order.** There is no order between records in different partitions. If the order of two kinds of events matters, give them the same key so they share a partition.
- **Delivery to every consumer of a group.** Inside a group each record goes to one member. Use separate groups for separate applications that each need every record.
- **Exactly-once side effects.** See above: idempotent consumers are your responsibility.
- **Unlimited memory of the past.** Data older than the retention setting is deleted, whether or not anybody read it.
- **Safety with weak settings.** With `acks=0` or `acks=1`, or with a replication factor of 1, a broker failure can lose acknowledged data. The guarantees above describe a sensibly configured cluster.
- **Valid data.** Kafka stores bytes. It does not check that a value is valid JSON or that a field exists; schemas are enforced by clients and a schema registry, covered in the serialization part.
- **Hard real-time latency.** Latency is typically low, but there is no upper bound that you can promise to a customer.

WATCH OUT

Do not read "at least once" as "exactly once in practice". Duplicates are rare in a healthy system, which makes them dangerous: the code path that handles them is the least tested one. Design every consumer so that processing the same event twice produces the same result as processing it once, and test that path on purpose.

INTERVIEW ANGLE

Does Kafka guarantee exactly-once delivery? Not by default. The default end-to-end behaviour is at least once. Kafka offers exactly-once semantics for the consume-transform-produce cycle inside Kafka by combining the idempotent producer, transactions and consumers that read only committed records (`isolation.level=read_committed`). Side effects outside Kafka still need idempotent consumers or a deduplication key.

KEY POINTS

What to remember from this chapter

- Order is guaranteed inside a partition only, so keys express your ordering requirements.
- Durability comes from replication: use `acks=all`, a replication factor of 3 and `min.insync.replicas=2`.
- Delivery is at least once by default, so consumers must tolerate duplicates.
- Retention, not consumption, decides how long data stays. Kafka stores bytes and does not validate them.

1.6 A Tour of the Kafka Ecosystem

Kafka is a core plus a large ecosystem. Knowing which piece solves which problem keeps you from writing code that a library already provides. This chapter is a map; the later parts of the book are the territory.

1.6.1 Shipped with Apache Kafka

Component	What it does	Covered in
Producer and Consumer APIs	Java client libraries (artifact <code>kafka-clients</code>) that write and read records	the parts on the Java producer and consumer
Admin API	Creates, inspects and changes topics, configurations, groups and ACLs from code	the producer and consumer parts, and the labs
Kafka Streams	A Java library for stateful stream processing that runs inside your own application, with no separate processing cluster	the part on Kafka Streams
Kafka Connect	A runtime with a REST API that runs source and sink connectors to move data between Kafka and other systems	the part on Kafka Connect
MirrorMaker 2	Replicates topics between clusters, built on Kafka Connect	the Connect part and the production part
Command-line tools	The <code>bin/kafka-*.sh</code> scripts for topics, groups, configs, logs and more	the hands-on part
Share groups	Queue-like consumption of a topic with per-record acknowledgement	the architecture part and the Spring production part

The parts that come in the Apache Kafka download.

1.6.2 Around Kafka

Category	Examples and what they do	Covered in
Schema registries	Confluent Schema Registry is the best known; Apicurio Registry and Karapace are open alternatives. They store schemas and check compatibility	the serialization part
SQL on streams	ksqlDB lets you write continuous queries in SQL on its own servers	the part on Kafka Streams and ksqlDB
Change data capture	Debezium provides connectors that turn database changes into Kafka events	the Connect and CDC part
Spring integration	Spring for Apache Kafka (<code>KafkaTemplate</code> , <code>@KafkaListener</code>)	the Spring parts
Other stream engines	Apache Flink and Apache Spark can read from and write to Kafka	not covered
Other languages	Clients based on the C library librdkafka, and pure-language clients for Go, Python, Node.js and more, exist	not covered
Managed services	Amazon MSK, Confluent Cloud, Aiven for Apache Kafka and the Kafka endpoint of Azure Event Hubs, among others	the production part
Kubernetes	Strimzi is an operator that runs Kafka on Kubernetes	the production part
UIs and tools	Community and commercial web UIs (for example AKHQ, Kafdrop, Kafbat UI) to browse topics and groups	the Docker Compose chapter
Monitoring	JMX metrics exported to Prometheus and Grafana; Cruise Control (an open-source tool from LinkedIn) helps to rebalance partitions	the production part

These are separate projects or products. Check their current status and licence before you adopt them.

The official documentation says there are "a plethora of tools that integrate with Kafka outside the main distribution" and keeps a community list on the project wiki. Treat every third-party tool as a dependency: look at its release activity, its licence, and whether it supports Kafka 4.x before you build on it.

TIP

When a problem sounds like a pipeline, look for a connector first. Copying rows from a database into Kafka, or topics into a search index, is almost never worth custom consumer code. Kafka Connect already handles offsets, retries, scaling and error routing. Write your own code for business logic, and use Connect for plumbing.

KEY POINTS

What to remember from this chapter

- Apache Kafka ships the clients, Kafka Streams, Kafka Connect, MirrorMaker 2 and the command-line tools.
- Schema registries, ksqldb, Debezium, Uls, operators and managed services are separate projects: check licence, maintenance and Kafka 4.x support.
- Use Connect for data movement and Streams for processing before you write consumers by hand.

1.7 A Short History and the Kafka 4.x Era

1.7.1 From LinkedIn to the Apache Software Foundation

Kafka was created at LinkedIn by a team that included Jay Kreps, Neha Narkhede and Jun Rao, to collect and distribute activity stream data and operational metrics. It was open-sourced in 2011 and became a top-level Apache project in

1. Several of its creators later founded Confluent, a company that builds

products around Kafka. The name is a reference to the writer Franz Kafka. Today Kafka is developed in the open under the Apache Software Foundation by contributors from many companies, and changes are discussed as **Kafka Improvement Proposals** (KIPs), each with a number such as KIP-500 or KIP-848.

1.7.2 From ZooKeeper to KRaft

For most of its life Kafka stored its cluster metadata (topics, partitions, leaders, configuration) in Apache ZooKeeper, a separate coordination service. That meant running and securing two distributed systems, and the metadata design limited how many partitions a cluster could handle and how fast it recovered from a controller failure. **KIP-500**, "Replace ZooKeeper with a Self-Managed Metadata Quorum", proposed that Kafka manage its own metadata in an internal replicated log, using a Raft-based protocol that became known as **KRaft**.

The road took several years: KRaft arrived as early access in Kafka 2.8, was declared production-ready in Kafka 3.3, and ZooKeeper mode was deprecated in Kafka 3.5. Kafka 3.9 was the last release that could still run with ZooKeeper, and Kafka 4.0 removed it. Since then KRaft is the only mode, which is why this book never mentions ZooKeeper again, except when you meet it in older material.

1.7.3 What each 4.x release added

Release	Announced	Highlights
4.0	March 2025	First release without ZooKeeper. The new consumer rebalance protocol (KIP-848) is generally available: broker side on by default, clients opt in with <code>group.protocol=consumer</code> . Share groups in early access (KIP-932). Brokers, Connect and tools need Java 17; clients and Streams need Java 11. Old protocol versions removed (KIP-896). Message formats v0 and v1, MirrorMaker 1 and <code>poll(long)</code> removed. Logging moves to Log4j 2
4.1	September 2025	Share groups in preview. Streams rebalance protocol (KIP-1071) in early access. Eligible leader replicas (ELR, KIP-966) enabled by default for new clusters
4.2	February 2026	Share groups production-ready. Streams rebalance protocol production-ready for its core features. Dead-letter-queue support in Kafka Streams exception handlers (KIP-1034). RENEW acknowledgement for share consumers (KIP-1222). Java 25 support. Consistent command-line option names (KIP-1147)
4.3	May 2026	Cordoning of brokers and log directories, so that no new partitions are placed there (KIP-1066). More configuration for share groups (KIP-1240). Partition size percentage metrics for storage monitoring (KIP-1257). Streams: a setting to remove stale state directories at startup (KIP-1259) and headers in state stores (KIP-1271). Follower fetch from a tiered offset (KIP-1023). The Scala wrapper for Kafka Streams is deprecated (KIP-1244)

Highlights of the Kafka 4.x releases (each KIP number is the proposal you can look up).

At the time of writing (October 2026) the supported lines are 4.3 (latest patch 4.3.1), 4.2 (4.2.2) and 4.1 (4.1.2). The book uses 4.3.1 for broker examples, and quotes defaults from the 4.2 documentation, which matches 4.3 unless stated.

1.7.4 Versioning and compatibility

Kafka versions are written major.minor.patch. A minor release (4.2, 4.3) adds features and arrives every few months; the four feature releases above were spread over about fourteen months. Patch releases (4.3.1) fix bugs. Features and removals are announced first as **deprecations**: the 4.2 release notes, for example, say that the old command-line spellings and some other items "will be removed in Kafka 5.0", which tells you today what to avoid.

The compatibility rules you should know are these.

- **Clients and brokers.** Since Kafka 4.0, a broker accepts only clients that are version 2.1 or newer, and 4.x clients need brokers of version 2.1 or newer (KIP-896). Clients from the 3.x line are fully compatible with 4.x brokers, and clients between 2.1 and 2.8 work with limits.
- **Java.** Brokers, Connect and the tools need Java 17 or newer. The Java clients and Kafka Streams need Java 11 or newer. Java 8 support ended with 4.0.
- **Upgrades.** A rolling upgrade restarts brokers one at a time and is finished by raising the feature level with `kafka-features.sh`; metadata downgrades are not supported. The production part walks through it.

Where to look things up: the **downloads page** (kafka.apache.org/downloads) links the release notes of each version, the **blog** (kafka.apache.org/blog) has a readable announcement for every release, the **upgrade guide** in the documentation lists every behaviour change, and the **KIP pages** on the Apache wiki explain why a change was made.

NOTE

Documentation is versioned. The documentation lives under a version-specific path, for example kafka.apache.org/42/ for 4.2. When a setting or a behaviour surprises you, check the page that matches your broker version.

KEY POINTS

What to remember from this chapter

- Kafka began at LinkedIn, was open-sourced in 2011 and became an Apache top-level project in 2012.
- KIP-500 replaced ZooKeeper by KRaft: early access in 2.8, production-ready in 3.3, deprecated ZooKeeper in 3.5, removed in 4.0.
- 4.x brought the new rebalance protocol, share groups (production-ready in 4.2) and many operational improvements.
- Clients and brokers must be 2.1 or newer to interoperate with 4.x; brokers need Java 17, clients Java 11.

1.8 How to Study This Book

This book is built for a learner who has never used Kafka and wants three things at the end: a mental model that survives interview questions, the ability to build real applications, and enough operational sense to run a cluster without fear. The path below gets you there in the order in which the ideas depend on each other.

1.8.1 The learning path

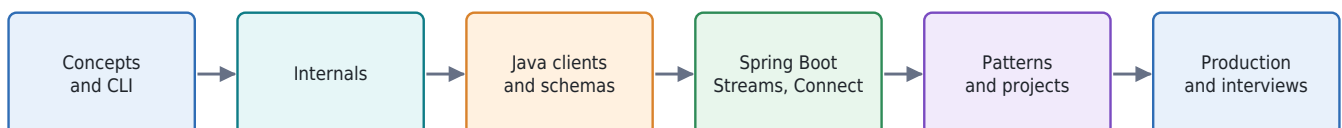


Figure 1.5. The route through the book. Fundamentals, hands-on work and internals build the mental model; the Java and Spring parts teach you to build; Streams, Connect, patterns and projects broaden and apply it; production and the revision kit finish the job.

Each part answers one question. What is Kafka and when do I use it? How do I run it and poke at it? How does it work inside? How do I write a producer and a consumer in plain Java? How do I model and evolve data? How do I do all of that with Spring Boot? How do I handle failure in production code? How do I process streams and move data in and out? Which design patterns are proven? Where is Kafka used? How do I put it all together? How do I operate it? And finally, how do I revise for an interview?

1.8.2 A four-week plan

The plan assumes about fifteen hours a week of reading and lab work. If you study part-time, stretch each week to two.

Week	Focus	Outcome
1	Kafka fundamentals, the hands-on labs, the architecture and internals	You can explain partitions, offsets, consumer groups, replication and the in-sync replica set from memory, and you ran every lab
2	The Java producer and consumer, serialization and schemas	You wrote a producer and a consumer, chose acknowledgements and commit strategies on purpose, and evolved a schema safely
3	Spring Boot with Spring Kafka, errors, retries and testing, then Kafka Streams	A Spring service publishes and consumes events with retries and a dead-letter topic; a Streams topology aggregates page views
4	Kafka Connect and CDC, design patterns, use cases, one end-to-end project, production, the revision kit	You can design an outbox or saga flow, you built one project, and you can answer the interview questions

A suggested four-week schedule. Do the labs, they are the point.

In a hurry before an interview? Read the fundamentals and the architecture parts, the reliability chapters of the producer and consumer parts, the patterns part, and then work through the revision kit. Spring-first reader? Do the fundamentals and the hands-on labs, then jump to the Spring parts and come back to the Java clients when something confuses you.

1.8.3 How the labs and listings work

- **Labs** are the amber banners. Each has a goal, numbered steps in dark terminal boxes, an explanation of what just happened, and a "try it yourself" exercise. Predict the output before you press Enter; the gap between your prediction and reality is where learning happens.
- **Terminal output** is typical output. Ids, offsets, timestamps and column spacing on your machine will differ.
- **Listings** use the package `com.shopstream` and the event names introduced in this book. Complete files include their imports; fragments say so.
- **Versions.** Broker examples use Kafka 4.3.1, plain Java uses `kafka-clients` 4.3.1, and the Spring examples take versions from Spring Boot 4.1.x and its Spring for Apache Kafka 4.1.x.

1.8.4 What to install

Tool	Version	Why
JDK	21 (17 is the minimum for brokers and tools)	Runs Kafka's tools and all the Java and Spring examples
Apache Kafka	4.3.1, Scala 2.13 binary	The command-line tools and a local broker
Docker with Compose	A current Docker Desktop or Docker Engine with the Compose plugin	Single-node and three-node clusters for the labs
Maven	A recent 3.9 or newer	Builds the Java and Spring projects (Gradle works too)
IDE	IntelliJ IDEA or VS Code	Writing and debugging the examples
Git, curl, jq	Any recent version	Fetching code, calling REST APIs, reading JSON

Everything you need, and why. Install the first four before the hands-on part.

Kafka's tools are shell scripts. On Windows, use WSL 2 or run the tools inside the Docker container. A laptop with several gigabytes of free memory is enough for everything except the three-node cluster, which starts six JVMs; close other heavy applications when you run it.

TIP

Keep a learning log. After every chapter write three lines: what problem it solves, one thing that surprised you, and one question you cannot yet answer. Review the questions at the start of the next session. The interview boxes in each chapter are a ready-made quiz: cover the answer and say yours out loud first.

KEY POINTS

What to remember from this chapter

- Study in dependency order: fundamentals, hands-on labs, internals, then the clients, then Spring, then Streams and Connect.
- Run every lab, and predict the output before you run it.
- Install a JDK 21, the Kafka 4.3.1 tarball, Docker with Compose and Maven before you start the hands-on part.

PART 2

Inside Kafka: Architecture and Internals

How Kafka stores, replicates, serves and coordinates data, so that you can predict its behaviour instead of memorising its settings.

In this part

- 2.1 Topics, Partitions and the Commit Log
- 2.2 Records, Batches and the Wire Format
- 2.3 Brokers, the Controller and KRaft
- 2.4 Replication: Leaders, Followers and the ISR

The earlier Parts showed what Kafka is and how to drive it from the command line. This Part opens the hood. Its goal is practical: when you meet a symptom (a consumer group that keeps rebalancing, a producer that times out, a partition reported as "under-replicated"), you should be able to say which internal mechanism is responsible and which setting controls it. It is also where most of the deep interview questions about Kafka come from.

Two ideas organise the material. The **data plane** is where records live and move: partitions, logs, segments, replication, produce and fetch requests. The **control plane** decides who does what: which broker leads which partition, which consumer reads which partition, which transaction is open. We start with partitions and records, then meet the controller quorum that manages cluster metadata, and then follow the data through replication, the request path, storage and retention. The last chapters cover the two coordination protocols that clients depend on, consumer groups and transactions, and finish by following one message from `send()` to a processed record, marking every place where it could be lost or duplicated.

2.1 Topics, Partitions and the Commit Log

A topic is the name under which related records are published, such as `orders` or `payments`. But a topic is not where the data physically lives. If a topic were one long sequence on one machine, its size, its write rate and its read rate would all be limited by that single machine, and every consumer would have to read the same single stream. Kafka's answer is to split each topic into **partitions**. A partition is a small, self-contained, append-only log; a topic is a named group of them. This one design decision explains most of what you will observe in practice: how Kafka scales, what it promises about ordering, why consumer groups behave the way they do, and why adding partitions later is risky.

2.1.1 The commit log

A **log**, in this sense, is not a text file full of debug messages. It is one of the oldest and simplest data structures in computing: a sequence of records that you can only append to at the end and read by position. Databases keep one internally (the write-ahead log, or commit log) so that they can recover after a crash. Kafka makes the log the product itself and calls each position an **offset**.

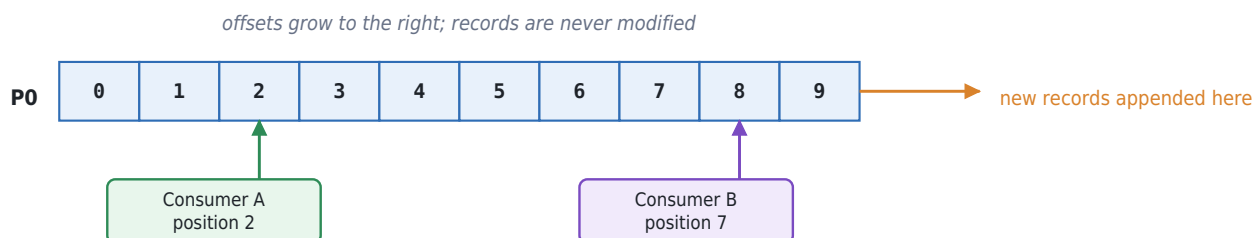


Figure 2.1. One partition is an append-only log. The producer adds at the end (here offset 9 is the newest record). Every consumer keeps its own position, so two consumers can read the same data at different speeds without affecting each other or the log.

Four properties of a partition do most of the work:

- **Append-only and immutable.** New records go at the end. A record is never modified in place; "updating" something means appending a newer record.

- **Sequential offsets.** The broker gives each record the next number (0, 1, 2, ...) as it is appended. Offsets are never reused, even after old records have been deleted. An offset only has meaning inside one partition: the triple (topic, partition, offset) identifies one record.
- **Read by position.** A consumer says "send me records starting at offset 42". The broker does not track which records have been delivered; the consumer remembers its own position. Records stay until retention removes them (by age or size), whether or not anybody has read them.
- **Sequential I/O.** Appending at the end and reading in order are both sequential access patterns, which disks, the operating system's page cache and networks handle best. The chapter on the storage engine shows how much of Kafka's speed comes from this.

Because nothing is removed when it is read, one log can serve many independent readers. The payment service, an analytics job and a brand new search indexer can all read `orders` at their own pace, and a new consumer can start from the beginning to rebuild its state. This "many readers, replayable" behaviour is the heart of the difference between a log and a traditional queue.

2.1.2 A topic is a set of partitions

A topic with N partitions is N independent logs numbered 0 to N-1. Each has its own offsets, so offset 5 in partition 0 and offset 5 in partition 2 are unrelated records. Kafka places each partition on a broker (its **leader**) and copies it to other brokers for safety, as the chapter on replication explains. A partition is never split across brokers.

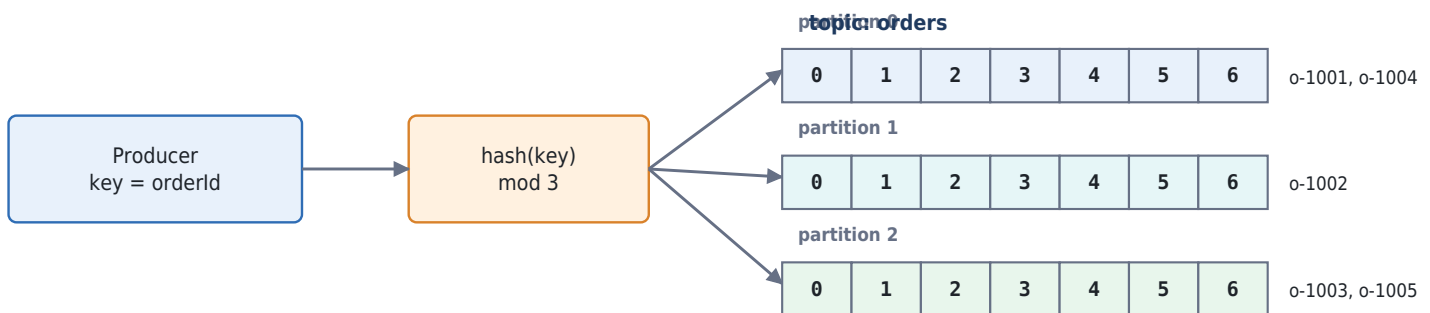


Figure 2.2. Topic orders with three partitions. A record that has a key goes to the partition chosen by hashing the key, so every event of one order lands in the same partition. Each partition has its own offsets and is read independently.

Describing a real topic shows the structure. The cluster below has three brokers with ids 1, 2 and 3, and the topic was created with three partitions and replication factor 3. The fields **Leader**, **Replicas** and **Isr** are explained in the chapter on replication; for now notice that each partition is its own line with its own leader.

Describe the orders topic (typical output; the real tool separates fields with tabs and prints Configs on the first line)

TERMINAL

```

$ bin/kafka-topics.sh --describe --topic orders --bootstrap-server localhost:9092
Topic: orders TopicId: 5kVv0Q6aTYm4d1WmTt0u2g PartitionCount: 3 ReplicationFactor: 3
  Configs: min.insync.replicas=2
Topic: orders Partition: 0 Leader: 2 Replicas: 2,3,1 Isr: 2,3,1
Topic: orders Partition: 1 Leader: 3 Replicas: 3,1,2 Isr: 3,1,2
Topic: orders Partition: 2 Leader: 1 Replicas: 1,2,3 Isr: 1,2,3
  
```

A partition is the unit of five different things, and it helps to keep them apart when you reason about Kafka.

Unit of	What it means in practice
Ordering	Kafka keeps records in order inside a partition. It makes no promise about order between partitions.
Parallelism	In a consumer group each partition is read by at most one member at a time (share groups, described later, are the exception), so the partition count is the ceiling for consumer parallelism. Producers also write to partitions in parallel.
Storage	A partition is a directory of files on a broker's disk. It cannot grow beyond what the leader's disks can hold, and spreading partitions evenly is how a cluster balances load.
Replication	Every partition has its own leader, its own followers and its own set of in-sync replicas.
Failure	When a broker fails, every partition it led needs a new leader. More partitions means more elections and more recovery work.

A partition is the unit of five different things.

2.1.3 Which partition does a record go to?

The producer decides, and it follows this order:

1. If the record names a partition explicitly (`new ProducerRecord<>(topic, partition, key, value)`), that partition is used.
2. Otherwise, if the record has a key, the partition is `toPositive(murmur2(keyBytes)) % numPartitions`. Murmur2 is a fast, non-cryptographic hash. It is applied to the **serialized** key bytes, and `toPositive` clears the sign bit so that the result is never negative.
3. Otherwise (no key) the producer's built-in logic spreads records across partitions. It is *sticky*: it keeps using one partition until about one batch worth of data (`batch.size`) has gone to it and then switches, because a few large batches are cheaper than many tiny ones. With the default `partitioner.adaptive.partitioning.enable=true` it also sends more records to partitions whose brokers respond faster.

The setting `partitioner.class` is `null` by default, which means "use the built-in logic above". The old `DefaultPartitioner` and `UniformStickyPartitioner` classes were removed in Kafka 4.0; their behaviour is simply built in now. If you want the key to be ignored for partitioning, there is `partitioner.ignore.keys` (default `false`).

Because the hash is computed over bytes, two producers that serialize the "same" key differently (say, one as plain text and one as JSON) send it to different partitions. Clients in other programming languages may also use a different default hash function. If a topic is written by more than one kind of client, check that they agree.

The listing below computes the partition for a few ShopStream order ids with Kafka's own hash function, once for three partitions and once for four.

WhichPartition.java

JAVA

```
package com.shopstream.demo;

import java.nio.charset.StandardCharsets;
import java.util.List;
import org.apache.kafka.common.utils.Utls;

public class WhichPartition {

    static int partitionFor(String key, int numPartitions) {
        byte[] keyBytes = key.getBytes(StandardCharsets.UTF_8);
        return Utls.toPositive(Utls.murmur2(keyBytes)) % numPartitions;
    }

    public static void main(String[] args) {
        List<String> keys = List.of("o-1001", "o-1002", "o-1003", "o-1004", "o-1005", "o-1006");
        for (String key : keys) {
            System.out.printf("%s 3 partitions -> %d 4 partitions -> %d%n",
                              key, partitionFor(key, 3), partitionFor(key, 4));
        }
    }
}
```

Output

```
o-1001 3 partitions -> 1 4 partitions -> 1
o-1002 3 partitions -> 1 4 partitions -> 3
o-1003 3 partitions -> 2 4 partitions -> 3
o-1004 3 partitions -> 2 4 partitions -> 1
o-1005 3 partitions -> 2 4 partitions -> 3
o-1006 3 partitions -> 1 4 partitions -> 3
```

NOTE

`Utls` is an internal helper class of the `kafka-clients` jar, not a supported public API, so use it only for experiments like this one. `StringSerializer` encodes strings as UTF-8, which is why the demo uses UTF-8 bytes too.

Going from three to four partitions moved five of the six keys. That is typical rather than unlucky. For a uniform hash, a key keeps its partition when the count goes from N to $N+1$ with probability only $1/(N+1)$. The next section shows why that matters.

2.1.4 Adding partitions: what changes and what breaks

You can increase the partition count of a topic but never decrease it; the documentation says plainly that Kafka does not support reducing the number of partitions.

Increase the partition count of orders from 3 to 6

TERMINAL

```
$ bin/kafka-topics.sh --alter --topic orders --partitions 6 --bootstrap-server localhost:9092
```

Adding partitions does not move any existing record. Old records stay in their old partitions, and only new records are routed with the new count. For a keyed topic this has consequences:

- **Per-key ordering breaks across the change.** Order `o-1003` had its history in partition 2. After the change its next event may go to partition 5, and a different consumer instance will process it, possibly before the old partition's backlog has been handled. For a short time the events of one order can be processed out of order.
- **Compacted topics keep stale copies.** Compaction works inside one partition. The old value of a key stays in the old partition forever (nothing newer arrives there to replace it), while the new value lives in the new partition. A reader that merges all partitions into a table may see two values for one key.
- **Stateful consumers lose their locality.** A Kafka Streams state store, a local cache or a per-partition aggregate was built on the assumption that a key always arrives in the same partition. Joins in Kafka Streams also assume that both topics have the same number of partitions.

GOTCHA

A consumer that has no committed offset for a newly added partition applies `auto.offset.reset`. With the default `latest` it starts at the end of that partition at the moment it learns about it. Records that producers appended in the gap (producers refresh their metadata, and consumers theirs, at different moments) are skipped. For topics that grow partitions, `earliest` is the safer choice.

The safe approaches are all variations on "decide generously at the start". If you must change the count of a topic whose key ordering matters, stop the producers, let the consumers drain the topic, add partitions, and restart. Or create a new topic with the desired count, copy or dual-write into it, and move the consumers over. Both are projects, not one-liners.

2.1.5 How many partitions?

There is no magic number, but there is a reasoned way to choose. Start from three questions.

1. **Throughput.** How much data per second must the topic absorb at peak? Measure what one partition sustains on your hardware (the producer and consumer performance tools in the CLI toolbox exist for this) and divide.
2. **Consumer parallelism.** A consumer group cannot use more consumers than there are partitions, so the partition count must be at least the largest number of consumer instances you will want to run. Measure what one consumer instance can process, including your own business logic, and divide the target throughput by it.
3. **Growth and key distribution.** Because the count cannot be changed freely later, size for the load you expect in a year or two, not for today. Prefer numbers with many divisors, such as 6, 12 or 24, so that 2, 3, 4 or 6 consumers share the partitions evenly.

The sizing rule of thumb

$\text{partitions} \geq \max(T / p, T / c)$ then add headroom, then round to a friendly number

T = peak throughput the topic must handle

p = throughput one partition sustains for producers (measure it)

c = throughput one consumer instance can process (measure it, with your real logic)

An illustrative example (made-up numbers, not a benchmark): ShopStream's `page-views` topic must absorb 10 MB/s at peak. Measurements say one partition accepts 20 MB/s of writes, but one consumer instance, doing real work per event, processes only 2 MB/s. Producers need $10/20$, so one partition; consumers need $10/2 = 5$. Take 5, double it for growth, round to 12. The consumers set the answer, not Kafka's write speed. The `orders` topic is the opposite case: it carries little data, so throughput alone would say one partition, but you may want up to six `payment-service` instances for availability and rolling deployments, so 6 is chosen for parallelism.

More partitions are not free. Every partition adds costs that grow with the count:

- Each replica of each partition is a directory with several open files per segment (log, offset index and time index) on a broker.

- Followers must fetch every partition they replicate, so replication work and the number of small requests grow.
- When a broker fails, all the partitions it led need new leaders, and when it comes back each one must be recovered and caught up.
- Cluster metadata, which every broker and client keeps and refreshes, grows with the partition count.
- Clients hold memory per partition: the producer keeps a batch per partition and the consumer buffers fetched data per partition.

KRaft (described in the next chapters) raised the number of partitions a cluster can manage compared with the ZooKeeper era, but it did not make partitions free. The production chapters on capacity planning give sizing guidance.

WATCH OUT

Do not create 1000 partitions "to be safe". Over-partitioning wastes memory and file handles, slows recovery after failures, and still cannot be undone, since partitions can never be removed. Pick a reasoned number, measure, and grow when the numbers say so.

2.1.6 Ordering, stated precisely

What exactly does Kafka guarantee about order?

- Records that one producer sends to one partition are appended in the order they were sent. With the idempotent producer (the default in Kafka 4.x), retries cannot reorder them.
- A consumer reads a partition in offset order.
- Therefore all records with the same key are read in the order they were produced, as long as they went to the same partition, which means as long as the partition count stayed the same.
- There is no order between partitions, between topics, or between two producers writing to the same partition at the same time (the order in which their batches reach the leader decides).

In ShopStream the topics `orders`, `payments`, `inventory` and `shipments` all use `orderId` as the key, so the events of one order are ordered within each topic. The `StockReserved` event of an order is still not ordered relative to its `PaymentAuthorized` event, because they are in different topics. Services must be written to cope with that, which is what the saga and state-machine patterns in the Part on design patterns are about.

INTERVIEW ANGLE

A topic has 6 partitions and you start 10 consumers in the same group. What happens, and how do you keep events of one customer in order? Six consumers each get one partition and the other four sit idle: a group's parallelism is capped by the partition count. To keep one customer's events in order, use the customer id as the record key. The same key always hashes to the same partition, one consumer reads that partition, and offsets within it are strictly increasing. The guarantee holds only while the partition count is unchanged.

KEY POINTS

- A topic is a set of independent append-only logs (partitions). Offsets are per partition, never reused, and (topic, partition, offset) identifies a record.
- The partition is the unit of ordering, parallelism, storage placement, replication and failure. Kafka orders records only within a partition.
- A keyed record goes to `murmur2(keyBytes) % numPartitions` (made positive); keyless records are spread by the producer's sticky, adaptive logic.
- Adding partitions never moves old data but re-routes new keys, which breaks per-key ordering, compaction assumptions and partition-local state. Partitions cannot be removed.
- Choose the count from measured throughput, the largest consumer count you will need and expected growth, then round to a number with many divisors.

2.2 Records, Batches and the Wire Format

When you call `send()` you think about one record: a key, a value, maybe a header or two. Kafka does not move or store records one at a time. Producers group records into **batches**, brokers append whole batches to the log, and consumers fetch whole batches back. Understanding the record and the batch explains several things that otherwise look odd: why compression is configured on the producer, why a "message size limit" is really a batch size limit, why offsets sometimes skip a number, and why a producer waits a few milliseconds before sending.

2.2.1 What a record contains

A record has the parts below. The topic and partition say where it lives; everything else is the record itself.

Part	Set by	Notes
key	producer	Optional bytes. Decides the partition (when none is given) and the identity of the record for log compaction. Null means "no key".
value	producer	Optional bytes: your payload. A null value is a tombstone : in a compacted topic it means "delete this key".
headers	producer	Zero or more pairs of a String name and an optional byte array, for metadata such as an event type, a trace id or a content type. Names may repeat. Headers do not affect partitioning.
timestamp	producer or broker	Milliseconds since the epoch. Which clock is used depends on the topic's timestamp type (below).
offset	broker	The record's position in its partition, assigned when it is appended. The producer cannot choose it.

The parts of a record.

Kafka never looks inside the key or the value. They are opaque byte arrays, and turning objects into bytes (and back) is the job of serializers, covered in the Part on serialization and schemas.

2.2.2 From records to record batches

Sending one record per network request would waste almost everything: the request header, a round trip, a checksum, an index update. So the producer collects records headed for the same partition into a **batch** and sends the batch as one unit. (The accumulator that does this is described in the Part on the Java producer; `batch.size` and `linger.ms` control it.) The batch is also the unit that the broker validates, appends and replicates, and that consumers fetch. If a consumer asks to start at offset 1 and offsets 0 to 2 live in one batch, the broker returns the whole batch and the client discards the records before offset 1.

Since version 0.11 Kafka has used one on-disk and on-the-wire format, called **magic v2** (the "magic byte" is the format version, and its value is 2). Kafka 4.0 removed support for the two older formats. A v2 batch looks like this:

Record batch layout (magic = 2)

baseOffset	int64	offset of the first record in the batch
batchLength	int32	number of bytes after this field
partitionLeaderEpoch	int32	stamped by the leader when it appends
magic	int8	format version (2)
crc	uint32	CRC-32C of everything from attributes on covered by the CRC
attributes	int16	compression, timestamp type, txn, control
lastOffsetDelta	int32	offset of the last record minus baseOffset
baseTimestamp	int64	timestamp of the first record
maxTimestamp	int64	largest timestamp in the batch
producerId	int64	-1 if the producer is not idempotent
producerEpoch	int16	used to fence old producers
baseSequence	int32	sequence number of the first record
recordsCount	int32	number of records
records (if compressed, this whole part is one compressed block) each record: length, attributes, timestampDelta, offsetDelta, key length + key, value length + value, headers		

The fixed header adds up to 61 bytes per batch (8+4+4+1+4+2+4+8+8+8+2+4+4), no matter how many records the batch holds. That is the main saving of batching: the per-record cost is only a few bytes of lengths and deltas. Four details are worth remembering.

- **Deltas keep records small.** Each record stores its offset and timestamp as a (variable-length) difference from `baseOffset` and `baseTimestamp`, so consecutive records cost a byte or two for each.
- **Producer fields enable idempotence.** `producerId`, `producerEpoch` and `baseSequence` let the broker detect duplicates and fence zombie producers. The chapter on delivery semantics builds on them.
- **The leader epoch tells replicas which "era" wrote the data.** The partition leader stamps `partitionLeaderEpoch` when it appends. The field sits before the checksum and is not covered by it, so the broker can set it without recomputing the CRC. The chapter on replication shows how epochs are used to find and remove diverging data after a leader change.
- **The attributes field packs flags.** The table below lists them.

Bits	Meaning
0 to 2	Compression codec: 0 none, 1 gzip, 2 snappy, 3 lz4, 4 zstd
3	Timestamp type: 0 CreateTime, 1 LogAppendTime
4	Transactional batch
5	Control batch (a transaction marker, never shown to applications)
6	The base timestamp field holds the delete horizon (used by log compaction)
7 to 15	Unused

The attributes field of a batch (16 bits).

A **control batch** holds a single control record: a commit or abort marker written when a transaction finishes. Consumers use it to filter out aborted data and never hand it to your code. But it does occupy an offset, so in a topic written with transactions you will see gaps of one offset where markers sit. Never assume that two consecutive records you receive have consecutive offsets.

2.2.3 Compression works on batches

Compression is set on the producer with `compression.type`: `none` (the default), `gzip`, `snappy`, `lz4` or `zstd`. The records of a batch are compressed together as one block, which is why batching and compression reinforce each other: a larger batch holds more similar-looking records (the same JSON field names over and over) and compresses better. The batch header stays uncompressed so the broker can read offsets, timestamps and producer fields without decompressing anything.

The broker has a topic setting, also called `compression.type`, whose default is `producer`. It means "keep whatever codec the producer used", so the broker can store the batch as it arrived and later serve it to consumers and followers unchanged. The consumer decompresses it. If you force a topic to a different codec from the one producers use, the broker must decompress and recompress every batch, which costs CPU on the busiest machines in your cluster. The sensible split is: choose the codec on the producer (the Part on the Java producer discusses the trade-offs between the four codecs) and leave the topic on `producer`.

2.2.4 Timestamps: CreateTime and LogAppendTime

Every record carries a timestamp. Which clock it comes from is a per-topic choice, `message.timestamp.type`:

- **CreateTime** (the default): the timestamp the producer put in the record, normally the time of the `send()` call, or a value you set explicitly. This is event time as seen by the producer.
- **LogAppendTime**: the broker overwrites the timestamp with its own clock when it appends the batch. Use it when you cannot trust the clocks of the producers and you do not need event time.

Setting	Default	What it does
<code>message.timestamp.type</code>	<code>CreateTime</code>	Which clock fills the timestamp field
<code>message.timestamp.before.max.ms</code>	<code>Long.MAX_VALUE</code>	How far in the past (compared with the broker's clock) a <code>CreateTime</code> may be before the broker rejects the record
<code>message.timestamp.after.max.ms</code>	<code>3600000</code> (1 hour)	How far in the future a <code>CreateTime</code> may be before the broker rejects the record

Topic settings that govern timestamps.

The "after" limit became one hour in Kafka 4.0 (before, it was effectively unlimited), because one producer with a wrong clock could write records stamped far in the future. Timestamps matter more than they seem: time-based retention looks at the timestamps in a segment, the time index behind "give me the offset for this time" lookups is built from them, and Kafka Streams uses them as event time for windows and joins. A bad clock therefore does not just make a record look odd; it can delay deletion and confuse time-based replay.

2.2.5 Size limits

The size limits all talk about batches, not individual records, and they live in three places.

Setting	Where	Default	What it limits
<code>max.request.size</code>	producer	1048576	The size of a produce request and, in effect, of one uncompressed batch. A record that is too large fails on the client with <code>RecordTooLargeException</code> .
<code>max.message.bytes</code>	topic	1048588	The largest record batch the broker accepts for this topic, measured after compression. Its default comes from the broker setting <code>message.max.bytes</code> .
<code>max.partition.fetch.bytes</code> , <code>fetch.max.bytes</code>	consumer	1048576, 52428800	How much data one fetch returns per partition and in total. They are soft limits (see below).

Where message size is limited.

The broker default 1048588 is exactly 1 MiB plus the 12-byte prefix (base offset and length) that precedes every batch. The consumer limits are deliberately soft. If the first batch of the first non-empty partition is larger than the limit, the broker still returns it, so a consumer can always make progress. A limit that is too small therefore slows a consumer down but does not strand it.

To allow larger messages for one topic you raise `max.message.bytes` on that topic and `max.request.size` on the producers that write to it. Before you do, ask whether the payload belongs in Kafka at all. Large messages inflate memory use on producers, brokers and consumers, delay everything queued behind them in the same partition, and make every fetch heavier. The claim-check pattern in the Part on design patterns (store the file elsewhere, send a reference) is usually the better answer.

2.2.6 Looking at a batch on disk

You can inspect real batches with the dump tool. The sample below shows one batch holding three `orders` records that were stored in partition 1. (The partition was chosen by the key hash as in the previous chapter: both `o-1001` and `o-1002` map to partition 1 of a three-partition topic.) The numbers are examples; the real tool prints each batch on a single long line, wrapped here to fit the page. The lab on looking inside the log in the hands-on Part shows how to find the right file.

Dump a segment file with its records (typical output, wrapped)

TERMINAL

```
$ bin/kafka-dump-log.sh --print-data-log \
  --files /var/lib/kafka/data/orders-1/00000000000000000000.log
Dumping /var/lib/kafka/data/orders-1/00000000000000000000.log
Starting offset: 0
baseOffset: 0 lastOffset: 2 count: 3 baseSequence: 0 lastSequence: 2 producerId: 3001
producerEpoch: 0 partitionLeaderEpoch: 0 isTransactional: false isControl: false
deleteHorizonMs: OptionalLong.empty position: 0 CreateTime: 1791454530131 size: 263
magic: 2 compresscodec: none crc: 2643010711 invalid: true
| offset: 0 CreateTime: 1791454530100 keySize: 6 valueSize: 58 sequence: 0 headerKeys: []
| key: o-1001 payload: {"type":"ORDER_CREATED","orderId":"o-1001","total":998.00}
| offset: 1 CreateTime: 1791454530112 keySize: 6 valueSize: 58 sequence: 1 headerKeys: []
| key: o-1002 payload: {"type":"ORDER_CREATED","orderId":"o-1002","total":250.00}
| offset: 2 CreateTime: 1791454530131 keySize: 6 valueSize: 45 sequence: 2 headerKeys: []
| key: o-1001 payload: {"type":"ORDER_CONFIRMED","orderId":"o-1001"}
```

What to notice: the three records share one batch header, so `baseOffset` is 0 and `lastOffset` is 2. The batch is 263 bytes in total, which is the 61-byte header plus the three small records. The batch `CreateTime` is the largest timestamp of the batch, and each record carries its own. `producerId` and the `sequence` numbers show that the producer was idempotent (the default); a producer with idempotence turned off would show -1 there. `compresscodec: none` means the producer did not compress.

INTERVIEW ANGLE

Is compression applied per message or per batch, and who does the work? Per batch. The producer compresses the records of one batch together, the broker normally stores the batch exactly as it arrived, and the consumer decompresses it. Bigger batches compress better, and because the topic default is `compression.type=producer` the broker avoids recompressing, so compression saves network, disk and replication bandwidth without moving the work onto the brokers. Forcing a different codec on the topic makes the broker recompress, which is usually a mistake.

KEY POINTS

- Kafka stores and moves record batches, not single records. The batch is the unit of compression, checksum, append, replication and fetch.
- A v2 batch has a fixed 61-byte header (base offset, timestamps, producer id, epoch, sequence, attributes, CRC-32C) followed by delta-encoded records.
- Compression is chosen by the producer and covers a whole batch; with the topic default `producer` the broker keeps the batch as received.
- `CreateTime` is the producer's clock and `LogAppendTime` the broker's. Retention, time-based offset lookups and Streams event time all depend on timestamps.
- Size limits apply to batches and exist on the producer, the topic and the consumer. Prefer small messages and the claim-check pattern to raising limits.

2.3 Brokers, the Controller and KRaft

A Kafka cluster is a group of **brokers**: JVM processes that store partitions and answer produce and fetch requests. But someone also has to hold the cluster's *metadata*: which topics exist, how many partitions each has, which brokers hold their replicas, who leads each partition, which brokers are alive, what the configs and ACLs are. Whoever holds that truth takes every important decision, so it must be consistent and must survive failures. For its first decade Kafka kept the metadata in a separate system, Apache ZooKeeper. Since Kafka 4.0 there is no ZooKeeper. Kafka stores its metadata in its own replicated log and keeps that log consistent with its own implementation of the Raft consensus protocol. This mode is called **KRaft** (Kafka Raft), and it is the only mode in the 4.x line.

2.3.1 The metadata log

In KRaft the cluster metadata is an event log, just like application data. It lives in an internal single-partition log named `__cluster_metadata`. Every change to the cluster is a record appended to it: a topic is created, a partition gets a new leader, a broker registers, a config changes, a feature level is raised. The current state of the cluster is simply the result of replaying that log from the beginning (or from a snapshot, below). It is not an ordinary topic: clients cannot read or write it, and it is replicated by the Raft protocol of the controller quorum rather than by the ISR mechanism described in the next chapter.

Every Kafka process is a **node** with a unique `node.id`, and the setting `process.roles` decides what it does.

process.roles	Meaning
broker	A data node. Stores partitions and serves produce, fetch, consumer group and other client requests.
controller	A metadata node. Takes part in the controller quorum that stores and manages the metadata log.
broker,controller	Combined mode: one process does both. Convenient for development and small setups; the documentation says it is not recommended for critical deployments.

The values of process.roles.

In a production cluster you run dedicated controller nodes (three or five of them) next to the brokers. Controllers do little work and need few resources; the KRaft documentation says that about 5 GB of memory and 5 GB of disk for the metadata log is sufficient for a typical cluster.

2.3.2 The controller quorum

The controller nodes form a **quorum**. One of them is the leader, called the **active controller**; the others are followers (standby controllers) that keep a copy of the metadata log. The protocol is Raft, in a variant adapted to Kafka:

- **Epochs and elections.** Every period of leadership has a number, the *epoch* (Raft calls it a term). A follower that does not hear from the leader for a short time (a few seconds at most, governed by the `controller.quorum.*` timeouts) becomes a candidate, asks the other voters for their votes in a new, higher epoch, and becomes leader if a majority agrees. A higher epoch always wins, so an old leader that comes back cannot cause damage.
- **Majority commit.** The leader appends a record to its own log; followers fetch it (pull, using the same Fetch request that ordinary replication uses). A record is **committed** when a majority of the voters have stored it, and only committed records are applied to the cluster state. A committed record survives any failure that leaves a majority alive, because at least one surviving voter has it.
- **One decision maker.** Only the active controller makes decisions. It processes admin requests (create a topic, change a config), notices that a broker has failed, picks new partition leaders, and writes the resulting records. Because the standby controllers already hold the log, taking over is just an election, not a reload of the cluster state from an external system.

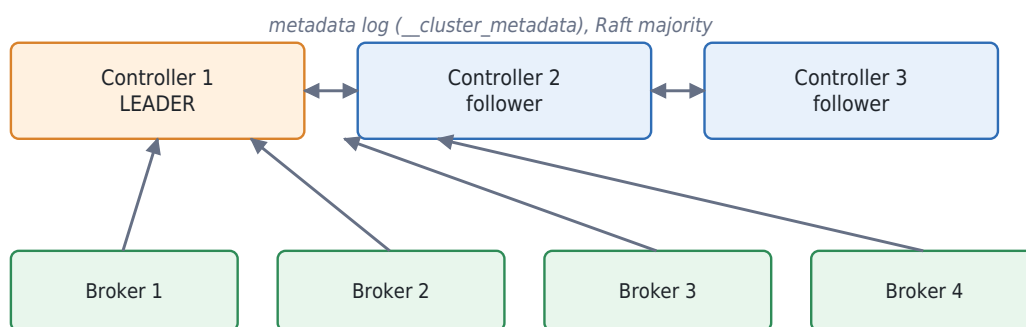


Figure 2.3. Three controllers form the quorum; one is the leader (the active controller). The standby controllers and every broker fetch the metadata log from the leader. Brokers are observers: they copy and apply the log but do not vote.

How many controllers? A quorum of $2N+1$ voters tolerates N failures, because a majority ($N+1$) must remain. Three controllers survive one failure, five survive two. Even numbers do not help: four voters need three for a majority and still tolerate only one failure, so you pay for a node you do not benefit from. Going beyond five rarely makes sense, since every metadata change waits for a majority and metadata changes are small and infrequent anyway.

Static or dynamic membership. Every node sets `controller.quorum.bootstrap.servers`, a list of controller endpoints that it uses to find the quorum. This is the dynamic quorum (KIP-853): controllers can be added with `bin/kafka-metadata-quorum.sh add-controller` and removed with `remove-controller`, and the documentation advises removing a controller from the quorum before you shut it down for good. The older static quorum lists every voter, with id, host and port, in `controller.quorum.voters`; its membership cannot change at runtime. The kind of quorum is decided when the storage is formatted (the hands-on Part shows the commands for a three-node cluster), and a feature level `kraft.version` of 1 or higher means dynamic.

2.3.3 How brokers use the metadata

Brokers are observers of the metadata log. Each broker fetches `__cluster_metadata` from the active controller, applies the records to an in-memory copy of the cluster state, and also keeps the log on disk (in `metadata.log.dir`, which defaults to the first entry of `log.dirs`). That has three useful consequences.

- **Clients talk to brokers only.** A producer or consumer asks any broker for metadata and gets the answer from that broker's in-memory copy: for each partition the leader, the replicas and the leader epoch, plus the broker endpoints. Clients never contact controllers.
- **Restarts are quick.** A broker that restarts loads its latest snapshot and replays only the log after it.
- **Propagation is simple.** Instead of the controller pushing updates to every broker, as it did in the ZooKeeper era, every broker pulls the same log and ends in the same state.

A **snapshot** is a file with the complete metadata state at one offset of the log. Each node writes snapshots periodically, which lets it delete the log before the snapshot. A new or far-behind node can fetch a snapshot instead of replaying months of history. Snapshot files end in `.checkpoint`.

Setting	Default	Meaning
<code>metadata.log.max.record.bytes.between.snapshots</code>	20971520	Take a new snapshot after this many bytes of new metadata
<code>metadata.log.max.snapshot.interval.ms</code>	3600000	...or after this long (1 hour) if there are unsnapshotted records
<code>metadata.max.retention.bytes</code>	104857600	Soft limit on the size of metadata log plus snapshots (100 MiB)
<code>metadata.max.retention.ms</code>	604800000	Soft limit on how long old metadata files are kept (7 days)

Settings for the metadata log (Kafka 4.2 documentation).

2.3.4 Broker lifecycle: registration, heartbeats and fencing

How does the cluster know which brokers are alive without ZooKeeper sessions? Through registration and heartbeats.

1. A starting broker contacts the controller quorum (using `controller.quorum.bootstrap.servers`) and **registers**: it announces its `node.id`, a fresh incarnation id, its listeners and the feature levels it supports. If this does not succeed within `initial.broker.registration.timeout.ms` (default 60000) the broker process exits.
2. A newly registered broker is **fenced**. It has not yet caught up on the metadata log, so it must not lead partitions. It fetches the log while sending heartbeats that tell the controller how far it has got.
3. When the broker has caught up, the active controller **unfences** it. It can now lead partitions and serve clients.
4. The broker keeps sending heartbeats every `broker.heartbeat.interval.ms` (default 2000). If the controller hears nothing for `broker.session.timeout.ms` (default 9000), it fences the broker again: the broker is removed from the in-sync replica sets of its partitions, and new leaders are elected for the partitions it led.

Controlled shutdown uses the same channel. When you stop a broker normally (`kafka-server-stop.sh` sends SIGTERM), the broker tells the controller it wants to shut down, the controller first moves partition leadership to other replicas, and only then does the broker stop. Clients see a short, orderly leader change instead of waiting for a session timeout. It is on by default (`controlled.shutdown.enable=true`).

Failure	What happens
A standby controller fails	Nothing visible. The quorum still has a majority (2 of 3), so metadata changes continue. Replace the node at leisure.
The active controller fails	The remaining controllers elect a new leader, typically within seconds. Brokers keep serving produce and fetch requests on their existing leaders, but metadata changes and new leader elections wait until the election is over.
A majority of controllers fails	No metadata change can be committed: no new topics, no leader elections, no broker registrations. Brokers can usually keep serving the partitions they already lead, but the cluster cannot react to the next failure. Restoring a majority of controllers resumes normal operation.
A broker fails	The controller fences it after missed heartbeats and elects new leaders for its partitions from their in-sync replicas (see the next chapter).

What happens when something fails.

2.3.5 What died with ZooKeeper

Operators who know older Kafka will look for familiar things. The table lists what changed.

Topic	ZooKeeper era	KRaft era
Metadata store	Znodes in a separate ZooKeeper ensemble	The <code>__cluster_metadata</code> log inside Kafka
Controller	One broker elected through ZooKeeper. After a failover it had to load the cluster state from ZooKeeper.	The leader of a Raft quorum. Standbys already hold the log, so failover is an election.
Metadata to brokers	The controller pushed updates to each broker by RPC	Each broker fetches the log and replays it
Broker liveness	A ZooKeeper session per broker	Heartbeats to the active controller
Systems to run and secure	Kafka plus a ZooKeeper ensemble	Kafka only

Topic	ZooKeeper era	KRaft era
Typical settings	<code>zookeeper.connect</code> , <code>broker.id</code>	<code>process.roles</code> , <code>node.id</code> , <code>controller.quorum.bootstrap.servers</code>
Version control of features	<code>inter.broker.protocol.version</code>	Feature levels, such as <code>metadata.version</code>

ZooKeeper era versus KRaft.

All `zookeeper.*` settings are gone, and so is `kafka-configs.sh --zookeeper`: tools talk to the cluster with `--bootstrap-server` (or `--bootstrap-controller` for tools that address a controller directly). Policy plug-ins such as `CreateTopicPolicy` now run in the controllers, so their jars must be on the controllers' class path.

2.3.6 Looking at the quorum

The tool `kafka-metadata-quorum.sh` shows who leads the metadata quorum and how far behind the others are. The values below are examples; after the summary the real output lists every controller (the voters) and every broker (the observers) with their ids and directory ids.

Describe the metadata quorum (typical output, abridged)

TERMINAL

```
$ bin/kafka-metadata-quorum.sh --bootstrap-server localhost:9092 describe --status
ClusterId:      Mk30EPKiRyCMvtGxTfJz0w
LeaderId:       3001
LeaderEpoch:   4
HighWatermark:  1873
MaxFollowerLag: 0
MaxFollowerLagTimeMs: -1
# CurrentVoters: one entry per controller; CurrentObservers: one entry per broker
```

Three more tools help when you want to look inside. `kafka-dump-log.sh` with `--cluster-metadata-decoder` decodes the segments and snapshots of the metadata directory into readable records. `kafka-metadata-shell.sh` opens a snapshot and lets you browse the cluster state with commands such as `ls` and `cat`. And `kafka-features.sh describe` shows the feature levels, including `metadata.version`, which replaces the old `inter.broker.protocol.version` and is how a cluster is moved to new capabilities after an upgrade.

INTERVIEW ANGLE

Why did Kafka drop ZooKeeper, and what replaced it? ZooKeeper was a second distributed system to deploy, secure, monitor and scale, and the metadata it held limited how many partitions a cluster could manage, because after a controller failover the new controller had to load all the state from ZooKeeper. KRaft stores the metadata in an internal log (`__cluster_metadata`) replicated by a Raft quorum of controller nodes. Standby controllers already hold the state, brokers keep up by fetching the log, and failover is an election instead of a reload. The result is one system to operate and faster recovery with more partitions.

KEY POINTS

- Kafka 4.x runs on KRaft only. Cluster metadata is the `__cluster_metadata` log, replicated by a quorum of controllers using Raft (epochs, majority commit, snapshots).
- `process.roles` is `broker`, `controller` or both. Use three (or five) dedicated controllers in production; $2N+1$ controllers tolerate N failures.
- Brokers are non-voting observers: they fetch and replay the metadata log and answer client metadata requests from memory. Clients never talk to controllers.
- A broker registers, starts fenced, is unfenced when caught up, and is fenced again if heartbeats stop for `broker.session.timeout.ms` (9 s). Controlled shutdown moves leaders away before the broker stops.
- Losing the controller majority freezes metadata changes (no elections, no topic creation) but does not by itself stop brokers from serving the partitions they lead.

2.4 Replication: Leaders, Followers and the ISR

Disks fail, machines reboot and racks lose power. A partition that lives on one broker disappears together with that broker. Kafka therefore keeps several copies of every partition, called **replicas**, on different brokers. The number of copies is the **replication factor** (RF), and RF 3 is the normal choice for data you care about. Replication is what lets a cluster lose a broker and carry on. It also raises the questions this chapter answers: which copy is in charge, when is a write "safe", and what happens to the data on a copy that was left behind.

2.4.1 Leaders and followers

For each partition exactly one replica is the **leader** and the others are **followers**. All writes go to the leader, and normally all reads do too (the section on reading from followers shows the exception). The ordered list of a partition's replicas is printed as `Replicas` by `kafka-topics.sh --describe`; its first entry is the **preferred leader**. Kafka assigns replicas round-robin so that every broker leads a fair share of the partitions. After failures have scrambled the leadership, `auto.leader.rebalance.enable` (default `true`) lets the controller move leaders back to their preferred replicas; it checks every `leader.imbalance.check.interval.seconds` (default 300).

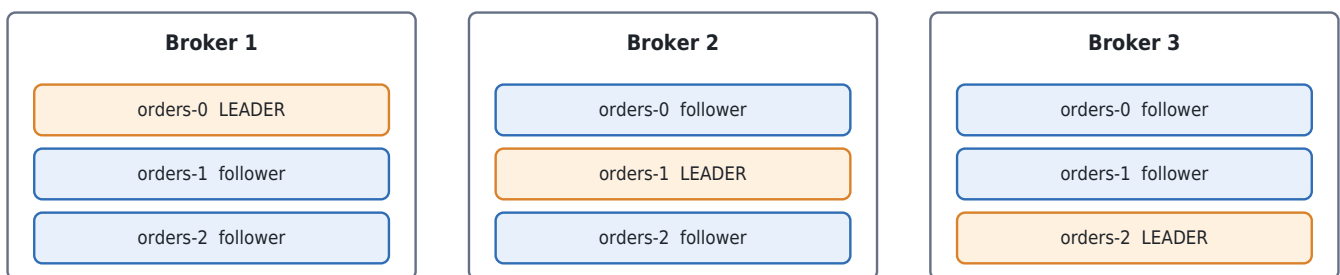


Figure 2.4. Three partitions of one topic replicated over three brokers (RF 3). Each broker leads one partition and follows the other two. Producers and consumers talk to the leaders; followers copy from them.

Followers **pull**. A follower runs fetcher threads that send Fetch requests to the leader, exactly like a consumer, and append the returned batches unchanged to their own log. Because the batch bytes are copied as they are, a follower's log has the same offsets as the leader's. Fetch requests long-poll: the leader holds one until `replica.fetch.min.bytes` (default 1) of data is available or `replica.fetch.wait.max.ms` (default 500) has passed. `num.replica.fetchers` (default 1) sets the number of fetcher threads per source broker. Pull replication has pleasant properties: the leader keeps no per-follower push state, a slow follower slows only itself, and replication uses the same fast path (page cache, zero copy) as consumers.

2.4.2 Log end offset, high watermark and committed records

Three terms describe the state of replication, and they come up in every discussion of Kafka's guarantees.

- The **log end offset (LEO)** of a replica is the offset that the next record appended to its log will get. Every replica has its own LEO.
- The **high watermark (HW)** is the offset below which every in-sync replica (next section) has the data. The leader computes it as the smallest LEO among the in-sync replicas.
- A record is **committed** when it lies below the HW. Consumers are only allowed to read committed records.

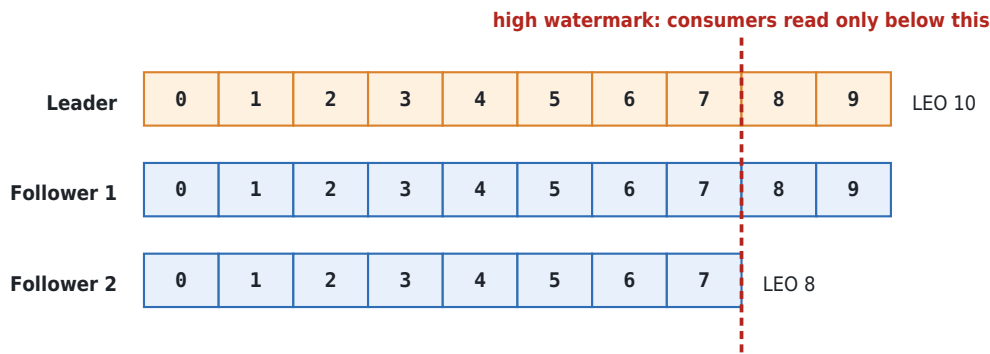


Figure 2.5. Each replica has its own log end offset (LEO). The high watermark is the smallest LEO among the in-sync replicas, and consumers may read only below it. Records above the high watermark exist on some replicas but are not committed yet.

How does the leader know the followers' LEOs? From the fetch requests themselves: a follower that asks for offset 7 is saying "I have everything below 7". The leader updates its view and advances the HW. It passes the HW back to the followers in its fetch responses, so a follower learns of a new HW one fetch round later than the leader does.

Why hide uncommitted records from consumers? Because a record that is not on every in-sync replica could vanish if the leader failed right now. If consumers could read it, two consumers might disagree about history after a failover. The HW guarantees that anything a consumer has seen will still exist after the leader changes.

A consumer has two positions of its own. Its **position** is the offset of the next record it will fetch. Its **committed offset** is the last position it saved to Kafka (in `__consumer_offsets`) as a bookmark for restarts. The distance between the end of the visible log and the consumer's position is the **lag**.

$$\text{consumer lag} = \text{log end offset} - \text{committed offset}$$



Figure 2.6. One partition from the consumer's point of view: the committed offset (the bookmark saved in Kafka), the current position, the high watermark and the log end offset. Consumer lag is the gap between what is visible and what has been processed.

GOTCHA

The `LOG-END-OFFSET` column of `kafka-consumer-groups.sh --describe` is the end of what consumers can see, normally the high watermark (for a `read_committed` consumer, the last stable offset). It is not the leader's raw LEO. That is the right number for computing lag, but the name is confusing.

2.4.3 The in-sync replicas (ISR)

Which followers should the HW wait for? Not all of them: one dead follower would freeze every write. Kafka waits for the **in-sync replicas (ISR)**. The documentation defines an in-sync replica by two conditions: it keeps an active session with the controller, so it receives metadata updates, and, if it is a follower, it replicates the leader's writes and does not fall "too far" behind. The leader is always a member of the ISR.

"Too far behind" is measured in time, not in messages. The leader removes a follower from the ISR if it has not fetched, or not caught up to the leader's log end offset, within `replica.lag.time.max.ms` (default 30000). The follower is added back once it has caught up. A broker that the controller fences (missed heartbeats, see the previous chapter) is removed from the ISRs sooner, after `broker.session.timeout.ms` (default 9000). ISR changes are committed through the controller (a leader asks for them with an `AlterPartition` request), so the ISR is part of the cluster metadata, which is why `describe` can show it.

The documentation states the consequence: a write is not considered committed until all in-sync replicas have received it, and with $f+1$ replicas a topic can tolerate f failures without losing committed messages. Compare that with a majority-vote scheme, which needs $2f+1$ replicas for the same tolerance. That economy is the reason Kafka uses an ISR.

2.4.4 acks and min.insync.replicas

The producer's `acks` setting decides when the leader answers, and the topic setting `min.insync.replicas` sets a floor on how small the ISR may be before the leader refuses to answer at all.

acks	The leader replies when	Acknowledged data can be lost if
0	It never replies; the producer does not wait	The leader fails or the request is dropped; nobody is told
1	The leader has appended the record to its own log	The leader fails before any follower has copied the record
all	Every replica in the ISR has the record, so it is committed	Every ISR member that held the record fails at the same time

What acks means for the leader.

`all` is the default in Kafka 4.x. But it means all *in-sync* replicas, and the ISR can shrink to the leader alone. The guard is `min.insync.replicas` (default 1): when `acks=all` and the ISR has fewer members than this, the leader rejects the write with `NotEnoughReplicasException` rather than accept a write that is protected by too few copies. The documentation notes that the setting takes effect only when the producer uses `acks=all`.

GOTCHA

With the default `min.insync.replicas=1`, `acks=all` is only as safe as `acks=1` whenever the ISR has shrunk to the leader. The producer gets an acknowledgement for data that exists on one machine.

The standard recipe for data you cannot afford to lose is RF 3, `min.insync.replicas=2`, producers with `acks=all`, and `unclean.leader.election.enable=false` (its default). A record is then acknowledged only when at least two brokers hold it, so any single failure leaves a copy; writes continue while two replicas remain in sync; and the cluster refuses writes instead of quietly lowering the protection. In general such a topic tolerates `min.insync.replicas - 1` simultaneous failures without losing acknowledged data, and `RF - min.insync.replicas` failures without losing the ability to write.

2.4.5 Leader election and leader epochs

When the controller finds that a leader has failed or been fenced, it elects one of the remaining ISR members as the new leader. Every ISR member holds all committed records, so an election never loses committed data.

Each period of leadership is numbered by a **leader epoch**, which grows by one at every change. The leader stamps the epoch into each batch it appends (the `partitionLeaderEpoch` field from the chapter on batches), and each replica keeps a small `leader-epoch-checkpoint` file recording the first offset of each epoch. Epochs solve a subtle problem. After a failover a replica may hold records that the new leader never received. They were not committed, or the new leader would have them, but they occupy offsets that the new leader will soon reuse for different records. The returning replica must find the point where its log diverges and cut it off. It asks the leader, with an `OffsetForLeaderEpoch` request, "where did my last epoch end in your log?" and truncates to that offset before it resumes fetching. (Older Kafka versions truncated to the high watermark instead, which in rare failure sequences left different records at the same offset on different replicas.)

Clients benefit from epochs as well. Requests carry the epoch the client knows, so a broker that has stopped being the leader recognises a stale request and rejects it. The client then refreshes its metadata and retries on the new leader.

2.4.6 Three failures, step by step

The scenarios use the `orders` topic with RF 3, `min.insync.replicas=2`, an idempotent producer with `acks=all`, and the defaults for timeouts.

Scenario 1: a follower crashes

Broker 1 leads a partition, broker 3 is a follower, and the ISR is {1,2,3}.

Timeline of scenario 1

t0	steady state	ISR {1,2,3}; all three replicas have offsets up to 100; HW 100
t1	broker 3 crashes	ISR is still {1,2,3}. The leader keeps appending (LEO 120) but the HW stays at 100, so acks=all produce requests wait for broker 3
t2	about 9 s later	the controller misses broker 3's heartbeats, fences it and removes it from the ISR: ISR {1,2}, HW jumps to 120, waiting requests complete
t3	degraded service	ISR size 2 ≥ min.insync.replicas 2, so writes keep succeeding. The partition is under-replicated (UnderReplicatedPartitions = 1)
t4	broker 3 restarts	it re-registers, catches up on metadata, checks against the leader's epoch history that its log is a prefix of the leader's, and fetches the 20 missing records
t5	caught up	the leader asks for broker 3 to rejoin the ISR: ISR {1,2,3}

The producer notices a pause of about nine seconds (with default settings) but no error. If broker 3 had not crashed but merely stalled while still heartbeating, the controller would not fence it; the leader would drop it after `replica.lag.time.max.ms`, so the pause could last up to 30 seconds. While broker 3 is down, `describe` shows it missing from the ISRs. The sample below also shows that broker 3 led partition 1, whose leadership moved to broker 1.

Describe orders while broker 3 is down (typical output, abridged)

TERMINAL

```
$ bin/kafka-topics.sh --describe --topic orders --bootstrap-server localhost:9092
Topic: orders TopicId: 5kVv0Q6aTYm4d1WmTt0u2g PartitionCount: 3 ReplicationFactor: 3
  Configs: min.insync.replicas=2
    Topic: orders Partition: 0 Leader: 2 Replicas: 2,3,1 Isr: 2,1
    Topic: orders Partition: 1 Leader: 1 Replicas: 3,1,2 Isr: 1,2
    Topic: orders Partition: 2 Leader: 1 Replicas: 1,2,3 Isr: 1,2
```

Scenario 2: the leader crashes with unreplicated records

This is the case that explains HW, epochs and idempotence at once. Broker 1 leads in epoch 0. The letters stand for records; **X** and **Y** are two recent records (in two separate batches) that the producer sent and has not yet seen acknowledged.

Timeline of scenario 2

offset:	0 1 2 3 4 5 6 7
t0	broker 1 (leader, epoch 0) [A A A A A X Y] LEO 8
	broker 2 (follower) [A A A A A X] LEO 7
	broker 3 (follower) [A A A A A] LEO 6 HW = 6, so only A is committed
t1	broker 1 crashes. The controller elects broker 2 from the ISR; epoch 1 begins.
t2	The producer's requests fail. It refreshes its metadata and resends X and Y to broker 2 with the same producer id and sequence numbers: X is already in broker 2's log: recognised as a duplicate, not appended again Y is new to broker 2: appended at offset 7 (epoch 1), call it Y'
	broker 2 (leader, epoch 1) [A A A A A X Y'] LEO 8
	broker 3 (follower) [A A A A A X Y'] after fetching; HW = 8, acks sent
t3	broker 1 restarts as a follower. Its log is [A A A A A X Y] and its Y is from epoch 0. It asks broker 2: "where did epoch 0 end?" Answer: offset 7. Broker 1 truncates its log to offset 7, then fetches Y' and joins the ISR again.

What was lost? Nothing that had been acknowledged: X and Y were never acknowledged, so the producer retried them. Other settings would have gone differently. With `acks=1`, broker 1 would have acknowledged Y on its own, and after the crash Y would be gone for good although the producer believes it was stored. Without idempotence, the retry of X would be appended a second time and stored twice. Three mechanisms cooperate here: `acks=all` prevents loss, the idempotent producer prevents duplicates, and leader epochs keep the replica logs identical.

Scenario 3: both followers fall behind

Now both followers become unreachable (a network partition, or two brokers down) while broker 1 stays up.

Timeline of scenario 3

t0	steady state	ISR {1,2,3}, min.insync.replicas = 2: writes succeed
t1	brokers 2 and 3 unreachable	after the controller fences them (about 9 s) or the leader drops them (30 s of lag), the ISR is {1}
t2	producers	acks=all and ISR size 1 < 2: the leader answers <code>NotEnoughReplicasException</code> . The producer retries until <code>delivery.timeout.ms</code> (120 s), then the send fails with a <code>TimeoutException</code> . Consumers still read below the HW.
t3a	broker 2 returns	it catches up and rejoins the ISR: ISR {1,2}; writes resume, nothing lost
t3b	broker 1 now fails too (worst case)	no in-sync replica is alive. With <code>unclean.leader.election.enable=false</code> the partition goes offline until a safe replica is back (broker 1, or with ELR a replica that left the ISR below the minimum): no data lost, but no availability either

This is the availability-versus-consistency decision in concrete form. With the default `min.insync.replicas=1` the leader would have kept accepting `acks=all` writes at t2 into an ISR of one, and the failure at t3b would have destroyed those writes. With `min.insync.replicas=2` it refuses them, and the producer sees errors instead of silently weaker durability.

2.4.7 Unclean leader election and eligible leader replicas

What if no in-sync replica is alive? The documentation names the two options. Wait for an ISR member to return (consistency over availability), or elect any replica that is alive, even one that is missing committed records (availability over consistency). The second is an *unclean* leader election. The topic and broker setting `unclean.leader.election.enable` chooses, and its default has been `false` since version 0.11, so Kafka waits. Turning it on means accepting that records can vanish and that offsets can be reused by different records. It can be reasonable for data such as metrics or logs where uptime matters more than completeness. For orders and payments it is the wrong choice.

Eligible leader replicas (ELR, KIP-966) narrow the gap between the two options. Under the "strict min ISR" rule, once the ISR falls below `min.insync.replicas` the high watermark can no longer advance, so nothing new can be committed. The replicas that were in the ISR at that moment therefore still hold every committed record even after they drop out of it. The controller remembers them as the partition's ELR. When it needs a leader it chooses from the ISR first, then from the ELR (a member that is not fenced), and only then falls back to the last known leader. ELR arrived in Kafka 4.0 and is enabled by default on new clusters from 4.1 (the feature level `eligible.leader.replicas.version`). One visible consequence is that `min.insync.replicas` becomes a cluster-level setting rather than a per-broker one, and changing it clears the ELR state. In daily work the rule is unchanged: set `min.insync.replicas` deliberately and leave unclean election off.

2.4.8 Reading from followers

By default every fetch goes to the leader. When clients and brokers sit in different availability zones, that can mean a lot of cross-zone traffic, which cloud providers often bill. Since Kafka 2.4 (KIP-392) a consumer can read from a follower in its own rack. You enable it with `broker.rack` on each broker, the broker setting `replica.selector.class=org.apache.kafka.common.replica.RackAwareReplicaSelector`, and `client.rack` on the consumer (the documentation says it corresponds with the broker's `broker.rack`). The leader answers the consumer's fetch with the preferred replica, and the consumer fetches from there. The data is still limited to the high watermark, which a follower learns slightly later than the leader, so records can appear to such a consumer a little later in exchange for cheaper traffic.

2.4.9 What to watch

Replication health is visible in a handful of metrics (the monitoring chapter in the production Part covers alerting).

MBean	What it tells you
<code>kafka.server:type=ReplicaManager,name=UnderReplicatedPartitions</code>	Partitions whose ISR is smaller than their replica list. Normal value 0.
<code>kafka.server:type=ReplicaManager,name=UnderMinIsrPartitionCount</code>	Partitions whose ISR is below <code>min.insync.replicas</code> . These reject <code>acks=all</code> writes. Normal value 0.
<code>kafka.server:type=ReplicaManager,name=IsrShrinksPerSec</code>	Rate of ISR shrinks. Expected when a broker goes down; otherwise 0.
<code>kafka.server:type=ReplicaManager,name=IsrExpandsPerSec</code>	Rate of ISR expansions. Follows shrinks once replicas catch up.
<code>kafka.controller:type=KafkaController,name=OfflinePartitionsCount</code>	Partitions with no leader at all, as seen by the controller.

Replication metrics from the Kafka 4.2 monitoring documentation.

On the command line, `kafka-topics.sh --describe --under-replicated-partitions` lists only the partitions whose ISR is smaller than the replica list. Frequent shrinks and expands for no obvious reason ("ISR flapping") usually point to a slow disk, a saturated network, long garbage-collection pauses, or a follower that cannot keep up with fetch traffic.

INTERVIEW ANGLE

What is the ISR, and why does Kafka not simply wait for all replicas? The ISR is the set of replicas, leader included, that are alive and have caught up within `replica.lag.time.max.ms`. A write is committed when every ISR member has it, and new leaders are chosen only from the ISR, so committed data survives. Waiting for all replicas would let a single dead or slow follower stall every write. Waiting for a majority, as Raft does, would need $2f+1$ replicas to tolerate f failures, while the ISR design needs $f+1$. The price is that the ISR can shrink, which is why `min.insync.replicas` exists.

INTERVIEW ANGLE

What happens when a partition leader dies? The controller notices (missed heartbeats, or a controlled shutdown), elects a new leader from the ISR and increases the leader epoch. Clients get a not-leader error, refresh their metadata and retry against the new leader. Committed records are safe because every ISR member has them. Records that only the old leader held were never acknowledged under `acks=all`, so the producer retries them, and idempotence prevents duplicates. When the old leader returns it truncates its log using the epoch history and becomes a follower. If no ISR member is alive the partition stays offline, unless unclean election is enabled.

KEY POINTS

- Each partition has one leader and $RF-1$ followers that pull from it. A follower's log is a byte-for-byte copy with the same offsets.
- The high watermark is the smallest log end offset among the in-sync replicas. Only records below it are committed and visible to consumers.
- The ISR contains the replicas that are alive and caught up (`replica.lag.time.max.ms`, 30 s). With $f+1$ replicas, f failures lose no committed data. New leaders come from the ISR.
- `acks=all` plus `min.insync.replicas=2` with $RF\ 3$ plus `unclean.leader.election.enable=false` is the durable recipe. With `min.insync.replicas=1`, `acks=all` can degrade to leader-only safety.
- Leader epochs let a returning replica find and truncate diverging records. Unclean election and ELR govern what happens when the ISR is gone.

PART 3

The Java Producer API

Plain kafka-clients, no Spring. Spring for Apache Kafka is built on exactly these classes.

In this part

- 3.1 The Producer at a Glance
- 3.2 What Happens Inside send()
- 3.3 Sending Records: Patterns and the ProducerRecord
- 3.4 Reliability: Acknowledgements, Retries and Idempotence
- 3.5 Throughput and Latency Tuning
- 3.6 Partitioning in Practice

3.1 The Producer at a Glance

Every event in Kafka was put there by a producer. In ShopStream, when a customer presses "Place order", the order-service saves the order in its database and then publishes an `OrderCreated` event to the `orders` topic. The object that does the publishing is a `KafkaProducer`, the producer class of the `kafka-clients` library. In this chapter you create a project, write the smallest useful producer, run it against your local broker and watch the record arrive.

A `KafkaProducer` is not a connection and it is not a message. It is a small engine that lives inside your JVM. It keeps a memory buffer of records that are waiting to be sent, runs a background thread that groups them into batches and ships the batches to the right brokers, and takes care of retries and cluster metadata. You create it once, share it between all threads of the application, and close it when the application stops.

3.1.1 Project setup

You need Java 21 (the client library itself runs on Java 11 or newer), Maven, and a broker on `localhost:9092`; the hands-on Part showed how to start one. The `kafka-clients` jar contains the producer, the consumer and the admin client. The clients write their logs through SLF4J, so the project also needs a logging backend. `slf4j-simple` is the smallest one. Without any backend you see no client logs at all, which makes problems hard to diagnose.

pom.xml (the parts that matter)

XML

```
<properties>
  <maven.compiler.release>21</maven.compiler.release>
</properties>

<dependencies>
  <dependency>
    <groupId>org.apache.kafka</groupId>
    <artifactId>kafka-clients</artifactId>
    <version>4.3.1</version>
  </dependency>
  <dependency>
    <groupId>org.slf4j</groupId>
    <artifactId>slf4j-api</artifactId>
    <version>2.0.17</version>
  </dependency>
  <dependency>
    <groupId>org.slf4j</groupId>
    <artifactId>slf4j-simple</artifactId>
    <version>2.0.17</version>
  </dependency>
</dependencies>
```

Two details. Client and broker versions do not have to match exactly, but with Kafka 4.x neither side may be older than 2.1. And `slf4j-api` is declared explicitly so that its version always matches `slf4j-simple`; a mismatch between the two makes SLF4J fall back to a silent no-operation logger.

3.1.2 Your first producer

Three settings have no default and must be supplied. `bootstrap.servers` is a list of `host:port` addresses used to find the cluster. `key.serializer` and `value.serializer` name the classes that turn the key and the value into bytes, because Kafka stores and transmits only bytes. We use `StringSerializer` for both and send the event as a JSON string. Part 6 covers richer formats.

```
FirstProducer.java JAVA

package com.shopstream.producer;

import java.util.Properties;
import org.apache.kafka.clients.producer.KafkaProducer;
import org.apache.kafka.clients.producer.ProducerConfig;
import org.apache.kafka.clients.producer.ProducerRecord;
import org.apache.kafka.common.serialization.StringSerializer;

public class FirstProducer {

    public static void main(String[] args) {
        Properties props = new Properties();
        props.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9092");
        props.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG, StringSerializer.class.getName());
        props.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG, StringSerializer.class.getName());

        String orderId = "o-1001";
        String json = """
            {"eventId": "6f1c2d4e-8a47-4d58-9d0e-3b2f6c1a9e10", "type": "ORDER_CREATED", \
            "occurredAt": "2026-10-08T10:15:30Z", "orderId": "%s", "customerId": "c-77", \
            "lines": [{"sku": "SKU-RED-SH0E-42", "qty": 2, "unitPrice": 499.00}], \
            "total": 998.00, "currency": "INR"}""".formatted(orderId);

        try (KafkaProducer<String, String> producer = new KafkaProducer<>(props)) {
            ProducerRecord<String, String> record =
                new ProducerRecord<>("orders", orderId, json);

            producer.send(record, (metadata, exception) -> {
                if (exception != null) {
                    exception.printStackTrace();
                } else {
                    System.out.printf("Sent %s to %s-%d at offset %d\n",
                        orderId, metadata.topic(), metadata.partition(), metadata.offset());
                }
            });
        } // close() blocks until every buffered record has been sent
    }
}
```

What to notice

- The constants in `ProducerConfig` (such as `BOOTSTRAP_SERVERS_CONFIG`) are just the string names, `"bootstrap.servers"` and so on. Prefer them: a misspelled constant does not compile, while a misspelled string key is only reported as one `WARN` line in the log ("These configurations ... were supplied but are not used yet") and the setting is silently ignored.
- The record has a topic, a key and a value. The key is the `orderId`, so every event of one order lands in the same partition and stays in order.
- `send()` returns immediately. The lambda is a `Callback`: Kafka calls it later, on the producer's background thread, with either a `RecordMetadata` (where the record landed) or an exception.
- The try-with-resources block calls `close()` at the end. `close()` waits until all buffered records have been transmitted and their callbacks have run. Without it this short program could exit before anything is sent.

3.1.3 Run it and look at the result

Create the topic with three partitions if it does not exist yet (the command is harmless if it does), then run the program. The partition and offset in the output are examples: yours depend on the key hash and on what the topic already contains.

Create the topic and run the producer

TERMINAL

```
$ bin/kafka-topics.sh --create --if-not-exists --topic orders --partitions 3 \
  --bootstrap-server localhost:9092
Created topic orders.
$ mvn -q compile exec:java -Dexec.mainClass=com.shopstream.producer.FirstProducer
[main] INFO org.apache.kafka.clients.producer.ProducerConfig - ProducerConfig values:
# (about a hundred lines with every setting and its value are skipped here)
[main] INFO org.apache.kafka.common.utils.AppInfoParser - Kafka version: 4.3.1
# (a few more startup messages are skipped here)
Sent o-1001 to orders-1 at offset 0
```

The client prints its complete configuration at INFO level when it starts. It is long, but worth reading once: it shows every default, including those you never set. Now read the record back with the console consumer. The `cut` at the end only trims the long JSON line for display; `--max-messages 1` makes the consumer exit after one record.

Read the record with the console consumer

TERMINAL

```
$ bin/kafka-console-consumer.sh --topic orders --from-beginning --max-messages 1 \
  --formatter-property print.key=true --bootstrap-server localhost:9092 | cut -c1-90
o-1001 {"eventId":"6f1c2d4e-8a47-4d58-9d0e-3b2f6c1a9e10","type":"ORDER_CREATED","occurredA
```

NOTE

Console tool options changed in Kafka 4.2 Kafka 4.2 renamed several options of the command-line tools to make them consistent (KIP-1147). The console consumer now uses `--formatter-property` where older versions used `--property`, `--command-property` instead of `--consumer-property`, and `--command-config` instead of `--consumer.config`. The old spellings still work in 4.2 but print a deprecation message, and they will be removed in the next major release. This book uses the new spellings; if you run an older broker distribution, switch back to the old ones.

GOTCHA

The program hangs for a minute, then prints a `TimeoutException` If `bootstrap.servers` is wrong or the broker is down, `send()` does not fail at once. The producer keeps trying to fetch metadata for the topic and gives up after `max.block.ms` (60 seconds). The error then arrives through the callback, not as an exception thrown by `send()`: `org.apache.kafka.common.errors.TimeoutException: Topic orders not present in metadata after 60000 ms.` The same message appears when the topic does not exist and the broker does not auto-create topics.

WATCH OUT

A typo in the topic name creates a new topic Brokers default to `auto.create.topics.enable=true`, and a producer asking for the metadata of an unknown topic is enough to trigger creation, with the broker defaults (one partition, replication factor one). A producer that writes to `order` instead of `orders` therefore "works", and the data sits in the wrong place. Production clusters usually turn auto-creation off and create topics on purpose.

WATCH OUT

Exiting without `close()` or `flush()` can lose the last records `send()` only places the record in a memory buffer. If the JVM exits before the background thread has transmitted the batch, the record is gone and nothing reports an error. Close the producer in `try-with-resources` or a `finally` block, and call `flush()` when you want to wait for everything sent so far without closing.

INTERVIEW ANGLE

Is `KafkaProducer` thread-safe? How many instances should an application have? Yes, `KafkaProducer` is thread-safe, and the Javadoc says that sharing one instance across threads is generally faster than having several. Most applications need one producer per distinct configuration (for example one for ordinary events and one transactional producer). Creating a producer per message is a classic mistake: every instance allocates a buffer, starts a network thread, opens connections and fetches metadata, and nothing gets batched.

KEY POINTS

- A `KafkaProducer` is a long-lived, thread-safe client: create one, share it, close it when the application stops.
- `bootstrap.servers`, `key.serializer` and `value.serializer` are mandatory; use the `ProducerConfig` constants to avoid typos.
- `send()` is asynchronous. The outcome arrives in a callback (or the returned `Future`), usually on the producer's background thread.
- Always `close()` (or `flush()`): records still in the buffer are lost if the JVM exits.
- Use a key such as `orderId` so related events share a partition and keep their order.

3.2 What Happens Inside `send()`

A common first picture is that `send()` makes a network call to a broker. It does not, and the difference explains most of the producer's behaviour: why it is fast, why a record can wait a few milliseconds before leaving, why a slow broker eventually blocks your code, and why records can vanish when you forget `close()`. This chapter follows one record from the moment you call `send()` until the broker has acknowledged it.



Figure 3.1. Everything up to the accumulator runs on the thread that calls `send()`. A separate sender thread drains full or lingering batches and talks to the brokers.

3.2.1 The journey of one record

The numbers below match the figure. Steps 1 to 4 run on your own thread inside `send()`; steps 5 to 7 run on the producer's sender thread.

1. **Metadata.** The producer must know how many partitions the topic has and which broker leads each partition. If it does not have this yet (the first send to a topic) it asks a broker and waits, for at most `max.block.ms`.
2. **Serialization.** The key and the value go through the configured serializers and become byte arrays.
3. **Partitioning.** The partitioner picks the partition number (next section).
4. **Accumulation.** The record is appended to a batch in the record accumulator, which keeps one queue of batches per partition. Batch memory comes from a pool of `buffer.memory` bytes. `send()` now returns a `Future<RecordMetadata>`; no network traffic has happened yet.
5. **Draining.** The sender thread, a background thread that each producer starts, repeatedly checks which batches are ready. It collects the ready batches of all partitions led by the same broker into a single produce request.
6. **Sending.** The request goes out over a connection to that broker. The sender does not wait for the answer before sending the next request: up to `max.in.flight.requests.per.connection` (5) requests may be outstanding on one connection.
7. **Completion.** On success the futures of all records in the batch complete and their callbacks run. On a retrievable error the batch is put back in the queue and tried again until `delivery.timeout.ms` (2 minutes) is used up; then the records fail with a `TimeoutException`.

3.2.2 Choosing the partition

The partition decides which log the record joins, and therefore its ordering neighbours and which consumer will read it. The producer applies these rules in order:

1. If the `ProducerRecord` names a partition explicitly, that partition is used.
2. Otherwise, if you configured `partitioner.class`, your partitioner decides (see the chapter on partitioning in practice).
3. Otherwise, if the record has a key (and `partitioner.ignore.keys` is `false`, the default), the partition is the murmur2 hash of the serialized key bytes, made positive, modulo the number of partitions.
4. Otherwise the built-in logic for keyless records applies.

Rule 3 is what gives you per-key ordering. The table shows real results of that formula for a topic with three partitions; the same key always gives the same partition as long as the partition count does not change.

Key	Partition	Note
o-1001	1	OrderCreated, OrderConfirmed of this order go here
o-1002	1	different order, same partition by chance
o-1003	2	a different order
o-1004	2	partition 2 now holds two orders
c-77	1	a customerId key (notifications topic)
c-78	0	another customer

Partition chosen for ShopStream keys by the default hash, topic with 3 partitions.

For records without a key, older clients spread records round-robin, one record at a time, which produced many tiny batches. Today's built-in logic is "sticky": it keeps writing to one partition until roughly one `batch.size` of data has gone there, then switches to another. Batches are full, requests are fewer and larger, and over time every partition gets a similar share. With `partitioner.adaptive.partitioning.enable=true` (the default) the producer also favours partitions on brokers that are answering faster. `partitioner.availability.timeout.ms` can additionally mark a partition unavailable when its broker makes no progress (the default 0 disables this).

GOTCHA

The hash is computed on bytes, not on your object. Partitioning uses the serialized key. If one service writes the key 42 with `StringSerializer` and another writes the integer 42 with `IntegerSerializer`, the byte arrays differ, so the "same" key lands in different partitions. Agree on one key format per topic, and treat it like part of the topic's contract.

3.2.3 Batches, linger and memory

Kafka is fast because it moves data in batches, and the accumulator is where batches are made. A batch holds records for one partition, up to `batch.size` bytes (16384 by default). That is an upper bound in bytes, not a count of records and not something the producer waits for. A record larger than `batch.size` simply gets a batch of its own.

The sender sends a batch as soon as one of these is true: the batch is full; it has waited `linger.ms` since its first record (5 ms by default since Kafka 4.0; it was 0 before); someone called `flush()` or `close()`; or the producer is short of buffer memory and needs to free some. Even with `linger.ms=0` batches form under load, because records pile up while the previous request is still in flight. A small delay such as 5 ms costs almost nothing in latency and usually raises throughput noticeably because batches are fuller and compress better.

3.2.4 Metadata, connections and advertised addresses

`bootstrap.servers` is only used for the first contact. The producer asks one of those brokers for the cluster metadata (all brokers, all partitions of the topic, the leader of each partition) and from then on connects directly to the leader of the partition it is writing to. The metadata is refreshed when the producer meets an error such as `NOT_LEADER_OR_FOLLOWER` after a leader election, and in any case every `metadata.max.age.ms` (5 minutes). Metadata of topics that have not been used for `metadata.max.idle.ms` (5 minutes) is dropped.

Give the producer a meaningful `client.id` such as `order-service`: it shows up in log lines, thread names and metrics (if you leave it empty the producer is called `producer-1`, `producer-2`, and so on). Two more practical consequences. List two or three brokers in `bootstrap.servers` so that one outage does not stop a restart. And remember that the producer must be able to reach every broker at the address the broker advertises, not only the bootstrap address. If the first connection works and the second times out, the cause is nearly always the broker's `advertised.listeners` setting, which is common with Docker and cloud networks. Part 14 returns to this.

3.2.5 When the broker is slower than your code

If your application produces faster than the network or the brokers accept, batches pile up in the accumulator until the `buffer.memory` pool (32 MB by default) is empty. From then on `send()` blocks the calling thread, waiting for memory, for at most `max.block.ms` (60 seconds). After that the record fails with a `BufferExhaustedException`, a subclass of `TimeoutException`. This is back-pressure. It is a feature, because an unbounded buffer would end in an `OutOfMemoryError` instead.

Your options when it happens are to slow the source down (a bounded queue in front of the producer, or pausing the consumer that feeds it), to send less data per record (compression, smaller events), or to add capacity (more partitions and brokers). Raising `buffer.memory` only postpones the problem if the brokers are the bottleneck.

The following settings govern the stages above. All values are the Kafka 4.x defaults.

Setting	Default	Stage / effect
<code>max.block.ms</code>	60000	How long <code>send()</code> , <code>partitionsFor()</code> and the transaction calls may block (metadata, buffer space)
<code>buffer.memory</code>	33554432	Total memory for unsent batches
<code>batch.size</code>	16384	Upper bound in bytes of one batch per partition
<code>linger.ms</code>	5	Wait time for a partly filled batch
<code>max.in.flight.requests.per.connection</code>	5	Outstanding requests per broker connection
<code>retry.backoff.ms</code> <code>retry.backoff.max.ms</code>	100 1000	Wait between retries; grows from the first to the second value
<code>delivery.timeout.ms</code>	120000	Upper bound for the whole send, including retries
<code>metadata.max.age.ms</code>	300000	Forced metadata refresh interval

Producer settings by pipeline stage.

3.2.6 Which thread does what

Because the work is split between two kinds of threads, some code runs where you might not expect. Serializers, the partitioner and interceptors run on your thread, so a slow custom serializer slows your caller. Network I/O, retries and callbacks run on the sender thread. The `send()` Javadoc says callbacks "will generally execute in the I/O thread of the producer and so should be reasonably fast or they will delay the sending of messages from other threads". The chapter on sending patterns returns to this.

Who does what

your thread	sender thread (one per producer)
-----	-----
<code>send(record)</code>	
1 wait for metadata (<code>max.block.ms</code>)	
2 serialize key and value	
3 choose the partition	
4 append to a batch → accumulator →	5 drain ready batches (one request per broker)
returns a Future	6 send, wait for the broker's response
	7 retry, or complete the Future and run the callback

INTERVIEW ANGLE

Walk me through what happens between `producer.send()` and the broker acknowledging the write. `send()` fetches topic metadata if needed, serializes the key and value, picks a partition (explicit partition, custom partitioner, murmur2 hash of the key, or sticky logic for keyless records) and appends the record to a per-partition batch in the record accumulator, then returns a Future. A background sender thread sends ready batches (full, or older than `linger.ms`) to the partition leaders, one request per broker, with up to five requests in flight per connection. The leader appends the batch to its log, waits for the replicas required by `acks`, and answers. The sender then completes the futures and runs callbacks, or retries retrievable errors until `delivery.timeout.ms` expires.

KEY POINTS

- `send()` does only in-memory work on your thread (metadata check, serialization, partitioning, append); a separate sender thread does all network I/O.
- Partition choice: explicit partition, then custom partitioner, then murmur2 of the key bytes, then sticky logic for keyless records.
- Batches are bounded by `batch.size` and released by fullness, `linger.ms`, `flush()` or memory pressure; batching is the main source of throughput.
- A full buffer blocks `send()` for up to `max.block.ms`, then fails; this is back-pressure, not a bug.
- Metadata is cached and refreshed on errors and every 5 minutes; every broker's advertised address must be reachable from the client.

3.3 Sending Records: Patterns and the `ProducerRecord`

Calling `send()` is easy. Deciding what to do with the answer is the real design question. Do you wait for the broker, ignore the outcome, or react later? The choice decides your throughput and also what happens to an order event when something goes wrong. This chapter shows the three patterns, what a `ProducerRecord` can carry, and the one rule about callbacks that you must not break.

3.3.1 Anatomy of a `ProducerRecord`

A `ProducerRecord<K,V>` is the unit you hand to the producer. It has six parts, and only the topic is required. There are six constructors for the combinations; the most common one is `new ProducerRecord<>(topic, key, value)`.

Part	Type	Meaning
topic	String	Destination topic. Required.
partition	Integer	Optional. If set, the record goes to exactly this partition and the partitioner is skipped.
timestamp	Long	Optional, epoch milliseconds. If not set, the producer stamps the record with its current time.
key	K	Optional. Drives the partition choice and is the identity of the record in compacted topics.
value	V	Optional. A null value is a tombstone: in a compacted topic it deletes the key.
headers	Headers	Optional list of name and byte-array pairs: metadata that travels with the record but is not part of key or value.

The parts of a `ProducerRecord`.

Three details deserve attention. First, the timestamp: by default it is the time the producer created the record (`CreateTime`). The topic setting `message.timestamp.type` decides whether the broker keeps it or overwrites it with its own append time. If you set an explicit timestamp, for example the event time when you re-import historical orders, note that since Kafka 4.0 the broker rejects timestamps more than one hour in the future by default (`message.timestamp.after.max.ms`).

Second, headers: use them for technical metadata such as a trace id, the event type or a schema version, so consumers can route or log without parsing the value. Add them with `record.headers().add(name, bytes)`. Headers become read-only once the record has been passed to `send()`; adding one afterwards throws `IllegalStateException`.

Third, an explicit partition: it works, but it couples your code to the partition count. If you later add partitions, your hand-made mapping is no longer what the hash would give, and nothing warns you. Prefer a key and let the partitioner decide.

3.3.2 Three ways to send

The class below wraps one shared producer and shows all patterns side by side. It depends on the `Producer` interface instead of `KafkaProducer`, which makes it easy to test with `MockProducer` later in this Part.

OrderEventPublisher.java

JAVA

```
package com.shopstream.producer;

import java.nio.charset.StandardCharsets;
import java.time.Instant;
import java.util.concurrent.ExecutionException;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.TimeoutException;
import org.apache.kafka.clients.producer.Callback;
import org.apache.kafka.clients.producer.Producer;
import org.apache.kafka.clients.producer.ProducerRecord;
import org.apache.kafka.clients.producer.RecordMetadata;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class OrderEventPublisher {

    private static final Logger log = LoggerFactory.getLogger(OrderEventPublisher.class);
    private static final String TOPIC = "orders";

    private final Producer<String, String> producer;

    public OrderEventPublisher(Producer<String, String> producer) {
        this.producer = producer;
    }

    /** 1. Fire and forget: nobody looks at the result, so failures go unnoticed. */
    public void fireAndForget(String orderId, String json) {
        producer.send(new ProducerRecord<>(TOPIC, orderId, json));
    }

    /** 2. Synchronous: wait for the broker's answer. Safe, but one round trip per record. */
    public RecordMetadata sendAndWait(String orderId, String json)
        throws InterruptedException, ExecutionException, TimeoutException {
        return producer.send(new ProducerRecord<>(TOPIC, orderId, json))
            .get(30, TimeUnit.SECONDS);
    }

    /** 3. Asynchronous with a callback: fast, and every failure is still seen. */
    public void sendAsync(String orderId, String json) {
        producer.send(new ProducerRecord<>(TOPIC, orderId, json), callbackFor(orderId));
    }

    /** 4. Full control: explicit event time and headers. */
    public void sendWithHeaders(String orderId, String json, String traceId, Instant occurredAt) {
        ProducerRecord<String, String> record =
            new ProducerRecord<>(TOPIC, null, occurredAt.toEpochMilli(), orderId, json);
        record.headers().add("traceId", traceId.getBytes(StandardCharsets.UTF_8));
        record.headers().add("eventType", "ORDER_CREATED".getBytes(StandardCharsets.UTF_8));
        producer.send(record, callbackFor(orderId));
    }

    private Callback callbackFor(String orderId) {
        return (metadata, exception) -> {
            if (exception != null) {
                log.error("Order {} was not published", orderId, exception);
                // keep it short: hand over to a retry queue, an outbox table or an alert
            } else {
                log.debug("Order {} stored at {}-{} offset {}",
                    orderId, metadata.topic(), metadata.partition(), metadata.offset());
            }
        };
    }
}
```

Pattern	Call	Learn the result	Use it for
Fire and forget	<code>send(record)</code>	never	Data you can afford to lose, such as best-effort metrics

Pattern	Call	Learn the result	Use it for
Synchronous	<code>send(record).get()</code>	before the next line	Rare: scripts, tests, very low volume where the caller must know
Asynchronous	<code>send(record, callback)</code>	in the callback	The default for services

The three patterns compared.

Be careful with the middle row. A loop of `send(...).get()` is slow by construction. Every iteration waits for a full network round trip plus replication, and because only one record is outstanding, the producer can never build a batch. As an illustration: if one round trip takes 5 ms, one thread cannot exceed about 200 records per second, however fast the broker is. The asynchronous pattern lets thousands of records share batches and requests.

In the fire-and-forget row, "forget" refers to your code only. The producer still retries according to its settings; you just never learn that the record finally failed.

3.3.3 Handling the result

Both the `Future` and the `Callback` deliver the same two things: a `RecordMetadata` on success (topic, partition, offset, timestamp) or an exception. With the future, `get()` wraps the Kafka exception in an `ExecutionException`; read `getCause()` to see the real one. Mind the name clash: `Future.get(timeout, unit)` throws `java.util.concurrent.TimeoutException`, which is a different class from Kafka's `org.apache.kafka.common.errors.TimeoutException`.

In a callback, exactly one of the two arguments is meaningful. On success `exception` is `null`. On failure the exception is set, and `metadata` is not null but its offset and other numeric fields are `-1` (the partition is `-1` too if none had been chosen yet). Always test `exception` first. Failures may also be thrown by `send()` itself, for example a `SerializationException` when the value does not fit the serializer; the chapter on reliability sorts the errors into those that are retried for you and those you must handle.

The producer guarantees that callbacks for records sent to the same partition run in the order the records were sent. There is no ordering guarantee between partitions.

WATCH OUT

Never block inside a callback. Callbacks run on the producer's single sender thread, the same thread that sends every batch for every partition. If a callback does slow work (a database call, an HTTP request) all sending of all threads stalls behind it. If a callback waits for another `send(...).get()`, it waits for the very thread it is running on and deadlocks. Keep callbacks to a few microseconds: log, increment a counter, or put the failed record on a queue that another thread processes.

3.3.4 Sending many records and waiting for them

For batch jobs you rarely want a result per record, but you do want to know when everything has been handled. `flush()` does that: it makes all buffered records ready to send immediately, even if `linger.ms` has not elapsed, and blocks until every request is complete. Note what "complete" means: `flush()` returns once all earlier sends have finished, successfully or not. It does not throw because of a failed record, so count failures in the callback and check the counter afterwards.

Publish a batch, then check the outcome

JAVA

```
AtomicInteger failures = new AtomicInteger();
for (OrderCreated event : events) {
    ProducerRecord<String, String> record =
        new ProducerRecord<>("orders", event.orderId(), toJson(event));
    producer.send(record, (metadata, exception) -> {
        if (exception != null) {
            failures.incrementAndGet();
        }
    });
}
producer.flush(); // every callback above has run when this returns
if (failures.get() > 0) {
    throw new IllegalStateException(failures.get() + " events were not published");
}
```

This is the producer half of a very common pattern in consumer applications: read a batch, produce results, `flush()`, check, and only then commit the consumer offsets. Never call `flush()` from inside a callback: it would wait for the very thread it is running on. Since Kafka 4.1 the producer detects this potential deadlock and prohibits the call.

INTERVIEW ANGLE

Why is `send().get()` in a loop slow, and what do you do instead? Each `get()` blocks the thread for a full request round trip (network, leader append, replication for `acks=all`) and keeps at most one record in flight, so batching and pipelining are lost. Use the asynchronous `send(record, callback)`, let the producer batch with `linger.ms` and `batch.size`, collect failures in the callback, and call `flush()` at a natural boundary such as the end of a job or before committing consumer offsets.

KEY POINTS

- A `ProducerRecord` has topic, optional partition, timestamp, key, value and headers; a null value is a tombstone.
- Prefer a key over an explicit partition; use headers for technical metadata, not business data.
- Fire-and-forget loses failures silently, synchronous `get()` costs throughput, and the asynchronous callback is the default for services.
- `get()` wraps the real exception in `ExecutionException`; in a callback test `exception` first.
- Callbacks run on the sender thread: never block in them and never wait for another `send()` there.
- `flush()` waits for completion, not for success; count failures in the callback.

3.4 Reliability: Acknowledgements, Retries and Idempotence

When the order-service publishes `OrderCreated`, three things can go wrong. The record can be lost (the customer paid, but no other service ever hears of the order). It can be duplicated (the warehouse reserves stock twice). Or two events of the same order can arrive in the wrong order. A reliable producer prevents all three. Part of the work is done by producer settings, part by topic settings on the broker side, and part by your own code, which must look at the result of every send.

3.4.1 acks: who must confirm the write

The `acks` setting decides when the broker may answer a produce request.

acks	The leader answers when ...	Consequence
0	immediately, without writing anything	Fastest. Failures are invisible, retries do not apply, and the returned offset is always -1.
1	it has written the record to its own log	If the leader dies before the followers copied the record, the record is lost.
all	all in-sync replicas have the record	The record survives as long as one in-sync replica survives. This is the default and the right choice for business events.

The three acks levels.

Notice the words "in-sync replicas". `acks=all` does not mean all replicas of the partition, only those currently keeping up with the leader (the ISR). If two of three replicas fall behind, the ISR is just the leader, and `acks=all` quietly behaves like `acks=1`. The topic setting `min.insync.replicas` closes this hole: it is the minimum ISR size for which the leader accepts a write when the producer asks for `acks=all`. The documented standard recipe is replication factor 3, `min.insync.replicas=2` and `acks=all`.

Replicas in sync	Result of a write	Meaning
3 of 3	Succeeds when all three have it	Normal operation
2 of 3	Succeeds when both in-sync replicas have it	One broker is down; the data is still on two machines
1 of 3	Rejected with <code>NotEnoughReplicasException</code>	Only the leader is left; the cluster refuses writes rather than weaken the guarantee

Writes to a topic with replication factor 3 and `min.insync.replicas=2`, `acks=all`.

Reads keep working in the last row; only writes are refused. This is a deliberate trade of availability for durability. The topic must be created with these settings, because the producer cannot enforce them on its own (this needs a cluster with at least three brokers):

Create a durable topic

TERMINAL

```
$ bin/kafka-topics.sh --create --topic orders --partitions 3 --replication-factor 3 \
  --config min.insync.replicas=2 --bootstrap-server localhost:9092
Created topic orders.
```

NOTE

The default `min.insync.replicas` is 1. Out of the box a topic has `min.insync.replicas=1`, so `acks=all` alone gives weaker durability than most people assume. Check the value on every important topic with `kafka-topics.sh --describe`. On clusters that use Eligible Leader Replicas, which Kafka enables by default for new 4.1 and later clusters, the exact semantics of this setting change somewhat; the operations Part explains the details.

3.4.2 Retries and timeouts

Transient problems (a leader election, a brief network break) are normal in a cluster, so the producer retries retrievable errors by itself. The `retries` setting defaults to 2147483647, which is practically unlimited, and the Kafka documentation advises leaving it alone. What really limits the effort is `delivery.timeout.ms` (default 120000, two minutes): the upper bound on the time from `send()` returning until the producer reports success or failure.

One record's time budget

```
send() returns
|<----- delivery.timeout.ms (default 120 s) ----->|
| wait in the batch | request 1 (up to 30 s) | backoff | request 2 (up to 30 s) | ...
| linger.ms        | request.timeout.ms    | 100 ms+ |
|
+-- answer arrives: the Future completes, the callback gets the RecordMetadata
+-- the budget is used up: the record fails with a TimeoutException
```

The settings fit together by one rule: `delivery.timeout.ms` must be at least `linger.ms + request.timeout.ms`. A record may wait `linger.ms` in its batch and then needs at least one full request attempt, otherwise batches would expire before they could be answered. The client checks this when it is created: a smaller `delivery.timeout.ms` that you set explicitly is rejected with a `ConfigException`, and if you left it unset the producer raises it for you and logs a warning. Either way a wrong combination shows up at start-up, not in production traffic. Between attempts the producer waits `retry.backoff.ms` (100 ms), growing up to `retry.backoff.max.ms` (1000 ms). The documentation also advises a `request.timeout.ms` above the brokers' `replica.lag.time.max.ms`, so that a slow follower is dropped from the ISR before the producer gives up on a request, which would trigger an unnecessary retry.

When the budget is used up, the callback receives a `TimeoutException`, with a message such as `Expiring 1 record(s) for orders-1: 120000 ms has passed since batch creation`. From this point the problem is yours.

GOTCHA

After a timeout you do not know what happened. A `TimeoutException` after the delivery timeout means "no answer in time", not "the broker did not write it". The write may have succeeded and only the response was lost. If your application now sends the same event again, the topic can contain it twice. Idempotence (next section) covers the producer's own retries, but not a second `send()` made by your code. Plan for this: make consumers idempotent (for example by remembering `eventId`) or use an outbox table, both covered later in the book.

3.4.3 Idempotence: retries without duplicates

Consider a produce request that the broker handled correctly, but the response was lost on the network. The producer sees a timeout and retries, and without protection the record would be appended twice. The idempotent producer prevents this. When it starts, the broker assigns it a producer id (PID). Every batch it sends carries the PID and a sequence number per partition. The broker remembers the latest sequence numbers per producer and partition (the last five batches, which is why `max.in.flight.requests.per.connection` may not exceed 5), so it can recognise a retried batch.

The broker removes the duplicate

```
producer (PID 42)                                broker, partition orders-1
batch, sequence 7 -----> append, remember (PID 42, seq 7)
      <----- response lost -----X
retry: batch, sequence 7 -----> already seen: do not append again
      <----- ack -----> (answer success)
next batch, sequence 8 -----> expected next number: append
```

If the broker instead sees a sequence number that skips ahead, data has been lost somewhere, and it answers with an `OutOfOrderSequenceException`. Retries also keep their order: with idempotence on, the producer preserves the order of batches within a partition even when requests fail and are repeated, for up to five requests in flight. Without idempotence, `retries` above zero combined with more than one request in flight can reorder records, because a retried first batch may land behind a second batch that succeeded. The old advice "set `max.in.flight.requests.per.connection=1` to keep order" is therefore obsolete.

Idempotence is on by default. It requires `acks=all`, `retries` above 0 and at most five in-flight requests. The documentation spells out two behaviours that matter: if you do not set `enable.idempotence` and set a conflicting option (for example `acks=1`), idempotence is silently switched off; if you set it explicitly to `true` and a conflicting option exists, the producer fails with a `ConfigException`. So set `enable.idempotence=true` explicitly in code, which turns a silent downgrade into an early error.

Know its limits. Idempotence removes duplicates caused by the producer's own retries, within one producer instance (one PID) and per partition. It does not help if your code sends the same event twice, if the application restarts (a new producer gets a new PID), or if you write to several partitions and need all-or-nothing. For the last two, see transactions later in this Part.

3.4.4 What errors look like

Errors reach you in two ways: `send()` may throw, or the `Future` and callback complete with an exception. The most important split is between retrievable and non-retrievable errors. A retrievable error is transient, and the producer retries it for you until `delivery.timeout.ms` expires; you only see it afterwards. A non-retrievable error is permanent, the producer gives up at once, and retrying the same record would only fail the same way.

Exception	Retried?	Cause and what to do
<code>TimeoutException</code> (also <code>BufferExhaustedException</code>)	yes	No answer in time, or no metadata or buffer space within <code>max.block.ms</code> . Outcome unknown. Resend later or park the event, and alert.
<code>NotEnoughReplicasException</code>	yes	ISR smaller than <code>min.insync.replicas</code> . Fix the brokers. Nothing was written.
<code>NotEnoughReplicasAfterAppendException</code>	yes	Same, but discovered after the append. The record may be in the leader's log; without idempotence the retry duplicates it.
<code>RecordTooLargeException</code>	no	Record larger than <code>max.request.size</code> or the broker and topic limits. Never retry. Park it and shrink the data (see the claim check pattern).
<code>SerializationException</code>	no	Thrown by <code>send()</code> itself: the object does not fit the serializer. A bug in your code.
<code>AuthorizationException</code>	no	Missing ACL for the topic. Stop and fix the permissions.
<code>ProducerFencedException</code>	no	Transactional only: another instance with the same <code>transactional.id</code> started. Close this producer.
<code>OutOfOrderSequenceException</code>	no	The broker's sequence check failed, so data may have been lost. For a transactional producer it is fatal; an idempotent-only producer may continue but loses the ordering guarantee. Investigate.

Common producer errors.

One small method can turn the table into code. A callback that only logs is the most common reliability bug; the useful thing is to decide, per error, whether to park, stop or resend.

ReliableProducerSupport.java

JAVA

```
package com.shopstream.producer;

import java.util.Properties;
import org.apache.kafka.clients.producer.ProducerConfig;
import org.apache.kafka.common.errors.AuthorizationException;
import org.apache.kafka.common.errors.RecordTooLargeException;
import org.apache.kafka.common.errors.SerializationException;
import org.apache.kafka.common.errors.TimeoutException;
import org.apache.kafka.common.serialization.StringSerializer;

public final class ReliableProducerSupport {
```

ReliableProducerSupport.java (continued)

JAVA

```

public enum Action { PARK, STOP, RESEND_LATER, ALERT }

private ReliableProducerSupport() { }

public static Properties props(String bootstrapServers) {
    Properties props = new Properties();
    props.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, bootstrapServers);
    props.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG, StringSerializer.class.getName());
    props.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG, StringSerializer.class.getName());
    props.put(ProducerConfig.CLIENT_ID_CONFIG, "order-service");
    props.put(ProducerConfig.ACKS_CONFIG, "all");
    props.put(ProducerConfig.ENABLE_IDEMPOTENCE_CONFIG, true);
    props.put(ProducerConfig.MAX_IN_FLIGHT_REQUESTS_PER_CONNECTION, 5);
    props.put(ProducerConfig.REQUEST_TIMEOUT_MS_CONFIG, 30_000);
    props.put(ProducerConfig.DELIVERY_TIMEOUT_MS_CONFIG, 120_000);
    return props;
}

/** Decide what to do with a failed send, based on the kind of exception. */
public static Action decide(Exception e) {
    return switch (e) {
        case RecordTooLargeException ex -> Action.PARK;           // retrying gives the same error
        case SerializationException ex -> Action.PARK;           // a bug: fix the code
        case AuthorizationException ex -> Action.STOP;           // fix the ACLs first
        case TimeoutException ex -> Action.RESEND_LATER;        // outcome unknown
        default -> Action.ALERT;
    };
}
}

```

What to notice

- Every value is spelled out even where it equals the default. That is deliberate: a reviewer can see the intended guarantees, and an explicit `enable.idempotence=true` fails fast on a conflicting setting.
- `decide()` needs a `default` branch because other exceptions exist. Anything unexpected should alert a human rather than be retried blindly.
- Which action to take is a business decision. For `RESEND_LATER`, the safe implementation writes the event to a table or queue and relies on idempotent consumers.

3.4.5 The "do not lose data" recipe

Reliability is the sum of several layers. The table lists every layer; if one is missing, the chain has a weak link.

Setting	Where	Value	Why
<code>replication.factor</code>	topic	3	Three copies; one broker can fail and two copies remain
<code>min.insync.replicas</code>	topic	2	A write needs two copies, otherwise it is rejected instead of weakened
<code>unclean.leader.election.enable</code>	topic	false (default)	Never promote a replica that is missing committed records
<code>acks</code>	producer	all (default)	Wait for every in-sync replica
<code>enable.idempotence</code>	producer	true (default)	No duplicates from retries; set it explicitly
<code>delivery.timeout.ms</code>	producer	120000 (default)	How long the producer keeps trying before your callback must act
<code>callback</code> or <code>outbox</code>	your code	check every result	A failed send must go somewhere: retry queue, outbox table, alert
<code>close()</code> and <code>flush()</code>	your code	on shutdown	Do not abandon buffered records

Settings for no data loss on important topics.

INTERVIEW ANGLE

Does `acks=all` guarantee that no data is ever lost? No. `acks=all` waits for all in-sync replicas, and if only the leader is in sync it behaves like `acks=1`. To be safe you also need replication factor 3 or more, `min.insync.replicas=2` or more, and `unclean.leader.election.enable=false`, so the broker refuses writes instead of weakening the guarantee. On the client side, the application must check every callback (a timeout means "outcome unknown"), and must close or flush the producer on shutdown. Even then, a catastrophic loss of all replicas is not covered.

KEY POINTS

- `acks=all` plus topic `min.insync.replicas=2` and replication factor 3 is the durable combination; `acks=all` alone depends on the current ISR size.
- The producer retries retrievable errors until `delivery.timeout.ms` (2 minutes); keep `delivery.timeout.ms` at least `linger.ms + request.timeout.ms`.
- The idempotent producer (on by default) removes duplicates caused by its own retries through producer id and sequence numbers, and keeps order for up to five requests in flight.
- Idempotence does not cover duplicate `send()` calls by your code, producer restarts, or multi-partition atomicity.
- After a `TimeoutException` the outcome is unknown; non-retriable errors such as `RecordTooLargeException` will fail every time.

3.5 Throughput and Latency Tuning

Two goals pull in opposite directions. Throughput is how many records or bytes per second you can push through. Latency is how long a single record takes from `send()` to acknowledgement. Almost every producer setting trades one against the other, and the exchange rate is batching. Every request has a fixed cost: a network round trip, request handling on the broker, a log append. Putting many records into one request spreads that cost and raises throughput, but collecting them takes time and adds latency.

3.5.1 The levers

Setting	Default	How it raises throughput	What it costs
<code>linger.ms</code>	5	Waiting lets batches fill up before they are sent	Up to this much added latency per record
<code>batch.size</code>	16384	Allows bigger batches when records arrive quickly	Memory for each partition that has an open batch
<code>compression.type</code>	none	Fewer bytes on the network and on disk; bigger batches compress better	CPU on the producer, and on every consumer that decompresses
<code>buffer.memory</code>	33554432	Absorbs bursts without blocking <code>send()</code>	Heap memory
<code>acks</code>	all	<code>1</code> does not wait for followers	Records can be lost when a leader fails

The main tuning settings (Kafka 4.x defaults).

Begin with batching. A batch is sent when it is full (`batch.size`) or when it has waited `linger.ms`, whichever happens first, so the useful knob depends on how fast records arrive. A worked example, with illustrative numbers: records of about 1 KB arrive at one partition every 0.5 ms. A 16 KB batch needs roughly 15 records, so it would take about 7.5 ms to fill. With `linger.ms=5` the timer fires first and each request carries about 10 records. Raising `linger.ms` to 10 would let batches fill completely, at the price of up to 7.5 ms extra delay. If instead records arrive so fast that batches are already full, `linger.ms` changes nothing and `batch.size` is the knob to raise.

GOTCHA

A big `batch.size` multiplies by the number of partitions. The producer keeps one open batch per partition that currently receives data, and allocates a buffer of `batch.size` for it in anticipation of more records. A producer that writes to 500 partitions with `batch.size=131072` can hold more than 62 MiB in open batches, which is already more than the default `buffer.memory` of 32 MiB. Then `send()` starts to block for memory even though the brokers are fine. Size `buffer.memory` as at least the number of active partitions times `batch.size`, plus headroom.

3.5.2 Compression

The producer compresses a whole batch at once and the broker stores it that way, so compression saves network traffic, disk space and replication traffic together. Because a bigger batch contains more repetition, more batching usually means a better compression ratio. JSON events compress well since field names repeat in every record; payloads that are already compressed or encrypted (images, encrypted blobs) do not.

compression.type	Compression ratio	CPU cost	Notes
none	none	none	Right for tiny records or already compressed data.
gzip	Highest of the four	Highest	Rarely worth it unless bandwidth is very expensive. Level: <code>compression.gzip.level</code> .
snappy	Moderate	Low	Fast, established choice.
lz4	Moderate	Lowest	Very fast; good when latency matters. Level: <code>compression.lz4.level</code> .
zstd	High, close to gzip	Moderate	Good all-round choice for bulk data. Level: <code>compression.zstd.level</code> (default 3).

Compression codecs, relative to each other. Always measure with your own data.

On the broker side, the topic setting `compression.type` defaults to `producer`, which means "keep the batch exactly as the producer compressed it". The broker then spends no CPU on it. If you set a specific codec on the topic that differs from the producer's, the broker has to decompress and recompress every batch, which costs CPU for no benefit in most cases.

LAB 3.1 · Measure batching and compression with the performance test

Kafka ships a load generator, `kafka-producer-perf-test.sh`. It needs a topic, a record count, a record size and a target rate (`-1` means "as fast as possible"). The commands assume the single local broker from the hands-on Part. Create a topic first.

Create the test topic

TERMINAL

```
$ bin/kafka-topics.sh --create --topic perf-test --partitions 3 --bootstrap-server localhost:9092
Created topic perf-test.
```

Run 1 uses the defaults. The tool prints a progress line every few seconds and a summary line at the end; we keep only the last line and split it at the commas so it fits the page.

Run 1: defaults

TERMINAL

```
$ bin/kafka-producer-perf-test.sh --topic perf-test --num-records 1000000 \
  --record-size 1024 --throughput -1 --bootstrap-server localhost:9092 | tail -1 | tr ',' '\n'
1000000 records sent
61236.5 records/sec (59.80 MB/sec)
9.18 ms avg latency
412.00 ms max latency
5 ms 50th
24 ms 95th
96 ms 99th
301 ms 99.9th.
```

Run 2 changes three settings. Put them in a file and pass it with `--command-config`.

tuned.properties

PROPERTIES

```
linger.ms=50
batch.size=131072
compression.type=lz4
```

Run 2: larger batches, longer linger, lz4

TERMINAL

```
$ bin/kafka-producer-perf-test.sh --topic perf-test --num-records 1000000 \
  --record-size 1024 --throughput -1 --bootstrap-server localhost:9092 \
  --command-config tuned.properties | tail -1 | tr ',' '\n'
1000000 records sent
118623.9 records/sec (115.84 MB/sec)
31.45 ms avg latency
529.00 ms max latency
22 ms 50th
71 ms 95th
190 ms 99th
402 ms 99.9th.
```

The numbers above are illustrative of the typical direction, not a measurement; yours will differ with hardware, record content and broker settings. What to look at: throughput (records/sec) roughly doubles in this example, while the average latency and the percentiles rise because each record now waits in a bigger batch. Whether that trade is good depends on the use case, which is the point of the next section. Two cautions. The tool fills records with random letters, so compression ratios on real data will differ (use `--payload-file` with sample events for a realistic test). And a one-broker laptop cluster says nothing about production capacity: compare settings against each other, not against an absolute number.

NOTE

Older option names In Kafka 4.2 `--producer.config` became `--command-config`, `--producer-props` became `--command-property`, and `--bootstrap-server` and `--reporting-interval` were added to the producer performance test (KIP-1147). Before 4.2 you pass the bootstrap address as `--producer-props bootstrap.servers=localhost:9092`. The old names still work in 4.x but are deprecated and will be removed in Kafka 5.0.

3.5.3 Let the producer's own metrics tell you what to change

The producer publishes metrics through JMX and through `producer.metrics()`. A handful of them show how batching behaves right now. All of them belong to the group `producer-metrics`.

Metric	Meaning	How to use it
batch-size-avg	Average bytes sent per partition per request	Close to <code>batch.size</code> : batches are full, raise <code>batch.size</code> . Far below: they leave early, raise <code>linger.ms</code> .
records-per-request-avg	Average records per request	A value near 1 means no real batching is happening.
compression-rate-avg	Average ratio of compressed to uncompressed batch size	Lower is better; a value near 1.0 means compression gains nothing.
record-queue-time-avg	Average time in ms that batches spend in the buffer	Roughly what lingering costs you in latency.
request-latency-avg	Average request latency in ms	Broker and network time per request.
bufferpool-wait-ratio waiting-threads	Share of time appenders wait for buffer memory; number of threads blocked for memory	Above zero means <code>send()</code> is blocking; raise <code>buffer.memory</code> or slow the source down.

Producer metrics for tuning.

3.5.4 A tuning method, and two ShopStream profiles

Tune by measurement, one change at a time. Decide whether you optimise throughput or latency. Record a baseline with realistic payloads. Change one setting (batching first, then compression), repeat the test, and read the metrics above to see why the result changed. Settings belong to a producer instance, so a service with two kinds of traffic can simply create two producers with different settings.

Aspect	payments (latency-critical)	page-views (bulk clickstream)
Goal	Fast confirmation for each customer	Maximum records per second at low cost
linger.ms	0 to 5	20 to 100
batch.size	default	64 KiB to 128 KiB
compression.type	none or lz4	lz4 or zstd

Aspect	payments (latency-critical)	page-views (bulk clickstream)
acks	all	all (use 1 only if losing some clicks is accepted)

Two profiles in ShopStream.

INTERVIEW ANGLE

How would you increase the throughput of a Kafka producer? Send asynchronously and share one producer. Increase batching: raise `linger.ms` and `batch.size` so that requests carry more records. Enable compression (`lz4` or `zstd`) to cut network and disk volume; it works best with big batches. Make sure `buffer.memory` is large enough for the number of active partitions, and add partitions or brokers if one partition leader is the bottleneck. Weaken `acks` only if losing data is acceptable. Measure with `kafka-producer-perf-test.sh` and the producer metrics, and expect latency to rise as batches grow.

KEY POINTS

- Batching is the main lever: a batch leaves when it is full (`batch.size`) or has waited `linger.ms`.
- Compression works per batch and is most effective with big batches; the broker keeps the producer's compression when the topic uses `compression.type=producer`.
- Open batches cost memory: partitions times `batch.size` must fit in `buffer.memory`.
- Measure with `kafka-producer-perf-test.sh`, compare runs against each other, and read `batch-size-avg`, `compression-rate-avg` and `record-queue-time-avg` to understand why.
- Use separate producer instances for latency-critical and bulk traffic.

3.6 Partitioning in Practice

Which partition a record lands in decides three things at once: the order in which consumers see it (Kafka orders records only within a partition), how much work each consumer gets (parallelism is limited by the number of partitions), and how evenly the brokers are loaded. The earlier chapter on `send()` showed the mechanics. This chapter is about the decisions: which key to choose, what goes wrong with hot partitions, why adding partitions is dangerous for keyed topics, and when (rarely) to write your own partitioner.

3.6.1 Choosing the key

The key is the unit of ordering. Pick the entity whose events must be processed in the order they happened, and use its id as the key. In ShopStream every event of one order, `OrderCreated`, then `PaymentAuthorized`, then `StockReserved`, must be seen in that sequence, so the topics `orders`, `payments`, `inventory` and `shipments` are keyed by `orderId`. The `notifications` topic is keyed by `customerId`, because messages to one customer should not overtake each other, and `page-views` by `sessionId`.

A good key has four properties. It identifies the thing whose order matters. It has many distinct values (high cardinality), so the hash can spread the load. Its traffic is reasonably even across values. And it is stable: the same entity must always produce the same key bytes. If order does not matter, send records without a key; the built-in sticky log then gives the best batching and balance.

Two things to remember about order. First, it exists only per partition: two keys in different partitions have no relative order. Second, order within a partition is the order in which records reach the producer's accumulator. If two threads send events for the same order concurrently, the producer cannot know which came "first". Let one thread (or one queue) own the events of a key, or sequence the sends. The ordering guarantee under retries comes from idempotence (on by default, see the reliability chapter), not from `max.in.flight.requests.per.connection=1`.

3.6.2 Hot partitions

A hot partition receives far more traffic than its siblings. Its leader broker works harder, its consumer falls behind while the others idle, and adding consumers does not help, because one partition can be read by only one consumer of a group. The usual causes are skewed keys (one very large customer or seller), a default key for "unknown" values (everything with the key `unknown` or an empty string lands together), and a key with too few distinct values (such as a country code with a handful of values).

Notice that adding partitions does not fix a hot key. A single key always hashes to a single partition: if one customer produces 30 percent of all events, the partition that owns that customer receives at least 30 percent of the traffic,

whatever the partition count. Detect the problem by comparing the end offsets of the partitions, or the lag per partition of your consumer group (a later chapter shows how).

Compare how much data each partition received

TERMINAL

```
$ bin/kafka-get-offsets.sh --bootstrap-server localhost:9092 --topic orders
orders:0:48210
orders:1:51377
orders:2:301954
```

In this example (output is illustrative: the format is `topic:partition:end-offset`) partition 2 holds about six times as many records as each of the others. The remedies, in order of preference: choose a better key (a more fine-grained entity, or a composite such as `customerId` plus an order number when strict per-customer order is not required); split the hot entity across several keys by adding a suffix ("salting", for example `c-corp-1#0` to `c-corp-1#3`), at the price of losing order between the pieces; or give the hot entity its own topic.

3.6.3 Adding partitions changes where keys go

Kafka lets you increase the number of partitions of an existing topic, but never decrease it.

Increase the partition count

TERMINAL

```
$ bin/kafka-topics.sh --alter --topic orders --partitions 6 --bootstrap-server localhost:9092
```

The command succeeds quietly, and the danger starts with the next record. The default partitioner computes the hash modulo the number of partitions, so a different count means a different result for most keys. The table shows real results of the default hash (murmur2 of the UTF-8 key bytes) for the same keys at different partition counts.

Key	3 partitions	4 partitions	6 partitions	12 partitions
o-1001	1	1	1	1
o-1002	1	3	1	7
o-1003	2	3	5	11
o-1004	2	1	5	5
c-77	1	2	4	10
c-78	0	1	3	9

Partition chosen by the default hash for the same key, at different partition counts.

Suppose `orders` grows from 3 to 4 partitions. Order `o-1002` was in partition 1, and from now on its new events go to partition 3. A consumer that reads partition 3 may see `OrderConfirmed` before the consumer of partition 1 has finished with `OrderCreated`: per-key order is broken at the boundary. Kafka Streams state, caches keyed by partition, and compacted topics (where one key now lives in two partitions) have similar problems. When the partition count doubles, each key either stays or moves to its old partition plus the old count (1 becomes 1 or 4, 2 becomes 2 or 5), which is neat but does not preserve order for the keys that move.

So plan the partition count for the future: estimate the growth, choose a number with many divisors (for example 6 or 12) that also exceeds the maximum number of consumers you expect to run, and avoid needing to change it. If you must grow a keyed topic, the safer route is to create a new topic with more partitions, copy the data, and move producers and consumers over together. For topics without keys the change is harmless.

WATCH OUT

Check the partition count before relying on key order. A keyed topic keeps its per-key ordering only while the partition count stays the same. Anyone with `kafka-topics.sh --alter` access can change it, and nothing in the producer or the broker warns you. Treat the partition count of a keyed topic as part of its contract and restrict who may alter it.

3.6.4 Writing a custom partitioner

You almost never need one. The built-in logic is good, and many reasons to customise are better solved otherwise. A reserved lane for urgent traffic is usually cleaner as a separate topic (own retention, own consumers, own access rules), and a fixed partition can be named in the `ProducerRecord`. A custom partitioner is justified when the rule must apply to every record of a producer, for example to line up with the partitioning of another system.

To show the mechanics, here is an express-lane partitioner for the `notifications` topic. Keys that start with `urgent:` always go to partition 0, so a dedicated consumer can read that partition with low delay; all other records are spread over the remaining partitions. Ordering between the urgent and the normal lane is given up on purpose.

```
ExpressLanePartitioner.java JAVA

package com.shopstream.producer;

import java.nio.charset.StandardCharsets;
import java.util.Arrays;
import java.util.Map;
import java.util.concurrent.ThreadLocalRandom;
import org.apache.kafka.clients.producer.Partitioner;
import org.apache.kafka.common.Cluster;

/** Keys that start with "urgent:" go to partition 0; everything else to partitions 1..n-1. */
public class ExpressLanePartitioner implements Partitioner {

    private static final byte[] URGENT = "urgent:".getBytes(StandardCharsets.UTF_8);

    @Override
    public void configure(Map<String, ?> configs) {
        // nothing to configure in this example
    }

    @Override
    public int partition(String topic, Object key, byte[] keyBytes,
                        Object value, byte[] valueBytes, Cluster cluster) {
        int partitions = cluster.partitionsForTopic(topic).size();
        if (partitions == 1) {
            return 0;
        }
        if (keyBytes == null) {
            return ThreadLocalRandom.current().nextInt(1, partitions);
        }
        if (keyBytes.length >= URGENT.length
            && Arrays.equals(keyBytes, 0, URGENT.length, URGENT, 0, URGENT.length)) {
            return 0;
        }
        return 1 + Math.floorMod(Arrays.hashCode(keyBytes), partitions - 1);
    }

    @Override
    public void close() {
        // nothing to release
    }
}
```

Register it with `props.put(ProducerConfig.PARTITIONER_CLASS_CONFIG, ExpressLanePartitioner.class.getName());`. The producer creates one instance and calls `configure` once, so the settings of the producer are visible there.

What to notice

- `partition()` receives both the original `key` and `value` objects and their serialized bytes, plus a `Cluster` snapshot. Compute the partition count on every call, because it can change after a metadata refresh.
- The result must be between 0 and the partition count minus one. A negative number is rejected immediately, and a number that is too large makes the send fail.
- One instance serves all threads that call `send()`, and it runs on the calling thread for every single record. It must be thread-safe, fast and free of blocking calls.
- Once you set `partitioner.class`, your code decides for keyless records too: the built-in sticky and adaptive logic is gone, and `partitioner.ignore.keys`, `partitioner.adaptive.partitioning.enable` and `partitioner.availability.timeout.ms` have no effect. That is why keyless records are picked at random here, which batches worse than sticky.
- `Arrays.hashCode` is not the default murmur2 hash. That is fine on a topic where every producer uses this partitioner. Mixing partitioners on one topic splits one key over several partitions.

GOTCHA

Other client libraries may hash differently. The Java producer hashes with murmur2. Producers written with other libraries may default to another function (the C client librdkafka, for example, defaults to a CRC32 based one and offers a murmur2 mode for compatibility with Java). If a Java service and a non-Java service both write keyed records to one topic, check that they place the same key in the same partition, or per-key ordering silently breaks.

INTERVIEW ANGLE

What happens to ordering if you add partitions to a topic that uses keys? The default partitioner takes the key hash modulo the partition count. With a new count, many keys map to different partitions. New events for such a key go to a new partition while its older events remain in the old one, so consumers can see them out of order, and stateful consumers (caches, Kafka Streams state) are invalidated. Avoid it by choosing the partition count with growth in mind, or by migrating to a new topic. Kafka cannot reduce the number of partitions.

KEY POINTS

- The key defines the unit of ordering; choose the entity whose events must stay in order (`orderId`, `customerId`, `sessionId`).
- A hot key makes a hot partition, and more partitions do not help: one key always maps to one partition.
- Adding partitions changes the hash-to-partition mapping for most keys and breaks per-key order across the change; plan the count up front.
- A custom `Partitioner` replaces all built-in logic including keyless handling, must be fast and thread-safe, and must agree with every other producer of the topic.
- Prefer simpler options first: a better key, an explicit partition, or a separate topic.

PART 4

Serialization, Schema Registry and Data Contracts

How objects become bytes, and how teams keep the shape of those bytes stable while systems change.

In this part

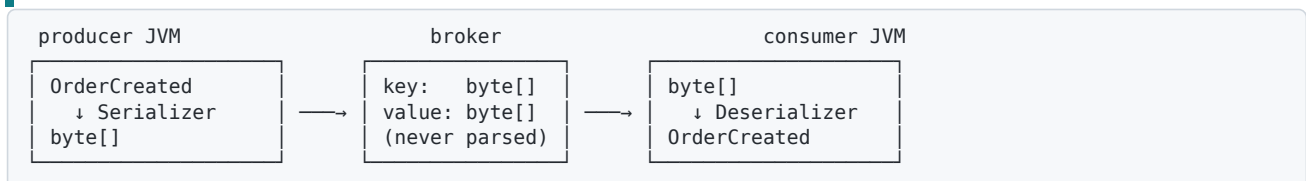
- 4.1 Serialization Basics
- 4.2 Why Schemas Matter
- 4.3 Schema Registry
- 4.4 Avro in Practice
- 4.5 Protobuf and JSON Schema
- 4.6 Evolving Schemas Safely

4.1 Serialization Basics

A Kafka broker never looks inside your messages. To the broker, the key and the value of a record are two arrays of bytes (either may also be `null`). That is a deliberate design choice: because the broker does not parse, validate or convert anything, it can append and serve data very quickly, and it can serve producers and consumers written in any language.

The price is that you have to decide how a Java object turns into bytes (**serialization**) and how those bytes turn back into an object (**deserialization**), and every producer and consumer of a topic has to make the same decision. This chapter shows where that decision lives in the Kafka client, which serializers ship with Kafka, and how to write your own. The chapters after it explain why "just send JSON" stops working as a system grows, and what to do instead.

4.1.1 Where serialization happens



The producer calls the key and value serializers inside `send()`, on the thread that called `send()`, before the record is placed into a batch. The consumer calls the deserializers inside `poll()`, one record at a time, while it builds the batch of records it returns to you. Both clients learn which classes to use from their configuration:

Setting	Client	What you provide
<code>key.serializer</code>	producer	A class implementing <code>Serializer</code> for the record key
<code>value.serializer</code>	producer	A class implementing <code>Serializer</code> for the record value
<code>key.deserializer</code>	consumer	A class implementing <code>Deserializer</code> for the record key
<code>value.deserializer</code>	consumer	A class implementing <code>Deserializer</code> for the record value

The settings that select the serializer and deserializer classes. None has a default, so a client refuses to start without them.

In Java you normally use the constants (`ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG`, `ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG`, `ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG`, `ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG`) and pass either a `Class` object or a fully qualified class name. Key and value are configured independently, so a topic can have a `String` key and a JSON value.

4.1.2 The Serializer and Deserializer interfaces

Both interfaces live in `org.apache.kafka.common.serialization`. Each has exactly one abstract method; everything else has a default implementation, so a minimal serializer is a single method.

Method	Purpose
<code>byte[] serialize(String topic, T data)</code>	Convert the object to bytes. The topic name is passed in so one class can behave differently per topic.
<code>byte[] serialize(String topic, Headers headers, T data)</code>	Same, but you also receive the record headers and may add some, for example a content type.
<code>T deserialize(String topic, byte[] data)</code>	Convert bytes back to an object. Returning <code>null</code> for <code>null</code> input is the recommended behaviour.
<code>T deserialize(String topic, Headers headers, byte[] data)</code>	Same, with the record headers (never <code>null</code> , possibly empty).
<code>T deserialize(String topic, Headers headers, ByteBuffer data)</code>	Same, but the payload arrives as a <code>ByteBuffer</code> .
<code>void configure(Map<String, ?> configs, boolean isKey)</code>	Called once with the client's configuration; <code>isKey</code> tells you whether you were chosen for keys or values.
<code>void close()</code>	Release resources. Both interfaces extend <code>Closeable</code> .

The methods of `Serializer<T>` and `Deserializer<T>` (Kafka 4.2 Javadoc). Only the first method of each interface is abstract.

Two details matter in practice. First, when you configure a class by name, Kafka creates the object by reflection, so the class needs a public constructor without parameters. Second, `configure` receives the **whole** client configuration, which is how a serializer can read its own settings: custom keys such as `schema.registry.url` (used by the Schema Registry serializers later in this Part) are simply additional entries in the producer or consumer properties.

GOTCHA

If you pass serializer or deserializer instances to the `KafkaProducer` or `KafkaConsumer` constructor, the client does not call `configure()` on them. The constructor Javadoc says so explicitly. An instance you create yourself must be fully configured before you hand it over. Only classes selected by name through `key.serializer` and friends are created and configured by the client.

4.1.3 Built-in serializers

Kafka ships serializers for the common simple types. They are all you need for keys such as `orderId` and for plain text or binary values.

Java type	Classes	What ends up in the byte array
String	<code>StringSerializer</code>	Text in UTF-8 by default; the encoding can be changed with a serializer encoding property
Integer / Short	<code>IntegerSerializer</code> , <code>ShortSerializer</code>	4 or 2 bytes, big-endian
Long	<code>LongSerializer</code>	8 bytes, big-endian
Float / Double	<code>FloatSerializer</code> , <code>DoubleSerializer</code>	4 or 8 bytes (IEEE 754)
Boolean	<code>BooleanSerializer</code>	one byte
UUID	<code>UUIDSerializer</code>	The 36-character text form of the UUID, as a string
byte[]	<code>ByteArraySerializer</code>	The array itself, unchanged
ByteBuffer	<code>ByteBufferSerializer</code>	The buffer content (the input position is rewound to zero first)
Bytes	<code>BytesSerializer</code>	The wrapped array of Kafka's <code>Bytes</code> helper class
Void	<code>VoidSerializer</code>	Always <code>null</code> ; use it when a topic has no key
List	<code>ListSerializer</code>	A list, each element written by an inner serializer

Serializers and deserializers included in `kafka-clients`. Each class named `XSerializer` has a matching `XDeserializer`.

To see what the broker stores, look at the key `o-1001` written once by `StringSerializer` and the number 1001 written once by `LongSerializer`:

```
StringSerializer("o-1001") → 6f 2d 31 30 30 31      (6 bytes: the UTF-8 characters)
LongSerializer(1001L)      → 00 00 00 00 00 00 03 e9 (8 bytes: 0x3e9 = 1001, big-endian)
```

The bytes carry no type information. Nothing in the second line says "this is a long". Whoever reads it must already know. Kafka Streams packages a serializer and a deserializer together as a `Serde` (for example `Serdes.String()` or `Serdes.serdeFrom(serializer, deserializer)`); that is covered in the Kafka Streams Part.

4.1.4 Writing your own: JSON with Jackson 3

To make the idea concrete we build a serializer pair for the `OrderCreated` record of the running example. JSON is a reasonable first format: it is readable and every language can parse it. We use Jackson 3 (package `tools.jackson`), the version that Spring Boot 4 and Spring for Apache Kafka 4 build on. Its `JsonMapper` is immutable and thread-safe once built, understands Java records, and has `java.time` support built in.

OrderCreatedSerializer.java

JAVA

```
package com.shopstream.serde;

import com.shopstream.events.OrderCreated;
import org.apache.kafka.common.errors.SerializationException;
import org.apache.kafka.common.serialization.Serializer;
import tools.jackson.core.JsonException;
import tools.jackson.databind.json.JsonMapper;

public class OrderCreatedSerializer implements Serializer<OrderCreated> {

    private final JsonMapper mapper = JsonMapper.builder().build();

    @Override
    public byte[] serialize(String topic, OrderCreated event) {
        if (event == null) {
            return null; // a null value must stay a Kafka null (a tombstone)
        }
        try {
            return mapper.writeValueAsBytes(event);
        } catch (JsonException e) {
            throw new SerializationException("Cannot serialize OrderCreated for " + topic, e);
        }
    }
}
```

OrderCreatedDeserializer.java

JAVA

```
package com.shopstream.serde;

import com.shopstream.events.OrderCreated;
import org.apache.kafka.common.errors.SerializationException;
import org.apache.kafka.common.serialization.Deserializer;
import tools.jackson.core.JsonException;
import tools.jackson.databind.json.JsonMapper;

public class OrderCreatedDeserializer implements Deserializer<OrderCreated> {

    private final JsonMapper mapper = JsonMapper.builder().build();

    @Override
    public OrderCreated deserialize(String topic, byte[] data) {
        if (data == null) {
            return null;
        }
        try {
            return mapper.readValue(data, OrderCreated.class);
        } catch (JsonException e) {
            throw new SerializationException("Cannot deserialize OrderCreated from " + topic, e);
        }
    }
}
```

What to notice. Both classes have an implicit public no-argument constructor, so Kafka can instantiate them from a class name. They keep no per-record state, which matters because one producer is shared by many threads and

therefore calls the same serializer concurrently. Failures are wrapped in Kafka's `SerializationException` so the client reports them in the usual way. And `null` goes in and out unchanged: Jackson would happily turn `null` into the four characters `null`, a perfectly non-null byte array, and the record would stop being a tombstone.

Wiring the classes into a plain Java producer takes three properties:

JAVA

```
Properties props = new Properties();
props.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9092");
props.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG, StringSerializer.class);
props.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG, OrderCreatedSerializer.class);
```

A serializer class per event type does not scale: ShopStream would need one pair per event type. Real projects use one generic serializer that handles many types. Spring for Apache Kafka ships `JacksonJsonSerializer` and `JacksonJsonDeserializer` for exactly that purpose (see the Spring Boot Part), and the Schema Registry serializers do the same for Avro, Protobuf and JSON Schema.

4.1.5 What goes wrong

Serialization failures are synchronous. If the object cannot be serialized, `send()` throws a `SerializationException` in the calling thread. It does not arrive through the callback or the returned future, because the record was never accepted for sending.

Deserialization failures happen inside `poll()`. When a deserializer throws, the consumer raises a `RecordDeserializationException` from `poll()`. The next chapter shows why that is dangerous.

Mismatched classes. If the producer writes the key with `StringSerializer` and a consumer reads it with `LongDeserializer`, the consumer fails because the byte count is not 8. The opposite mistake is worse: `StringDeserializer` accepts any bytes, so reading a `Long` key with it produces unreadable text and no error at all.

Java native serialization. `ObjectOutputStream` works, and you should not use it: the bytes are tied to exact class versions, only Java can read them, and deserializing untrusted bytes is a well-known security hole.

INTERVIEW ANGLE

Why does Kafka store byte arrays instead of typed objects? Because the broker is deliberately a dumb, fast log. It never parses a record, so it needs no knowledge of your classes or languages, it can send log segments to consumers straight from disk and page cache, and it treats every topic the same. The trade-off is that nothing on the server validates the content. The contract between producers and consumers has to be enforced at the edges, which is what schemas and the Schema Registry are for.

KEY POINTS

Serialization basics

- Kafka stores and transmits `byte[]` keys and values; the broker never interprets them.
- Serializers run inside `send()`, deserializers inside `poll()`. Key and value have separate settings that have no default.
- Implement `Serializer.serialize(topic, data)` or `Deserializer.deserialize(topic, data)`; always pass `null` through unchanged and keep the class stateless and thread-safe.
- Instances passed to a client constructor are not configured by the client; classes selected by name are.
- The bytes carry no type information, so producers and consumers must agree on the format by some other means. That is the subject of the next chapters.

4.2 Why Schemas Matter

JSON is the easiest way to start: people can read it, every language can parse it and no extra infrastructure is needed. For one service talking to itself that is fine. On a Kafka cluster shared by many teams, "just send JSON" hides a coupling that eventually hurts. This chapter describes the problem. The chapters that follow introduce the tools that solve it.

4.2.1 A Friday-afternoon incident

ShopStream's `order-service` publishes `OrderCreated` events to the `orders` topic. Three other applications read them: `payment-service`, `notification-service` and the `analytics-app`. This is the event as everyone has known it for months:

```
JSON
{
  "eventId": "6f1c2d4e-8a47-4d58-9d0e-3b2f6c1a9e10",
  "type": "ORDER_CREATED",
  "occurredAt": "2026-10-08T10:15:30Z",
  "orderId": "o-1001",
  "customerId": "c-77",
  "lines": [ { "sku": "SKU-RED-SHOE-42", "qty": 2, "unitPrice": 499.00 } ],
  "total": 998.00,
  "currency": "INR"
}
```

On Friday the order team decides to report tax separately. A developer renames `total` to `grandTotal` and adds a `tax` field. The unit tests of `order-service` pass, the build is green, the service is deployed. The new events look like this:

```
JSON
{
  "eventId": "0b7e5a10-52c3-4c0e-9f1d-77a1c4f1b2aa",
  "type": "ORDER_CREATED",
  "occurredAt": "2026-10-09T16:02:11Z",
  "orderId": "o-1002",
  "customerId": "c-12",
  "lines": [ { "sku": "SKU-BLUE-CAP-01", "qty": 1, "unitPrice": 1180.00 } ],
  "grandTotal": 1180.00,
  "tax": 180.00,
  "currency": "INR"
}
```

What happens next depends on how each consumer was written. `payment-service` deserializes into a record with a `BigDecimal total`; the field is missing, so `total` is `null`, the first call on it throws a `NullPointerException`, and the listener fails on every new order. `notification-service` formats the amount into an e-mail and happily sends "Your order total is null". The `analytics-app` sums the field with a null guard, so the revenue dashboard quietly shows zero from Friday afternoon onwards. Three consumers, three different symptoms, one of them completely silent.

Nothing in Kafka could have prevented this. The broker saw bytes, and bytes do not have a "wrong" shape. The root cause is that the agreement between producer and consumers existed only in people's heads and in scattered copies of a sample message.

4.2.2 What goes missing without a schema

A **schema** is a machine-readable description of a record: which fields exist, their names, their types and which of them are optional. Without one, several problems appear sooner or later.

Problem	Example	Consequence
Structural drift	A field is renamed or removed	Consumers read <code>null</code> or fail, sometimes silently
Type drift	<code>qty</code> changes from 2 to "2"	Parse errors, or wrong arithmetic
Ambiguous types	JSON has one "number" type; 499.00 may be read as a binary double	Money rounding errors unless every consumer picks a decimal type
Absent versus null	A missing field, a <code>null</code> field and 0	Each consumer interprets them differently
Repeated names	Every record repeats <code>"customerId"</code> , <code>"unitPrice"</code> and so on	Larger records on the wire and on disk

Problem	Example	Consequence
No documentation	The meaning of fields lives in someone's memory	A new team reverse-engineers the format from samples

Problems that a schema removes or makes visible.

4.2.3 Data lives longer than code

A REST API can be versioned and the old version retired a month later, because requests are short-lived. Kafka records are not. They stay in the log for the retention period (seven days by default), forever in a compacted topic, and in an event-sourced design they are the system of record. A consumer that starts next year, or a new service that replays the topic from the beginning, must be able to read **every** record ever written, including those written under the first version of the format.

That has two consequences. Producers must evolve the format in ways that old and new readers both survive. And each record must be traceable to the version of the format that wrote it. Schema registries solve both: they check changes before they happen and they tag every record with a schema identifier. The next chapter shows how.

4.2.4 What a data contract is

DEFINITION

Data contract: an explicit, versioned agreement between the owner of a data stream and its consumers. It consists of the schema (names, types, defaults), the meaning of the fields (units, currency, allowed values), the rules for changing the schema (the compatibility mode), and an owner who answers questions about it.

The schema is the part a machine can check. The rest is documentation, review and conversation, and it matters just as much. A schema cannot notice that **total** silently changed from rupees to paise: the type is still a number, only the meaning changed. Treat the topic as a public API of the owning team. Changes go through review, and the schema file lives in version control next to the code that produces the events.

4.2.5 Poison pills

A **poison pill** is a record that a consumer cannot process, and that it will meet again every time it tries. The most common cause is bytes that the deserializer cannot decode: a producer wrote JSON where Avro was expected, a field has the wrong type, or the message is truncated. When the deserializer throws, `poll()` throws a `RecordDeserializationException`. The consumer does not move past the broken record, so the next `poll()` fetches the same record and fails the same way. The group stops making progress on that partition, and the lag grows.

The exception tells you exactly where the record sits (`topicPartition()` and `offset()`), and its `valueBuffer()` and `keyBuffer()` return the raw bytes. A hand-written escape hatch looks like this:

JAVA

```
while (running) {
    try {
        ConsumerRecords<String, OrderCreated> records = consumer.poll(Duration.ofMillis(500));
        for (ConsumerRecord<String, OrderCreated> record : records) {
            process(record.value());
        }
    } catch (RecordDeserializationException e) {
        log.error("Skipping undecodable record at {} offset {}", e.topicPartition(), e.offset(), e);
        consumer.seek(e.topicPartition(), e.offset() + 1);
    }
}
```

This loop skips the bad record and carries on. That keeps the service alive, but the record is lost unless you also copy `e.valueBuffer()` to a dead-letter topic. Spring for Apache Kafka automates the whole pattern with `ErrorHandlingDeserializer` and dead-letter publishing (see the error handling chapters in the Spring Kafka Parts). A good schema practice shrinks the problem at the source: when producers can only write records that match a registered, compatible schema, undecodable records almost never reach the topic.

INTERVIEW ANGLE

What is a poison pill in Kafka, and how do you deal with it? A poison pill is a record that makes a consumer fail every time it is fetched, typically because it cannot be deserialized, so the consumer never advances past it. Defences, from cheapest to best: catch the deserialization exception and skip the offset (and keep the bytes in a dead-letter topic), wrap the deserializer so failures become values the error handler can route (Spring's `ErrorHandlerDeserializer`), and prevent bad records in the first place with schemas and compatibility checks on the producer side.

4.2.6 Three schema formats

Three formats dominate the Kafka world. **Apache Avro** and **Protocol Buffers (Protobuf)** are compact binary formats; **JSON Schema** describes ordinary JSON text. All three define fields, types and defaults, and all three can be managed by a schema registry. Later chapters compare them in detail and show Avro and Protobuf in working code. At this point you only need to remember the idea: the schema is written once, stored centrally, checked automatically, and referenced from each record by a small identifier instead of being repeated.

4.2.7 Do you always need one?

No. If one team owns both the producer and the only consumer, if the topic is a prototype or a short-lived log, or if you are learning, plain JSON (or plain strings) is a sensible choice and a registry is extra infrastructure to run, secure and keep available. The balance tips when several teams share a topic, when records are retained for a long time or replayed, when the data feeds analytics or other downstream systems, or when a bad deploy would cost real money. The rest of this Part assumes that you are in the second group.

KEY POINTS

Why schemas matter

- The broker cannot protect consumers from format changes; without a schema the contract exists only in people's heads.
- Kafka data outlives code. Consumers must read every record ever written, so formats must evolve compatibly and records must reveal which version wrote them.
- A data contract is schema plus meaning, change rules and an owner. The schema can be checked by machines; the meaning needs review.
- A poison pill blocks a consumer on one undecodable record. Skip-and-dead-letter handles it; schemas and compatibility checks prevent most cases.
- Schemas cost infrastructure and discipline. Use them when several teams, long retention or money are involved.

4.3 Schema Registry

The previous chapter ended with an idea: write the schema once, store it centrally, check every change automatically, and let each record point to its schema with a small identifier instead of carrying it. The component that implements this idea is a **schema registry**. This chapter explains how the best-known one, Confluent Schema Registry, works and how to talk to it. Apache Kafka itself does not include a registry; it is a separate service that you run next to your brokers.

4.3.1 What the registry does

A registry is a small web service with a REST API. It has four jobs: it stores schemas, it gives each distinct schema a numeric ID, it checks new schemas for compatibility with earlier ones, and it hands schemas back to clients that ask for them by ID. It keeps its data in a Kafka topic, `_schemas` by default, which is a single-partition compacted topic. The service instances themselves hold no state on disk, so you can restart them or run several side by side. One instance acts as the primary, chosen through the Kafka group protocol (`leader.eligibility` controls which nodes may become primary), and every instance can answer reads.

4.3.2 Subjects, versions and IDs

Three words appear everywhere in registry documentation. A **subject** is a named scope for schema evolution: the list of versions of one kind of data. A **version** is a position within a subject's sequence (1, 2, 3, ...). A **schema ID** is the globally unique identifier of a schema definition.

subject "order-status-value"	globally unique schema IDs
version 1 → schema ID 1	ID 1: OrderStatus
subject "orders-value"	
version 1 → schema ID 2	ID 2: OrderCreated, the first layout
version 2 → schema ID 3	ID 3: OrderCreated plus new fields

Versions count per subject; IDs count across the whole registry, in order of registration. A registered schema never changes: a new layout becomes a new version with a new ID. If the same schema text is registered under a second subject, the registry returns the existing ID.

4.3.3 The wire format

When a Schema Registry serializer writes a record, it prepends five bytes to the encoded data. The first byte is a format marker (the "magic byte", always 0 today). The next four bytes are the schema ID as a big-endian integer. The rest is the payload in the chosen encoding. For Protobuf the schema ID is followed by message indexes, so the reader knows which message type inside the schema was written.

byte 0	bytes 1-4	bytes 5 ...
0x00 magic	schema ID (big-endian) e.g. 00 00 00 01 = 1	payload: Avro binary / Protobuf / JSON text, depending on the format

Two consequences follow. A record written this way cannot be read with a plain `StringDeserializer` or `JacksonJsonDeserializer`; every reader must understand the five-byte prefix. And the other way round: if you point a registry deserializer at a topic that contains plain JSON, it fails because the first byte is not the expected marker, with an error about an unknown magic byte. Seeing that message almost always means "this topic was not written by a registry serializer".

4.3.4 How producers and consumers use the registry

The clients hide all registry traffic inside the serializer and deserializer classes. The producer side works like this:

1. `serialize(topic, value)` derives the **subject** from the topic name and the naming strategy (by default `orders-value` for the value of topic `orders`).
2. With `auto.register.schemas` switched on (the default), the serializer registers the value's schema under that subject. The registry checks compatibility with the existing versions and returns the schema ID, or rejects the schema with an HTTP 409 error if it is incompatible. With auto-registration switched off, the serializer only looks up the ID of a schema that must already exist.
3. The serializer writes magic byte, ID and the encoded payload, and the producer sends the bytes as usual.

The consumer side is shorter:

1. The deserializer reads the schema ID from the first five bytes.
2. It fetches the schema with that ID from the registry (`GET /schemas/ids/{id}`).
3. It decodes the payload with that **writer schema**, and converts to the reader's own schema (for example, your generated class) if the two differ.

Both sides cache what they learn, so the registry is contacted only the first time a client sees a schema, not once per record. That keeps the registry out of the hot path. It also means a consumer that is already running keeps working during a short registry outage, but a *new* consumer, or one that meets a new schema ID, cannot decode anything until the registry is back. In production, run at least two registry instances.

4.3.5 Compatibility modes

Every subject has a compatibility mode that decides which new schemas the registry accepts. The terms are always from the point of view of the *new* schema: "backward" means the new schema looks backward at old data. For Avro, call a field **optional** when it has a default value.

Mode	Guarantee	Typical safe change (Avro)	Upgrade first
BACKWARD	Consumers on the new schema can read data written with the previous schema	Delete a field; add a field with a default	Consumers
BACKWARD_TRANSITIVE	Same, for all earlier versions	Same	Consumers
FORWARD	Consumers on the previous schema can read data written with the new schema	Add a field; delete a field that has a default	Producers
FORWARD_TRANSITIVE	Same, for all earlier versions	Same	Producers
FULL	Both of the above, against the previous version	Add or delete a field that has a default	Any order
FULL_TRANSITIVE	Both, against all earlier versions	Same	Any order
NONE	No check at all	Anything	Coordinate by hand

Compatibility modes. A "transitive" mode checks against all earlier versions instead of only the latest one.

The default is **BACKWARD**; the server-side setting is `schema.compatibility.level` (default `backward`) and individual subjects can override it. **BACKWARD** fits Kafka well because a consumer can always rewind: after you upgrade consumers first, they can read both the old records still in the topic and the new ones. Note that plain **BACKWARD** only compares with the previous version. If you replay topics from the beginning, or keep data for a very long time, prefer **BACKWARD_TRANSITIVE** (or the **FULL** variants when producers and consumers deploy independently).

4.3.6 Subject naming strategies

The subject is derived by a pluggable strategy, selected separately for keys and values by `key.subject.name.strategy` and `value.subject.name.strategy`. The classes live in the package `io.confluent.kafka.serializers.subject`.

Strategy	Subject name	Choose it when
TopicNameStrategy	<code>`orders-value` (topic name plus <code>`-key`</code> or <code>`-value`</code>)</code>	Every topic holds one type of record. This is the default.
RecordNameStrategy	<code>`com.shopstream.events.avro.OrderCreated` (the full record name)</code>	One topic holds several record types, or one type is used on several topics and should evolve as a single unit
TopicRecordNameStrategy	<code>`orders-com.shopstream.events.avro.OrderCreated`</code>	One topic holds several record types, and each topic should evolve independently

The three subject naming strategies.

The ShopStream `orders` topic carries `OrderCreated`, `OrderConfirmed` and `OrderCancelled`. With `TopicNameStrategy` all three would be registered under `orders-value` and the registry would check each against the others, which fails immediately because they are different records. For such a multi-type topic use `RecordNameStrategy` or `TopicRecordNameStrategy`. The price: compatibility is checked per record type, not per topic, so nothing stops someone from adding a fourth type to the topic.

4.3.7 Running a registry locally

The following Compose file starts a single-node Kafka broker with two listeners (one for your laptop on port 9092, one for other containers) and a Schema Registry connected to it. The Kafka environment variables follow the rule of the `apache/kafka` image: prefix with `KAFKA_` and replace dots by underscores. The registry image uses the prefix `SCHEMA_REGISTRY_` with the same idea; the Confluent documentation pins an explicit image tag, and so should you.

`docker-compose.yml`

YAML

```
services:
  broker:
    image: apache/kafka:4.3.1
    container_name: broker
    ports:
```

docker-compose.yml (continued)

YAML

```

- "9092:9092"
environment:
  KAFKA_NODE_ID: 1
  KAFKA_PROCESS_ROLES: broker,controller
  KAFKA_LISTENERS: PLAINTEXT://:19092,PLAINTEXT_HOST://:9092,CONTROLLER://:9093
  KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://broker:19092,PLAINTEXT_HOST://localhost:9092
  KAFKA_INTER_BROKER_LISTENER_NAME: PLAINTEXT
  KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
  KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: >-
    CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT,PLAINTEXT_HOST:PLAINTEXT
  KAFKA_CONTROLLER_QUORUM_VOTERS: 1@broker:9093
  KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
  KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 1
  KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: 1
  KAFKA_GROUP_INITIAL_REBALANCE_DELAY_MS: 0

schema-registry:
  image: confluentinc/cp-schema-registry:8.3.2
  container_name: schema-registry
  depends_on:
    - broker
  ports:
    - "8081:8081"
  environment:
    SCHEMA_REGISTRY_HOST_NAME: schema-registry
    SCHEMA_REGISTRY_KAFKASTORE_BOOTSTRAP_SERVERS: broker:19092
    SCHEMA_REGISTRY_LISTENERS: http://0.0.0.0:8081

```

What to notice. The registry reaches the broker through the container-network listener `broker:19092`, while your own applications use `localhost:9092`; a single advertised address could not serve both. The registry creates its `_schemas` topic on first start (replication factor 3 by default, reduced to the number of live brokers, so one here). `depends_on` only orders container start, not readiness: if the registry exits because the broker was not yet reachable, start it again.

4.3.8 Talking to the registry with curl

Everything the serializers do can be done by hand with the REST API. We register a first schema for the compacted `order-status` topic. The schema lives in a file, as it would in a project:

order-status.avsc

JSON

```

{
  "type": "record",
  "name": "OrderStatus",
  "namespace": "com.shopstream.events.avro",
  "fields": [
    { "name": "orderId", "type": "string" },
    { "name": "status", "type": "string" }
  ]
}

```

The API expects the schema as a JSON string inside a JSON body, which is awkward to type. `jq` builds the body from the file for us. Requests use the content type `application/vnd.schemaregistry.v1+json`.

Register a schema and look at it

TERMINAL

```

$ curl -s http://localhost:8081/subjects
[]
$ jq -n --rawfile s order-status.avsc '{schema: $s}' | \
  curl -s -X POST -H "Content-Type: application/vnd.schemaregistry.v1+json" \
    --data @- http://localhost:8081/subjects/order-status-value/versions
{"id":1}
$ curl -s http://localhost:8081/subjects/order-status-value/versions
[1]
$ curl -s http://localhost:8081/subjects/order-status-value/versions/latest | \
  jq '{subject, version, id}'
{
  "subject": "order-status-value",
  "version": 1,
  "id": 1
}
$ curl -s http://localhost:8081/config
{"compatibilityLevel":"BACKWARD"}

```

Registering the same schema again returns the same ID and creates no new version. To change the compatibility mode of one subject, send a **PUT** to `/config/{subject}`:

Override the mode for one subject

TERMINAL

```
$ curl -s -X PUT -H "Content-Type: application/vnd.schemaregistry.v1+json" \
  --data '{"compatibility": "FULL"}' \
  http://localhost:8081/config/order-status-value
{"compatibility":"FULL"}
```

The endpoints you will use most are summarised below. IDs, versions and outputs in this book are examples.

Endpoint	Purpose
GET /subjects	List all subjects
POST /subjects/{subject}/versions	Register a schema (checked for compatibility); returns the ID
GET /subjects/{subject}/versions/{version}	Fetch one version; latest is accepted
GET /schemas/ids/{id}	Fetch a schema by its global ID (what consumers do)
POST /compatibility/subjects/{subject}/versions/{version}	Test a schema against a version without registering it
GET /config and PUT /config	Read or set the global compatibility mode
GET /config/{subject} and PUT /config/{subject}	Read or set the mode of one subject

The Schema Registry REST endpoints used in this book.

4.3.9 Alternatives and licensing

Confluent Schema Registry is distributed under the Confluent Community License. Confluent describes this licence as source-available and not approved by the OSI: you can read, modify and redistribute the code, but you may not use it to offer a competing hosted service. Apache Kafka itself stays under the Apache License 2.0, and the serializer artifacts on Maven are listed under Apache 2.0. Read the licence files of the version you plan to run; this paragraph is a summary, not legal advice.

Two open alternatives exist. **Apicurio Registry** is released under Apache 2.0, manages Avro, Protobuf, JSON Schema and several other artifact types, and can store its data in Kafka, PostgreSQL or SQL Server. **Karapace**, from Aiven, describes itself as a free and open source drop-in replacement for the Confluent Schema Registry and REST Proxy. Both are worth a look if the licence or your platform matters; the rest of this book uses the Confluent REST API and clients because they are the common denominator.

WATCH OUT

Never delete or recreate the `_schemas` topic, and give it production-grade settings. Every record in your topics refers to a schema ID. If the registry loses its topic, those IDs mean nothing and the data becomes undecodable. Keep it compacted, replicate it (a replication factor of 3 with `min.insync.replicas=2` on a three-broker cluster) and disable unclean leader election.

INTERVIEW ANGLE

What does a Schema Registry add to Kafka, and what happens at runtime? It stores versioned schemas per subject, assigns each one a global ID, enforces a compatibility mode when schemas are registered, and serves schemas by ID. At runtime the producer's serializer registers or looks up the schema once, caches the ID and prefixes every record with a magic byte and the four-byte ID. The consumer's deserializer reads the ID, fetches and caches the writer schema and decodes the payload. The broker never sees any of this; it still stores bytes.

KEY POINTS

Schema Registry

- A registry is a separate REST service backed by the compacted `_schemas` topic. Subjects hold versions; IDs are global.
- Each record carries a five-byte prefix: magic byte 0 and a big-endian schema ID. Readers must understand it.
- Clients cache IDs and schemas, so the registry is off the hot path, but it must be highly available.
- The default compatibility mode is **BACKWARD**; use the transitive variants when you replay old data. Mode names describe the new schema's view of the old one.
- `TopicNameStrategy` (default) suits one record type per topic; multi-type topics need `RecordNameStrategy` or `TopicRecordNameStrategy`.

4.4 Avro in Practice

Apache Avro is the format most often used together with Kafka and a schema registry. It stores a record as a compact binary sequence of field values in the order the schema lists them, with no field names and no per-field markers. That is why Avro data is small, and also why you cannot read it without the schema that wrote it. The registry supplies that schema.

4.4.1 A schema for OrderCreated

Avro schemas are JSON documents with a `.avsc` extension. Here is the canonical `OrderCreated` event of the running example. The `type` field of the JSON envelope disappears: in Avro the record name already says what the record is.

OrderCreated.avsc

JSON

```
{
  "type": "record",
  "name": "OrderCreated",
  "namespace": "com.shopstream.events.avro",
  "doc": "An order was accepted by order-service.",
  "fields": [
    { "name": "eventId", "type": "string" },
    { "name": "orderId", "type": "string" },
    { "name": "customerId", "type": "string" },
    { "name": "lines", "type": { "type": "array", "items": {
      "type": "record",
      "name": "OrderLine",
      "fields": [
        { "name": "sku", "type": "string" },
        { "name": "qty", "type": "int" },
        { "name": "unitPrice", "type": { "type": "bytes", "logicalType": "decimal",
          "precision": 12, "scale": 2 } }
      ] } } },
    { "name": "total", "type": { "type": "bytes", "logicalType": "decimal",
      "precision": 12, "scale": 2 } },
    { "name": "currency", "type": "string", "default": "INR" },
    { "name": "occurredAt", "type": { "type": "long", "logicalType": "timestamp-millis" } }
  ]
}
```

What to notice. Avro has primitive types (`null`, `boolean`, `int`, `long`, `float`, `double`, `bytes`, `string`) and complex ones (`record`, `enum`, `array`, `map`, `union`, `fixed`). **Logical types** add meaning on top of a primitive: `decimal` is stored as `bytes` with a stated precision and scale, which is the right way to carry money; `timestamp-millis` is a `long` counting milliseconds since the Unix epoch. The `currency` field has a **default**. Defaults are what make schema evolution possible, as the last chapter of this Part shows.

As a size check, the compact JSON text of this example event (without the `type` field) is 220 bytes. The Avro encoding of the same data is 86 bytes, plus the five-byte registry prefix: 91 bytes in total. The saving comes from leaving out field names and from binary numbers; with real messages it varies, and compression narrows the gap.

```
00 | 00 00 00 02 | 48 | 36 66 31 63 32 64 34 65 2d 38 61 34 37 2d ...
|               |               |
|               |               |   L the characters "6f1c2d4e-8a47-" of the eventId
|               |               |   L the string length 36, zigzag-encoded (0x48 = 72)
|               |               |   L schema ID 2 (ID 1 is the OrderStatus schema registered earlier), big-endian
|               |               |
|               |               |   L magic byte 0
```

4.4.2 Generating Java classes and configuring the build

You can use Avro with generic records (a map-like `GenericRecord` that you fill by field name), but for application code generated classes are much safer: the compiler checks field names and types. The `avro-maven-plugin` reads every `.avsc` file under `src/main/avro` and writes Java sources during the `generate-sources` phase.

pom.xml (the parts that matter)

XML

```
<properties>
  <avro.version>1.12.2</avro.version>
</properties>

<repositories>
  <repository>
    <id>confluent</id>
```

pom.xml (the parts that matter) (continued)

XML

```

    <url>https://packages.confluent.io/maven/</url>
  </repository>
</repositories>

<dependencies>
  <dependency>
    <groupId>org.apache.kafka</groupId>
    <artifactId>kafka-clients</artifactId>
    <version>4.3.1</version>
  </dependency>
  <dependency>
    <groupId>io.confluent</groupId>
    <artifactId>kafka-avro-serializer</artifactId>
    <version>8.3.2</version>
  </dependency>
  <dependency>
    <groupId>org.apache.avro</groupId>
    <artifactId>avro</artifactId>
    <version>${avro.version}</version>
  </dependency>
</dependencies>

<build>
  <plugins>
    <plugin>
      <groupId>org.apache.avro</groupId>
      <artifactId>avro-maven-plugin</artifactId>
      <version>${avro.version}</version>
      <executions>
        <execution>
          <phase>generate-sources</phase>
          <goals>
            <goal>schema</goal>
          </goals>
          <configuration>
            <sourceDirectory>${project.basedir}/src/main/avro/</sourceDirectory>
            <stringType>String</stringType>
            <enableDecimalLogicalType>true</enableDecimalLogicalType>
          </configuration>
        </execution>
      </executions>
    </plugin>
  </plugins>
</build>

```

What to notice. The Confluent serializer libraries are not on Maven Central; they come from Confluent's repository <https://packages.confluent.io/maven/>, so the build needs the `<repositories>` block. Their versions follow Confluent Platform releases (for example 8.3.2), not Apache Kafka. Keep the plugin and the runtime library on the same Avro version. `stringType=String` makes generated getters return `java.lang.String` instead of `CharSequence`, and `enableDecimalLogicalType` asks the generator for `java.math.BigDecimal` fields; without it, decimals may come out as `ByteBuffer`. The generated classes land in `com.shopstream.events.avro`, next to (not instead of) the hand-written records of the JSON examples.

4.4.3 Producing and consuming Avro

Only three things change compared with the String or JSON producers of earlier chapters: the value serializer class, the registry address, and the value type.

AvroOrderProducer.java

JAVA

```

package com.shopstream.avro;

import com.shopstream.events.avro.OrderCreated;
import com.shopstream.events.avro.OrderLine;
import io.confluent.kafka.serializers.KafkaAvroSerializer;
import java.math.BigDecimal;
import java.time.Instant;
import java.util.List;
import java.util.Properties;
import org.apache.kafka.clients.producer.KafkaProducer;
import org.apache.kafka.clients.producer.ProducerConfig;
import org.apache.kafka.clients.producer.ProducerRecord;

```

AvroOrderProducer.java (continued)

JAVA

```
import org.apache.kafka.common.serialization.StringSerializer;

public class AvroOrderProducer {

    public static void main(String[] args) {
        Properties props = new Properties();
        props.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9092");
        props.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG, StringSerializer.class);
        props.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG, KafkaAvroSerializer.class);
        props.put("schema.registry.url", "http://localhost:8081");

        OrderLine line = OrderLine.newBuilder()
            .setSku("SKU-RED-SHOE-42")
            .setQty(2)
            .setUnitPrice(new BigDecimal("499.00"))
            .build();

        OrderCreated event = OrderCreated.newBuilder()
            .setEventId("6f1c2d4e-8a47-4d58-9d0e-3b2f6c1a9e10")
            .setOrderId("o-1001")
            .setCustomerId("c-77")
            .setLines(List.of(line))
            .setTotal(new BigDecimal("998.00"))
            .setOccurredAt(Instant.parse("2026-10-08T10:15:30Z"))
            .build();

        try (KafkaProducer<String, OrderCreated> producer = new KafkaProducer<>(props)) {
            producer.send(new ProducerRecord<>("orders", event.getId(), event));
            producer.flush();
        }
    }
}
```

On the first `send()` the serializer registers the schema under `orders-value` and caches the ID. The builder applies the default for `currency` (it was not set) and refuses to build if a field without a default is missing. The consumer needs one extra switch:

JAVA

```
Properties props = new Properties();
props.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9092");
props.put(ConsumerConfig.GROUP_ID_CONFIG, "notification-service");
props.put(ConsumerConfig.AUTO_OFFSET_RESET_CONFIG, "earliest");
props.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG, StringDeserializer.class);
props.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG, KafkaAvroDeserializer.class);
props.put("schema.registry.url", "http://localhost:8081");
props.put("specific.avro.reader", true);

try (KafkaConsumer<String, OrderCreated> consumer = new KafkaConsumer<>(props)) {
    consumer.subscribe(List.of("orders"));
    for (ConsumerRecord<String, OrderCreated> r : consumer.poll(Duration.ofSeconds(5))) {
        System.out.printf("%s total=%s\n", r.key(), r.value().getTotal());
    }
}
}
```

`specific.avro.reader=true` tells the deserializer to return instances of the generated class. Its default is `false`, in which case you receive a `GenericRecord` and a `ClassCastException` as soon as your code assigns it to `OrderCreated`.

Setting	Side	Meaning
<code>schema.registry.url</code>	both	Address(es) of the registry
<code>auto.register.schemas</code>	producer	Whether the serializer registers unknown schemas itself (on by default)
<code>use.latest.version</code>	producer	Use the latest registered version instead of the record's own schema; only applies when auto-registration is off
<code>specific.avro.reader</code>	consumer	Return generated classes instead of <code>GenericRecord</code> (default false)
<code>key.subject.name.strategy</code> , <code>value.subject.name.strategy</code>	both	How the subject name is derived (default <code>TopicNameStrategy</code>)

The Schema Registry settings you meet most often on Avro clients.

4.4.4 Gotchas

WATCH OUT

Turn off `auto.register.schemas` in production and register schemas from your build pipeline. With auto-registration on, any producer that runs with a changed class registers the new schema as a side effect of its first `send()`. It will still be compatibility-checked, but schema changes then happen at deploy time without review, and each environment drifts on its own. With it off, the serializer only looks up schemas that already exist, and a deployment with an unregistered schema fails loudly at the first record. Register from CI with the REST API or Confluent's Maven plugin.

GOTCHA

A `BigDecimal` with the wrong scale is rejected. The schema above says `scale: 2`. Avro encodes the unscaled integer, so `new BigDecimal("998")` (scale 0) cannot be written under that schema and the serializer throws. Normalise with `setScale(2)` before building the event. The same goes for nulls: a field of type `string` cannot be `null`; for optional values use a union with `null` first, and a default of `null`.

Two more habits help. Put `"null"` first in a union whose default is `null`, because Avro requires the default to match the first branch of the union. And give every field you add later a default: that single rule keeps almost all changes compatible, as the final chapter of this Part shows.

INTERVIEW ANGLE

Why does Avro need the writer's schema to read data, and how do the Kafka serializers provide it? Avro binary data contains only field values in schema order, with no names or types, so a reader without the writer's schema cannot tell where one field ends. The Confluent serializers solve this by registering the schema and prefixing each record with its ID. The deserializer fetches the writer schema by ID, then resolves it against the reader schema: fields missing in the data are filled from the reader's defaults, and fields the reader does not know are skipped.

KEY POINTS

Avro in practice

- Avro encodes only values, in schema order; the writer schema is required for reading, supplied via the registry ID.
- Use `decimal` for money and `timestamp-millis` for instants; give new fields defaults.
- The `avro-maven-plugin` generates Java classes; the Confluent serializers come from Confluent's Maven repository and are versioned with Confluent Platform.
- Producer: `KafkaAvroSerializer` plus `schema.registry.url`. Consumer: `KafkaAvroDeserializer` plus `specific.avro.reader=true` for generated classes.
- In production switch `auto.register.schemas` off and register schemas from CI.

4.5 Protobuf and JSON Schema

Avro is a good default, but the registry also manages two other formats. **Protocol Buffers (Protobuf)** is Google's binary format, popular where services already talk gRPC. **JSON Schema** keeps the data as JSON text and adds validation. This chapter shows both with the same `OrderCreated` event, compares all four options, and gives rules for choosing.

4.5.1 Protocol Buffers

A Protobuf schema is a `.proto` file. Unlike Avro, the bytes on the wire do not contain field names or a schema-order convention: every field is written as a **field number** plus a value. The numbers, not the names, identify fields, so a number must never change meaning once data exists.

order_created.proto

PROTO

```
syntax = "proto3";

package com.shopstream.events;

import "google/protobuf/timestamp.proto";

option java_package = "com.shopstream.events.proto";
option java_multiple_files = true;

message OrderLine {
```

order_created.proto (continued)

PROTO

```

string sku = 1;
int32 qty = 2;
int64 unit_price_minor = 3; // amount in paise: 49900 means INR 499.00
}

message OrderCreated {
  string event_id = 1;
  string order_id = 2;
  string customer_id = 3;
  repeated OrderLine lines = 4;
  int64 total_minor = 5;
  string currency = 6;
  google.protobuf.Timestamp occurred_at = 7;
}

```

What to notice. Protobuf has no decimal type, so money is carried as an integer in the smallest currency unit (a common convention, but you must document it in the contract). Timestamps use the well-known type `google.protobuf.Timestamp`. A proto3 scalar field that was never set reads as its zero value ("", 0), so you cannot tell "not set" from "set to zero" unless you declare the field `optional`. The `protoc` compiler generates the Java classes; build tools offer plugins for it.

The Confluent classes for Protobuf live in the artifact `io.confluent:kafka-protobuf-serializer`, from the same Confluent repository and with the same version numbers as the Avro serializer:

JAVA

```

// producer
props.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG, KafkaProtobufSerializer.class);
props.put("schema.registry.url", "http://localhost:8081");

OrderCreated event = OrderCreated.newBuilder()
    .setEventId("6f1c2d4e-8a47-4d58-9d0e-3b2f6c1a9e10")
    .setOrderId("o-1001")
    .setCustomerId("c-77")
    .addLines(OrderLine.newBuilder().setSku("SKU-RED-SH0E-42").setQty(2)
        .setUnitPriceMinor(49900L))
    .setTotalMinor(99800L)
    .setCurrency("INR")
    .build();

// consumer
props.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG, KafkaProtobufDeserializer.class);
props.put("specific.protobuf.value.type", OrderCreated.class.getName());

```

Both classes are in the package `io.confluent.kafka.serializers.protobuf`. Without `specific.protobuf.value.type` the deserializer cannot know your generated class and returns a generic `DynamicMessage`; the setting for keys is `specific.protobuf.key.type`. The wire format adds *message indexes* after the schema ID, because one `.proto` file may define several messages and the reader must know which one was written.

NOTE

Protobuf support has a few sharp edges. Confluent's documentation states that Protobuf does not allow null values on message fields and recommends `optional` fields where you need presence, and it recommends the `BACKWARD_TRANSITIVE` mode for Protobuf subjects because adding new message types is not forward compatible. At the time of writing it also states that Google Protobuf v4 is not yet supported by these serializers, so check the supported `protobuf-java` version before upgrading it.

4.5.2 JSON Schema

If your consumers must keep receiving JSON (a browser client, an existing integration, a team that wants to read records with its eyes), JSON Schema gives you a registered, versioned, validated description without changing the text format. A schema for the running example looks like this:

OrderCreated.schema.json

JSON

```

{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "OrderCreated",
  "type": "object",
  "properties": {

```

OrderCreated.schema.json (continued)

JSON

```

    "eventId": { "type": "string" },
    "orderId": { "type": "string" },
    "customerId": { "type": "string" },
    "lines": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "sku": { "type": "string" },
          "qty": { "type": "integer", "minimum": 1 },
          "unitPrice": { "type": "number" }
        },
        "required": ["sku", "qty", "unitPrice"]
      }
    },
    "total": { "type": "number" },
    "currency": { "type": "string" },
    "occurredAt": { "type": "string", "format": "date-time" }
  },
  "required": ["eventId", "orderId", "customerId", "lines", "total", "occurredAt"]
}

```

The classes are `io.confluent.kafka.serializers.json.KafkaJsonSchemaSerializer` and `KafkaJsonSchemaDeserializer` (artifact `io.confluent:kafka-json-schema-serializer`). The serializer can derive a schema from a Java class annotated for Jackson, or accept a schema you supply. Three settings are worth knowing: `json.fail.invalid.schema` (default `false`; set it to `true` to make the serializer fail when a value does not match the schema), `json.value.type` (the Java class the deserializer should create; without it you receive a `LinkedHashMap` for objects) and `json.oneof.for.nullables` (default `true`, which controls how nullable fields appear in derived schemas).

GOTCHA

JSON Schema through the registry is no longer plain JSON on the wire. The serializer still writes JSON text, but it puts the same magic byte and four-byte schema ID in front of it. A consumer using a plain JSON deserializer will fail on the first byte. Everybody on the topic needs the registry-aware deserializer, which is exactly the migration cost you hoped to avoid by staying with JSON.

4.5.3 Choosing a format

Aspect	Avro	Protobuf	JSON Schema	Plain JSON
Encoding	Compact binary, values only	Compact binary, field number plus value	JSON text	JSON text
Size on the wire	Smallest; no names, no tags	Small; one tag per field	Largest; names repeated per record	Largest
Needs a schema to read	Yes (writer schema, from the registry)	Yes (generated class or registry)	No, but needed for validation	No
Evolution mechanism	Reader/writer schema resolution by field name; defaults	Field numbers; add new numbers, never reuse	Rules depend on open or closed content model	None; by convention only
Nulls and defaults	Explicit: union with <code>null</code> , declared defaults	Zero values; <code>optional</code> for presence	<code>required</code> list; nullable types	Anything goes
Code generation	Optional (Maven or Gradle plugin)	Normal (<code>protoc</code>)	Rare; POJOs or maps	None
Readable by eye	No	No	Yes (after the prefix)	Yes
Registry support	Yes	Yes	Yes	Not applicable

Avro, Protobuf, JSON Schema and plain JSON compared. "Registry" means a schema registry such as Confluent Schema Registry.

There is no universally best format, but there are good defaults. Choose **Avro** when Kafka is the centre of your data flow: it has the longest history in the Kafka ecosystem, its reader/writer resolution suits long-retention topics, and Kafka Connect, Kafka Streams and ksqlDB all work with it. Choose **Protobuf** when your organisation already defines contracts in `.proto` files for gRPC, or when you need strong code generation in many languages. Choose **JSON Schema** when readability and compatibility with existing JSON consumers outweigh size. Stay with **plain JSON** only for prototypes, local experiments and topics with a single owner.

TIP

Pick one format per organisation (or at least per domain) and stick with it. Mixed formats multiply the tooling every team must learn, the monitoring you need, and the number of ways a consumer can misread a topic.

KEY POINTS

Protobuf and JSON Schema

- Protobuf identifies fields by number: add new fields with new numbers, never reuse or retype one. It has no decimal type, so money usually travels as minor units.
- Confluent's Protobuf serializer needs `specific.protobuf.value.type` to return generated classes (otherwise `DynamicMessage`). Use `BACKWARD_TRANSITIVE`.
- JSON Schema keeps JSON text but adds a magic byte and schema ID, so plain JSON readers can no longer consume it.
- Avro is a strong default for Kafka-centred pipelines, Protobuf fits gRPC-heavy organisations, JSON Schema fits JSON-bound consumers.

4.6 Evolving Schemas Safely

A schema is not finished the day it is registered. Fields get added, requirements change, and the Friday incident of the earlier chapter will happen again unless changes follow rules. This chapter turns the compatibility modes into everyday practice: which changes are safe, how to roll them out, how to test them before they reach production, and what to do when a change cannot be made compatible.

4.6.1 Which changes are safe

The table lists common Avro changes and whether each mode accepts them (according to the Confluent documentation). A "field with a default" is called optional; a field without a default is required.

Change	BACKWARD	FORWARD	FULL
Add an optional field (with default)	✓	✓	✓
Add a required field (no default)	✗	✓	✗
Remove an optional field	✓	✓	✓
Remove a required field	✓	✗	✗
Add a variant to a union	✓	✗	✗
Remove a variant from a union	✗	✓	✗
Widen a scalar type (for example int to long)	✓	✗	✗
Narrow a scalar type	✗	✓	✗
Rename a field	✗	✗	✗

Avro schema changes and the compatibility modes that accept them.

Read the table with the Avro resolution rules in mind. A reader that expects a field the data does not contain uses the field's default, or fails if there is none: that is why adding a required field breaks **BACKWARD**. A field in the data that the reader does not know is simply skipped: that is why removing fields is fine for **BACKWARD**. A rename looks like a removal plus an addition, and therefore breaks old data or old readers. Avro **aliases** can tell a reader that a new field name stands for an old one, but they help only one direction, so treat a rename as a multi-step change (below).

Enums deserve a warning of their own. If a producer adds a symbol to an enum, old readers meet a symbol they do not know and fail, unless the reader schema declares a default symbol for the enum. Prefer a plain **string** for values that will grow, or declare an enum default from the first day.

For Protobuf the rules are shorter: add new fields with new numbers, never change the number or type of an existing field, and when you delete a field, mark its number and name as **reserved** so nobody reuses them:

PROTO

```
message OrderCreated {
  reserved 5;
  reserved "total_minor";
  // ...
}
```

4.6.2 The Friday incident, solved by expand and contract

Back to the renamed `total`. With the registry in place the developer's change would have been rejected: removing the required field `total` and adding a required `grandTotal` is not accepted under **FORWARD** or **FULL**, and a new required field is refused under **BACKWARD**. The registry forces the safer path, **expand and contract**:

1. **Expand.** Add the new fields as optional, keep the old one. Producers fill both `total` and `grandTotal`.
2. **Migrate.** Consumers switch to `grandTotal` one at a time, falling back to `total` while old records still exist.
3. **Contract.** After every consumer has moved (and old records are no longer needed, or a transitive check allows it), remove `total`.

The fields added in step 1 use the idiom for optional values in Avro, a union that starts with `null` and has the default `null`:

fields added in version 2

JSON

```
[
  {
    "name": "grandTotal",
    "type": ["null", { "type": "bytes", "logicalType": "decimal", "precision": 12, "scale": 2 }],
    "default": null },
  {
    "name": "tax",
    "type": ["null", { "type": "bytes", "logicalType": "decimal", "precision": 12, "scale": 2 }],
    "default": null }
]
```

4.6.3 Testing a change before you ship it

The registry can judge a candidate schema without registering it. Save two variants of `OrderCreated.avsc`: variant A appends the two fields above without the `"default": null` line (required fields), variant B appends them exactly as shown. Then ask the registry:

Compatibility checks against the latest version of orders-value

TERMINAL

```
$ jq -n --rawfile s OrderCreated-a.avsc '{schema: $s}' | \
  curl -s -X POST -H "Content-Type: application/vnd.schemaregistry.v1+json" \
    --data @- http://localhost:8081/compatibility/subjects/orders-value/versions/latest
{"is_compatible":false}
$ jq -n --rawfile s OrderCreated-b.avsc '{schema: $s}' | \
  curl -s -X POST -H "Content-Type: application/vnd.schemaregistry.v1+json" \
    --data @- http://localhost:8081/compatibility/subjects/orders-value/versions/latest
{"is_compatible":true}
```

Variant A is rejected under the default **BACKWARD** mode because consumers on the new schema could not read the existing records, which have no `grandTotal`. Variant B passes. Appending `?verbose=true` to the URL asks the registry to include messages that explain a rejection.

4.6.4 Rolling out a compatible change

Compatible does not mean "deploy everything at once". The order depends on the mode, and it is the same order every time:

- **BACKWARD** (the default): upgrade the **consumers** first. They then understand old and new records. Only afterwards let producers write the new version.
- **FORWARD**: upgrade the **producers** first. The consumers still on the old schema can read what the producers now write, and they upgrade whenever they like.
- **FULL**: upgrade in any order.
- **Transitive variants**: the same orders, but the guarantee covers every earlier version, which is what you need if consumers may replay the whole topic.

A mnemonic: the mode name says who can read what from the point of view of the new schema. **BACKWARD** means "new reads old", so the readers go first. **FORWARD** means "old reads new", so the writers go first.

WATCH OUT

Do not evolve key schemas casually. The default partitioner hashes the serialized key bytes. If a schema change alters how the same logical key is encoded, records for that key suddenly land in a different partition, and per-key ordering is broken at that moment. Keep keys simple: a plain string or number (`orderId`) rather than a record, and treat the key format as frozen.

4.6.5 When a change cannot be compatible

Some changes are inherently breaking: splitting one event into two, changing the meaning of a field, switching the unit of an amount, changing the key. For those you cannot rely on the registry. The usual options are:

Option	How it works	Cost
New event type	Introduce <code>OrderCreatedV2</code> (a new record name) on the same topic with a multi-type naming strategy	Consumers must handle both types for a while
New topic	Publish to <code>orders-v2</code> (often through a Kafka Streams job or by dual-writing) and move consumers across; delete the old topic when it is no longer read	Duplicated data, ordering across the two topics, migration effort
Set the mode to <code>NONE</code> for one change	Lets the registry accept anything	Only with a coordinated, full stop of readers; the safety net is gone for that moment

Options for a breaking change.

Prefer prevention: review the meaning of fields as carefully as their types, document units and currencies in the schema's `doc` attributes, and treat new topics as an expensive tool that you use rarely.

4.6.6 Checking compatibility in CI

Compatibility should be checked on the pull request, not discovered in production. The simplest version is a script that asks the registry about every schema file and fails the build on a `false` answer. Run it against a registry that holds the current production schemas (a staging registry that mirrors production is the usual choice).

check-schemas.sh

BASH

```
#!/usr/bin/env bash
set -euo pipefail

REGISTRY="${REGISTRY:-http://localhost:8081}"

check() {
  local subject="$1" file="$2" result
  result=$(jq -n --rawfile s "$file" '{schema: $s}' | \
    curl -s -X POST -H "Content-Type: application/vnd.schemaregistry.v1+json" \
      --data @- "$REGISTRY/compatibility/subjects/$subject/versions/latest")
  if [ "$(echo "$result" | jq -r '.is_compatible')" != "true" ]; then
    echo "INCOMPATIBLE: $file against $subject: $result" >&2
    return 1
  fi
  echo "ok: $file against $subject"
}

check orders-value src/main/avro/OrderCreated.avsc
check order-status-value src/main/avro/OrderStatus.avsc
```

The check needs an existing version to compare with, so the very first version of a subject is registered by a deliberate step. Confluent also publishes a Maven plugin, `io.confluent:kafka-schema-registry-maven-plugin`, whose goals include `validate` (syntax), `test-compatibility` (local files against a registry), `register` and `set-compatibility`. Its `subjects` configuration maps a subject name to a schema file, which lets the same build both test and, after review, register the schemas.

INTERVIEW ANGLE

You use BACKWARD compatibility. In which order do you deploy a new schema version, and why? Consumers first, then producers. BACKWARD means a consumer on the new schema can read data written with the previous schema, so upgraded consumers handle old and new records. If producers went first, consumers on the old schema could receive records with fields they do not expect. For FORWARD it is the other way round, and with FULL the order does not matter. If consumers may replay the whole topic, use the TRANSITIVE variant so the guarantee covers every earlier version, not only the last.

KEY POINTS

Evolving schemas safely

- Give every new field a default. Adding and removing optional fields is the safest change in every mode.
- Never rename or retype a field in place. Use expand and contract: add, migrate consumers, remove.
- BACKWARD: consumers first. FORWARD: producers first. FULL: any order. Use TRANSITIVE when old data may be replayed.
- Test candidate schemas with the registry's `/compatibility` endpoint (or the Maven plugin) in CI, not in production.
- Keep key formats frozen. For truly breaking changes use a new event type or a new topic, and plan the migration.

PART 5

Spring Boot and Spring for Apache Kafka

From plain Kafka clients to productive Spring code: templates, listeners, containers, configuration, serialization and a small project.

In this part

5.1 Spring Boot and Kafka: Project Setup

5.2 Producing with KafkaTemplate

5.1 Spring Boot and Kafka: Project Setup

The plain Kafka clients of the earlier Parts are powerful and low-level. You create the producer and the consumer, you run the `poll()` loop on a thread of your own, you commit offsets, you react to rebalances and you shut everything down cleanly. **Spring for Apache Kafka** (the `spring-kafka` project) keeps the same clients underneath and takes over the repetitive parts: a `KafkaTemplate` for sending, the `@KafkaListener` annotation for receiving, listener containers that own the poll loops, error handling and retries, and integration with transactions, testing and observability. **Spring Boot** then configures all of it from `application.yml`.

The versions in this Part are Spring Boot 4.1.x (built on Spring Framework 7), Spring for Apache Kafka 4.1.x, Java 21, and the Kafka client library that the Spring Boot BOM manages. Do not pin the client version yourself in a Boot project. The broker is the Kafka 4.3.1 setup from the hands-on Part.

5.1.1 What Spring adds

Concern	Plain Kafka client	With Spring Kafka
Sending	Create a <code>KafkaProducer</code> , call <code>send</code> , pass a callback	Inject a <code>KafkaTemplate</code> ; <code>send</code> returns a <code>CompletableFuture</code>
Receiving	Write the <code>poll()</code> loop and the thread around it	Annotate a method with <code>@KafkaListener</code> ; a container runs the loop
Threads	You start and stop them	Containers manage them; <code>concurrency</code> adds consumers
Offsets	Call <code>commitSync</code> or <code>commitAsync</code>	Choose an acknowledgment mode on the container
Failures	<code>try/catch</code> inside the loop	Error handlers, back-off, retry topics, dead-letter topics
Topics	Admin client calls	<code>NewTopic</code> beans created at startup
Configuration	<code>Properties</code> objects	<code>spring.kafka.*</code> properties, or your own beans

What changes when you move from the plain clients to Spring for Apache Kafka.

5.1.2 Creating the project

The project below is the `order-service` of ShopStream. It needs the web starter for its REST API and the Kafka starter. In Spring Boot 4 the Kafka support is packaged in `spring-boot-starter-kafka`, with `spring-boot-starter-kafka-test` for tests, and the starter for Spring MVC is called `spring-boot-starter-webmvc` (the older name `spring-boot-starter-web` still exists but is deprecated).

pom.xml (the parts that matter)

XML

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>4.1.1</version>
</parent>

<properties>
  <java.version>21</java.version>
</properties>

<dependencies>
```

pom.xml (the parts that matter) (continued)

XML

```

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webmvc</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-kafka</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-jackson</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-kafka-test</artifactId>
  <scope>test</scope>
</dependency>
</dependencies>

<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>

```

What to notice. None of the dependencies has a version: the Boot parent manages them, including `spring-kafka` and the Kafka client. `spring-boot-starter-jackson` brings Jackson 3 (package `tools.jackson`), which Spring Kafka's JSON classes need; a web starter usually pulls it in already, but a worker service without a web layer, like the notification service later in this Part, should declare it explicitly.

The application class is the usual one; no Kafka annotation is needed, because auto-configuration enables the Kafka infrastructure when it sees the classes on the classpath.

OrderServiceApplication.java

JAVA

```

package com.shopstream.order;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class OrderServiceApplication {

    public static void main(String[] args) {
        SpringApplication.run(OrderServiceApplication.class, args);
    }
}

```

5.1.3 Configuration in application.yml

Everything under `spring.kafka` is bound to a `KafkaProperties` object. The sections mirror the client types:

Namespace	What it configures
<code>spring.kafka.bootstrap-servers</code>	The broker addresses, shared by all clients
<code>spring.kafka.producer.*</code>	Producer settings such as <code>acks</code> , <code>batch-size</code> , <code>compression-type</code> , <code>key-serializer</code> , <code>value-serializer</code> , <code>transaction-id-prefix</code>
<code>spring.kafka.consumer.*</code>	Consumer settings such as <code>group-id</code> , <code>auto-offset-reset</code> , <code>enable-auto-commit</code> , <code>max-poll-records</code> , <code>isolation-level</code> , <code>key-deserializer</code> , <code>value-deserializer</code>
<code>spring.kafka.listener.*</code>	The listener containers: <code>ack-mode</code> , <code>concurrency</code> , <code>type</code> , <code>poll-timeout</code> , <code>idle-event-interval</code> , <code>missing-topics-fatal</code>
<code>spring.kafka.admin.*</code>	The <code>KafkaAdmin</code> that creates topics: <code>auto-create</code> , <code>fail-fast</code> , <code>modify-topic-configs</code>
<code>spring.kafka.template.*</code>	Defaults of the <code>KafkaTemplate</code> , for example <code>default-topic</code>
<code>spring.kafka.properties.*</code>	Any client property, passed through to all clients (producer, consumer, admin)

Namespace	What it configures
<code>spring.kafka.producer.properties.*</code>	Any producer property, with the real Kafka name as key

The namespaces under `spring.kafka`.

The dedicated properties cover the common cases. For everything else use the `properties` pass-through with the original Kafka key. The Boot reference puts it this way: properties that do not include a client type (`producer`, `consumer`, `admin` or `streams`) in their name are considered common and apply to all clients. Here is a configuration for the order service:

```
application.yml
spring:
  application:
    name: order-service
  kafka:
    bootstrap-servers: localhost:9092
    producer:
      acks: all
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.springframework.kafka.support.serializer.JsonSerializer
      properties:
        spring.json.add.type.headers: false
        linger.ms: 5
```

The producer's `acks` and idempotence defaults already come from the Kafka 4 client (`acks=all`, `enable.idempotence=true`); the line is there to make the intent visible. `JsonSerializer` is Spring's generic JSON serializer, and `spring.json.add.type.headers` is one of its settings, explained in the next chapter. Boot's own defaults for keys and values, when you configure nothing, are the `String` serializer and deserializer.

5.1.4 What auto-configuration gives you

With `spring-kafka` on the classpath Boot creates, unless you define a bean of the same type yourself:

- a `KafkaAdmin`, which creates the topics declared as `NewTopic` beans;
- a `ProducerFactory` and a `KafkaTemplate`, built from `spring.kafka.producer.*`;
- a `ConsumerFactory` and a `ConcurrentKafkaListenerContainerFactory` (bean name `kafkaListenerContainerFactory`), built from `spring.kafka.consumer.*` and `spring.kafka.listener.*`, and the switch that turns `@KafkaListener` processing on (what `@EnableKafka` does in a non-Boot application);
- a `KafkaTransactionManager`, but only if `spring.kafka.producer.transaction-id-prefix` is set;
- with Kafka Streams on the classpath and `@EnableKafkaStreams`, the Streams configuration (a later Part).

Boot also attaches some beans to the container factory automatically if it finds exactly one of them: a `CommonErrorHandler`, a `RecordFilterStrategy`, an `AfterRollbackProcessor`, a rebalance listener and a record message converter. That is the least-effort way to customise listener behaviour, and the next chapters use it.

5.1.5 Declaring topics

A `NewTopic` bean tells the `KafkaAdmin` to create a topic during startup if it does not exist. `TopicBuilder` makes these beans readable.

```
KafkaTopicConfig.java
package com.shopstream.order;

import org.apache.kafka.clients.admin.NewTopic;
import org.apache.kafka.common.config.TopicConfig;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.kafka.config.TopicBuilder;

@Configuration
public class KafkaTopicConfig {

    @Bean
    NewTopic ordersTopic() {
        return TopicBuilder.name("orders")
            .partitions(3)
            .replicas(1)
            .build();
    }
}
```

KafkaTopicConfig.java (continued)

JAVA

```

        .config(TopicConfig.MIN_IN_SYNC_REPLICAS_CONFIG, "1")
        .build();
    }

    @Bean
    NewTopic orderStatusTopic() {
        return TopicBuilder.name("order-status")
            .partitions(3)
            .replicas(1)
            .compact()
            .build();
    }
}

```

If the broker is not reachable at startup, the admin logs a message and the application context still starts (set the admin's `fail-fast` property to make this fatal). If a topic exists with fewer partitions than declared, the admin increases the partition count; it never reduces it. For the local single-broker setup `replicas(1)` is the only option; in production use at least 3 replicas and a higher `min.insync.replicas`. Omitting `partitions()` or `replicas()` makes the broker's defaults apply.

WATCH OUT

Declared topics are a convenience for development and tests, not a topic-management strategy. Partition counts, replication factors and configs of production topics usually come from reviewed infrastructure code, because a wrong value is expensive: the partition count can only grow, and growing it changes which partition each key maps to. Switch `auto-create` off for the admin in environments where the application must not create topics.

GOTCHA

A new consumer group starts at the end of the topic by default. Kafka's `auto.offset.reset` defaults to `latest`, and Boot does not change it. A listener started for the first time sees only records produced after it joined, which looks as if "messages disappear". During development set `spring.kafka.consumer.auto-offset-reset: earliest`, and decide deliberately for production.

KEY POINTS

Project setup

- Spring for Apache Kafka wraps the plain clients: a template for sending, annotated listeners and containers for receiving. Boot configures everything from `spring.kafka.*`.
- In Boot 4 add `spring-boot-starter-kafka` (and `spring-boot-starter-kafka-test` for tests); versions come from the Boot BOM.
- Dedicated properties cover the common settings; `spring.kafka.properties.*` and `spring.kafka.producer.properties.*` pass any Kafka property through.
- Auto-configuration creates `KafkaAdmin`, `KafkaTemplate`, the factories and the listener container factory, and backs off when you define your own beans.
- `NewTopic` beans create missing topics at startup; partitions can grow but never shrink.

5.2 Producing with KafkaTemplate

`KafkaTemplate<K, V>` is the class you use to send records from Spring code. It wraps a Kafka producer obtained from a `ProducerFactory`. The default factory keeps a single shared producer, and the template is thread-safe, so you create one template per configuration (Boot creates it for you) and inject it wherever you need it. Never create a template per request.

5.2.1 The send methods

All send methods return a `CompletableFuture<SendResult<K, V>>`. They differ in how much of the record you specify.

Method	Typical use
<code>send(String topic, V data)</code>	No key; records are spread over the partitions
<code>send(String topic, K key, V data)</code>	The normal case; the key decides the partition

Method	Typical use
<code>send(String topic, Integer partition, K key, V data)</code>	Pin a record to a partition
<code>send(String topic, Integer partition, Long timestamp, K key, V data)</code>	Also set the record timestamp, for example when importing history
<code>send(ProducerRecord<K, V> record)</code>	Full control, including headers
<code>send(Message<?> message)</code>	Spring Messaging style; topic, key and partition travel in headers
<code>sendDefault(...)</code> in four variants	Same as the first four, but to the template's default topic

The `send` and `sendDefault` overloads (all return `CompletableFuture<SendResult<K, V>>`).

`sendDefault` needs a default topic, set with `spring.kafka.template.default-topic` or `setDefaultTopic`. The template also offers `flush()`, `partitionsFor(topic)` and `metrics()`, and methods for transactions that Part 8 covers.

5.2.2 What the future tells you

`send` returns before the broker has answered. The record has been handed to the producer, which batches it (waiting up to `linger.ms`, 5 ms by default) and sends it in the background. The returned future completes when the producer knows the outcome, and the `SendResult` it carries holds the `ProducerRecord` that was sent and the `RecordMetadata` assigned by the broker: topic, partition, offset and timestamp. On failure the future completes exceptionally, typically with a `KafkaProducerException` that carries the failed record.

Your choices for handling the result:

- **Fire and forget.** Ignore the future. By default the template logs failures through a `LoggingProducerListener`, but your code will never know. Acceptable for metrics or best-effort events only.
- **Callback.** Attach `whenComplete` and react asynchronously. This does not block the calling thread and is the right default.
- **Wait.** Call `future.get(timeout, unit)` to block until the broker has answered. It turns throughput into latency, so use it where the caller truly needs the guarantee, for example before returning success to a client.
- **Producer listener.** Register a `ProducerListener` on the template (methods `onSuccess` and `onError`) to get one central callback for every send instead of handling each future.

5.2.3 Keys, headers and JSON

For `ShopStream` every event of an order uses the `orderId` as the key. All records with the same key go to the same partition, so a consumer sees the events of one order in the order they were sent. Headers carry metadata that is not part of the business payload; with the `ProducerRecord` overload you add them directly:

JAVA

```
ProducerRecord<String, OrderCreated> record =
    new ProducerRecord<>("orders", event.orderId(), event);
record.headers().add("source", "order-service".getBytes(StandardCharsets.UTF_8));
kafkaTemplate.send(record);
```

The `Message` overload is the Spring Messaging way to say the same thing. `KafkaHeaders.TOPIC`, `KafkaHeaders.KEY`, `KafkaHeaders.PARTITION` and `KafkaHeaders.TIMESTAMP` control the record; other headers become Kafka record headers:

JAVA

```
Message<OrderCreated> message = MessageBuilder.withPayload(event)
    .setHeader(KafkaHeaders.TOPIC, "orders")
    .setHeader(KafkaHeaders.KEY, event.orderId())
    .setHeader("source", "order-service")
    .build();
kafkaTemplate.send(message);
```

With `JacksonJsonSerializer` as the value serializer the event record becomes JSON text. By default the serializer also adds a record header named `__TypeId__` with the Java class name, so that a consumer's `JacksonJsonDeserializer` can rebuild the right class. That is convenient inside one codebase and a trap between services: the consumer then depends on the producer's package and class names. The next chapters show how to switch the header off (`spring.json.add.type.headers=false`) and give each consumer its own target type.

5.2.4 The event records and the order service

The events are plain Java records in the shared package `com.shopstream.events` (in a real project they live in a small library that producers and consumers both depend on, or are generated from a schema as in the previous Part).

OrderLine.java

JAVA

```
package com.shopstream.events;

import java.math.BigDecimal;

public record OrderLine(String sku, int qty, BigDecimal unitPrice) { }
```

OrderCreated.java

JAVA

```
package com.shopstream.events;

import java.math.BigDecimal;
import java.time.Instant;
import java.util.List;

public record OrderCreated(String eventId, String orderId, String customerId,
    List<OrderLine> lines, BigDecimal total, String currency,
    Instant occurredAt) { }
```

The publisher wraps the template. It builds the record, attaches the header, and logs the outcome in a callback:

OrderEventPublisher.java

JAVA

```
package com.shopstream.order;

import com.shopstream.events.OrderCreated;
import java.nio.charset.StandardCharsets;
import org.apache.kafka.clients.producer.ProducerRecord;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.kafka.core.KafkaTemplate;
import org.springframework.stereotype.Service;

@Service
public class OrderEventPublisher {

    private static final Logger log = LoggerFactory.getLogger(OrderEventPublisher.class);
    private static final String TOPIC = "orders";

    private final KafkaTemplate<String, OrderCreated> kafkaTemplate;

    public OrderEventPublisher(KafkaTemplate<String, OrderCreated> kafkaTemplate) {
        this.kafkaTemplate = kafkaTemplate;
    }

    public void publish(OrderCreated event) {
        ProducerRecord<String, OrderCreated> record =
            new ProducerRecord<>(TOPIC, event.orderId(), event);
        record.headers().add("source", "order-service".getBytes(StandardCharsets.UTF_8));

        kafkaTemplate.send(record).whenComplete((result, ex) -> {
            if (ex != null) {
                log.error("Could not publish order {}", event.orderId(), ex);
            } else {
                var meta = result.getRecordMetadata();
                log.info("Published order {} to {}-{} at offset {}",
                    event.orderId(), meta.topic(), meta.partition(), meta.offset());
            }
        });
    }
}
```

The service turns a request into an event and the controller exposes it over HTTP:

OrderService.java

JAVA

```
package com.shopstream.order;

import com.shopstream.events.OrderCreated;
import com.shopstream.events.OrderLine;
import java.math.BigDecimal;
```

OrderService.java (continued)

JAVA

```

import java.time.Instant;
import java.util.List;
import java.util.UUID;
import java.util.concurrent.atomic.AtomicLong;
import org.springframework.stereotype.Service;

@Service
public class OrderService {

    public record CreateOrderRequest(String customerId, List<OrderLine> lines) { }

    private final AtomicLong sequence = new AtomicLong(1000);
    private final OrderEventPublisher publisher;

    public OrderService(OrderEventPublisher publisher) {
        this.publisher = publisher;
    }

    public OrderCreated create(CreateOrderRequest request) {
        BigDecimal total = request.lines().stream()
            .map(line -> line.unitPrice().multiply(BigDecimal.valueOf(line.qty())))
            .reduce(BigDecimal.ZERO, BigDecimal::add);
        OrderCreated event = new OrderCreated(
            UUID.randomUUID().toString(),
            "o-" + sequence.incrementAndGet(),
            request.customerId(),
            request.lines(),
            total,
            "INR",
            Instant.now());
        publisher.publish(event);
        return event;
    }
}

```

OrderController.java

JAVA

```

package com.shopstream.order;

import com.shopstream.events.OrderCreated;
import com.shopstream.order.OrderService.CreateOrderRequest;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/orders")
public class OrderController {

    public record OrderAccepted(String orderId, String status) { }

    private final OrderService orderService;

    public OrderController(OrderService orderService) {
        this.orderService = orderService;
    }

    @PostMapping
    public ResponseEntity<OrderAccepted> create(@RequestBody CreateOrderRequest request) {
        OrderCreated event = orderService.create(request);
        return ResponseEntity.accepted().body(new OrderAccepted(event.orderId(), "ACCEPTED"));
    }
}

```

What to notice. The template is injected through the constructor, with the key and value types spelled out; Boot's single `KafkaTemplate` bean satisfies it. The record key is the order id. `whenComplete` runs on the producer's I/O thread, so keep the callback short and never block in it. The HTTP request returns `202 Accepted` without waiting for Kafka. Orders get ids like `o-1001` from an in-memory counter, which is only good enough for a demo.

5.2.5 Running it

Start the broker, the application, and a console consumer on the `orders` topic. The application creates the topic at startup. Then post an order:

Create an order and read the event

TERMINAL

```
$ mvn spring-boot:run
$ curl -s -X POST http://localhost:8080/orders -H "Content-Type: application/json" \
  -d '{"customerId":"c-77","lines":[{"sku":"SKU-RED-SHOE-42","qty":2,"unitPrice":499.00}]}'
{"orderId":"o-1001","status":"ACCEPTED"}
$ bin/kafka-console-consumer.sh --topic orders --from-beginning --max-messages 1 \
  --formatter-property print.key=true --formatter-property key.separator=" | " \
  --bootstrap-server localhost:9092 | cut -c1-100
o-1001 | {"eventId":"5d0e9a1c-3b77-4e0f-8a52-9c1d7e4b2f10","orderId":"o-1001","customerId":"c-
```

In the application log you should see a line like `Published order o-1001 to orders-1 at offset 0`. The partition number depends on the hash of the key, and the order of the JSON fields depends on the mapper configuration; both are examples here.

WATCH OUT

A successful `send` does not mean the order is safely stored. The controller above answers `202` even if Kafka is down: the failure only appears in the log. And if the service had saved the order in a database and then published the event, a crash between the two steps would leave the database and the topic disagreeing (the dual-write problem). The outbox pattern in the design patterns Part is the standard cure. For events where loss is unacceptable, at least wait for the future before replying, and keep the producer defaults (`acks=all`, idempotence on).

INTERVIEW ANGLE

Is `KafkaTemplate.send` synchronous? How do you know a message arrived? No. It returns a `CompletableFuture` immediately after handing the record to the producer's buffer. The future completes with a `SendResult` (carrying topic, partition and offset) once the broker acknowledges according to `acks`, or exceptionally after the producer gave up (`delivery.timeout.ms`). You attach `whenComplete` for asynchronous handling or call `get` with a timeout to wait. With `acks=all`, a successful result means all in-sync replicas have the record.

KEY POINTS

Producing with `KafkaTemplate`

- One thread-safe `KafkaTemplate` wraps a shared producer; inject it, never create one per request.
- `send` returns a `CompletableFuture<SendResult>` immediately; handle success and failure with `whenComplete`, or wait with `get` when the caller needs the guarantee.
- Use the order id (or another business key) as record key for per-entity ordering; use `ProducerRecord` or `Message` to add headers.
- `JacksonJsonSerializer` writes JSON and adds a type header by default; across services prefer to switch it off.
- A 202 response only means the record was handed over. Real delivery guarantees need the future, the producer defaults, and for database changes the outbox pattern.

PART 6

Spring Kafka in Production: Errors, Retries, Transactions, Testing and Observability

What separates a demo from a service you can run at 3 a.m.: failure handling, duplicates, transactions, tests and metrics.

In this part

- 6.1** Error Handling in Spring Kafka Listeners
- 6.2** Non-Blocking Retries with `@RetryableTopic`
- 6.3** Idempotent Consumers and the Inbox Pattern
- 6.4** Transactions in Spring Kafka

6.1 Error Handling in Spring Kafka Listeners

A listener that works on your laptop will meet failure in production, probably within the first week. A database times out. A producer publishes a malformed JSON document. A deploy ships a bug that throws a `NullPointerException` for one customer. What the consumer does next decides whether you get a blip on a dashboard, a stuck partition, silently lost records, or a service that has stopped consuming while its health check still says UP.

This chapter explains the error-handling model of the Spring Kafka listener container, shows how to configure retries and dead-letter topics, and ends with a decision table for a choice every team faces sooner or later: block the partition and retry in place, or move the record aside and keep going.

6.1.1 What happens when a listener throws

A `@KafkaListener` method is only a method. The code that calls it is the **listener container**: a loop on its own thread that polls the broker, calls your method for each record and commits offsets. When your method throws, the container must answer three questions:

1. Should this record be tried again?
2. If so, after how long and how many times?
3. If we give up, what happens to the record and to its offset?

Spring answers with one interface, `CommonErrorHandler` ("common" because the same interface serves record and batch listeners). Its default implementation, and the one you will almost always use, is `DefaultErrorHandler`. Spring Boot attaches any `CommonErrorHandler` bean to the auto-configured listener container factory, so declaring a bean is all the wiring you need.

Class	Behaviour
<code>DefaultErrorHandler</code>	Retries with a back-off, then recovers (log, dead-letter or custom). Preserves order. The subject of this chapter
<code>CommonContainerStoppingErrorHandler</code>	Stops the container when the listener throws. Fail fast: nothing is skipped, but someone must restart the consumer
<code>CommonLoggingErrorHandler</code>	Logs the failure and moves on: no retry, the failed record is skipped
<code>CommonDelegatingErrorHandler</code>	Chooses a delegate by exception type, for example the default handler for most failures and the stopping handler for a fatal one
<code>CommonMixedErrorHandler</code>	Uses one handler for record listeners and another for batch listeners

The `CommonErrorHandler` implementations that ship with Spring Kafka.

DEFINITION

A **poison pill** (or poison message) is a record that can never be processed successfully, for example malformed JSON or an order that references a product that does not exist. If a consumer retries it forever, nothing behind it on that partition is ever processed.

The failures you meet in practice fall into a few classes, and they deserve different treatment.

Failure	Example	Right reaction
Deserialization	Bytes are not valid JSON; unknown Avro schema id	Never retry. Dead-letter the record, fix the producer
Conversion	Payload cannot be converted to the method parameter	Never retry. Dead-letter it
Business validation	Order with quantity 0, unknown SKU	Never retry. Dead-letter it, or handle it in code and return normally
Transient dependency	Database timeout, HTTP 503 from a payment gateway	Retry after a delay; dead-letter only when retries run out
Bug in your code	<code>NullPointerException</code> on a field nobody expected to be null	Retrying rarely helps. Dead-letter, fix, then replay
Error, not Exception	<code>OutOfMemoryError</code> , <code>NoClassDefFoundError</code>	Bypasses the error handler entirely (see the gotcha below)

Typical failures and the right reaction.

6.1.2 How DefaultErrorHandler reacts

Suppose `poll()` returns five records from partition `orders-0`, offsets 40 to 44, and the listener throws on offset 42. This is what the default handler does, in order:

```
offsets returned by poll():  40  41  42  43  44
listener result:           ok  ok  FAIL  -  -
```

1. commit the offsets of the records before the failure → 40 and 41 are done
2. wait for the next back-off interval (the thread sleeps)
3. seek the partition back to the failed record → next `poll()` returns 42, 43, 44
4. call the listener with 42 again
 - if it succeeds: carry on with 43 and 44
 - if it fails again: repeat from step 2 until the back-off is exhausted
5. back-off exhausted: hand record 42 to the recoverer (default: log it at ERROR; better: publish it to a dead-letter topic)
6. commit the offset of the recovered record and seek past it → 43 and 44 are processed

If the recoverer itself throws (for example the dead-letter topic does not exist and cannot be created), the handler seeks as if retries were not exhausted and, by default, resets the back-off, so the whole retry sequence starts again. The record is not lost, but the partition stays stuck until the recoverer works.

Three consequences follow from this design.

Retries happen in the consumer thread. While the handler sleeps, the thread does not call `poll()`, so every partition assigned to that thread waits. If the sleeping lasts longer than `max.poll.interval.ms` (5 minutes by default), the broker decides the consumer is dead and triggers a rebalance. Keep the sum of all back-off intervals well below that limit. Spring also ships a `ContainerPausingBackOffHandler`, which pauses the container instead of sleeping and is meant for delays longer than `max.poll.interval.ms`; for such long delays the better tool is usually non-blocking retry, covered in the next chapter.

Order is preserved. Record 43 is not touched until 42 is processed or recovered. That is a feature when order matters (all events of one order share a partition) and a cost when one bad record holds up everything behind it.

The default is dangerous. With no bean configured you get `FixedBackOff(0L, 9)`: ten delivery attempts in total with no delay between them, then one ERROR log line, and the record is skipped. A transient outage lasting more than a few milliseconds therefore loses data. Always configure a back-off and a recoverer.

6.1.3 Configuring back-off and recovery

The building blocks come from two places. The back-off classes (`FixedBackOff`, `ExponentialBackOff`) are plain Spring Framework classes in `org.springframework.util.backoff`; the error handler, recoverers and `ExponentialBackOffWithMaxRetries` are Spring Kafka.

Class	What the numbers mean	Example
<code>FixedBackOff(interval, maxAttempts)</code>	Wait <code>interval</code> ms between tries; <code>maxAttempts</code> is the number of retries after the first delivery	<code>new FixedBackOff(1000L, 2L) = 3 deliveries, 1 s apart</code>

Class	What the numbers mean	Example
ExponentialBackOff	Initial interval, multiplier, maximum interval, maximum elapsed time (effectively unlimited unless you set it)	Use only if you also bound the elapsed time
ExponentialBackOffWithMaxRetries(maxRetries)	Same, but bounded by a number of retries instead of elapsed time	(6), 1 s, x2, max 10 s: waits of 1, 2, 4, 8, 10, 10 s

Back-off implementations you will use.

The listing below is the configuration `inventory-service` uses for its `orders` listener. It retries transient failures with exponential back-off, sends records that still fail to `orders-dlt`, and skips retries entirely for validation errors. The validation exception is a small class of our own:

InvalidOrderException.java

JAVA

```
package com.shopstream.inventory;

/** Thrown when an order can never be processed, however often we retry (bad SKU, quantity 0). */
public class InvalidOrderException extends RuntimeException {

    public InvalidOrderException(String message) {
        super(message);
    }
}
```

ErrorHandlingConfig.java

JAVA

```
package com.shopstream.inventory;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.kafka.core.KafkaTemplate;
import org.springframework.kafka.listener.CommonErrorHandler;
import org.springframework.kafka.listener.DeadLetterPublishingRecoverer;
import org.springframework.kafka.listener.DefaultErrorHandler;
import org.springframework.kafka.support.ExponentialBackOffWithMaxRetries;

@Configuration
public class ErrorHandlingConfig {

    @Bean
    public CommonErrorHandler orderErrorHandler(KafkaTemplate<String, Object> template) {
        // Where exhausted records go: <original topic>-dlt, same partition number
        var recoverer = new DeadLetterPublishingRecoverer(template);

        // When to try again: 1 s, 2 s, 4 s, 8 s, 10 s, 10 s (6 retries, 7 deliveries)
        var backOff = new ExponentialBackOffWithMaxRetries(6);
        backOff.setInitialInterval(1_000L);
        backOff.setMultiplier(2.0);
        backOff.setMaxInterval(10_000L);

        var handler = new DefaultErrorHandler(recoverer, backOff);
        // Retrying a validation failure cannot help: go to the DLT on the first failure
        handler.addNotRetryableExceptions(InvalidOrderException.class);
        return handler;
    }
}
```

What to notice. `DeadLetterPublishingRecoverer` needs a `KafkaTemplate`; the one Spring Boot auto-configures is enough as long as its serializers can write the failed record's key and value (the section on deserialization failures below shows a case where they cannot). `addNotRetryableExceptions` adds to the default list of non-retryable types rather than replacing it. The container wraps whatever your method throws in a `ListenerExecutionFailedException`, and the handler's classification looks through the cause chain, so `InvalidOrderException` is recognised even though it is wrapped. Finally, the bean's type is `CommonErrorHandler`; that is all Boot needs to attach it to the container factory.

Two hooks help you see what the handler is doing. `setRetryListeners(...)` registers a `RetryListener`, whose `failedDelivery(record, exception, deliveryAttempt)` method is called after every failed attempt and whose `recovered(...)` method after a successful recovery; that is the natural place to count retries for metrics. And if you set the container property `deliveryAttemptHeader` to `true`, each record handed to a record listener carries a `kafka_deliveryAttempt` header, an integer that starts at 1.

The listener it protects is ordinary:

OrderCreatedListener.java

JAVA

```

package com.shopstream.inventory;

import com.shopstream.events.OrderCreated;
import org.springframework.kafka.annotation.KafkaListener;
import org.springframework.stereotype.Component;

@Component
public class OrderCreatedListener {

    private final InventoryService inventory;

    public OrderCreatedListener(InventoryService inventory) {
        this.inventory = inventory;
    }

    @KafkaListener(topics = "orders", groupId = "inventory-service")
    public void onOrderCreated(OrderCreated event) {
        // may throw InvalidOrderException (permanent) or a transient DataAccessException
        inventory.reserve(event);
    }
}

```

6.1.4 Which exceptions are retried

`DefaultErrorHandler` treats a short list of exception types as **fatal**: retrying cannot help, so the recoverer is called on the first failure.

- `DeserializationException`
- `MessageConversionException` and `ConversionException`
- `MethodArgumentResolutionException`
- `NoSuchMethodException`
- `ClassCastException`

Everything else is retried according to the back-off. You extend the fatal list with `addNotRetryableExceptions(...)`, as above, or replace the whole classification with `setClassifications(...)`. A useful default policy for a service is the reverse of what beginners expect: decide which exceptions are *transient* (timeouts, connection failures, optimistic-lock conflicts), and treat the validation and mapping exceptions of your own domain as not retryable.

Two smaller settings change what "retry" means. `setResetStateOnExceptionChange` (true by default) restarts the retry sequence, including the choice of back-off, if the failing record throws a different exception type from the previous attempt. `setCommitRecovered` matters only with `AckMode.MANUAL_IMMEDIATE`: it makes the handler commit the offset of a recovered record itself.

GOTCHA

The `DefaultErrorHandler` handles only exceptions that extend `RuntimeException`. An `Error` (`OutOfMemoryError`, `NoClassDefFoundError`, `StackOverflowError`, or a custom class that extends `Error` by mistake) bypasses the handler: the consumer thread terminates, the Kafka connection is closed and no retry or recovery happens. The Spring Kafka documentation warns that the application may keep reporting a healthy status even though its consumers are gone. Alert on consumer lag and on a drop in consumption rate, not only on health endpoints.

There are in fact two layers of error handling. A `@KafkaListener(errorHandler = "beanName")` can name a `KafkaListenerErrorHandler` bean that runs right after your method fails; it can translate the exception, return a fallback value (which is sent to the `@SendTo` destination, if any), or rethrow. Whatever it rethrows reaches the container-level `CommonErrorHandler` described above. Use the listener-level handler for per-method translation, and the container-level handler for retry and recovery policy.

6.1.5 Dead-letter topics

A **dead-letter topic** (DLT) is an ordinary topic that collects records the consumer gave up on, together with information about why. It turns "log and lose" into "park and inspect", and it lets the main flow continue.

`DeadLetterPublishingRecoverer` is the standard recoverer that does the parking.

By default it publishes to `<original topic>-dlt` (so `orders-dlt`) and uses the **same partition number** as the failed record. The DLT therefore needs at least as many partitions as the original topic, or the send fails. Declare it explicitly, with the same partition count:

DltTopicConfig.java

JAVA

```
package com.shopstream.inventory;

import org.apache.kafka.clients.admin.NewTopic;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.kafka.config.TopicBuilder;

@Configuration
public class DltTopicConfig {

    @Bean
    public NewTopic ordersDlt() {
        // same number of partitions as "orders"; use replicas(3) on a real cluster
        return TopicBuilder.name("orders-dlt").partitions(3).replicas(1).build();
    }
}
```

If you prefer a different layout, pass a destination resolver as the second constructor argument: a `BiFunction<ConsumerRecord<?,?>, Exception, TopicPartition>`. Returning a `TopicPartition` with a negative partition number lets the producer choose the partition, which removes the "at least as many partitions" requirement. You can also route by exception type, for example validation failures to `orders-invalid` and everything else to `orders-dlt`.

The recoverer copies the key and value of the failed record and adds headers that describe the failure. The constants live in `KafkaHeaders`:

Constant	Content
DLT_ORIGINAL_TOPIC, DLT_ORIGINAL_PARTITION, DLT_ORIGINAL_OFFSET	Where the record came from
DLT_ORIGINAL_TIMESTAMP, DLT_ORIGINAL_TIMESTAMP_TYPE	The timestamp of the original record
DLT_ORIGINAL_CONSUMER_GROUP	The group whose listener failed
DLT_EXCEPTION_FQCN, DLT_EXCEPTION_CAUSE_FQCN	Class names of the exception and of its cause
DLT_EXCEPTION_MESSAGE, DLT_EXCEPTION_STACKTRACE	Message and stack trace
DLT_KEY_EXCEPTION_*	The same set for a key deserialization failure

Headers added by DeadLetterPublishingRecoverer.

Header values are bytes: strings are UTF-8, and the numeric ones (partition, offset) are binary. A small listener that logs dead letters, which is usually the first thing a team builds, reads them from the record:

OrderDeadLetterListener.java

JAVA

```
package com.shopstream.inventory;

import java.nio.charset.StandardCharsets;
import org.apache.kafka.clients.consumer.ConsumerRecord;
import org.apache.kafka.common.header.Header;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.kafka.annotation.KafkaListener;
import org.springframework.kafka.support.KafkaHeaders;
import org.springframework.stereotype.Component;

@Component
public class OrderDeadLetterListener {

    private static final Logger log = LoggerFactory.getLogger(OrderDeadLetterListener.class);

    // Read the DLT as plain strings: a malformed payload must not fail here as well
    @KafkaListener(
        topics = "orders-dlt",
        groupId = "inventory-service-dlt",
        properties = {

```

OrderDeadLetterListener.java (continued)

JAVA

```

        "key.deserializer=org.apache.kafka.common.serialization.StringDeserializer",
        "value.deserializer=org.apache.kafka.common.serialization.StringDeserializer"
    })
    public void onDeadLetter(ConsumerRecord<String, String> record) {
        log.error("Dead letter: key={} cause={} message={} value={}",
            record.key(),
            header(record, KafkaHeaders.DLT_EXCEPTION_FQCN),
            header(record, KafkaHeaders.DLT_EXCEPTION_MESSAGE),
            record.value());
    }

    private static String header(ConsumerRecord<?, ?> record, String name) {
        Header h = record.headers().lastHeader(name);
        return h == null ? "n/a" : new String(h.value(), StandardCharsets.UTF_8);
    }
}

```

TIP

Put an alert on the *rate of records arriving in the DLT*, not on individual error log lines. A DLT that receives traffic is a bug report with a payload attached. Keep its retention long enough for people to react (days, not hours), and decide up front who owns the replay: a small tool or a one-off consumer that reads `orders-dlt` and republishes to `orders` once the bug is fixed.

6.1.6 Deserialization failures and ErrorHandlerDeserializer

Everything above happens after the record has been turned into a Java object. Deserialization happens earlier, inside `poll()`. If the deserializer throws there, the container never receives a record it could hand to the error handler. The consumer's position does not advance, the next `poll()` fetches the same bytes, and the same exception is thrown again: a poison pill that stalls the partition for good.

`ErrorHandlerDeserializer` fixes this by wrapping the real deserializer. It catches the exception, returns null for the value, and attaches the failure, together with the raw bytes, to a record header. The container sees the header, turns it into a `DeserializationException` and gives it to the error handler. That exception is on the fatal list, so the recoverer is called immediately and the offset moves on. With Spring Boot you configure it in `application.yml`:

application.yml

YAML

```

spring:
  kafka:
    bootstrap-servers: localhost:9092
    consumer:
      group-id: inventory-service
      auto-offset-reset: earliest
      key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
      value-deserializer: org.springframework.kafka.support.serializer.ErrorHandlerDeserializer
      properties:
        # folded onto two lines only to fit the page
        spring.deserializer.value.delegate.class: >-
          org.springframework.kafka.support.serializer.JsonDeserializer
        spring.json.value.default.type: com.shopstream.events.OrderCreated
        spring.json.trusted.packages: com.shopstream.events

```

The wrapper is configured as the deserializer, and the real one is named in `spring.deserializer.value.delegate.class`. The remaining `spring.json.*` properties belong to the delegate. Keys here are plain strings and need no wrapper; a key deserializer can be wrapped in the same way with `spring.deserializer.key.delegate.class`.

GOTCHA

When the failure is a deserialization error, the recoverer restores the original *bytes* that could not be deserialized as the value of the dead-letter record, so the exact payload is preserved for inspection and replay. That means the template used for dead-lettering must be able to serialize `byte[]` as well as your normal event objects. A template whose value serializer is only the JSON serializer cannot publish those bytes unchanged. The Spring Kafka documentation recommends a template whose value type is `Object` and whose serializer chooses by type.

The listing below defines such a template for a consumer-only service. Spring Boot creates its own `KafkaTemplate` only when you have not defined one, so this bean replaces the auto-configured one and is used by the recoverer and, as the next chapter shows, by the retry topics.

DeadLetterTemplateConfig.java

JAVA

```
package com.shopstream.inventory;

import com.shopstream.events.OrderCreated;
import java.util.Map;
import org.apache.kafka.clients.producer.ProducerConfig;
import org.apache.kafka.common.serialization.ByteArraySerializer;
import org.apache.kafka.common.serialization.StringSerializer;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.kafka.core.DefaultKafkaProducerFactory;
import org.springframework.kafka.core.KafkaTemplate;
import org.springframework.kafka.support.serializer.DelegatingByTypeSerializer;
import org.springframework.kafka.support.serializer.JsonSerializer;

@Configuration
public class DeadLetterTemplateConfig {

    @Bean
    public KafkaTemplate<String, Object> kafkaTemplate(
        @Value("${spring.kafka.bootstrap-servers}") String bootstrapServers) {
        // raw bytes (from a failed deserialization) are written as they are,
        // OrderCreated objects are written as JSON
        var valueSerializer = new DelegatingByTypeSerializer(Map.of(
            byte[].class, new ByteArraySerializer(),
            OrderCreated.class, new JsonSerializer<Object>());
        var producerFactory = new DefaultKafkaProducerFactory<String, Object>(
            Map.of(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, bootstrapServers),
            new StringSerializer(), valueSerializer);
        return new KafkaTemplate<>(producerFactory);
    }
}
```

The alternative is to give `DeadLetterPublishingRecoverer` several templates in a `LinkedHashMap`, keyed by the type of value each one can write (for example `byte[].class` for a byte-array template). The recoverer examines the keys in order and uses the first template that fits the value, which is why a `LinkedHashMap` is recommended.

Two limits to remember. The wrapper only helps with exceptions thrown by the deserializer; a record that deserializes fine but fails validation is handled by the normal path. And batch listeners do not get automatic routing of deserialization failures, so with a batch listener you must inspect the records yourself (next section).

6.1.7 Batch listeners and BatchListenerFailedException

A batch listener receives a `List` of records from one poll. If it throws an ordinary exception, the error handler cannot know which record was the culprit, so it retries the complete batch from memory with the same back-off (pausing the consumer and polling between attempts, so the group does not rebalance). When the retries run out, the recoverer is called for every record in the batch, so one bad record can send all of its neighbours to the DLT. Records that succeeded the first time are redelivered on every attempt, which means duplicates unless the listener is idempotent. To avoid all that, tell the container which record failed by throwing `BatchListenerFailedException`:

OrderBatchListener.java

JAVA

```
package com.shopstream.search;

import com.shopstream.events.OrderCreated;
import java.util.List;
import org.apache.kafka.clients.consumer.ConsumerRecord;
import org.springframework.kafka.annotation.KafkaListener;
import org.springframework.kafka.listener.BatchListenerFailedException;
import org.springframework.stereotype.Component;

@Component
public class OrderBatchListener {

    private final SearchIndex index;

    public OrderBatchListener(SearchIndex index) {
```

OrderBatchListener.java (continued)

JAVA

```

        this.index = index;
    }

    @KafkaListener(topics = "orders", groupId = "search-indexer", batch = "true")
    public void onOrders(List<ConsumerRecord<String, OrderCreated>> records) {
        for (ConsumerRecord<String, OrderCreated> record : records) {
            if (record.value() == null) {
                // ErrorHandlingDeserializer hands over a failed record with a null value
                throw new BatchListenerFailedException("Deserialization failed", record);
            }
            try {
                index.upsert(record.value());
            }
            catch (Exception e) {
                // tells the container which record failed
                throw new BatchListenerFailedException("Indexing failed", e, record);
            }
        }
    }
}

```

With the exception in hand, the `DefaultErrorHandler` commits the offsets of all records before the failed one, retries the failed record and everything after it according to the back-off and, when retries run out, publishes only the failed record to the DLT. Its offset is committed and the remaining records are redelivered. If your listener receives plain objects (`List<OrderCreated>`) and has no `ConsumerRecord`, use the constructor that takes the index of the failed element instead of the record. In the batch case, `AckMode.BATCH` is the common choice; with `MANUAL_IMMEDIATE` you need `setCommitRecovered(true)` for the recovered offset to be committed.

The null check at the top of the loop is the manual handling of deserialization failures promised in the previous section. With `ErrorHandlingDeserializer`, a record that could not be deserialized still appears in the batch, but with a null value, and nobody routes it for you. Throwing `BatchListenerFailedException` for it sends it to the DLT with the original bytes restored. If null is a legitimate value on your topic (a tombstone on a compacted topic), look at the exception header instead of the value alone.

NOTE

The `batch = "true"` attribute makes this single listener a batch listener. You can also switch every listener at once with `spring.kafka.listener.type=batch`. Batch listeners cannot use non-blocking retries (next chapter).

6.1.8 Blocking or non-blocking retries?

Everything in this chapter so far is **blocking retry**: the partition waits while the record is retried. The alternative, **non-blocking retry**, publishes the failed record to a retry topic with a delay and moves on; the next chapter covers it in detail. Which one fits depends on how long the failure lasts and on whether order matters.

Question	Blocking (DefaultErrorHandler back-off)	Non-blocking (retry topics)
Order within a partition	Preserved	Lost: a failed record is processed later than its successors
Effect on other records	Everything behind the failing record waits	Main flow continues at full speed
Practical delay	Seconds; must stay below <code>max.poll.interval.ms</code>	Seconds to hours, one topic per delay
Extra infrastructure	None, or a DLT	Retry topics plus a DLT, and more consumers
Batch listeners	Supported (with <code>BatchListenerFailedException</code>)	Not supported
Container transactions	Supported	Cannot be combined
Good fit	Brief, transient failures where order matters (a locked row, a failover)	Slow-recovering dependencies (a payment gateway outage) and records that can be processed out of order

Blocking versus non-blocking retry.

The two combine well. A common design is two or three quick blocking retries (a second or two each) to absorb blips, then non-blocking retry topics with growing delays for longer outages, then the DLT. Spring supports this combination directly; the next chapter shows how.

INTERVIEW ANGLE

What happens in Spring Kafka when a `@KafkaListener` method throws, and how would you make that safe for production? By default the `DefaultErrorHandler` commits the offsets before the failed record, seeks back so the record is redelivered and retries up to ten deliveries in total with no delay; then it logs the record at ERROR and skips it, which silently loses data. For production I configure a back-off (for example exponential with a retry limit), a `DeadLetterPublishingRecoverer` so exhausted records land in `<topic>-dlt` with failure headers, a list of non-retryable exceptions for validation errors, and `ErrorHandlingDeserializer` so a malformed record cannot become a poison pill. I alert on DLT traffic and consumer lag.

INTERVIEW ANGLE

Why can retrying inside the listener container cause a rebalance? Retries run on the consumer thread, which sleeps during the back-off and does not call `poll()`. If the time between two polls exceeds `max.poll.interval.ms`, the consumer is removed from the group and its partitions are reassigned, so the same records can be processed twice. Keep back-offs short, use `ContainerPausingBackOffHandler`, or move long delays to retry topics.

KEY POINTS

- The container catches listener exceptions and asks a `CommonErrorHandler`; the default `DefaultErrorHandler` tries a record ten times with no delay and then skips it with an ERROR log. Replace that default in every real service.
- Blocking retries preserve order but stall the partition and must fit inside `max.poll.interval.ms`; the offsets before the failed record are committed, and the failed record is re-fetched by a seek.
- `DeadLetterPublishingRecoverer` publishes to `<topic>-dlt` on the same partition number, with headers that describe the failure; create the DLT with enough partitions and monitor its traffic.
- Mark validation and conversion errors as not retryable, and wrap deserializers in `ErrorHandlingDeserializer` so one bad record cannot stall a partition.
- Only `RuntimeExceptions` reach the handler: an `Error` kills the consumer thread. Batch listeners should throw `BatchListenerFailedException` to name the failed record.

6.2 Non-Blocking Retries with `@RetryableTopic`

Blocking retries are simple and keep order, but they hold a partition hostage. Imagine that the payment gateway behind `payment-service` goes down for ten minutes. With blocking retry, every record behind the failing one waits, consumer lag climbs, and a back-off longer than `max.poll.interval.ms` gets the consumer thrown out of its group. With the default of ten immediate attempts you would instead skip the orders and lose them.

Non-blocking retry gets the failed record out of the way. Instead of sleeping in the consumer thread, the framework publishes the record to a **retry topic**, commits the offset on the main topic and carries on with the next record. A second consumer reads the retry topic and processes the record again once its delay has elapsed. If that fails too, the record moves to the next retry topic, and after the last attempt to a dead-letter topic. In Spring Kafka the whole arrangement is switched on by one annotation, `@RetryableTopic`.

6.2.1 How the pattern works

Take a record on `orders` that fails because a downstream service is briefly unavailable. This is what happens:

1. The listener method throws. The framework publishes the record, with a header that carries the time at which it is due, to the first retry topic, and the offset on `orders` moves on. The main consumer is free again.
2. A listener container on the retry topic reads the record. If it is not yet due, the container pauses that partition (the consumer keeps polling, so it stays in the group) and resumes it when the due time arrives.
3. Your *same listener method* is called with the record again. If it succeeds, the story ends. If it fails, the record is published to the next retry topic, with a longer delay.
4. After the last attempt the record is published to the dead-letter topic, where a method annotated with `@DltHandler` (or a default handler that only logs) receives it.

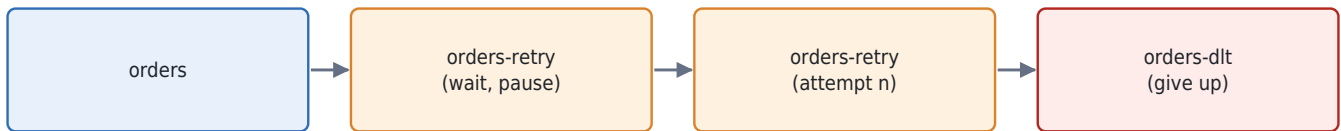


Figure 6.1. The main topic, the retry topics and the dead-letter topic. A failed record moves one step along the chain, each step with a longer delay, and the main topic never waits for it.

The framework creates the retry topics and the extra listener containers for you. Everything underneath is ordinary Kafka, which makes the behaviour predictable, and it also explains the price you pay. The record is no longer processed in its original position, so **ordering is lost**: successors on the same partition are handled while the failed record waits in a retry topic. The feature is not supported for batch listeners and cannot be combined with container transactions (if the listener throws, the container transaction commits and the record is forwarded to the retry topic). And you now operate several extra topics per listener.

6.2.2 A first example

The listing is the `orders` listener of `inventory-service` from the previous chapter, now with non-blocking retry. The only new code is the annotation and the `@DltHandler` method.

OrderRetryListener.java

JAVA

```

package com.shopstream.inventory;

import com.shopstream.events.OrderCreated;
import org.apache.kafka.clients.consumer.ConsumerRecord;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.kafka.annotation.BackOff;
import org.springframework.kafka.annotation.DltHandler;
import org.springframework.kafka.annotation.KafkaListener;
import org.springframework.kafka.annotation.RetryableTopic;
import org.springframework.stereotype.Component;

@Component
public class OrderRetryListener {

    private static final Logger log = LoggerFactory.getLogger(OrderRetryListener.class);

    private final InventoryService inventory;

    public OrderRetryListener(InventoryService inventory) {
        this.inventory = inventory;
    }

    @RetryableTopic(
        kafkaTemplate = "kafkaTemplate",
        attempts = "4",
        backOff = @BackOff(delay = 1000, multiplier = 2.0, maxDelay = 16000),
        exclude = InvalidOrderException.class,
        numPartitions = "3")
    @KafkaListener(topics = "orders", groupId = "inventory-service")
    public void onOrderCreated(OrderCreated event) {
        inventory.reserve(event);
    }

    @DltHandler
    public void onDeadLetter(ConsumerRecord<String, OrderCreated> record) {
        log.error("Order {} failed all retries (dead letter {}-{}@{})",
            record.key(), record.topic(), record.partition(), record.offset());
    }
}

```

What to notice. `attempts` is a *string* attribute and counts the first delivery, so "4" means one attempt on `orders` plus three on retry topics. `@BackOff` is Spring Kafka's own annotation from `org.springframework.kafka.annotation`; older tutorials show `@Backoff` from Spring Retry, a dependency that Spring Kafka 4 no longer uses. `exclude` names exceptions that must not be retried: a record that fails with `InvalidOrderException` goes straight to the dead-letter topic. `numPartitions = "3"` gives the retry topics and the dead-letter topic as many partitions as `orders` (see the sizing warning below). `kafkaTemplate` names the bean that publishes to the retry topics.

With exponential back-off of 1 s, factor 2 and four attempts, Spring Kafka creates three retry topics and a dead-letter topic. On a local broker the topic list looks like this (internal topics included):

```

Topics created by the framework at startup TERMINAL

$ bin/kafka-topics.sh --list --bootstrap-server localhost:9092
__consumer_offsets
orders
orders-dlt
orders-retry-1000
orders-retry-2000
orders-retry-4000

```

The life of one record that keeps failing looks like the table below. The delays are lower bounds: Spring guarantees that a record is never processed before it is due, not that it is processed exactly on time.

Time	Topic	Attempt	Outcome
0 s	orders	1	Fails, published to orders-retry-1000
about 1 s	orders-retry-1000	2	Fails, published to orders-retry-2000
about 3 s	orders-retry-2000	3	Fails, published to orders-retry-4000
about 7 s	orders-retry-4000	4	Fails, published to orders-dlt
immediately	orders-dlt	-	The @DltHandler method runs

Timeline of one failing record (approximate).

6.2.3 The attributes that matter

@RetryableTopic has about twenty elements. These are the ones you will use, with their defaults:

Attribute	Default	Meaning
attempts	"3"	Total attempts before the record goes to the DLT, including the first one. A String
backOff	@BackOff (1000 ms fixed)	Delay between attempts: delay , multiplier , maxDelay , jitter
kafkaTemplate	" "	Bean name of the template that publishes to retry topics and DLT
numPartitions	"1"	Partitions of the retry topics and the DLT
replicationFactor	"-1"	Replication factor; -1 means the broker default
autoCreateTopics	"true"	Let the framework create the topics through KafkaAdmin
include, exclude	none	Exceptions to retry, or not to retry (straight to the DLT)
traversingCauses	" "	Look through the cause chain; effectively true once include or exclude is set
timeout	" "	Give up after this many milliseconds in total, whatever attempts says
dltStrategy	ALWAYS_RETRY_ON_ERROR	What happens if DLT processing fails (see below)
retryTopicSuffix, dltTopicSuffix	"-retry", "-dlt"	Name suffixes
topicSuffixingStrategy	SUFFIX_WITH_DELAY_VALUE	Suffix retry topics with the delay, or with an index
sameIntervalTopicReuseStrategy	SINGLE_TOPIC	One topic for attempts with the same delay, or one topic per attempt
concurrency	" "	Concurrency of retry and DLT containers; defaults to the main container's
autoStartDltHandler	" "	Set to "false" to keep the DLT container stopped until you start it
exceptionBasedDltRouting	none	Send particular exceptions to a custom DLT

The most useful @RetryableTopic attributes.

6.2.4 Topic naming and sizing

Retry topics are named after the main topic, a suffix, and the delay or an index. Since Spring Kafka 4.1, attempts that share the same delay reuse one topic by default.

Back-off configuration	Topics created
Fixed delay, default (<code>SINGLE_TOPIC</code>)	<code>orders-retry</code> , <code>orders-dlt</code>
Fixed delay with <code>MULTIPLE_TOPICS</code>	<code>orders-retry-0</code> , <code>orders-retry-1</code> , and so on, then <code>orders-dlt</code>
Exponential 1000 ms, factor 2, 4 attempts	<code>orders-retry-1000</code> , <code>orders-retry-2000</code> , <code>orders-retry-4000</code> , <code>orders-dlt</code>
Exponential, 1000 ms, factor 2, maxDelay 16000	<code>orders-retry-1000</code> , <code>-2000</code> , <code>-4000</code> , <code>-8000</code> , <code>-16000</code> ; the last one is reused for all further attempts
<code>SUFFIX_WITH_INDEX_VALUE</code>	<code>orders-retry-0</code> , <code>orders-retry-1</code> , and so on
<code>retryTopicSuffix = "-wait"</code>	<code>orders-wait-1000</code> , and so on

Topic names for the main topic `orders`.

The reuse rule keeps the topic count small. According to the reference documentation, an exponential policy from 1 s to a 16 s maximum that keeps trying for a whole hour needs only five retry topics, because attempts at the maximum delay all use the last one.

WATCH OUT

By default the retry topics and the DLT are created with **one partition** and the broker's default replication factor. For a three-partition `orders` topic that squeezes every retried record through a single partition and a single consumer thread. Set `numPartitions` to the partition count of the main topic, as the example does, and make sure `replicationFactor` (or the broker default) gives you real redundancy in production.

The framework creates the topics by registering `NewTopic` beans that `KafkaAdmin` applies at startup. That is convenient locally and often forbidden in production, where applications may not have permission to create topics. Set `autoCreateTopics = "false"` and provision the topics with your infrastructure tooling, using the names from the table above. Remember to grant the service permission to read and write all of them.

6.2.5 Which failures are retried, and what the DLT does

By default every exception is retried. `include` and `exclude` narrow that down; with either present, the cause chain is searched, because the listener container wraps your exception in a `ListenerExecutionFailedException`. In addition, the same list of fatal exceptions as in `DefaultErrorHandler` (deserialization, conversion and argument-resolution errors, `ClassCastException`, `NoSuchMethodException`) bypasses the retry topics and goes straight to the DLT. You can change that list globally, as shown in the configuration section below.

The method annotated with `@DltHandler` processes dead letters. It must live in the same class as the `@RetryableTopic` method, and one handler serves all retryable methods of that class. If you write none, a default handler logs the record. What if the DLT handler itself fails? That is decided by `dltStrategy`:

dltStrategy	Behaviour
<code>ALWAYS_RETRY_ON_ERROR</code>	Default. A record whose DLT processing fails is published back to the DLT so that it does not block other dead letters. Not done for fatal exceptions such as <code>DeserializationException</code> , which would only fail again
<code>FAIL_ON_ERROR</code>	The DLT consumer gives up without forwarding the record
<code>NO_DLT</code>	No DLT is created; after the last attempt the record is dropped

DLT strategies.

Two further options are worth knowing. `autoStartDltHandler = "false"` keeps the DLT container stopped, for example until an operator is ready to look at dead letters; you start it later through the `KafkaListenerEndpointRegistry`. And `exceptionBasedDltRouting` sends chosen exception types to a custom DLT whose suffix goes before the normal one, so deserialization failures can land in `orders-deserialization-dlt` instead of `orders-dlt`.

By default only the first set of "original" headers and the last set of exception headers are kept on a record as it travels through the retry topics. That prevents the record from growing with every attempt (the stack trace header alone can be large).

6.2.6 Configuring the template and global behaviour

The retry machinery republishes records, so it needs a `KafkaTemplate`. If you do not name one in `kafkaTemplate`, Spring looks for a bean called `defaultRetryTopicKafkaTemplate` and throws an exception if there is none. Spring Boot's auto-configured template is called `kafkaTemplate`, which is why the example names it explicitly. If your consumer uses `ErrorHandlingDeserializer`, the template must also be able to write raw `byte[]` values, because deserialization failures arrive as bytes; the `KafkaTemplate<String, Object>` with a `DelegatingByTypeSerializer` from the previous chapter is exactly that.

For settings that apply to every retryable listener in the application, extend `RetryTopicConfigurationSupport` in one `@Configuration` class and override the hooks you need. The listing below does two things. It adds **blocking retries in front of the retry topics**: a failure with a transient database exception is first retried in place, twice, 200 ms apart, which absorbs a lock timeout or failover blip without any topic hop; only if it still fails does the record travel to the first retry topic. And it declares `InvalidOrderException` fatal, so that it skips all retries.

```
RetryConfig.java JAVA

package com.shopstream.inventory;

import java.util.List;
import org.springframework.context.annotation.Configuration;
import org.springframework.dao.TransientDataAccessException;
import org.springframework.kafka.annotation.EnableKafka;
import org.springframework.kafka.retrytopic.RetryTopicConfigurationSupport;
import org.springframework.util.backoff.FixedBackOff;

@EnableKafka
@Configuration
public class RetryConfig extends RetryTopicConfigurationSupport {

    @Override
    protected void configureBlockingRetries(BlockingRetriesConfigurer blockingRetries) {
        blockingRetries
            .retryOn(TransientDataAccessException.class)
            .backOff(new FixedBackOff(200L, 2L));
    }

    @Override
    protected void manageNonBlockingFatalExceptions(
        List<Class<? extends Throwable>> nonBlockingFatalExceptions) {
        nonBlockingFatalExceptions.add(InvalidOrderException.class);
    }
}
```

The two lists behave differently, and mixing them up is a common mistake. Blocking retries are an **allow-list**: only the exceptions you name are retried in place. The non-blocking fatal list is a **deny-list**: exceptions you name skip the retry topics, everything else is retried. Combining them gives every behaviour you might want: retried only in place, retried in place and then via topics, or never retried.

Use only one global mechanism: either extend `RetryTopicConfigurationSupport` (once in the application) or annotate a configuration class with `@EnableKafkaRetryTopic`, which imports the infrastructure and exposes its components as beans. For simple cases you need neither, because `@RetryableTopic` alone bootstraps everything with defaults.

6.2.7 Configuring retries without annotations

Instead of annotating every method, you can declare a `RetryTopicConfiguration` bean that applies to all listeners of the topics it includes. This keeps the policy in one place and works for listeners you cannot annotate.

```
OrdersRetryConfiguration.java JAVA

package com.shopstream.inventory;

import java.util.List;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.kafka.core.KafkaTemplate;
import org.springframework.kafka.retrytopic.RetryTopicConfiguration;
import org.springframework.kafka.retrytopic.RetryTopicConfigurationBuilder;

@Configuration
public class OrdersRetryConfiguration {
```

OrdersRetryConfiguration.java (continued)

JAVA

```

@Bean
public RetryTopicConfiguration ordersRetry(KafkaTemplate<String, Object> template) {
    return RetryTopicConfigurationBuilder
        .newInstance()
        .exponentialBackoff(1_000, 2, 16_000)
        .maxAttempts(4)
        .autoCreateTopicsWith(3, (short) 1)
        .includeTopics(List.of("orders"))
        .notRetryOn(InvalidOrderException.class)
        .create(template);
}

```

The builder mirrors the annotation: `fixedBackOff(...)`, `retryTopicSuffix(...)`, `dltHandlerMethod("beanName", "methodName")`, `doNotRetryOnDltFailure()`, `doNotConfigureDlt()`, `timeoutAfter(...)` and `autoStartDltHandler(false)` are the builder spellings of the attributes above. A configuration that includes no topics applies to every listener, and the `autoCreateTopicsWith(3, (short) 1)` call sets partitions and replication factor (here one replica, for a local broker).

6.2.8 Consequences you must design for

Ordering is gone. Suppose `OrderCreated` for `o-1001` fails and is parked in `orders-retry-1000`. The next event with the same key, `OrderCancelled` for `o-1001`, is processed immediately from `orders`. One second later the retry delivers `OrderCreated`, and a naive handler reserves stock for an order that is already cancelled. Handlers behind retry topics must therefore check state before they act (is the order still open? is this event older than the stored version?) or be written as idempotent upserts. When per-key order is essential, stay with blocking retries.

Delivery stays at-least-once. The framework publishes to the next topic and then commits, so a crash in between can produce a duplicate. The idempotency techniques of the next chapter apply to retry topics as much as to the main topic.

Delays are best effort. All waiting is done by the consumer thread. A retry consumer that handles several partitions, or a slow listener, delays later records. For very short delays (around a second or less), offset commits and other housekeeping can add noticeable jitter. Use `ackMode RECORD` (`spring.kafka.listener.ack-mode=record`), which the documentation suggests for retry topics.

Topics multiply. Every listener with its own policy adds retry topics and containers. Keep policies uniform, prefer the global `RetryTopicConfiguration` style, and put the DLT of each service on a dashboard.

INTERVIEW ANGLE

How do non-blocking retries work in Spring Kafka, and what do you give up compared with blocking retries?

`@RetryableTopic` makes the framework publish a failed record to a retry topic with a due time and commit the offset on the main topic. A container on the retry topic pauses the partition until the record is due and then calls the same listener again; after the last attempt the record goes to a dead-letter topic. The main flow never waits, and delays can be long without causing rebalances. The price is lost ordering, extra topics and consumers, no support for batch listeners or container transactions, and best-effort delay precision.

KEY POINTS

- `@RetryableTopic` turns a failure into a hop to a retry topic with a delay; the main partition keeps flowing, and after the last attempt the record lands in `<topic>-dlt`.
- `attempts` is a String that includes the first delivery; set `numPartitions` to match the main topic and decide whether the framework may create topics at all (`autoCreateTopics`).
- Retry topics are named by delay (`orders-retry-1000`) or index; since Spring Kafka 4.1 attempts with the same delay share a topic.
- Use `@DltHandler`, `dltStrategy` and `exceptionBasedDltRouting` to decide what happens to dead letters; combine quick blocking retries with retry topics through `RetryTopicConfigurationSupport`.
- Ordering is lost and delivery is at-least-once: write handlers that check state and tolerate duplicates.

6.3 Idempotent Consumers and the Inbox Pattern

In practice a Kafka consumer gets every record **at least once**. Offsets are committed after the work is done, so if the process dies in between, the next owner of the partition reads the record again. That is the correct trade-off, since the alternative is losing data, but it means duplicates are a normal event rather than an exception. They come from several places:

- A consumer processes a record and crashes, or is removed by a rebalance, before its offset is committed.
- Blocking retries redeliver a record, and retry topics forward it at least once.
- An upstream service publishes the same business event twice, for example an outbox relay that resends after a timeout. The idempotent producer protects against duplicates caused by its *own* retries within one session, not against your application producing the same event again.
- People replay a dead-letter topic or reset offsets to repair a bug.

Correctness therefore has to come from the consumer. Processing the same event twice must leave the system in the same state as processing it once.

DEFINITION

An operation is **idempotent** if applying it several times has the same effect as applying it once. "Set the order status to CONFIRMED" is idempotent. "Add 2 to the reserved stock" is not.

6.3.1 Three ways to get there

Technique	Example	Weakness
Naturally idempotent write	<code>UPDATE orders SET status = 'CONFIRMED' WHERE id = ?</code> , or an upsert by key	Does not help with "add 1", "send an email", "charge a card"
State or version check	Apply <code>PaymentAuthorized</code> only if the order is still <code>PENDING</code> ; ignore an event whose version is not newer than the stored one	Needs a state machine or a version in every event and table
Event-id deduplication	A processed-event table with a unique key, written in the same transaction as the effect	Extra table that needs cleanup; covers only effects inside that one database

Techniques for idempotent processing, cheapest first.

Prefer the first two when they fit, since they need no extra storage. The third is the general-purpose tool, and the rest of this chapter builds it.

6.3.2 Deduplicating with a processed-event table

The idea is to record "I have handled event X" in the same database transaction that applies the effect. Either both happen or neither does, so after a crash the table tells you the truth. The table needs one thing above all: a unique key.

V2_processed_event.sql

SQL

```
CREATE TABLE processed_event (
  consumer    VARCHAR(100) NOT NULL,
  event_id    VARCHAR(64)  NOT NULL,
  processed_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  PRIMARY KEY (consumer, event_id)
);

CREATE INDEX idx_processed_event_processed_at ON processed_event (processed_at);
```

The `consumer` column names the logical handler (say `inventory-reserve`), so several handlers in one database can process the same event without colliding. The index serves the cleanup job later. The service below uses Spring's `JdbcTemplate` and PostgreSQL's `INSERT ... ON CONFLICT DO NOTHING`:

InventoryService.java

JAVA

```
package com.shopstream.inventory;

import com.shopstream.events.OrderCreated;
import com.shopstream.events.OrderLine;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.jdbc.core.simple.JdbcTemplate;
```

InventoryService.java (continued)

JAVA

```

import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class InventoryService {

    private static final Logger log = LoggerFactory.getLogger(InventoryService.class);
    private static final String CONSUMER = "inventory-reserve";

    private final JdbcClient jdbc;

    public InventoryService(JdbcClient jdbc) {
        this.jdbc = jdbc;
    }

    @Transactional
    public void reserve(OrderCreated event) {
        int inserted = jdbc.sql("""
            INSERT INTO processed_event (consumer, event_id)
            VALUES (:consumer, :eventId)
            ON CONFLICT (consumer, event_id) DO NOTHING
            """)
            .param("consumer", CONSUMER)
            .param("eventId", event.eventId())
            .update();

        if (inserted == 0) {
            log.info("Event {} already processed, skipping", event.eventId());
            return;
        }

        for (OrderLine line : event.lines()) {
            int updated = jdbc.sql("""
                UPDATE stock SET reserved = reserved + :qty
                WHERE sku = :sku AND on_hand - reserved >= :qty
                """)
                .param("qty", line.qty())
                .param("sku", line.sku())
                .update();

            if (updated == 0) {
                // rolls back the transaction, including the processed_event row
                throw new InvalidOrderException("Unknown SKU or not enough stock: " + line.sku());
            }
        }
    }
}

```

What to notice. The dedupe row is inserted *first*, and the number of affected rows is the answer: 1 means "first time, go ahead", 0 means "seen before". A "select, then insert if absent" sequence has a race window in which two consumer instances both see nothing; the primary key closes it, because the second insert waits for the first transaction to finish and then finds the row. The insert and the stock update share one `@Transactional` method, so an exception rolls back both and the event can be retried later as if it had never been seen. And the key is `eventId`, the UUID in the event envelope, not the Kafka offset: an offset identifies a position in one topic, whereas a copy of the same event that travels through a retry topic, a mirror or a replay gets a different offset.

```

poll → record r (eventId e1)
BEGIN
  INSERT processed_event e1      → 1 row    first delivery
  UPDATE stock ...
COMMIT
— crash before the offset is committed —
poll → record r again (eventId e1)
BEGIN
  INSERT processed_event e1      → 0 rows    conflict: skip the business logic
COMMIT
offset committed, consumer moves on

```

What should the code do on a duplicate? Return normally. The offset then commits and the consumer moves on. Do not throw (a retry would see the same duplicate) and do not dead-letter it (nothing is wrong). Log at info level, and count duplicates in a metric: a sudden rise means rebalances or an upstream replay and is worth a look.

WATCH OUT

Deduplication protects only effects inside the same database transaction. If handling the event also publishes a new event, say `StockReserved`, a plain `KafkaTemplate#send` after the commit can be lost in a crash, and the redelivery will then skip the duplicate and never publish it. Write the outgoing event to an outbox table in the same transaction and let a relay publish it (see the chapter on the outbox pattern in the design-patterns Part).

6.3.3 Keeping the table small

The table grows with every event, so it needs a cleanup job. The rule for the horizon: keep rows at least as long as a duplicate can still arrive. That is the longest of the topic's retention (7 days by default), the longest retry delay, and the window in which you might replay a DLT or rewind offsets. A horizon of 24 hours is fatal if you can reset offsets to last week. A simple job deletes in one statement; for high volumes, delete in batches or partition the table by date and drop old partitions.

ProcessedEventCleanup.java

JAVA

```
package com.shopstream.inventory;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.jdbc.core.simple.JdbcClient;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Component;
import org.springframework.transaction.annotation.Transactional;

@Component
public class ProcessedEventCleanup {

    private static final Logger log = LoggerFactory.getLogger(ProcessedEventCleanup.class);

    private final JdbcClient jdbc;

    public ProcessedEventCleanup(JdbcClient jdbc) {
        this.jdbc = jdbc;
    }

    // needs @EnableScheduling on a configuration class
    @Scheduled(cron = "0 15 3 * * *")
    @Transactional
    public void purge() {
        int deleted = jdbc
            .sql("DELETE FROM processed_event WHERE processed_at < now() - interval '14 days'")
            .update();
        log.info("Purged {} processed_event rows older than 14 days", deleted);
    }
}
```

6.3.4 The full inbox pattern

The table above is the lightweight form, sometimes called an *idempotent receiver*. The full **inbox pattern** goes one step further: the listener only stores the incoming message (payload and event id) in an **inbox** table, protected by the same unique key, commits the Kafka offset at once, and a separate worker processes the rows. This decouples consumption speed from processing speed, gives you an audit trail and lets you retry or inspect messages with SQL. The price is another moving part and delayed processing. Start with the lightweight form; move to a full inbox when processing is slow, needs its own retry policy, or must be queryable.

INTERVIEW ANGLE

How do you make a Kafka consumer idempotent, and why is it necessary? Delivery to a consumer is at least once, so duplicates arrive after crashes, rebalances, retries and replays. The consumer must make repeated processing harmless: use naturally idempotent writes or state checks where possible, otherwise store the event id in a processed-event table with a unique key in the same database transaction as the business change, treat a unique-key conflict as "already done", and purge old rows after a horizon that covers topic retention and replay windows.

KEY POINTS

- Consumers see records at least once; duplicates come from crashes, rebalances, retries, upstream republishing and replays, so idempotence is the consumer's job.
- Prefer naturally idempotent writes and state or version checks; use a processed-event table with a unique key when they are not enough.
- Insert the event id first and write it in the same transaction as the effect; use the event id from the envelope, never the offset.
- On a duplicate, return normally and count it. Deduplication does not cover events you publish: use an outbox for those.
- Purge rows older than a horizon that exceeds retention and replay windows.

6.4 Transactions in Spring Kafka

Sometimes several writes belong together. A service publishes two events that must both appear or neither. A processor reads from `orders` and writes to `payments`, and a crash must neither lose nor duplicate a result. A service updates its database and announces the change. Kafka transactions answer the first two cases, inside Kafka. They do not answer the third, and understanding where exactly the boundary lies is the point of this chapter. The mechanics (producer id, `transactional.id`, epochs, commit markers, the last stable offset) are covered in the earlier Parts on the producer and on internals; here we look at how Spring Kafka wires them up.

6.4.1 Producer-only transactions

Transactions are switched on by giving the producer factory a `transactionIdPrefix`. With Spring Boot that is one property:

```
application.yml
spring:
  kafka:
    bootstrap-servers: localhost:9092
    producer:
      # transactional.id = prefix + n; must be unique per application instance
      transaction-id-prefix: order-service-${HOSTNAME:local}-
```

Boot then creates the producer factory with a cache of transactional producers (`transactional.id` is the prefix plus a counter) and a `KafkaTransactionManager`, and it wires that manager into the listener container factory. Two consequences are worth knowing before you write any code:

- **Every send needs a transaction.** A transactional `KafkaTemplate` that is used outside a transaction throws an `IllegalStateException`. A transaction is started by `executeInTransaction`, by a `@Transactional` method, by a `TransactionTemplate`, or by a listener container. If you really want some non-transactional sends, set `spring.kafka.template.allow-non-transactional=true`.
- **The prefix must be unique per instance.** Two running instances with the same prefix would use the same `transactional.id` and fence each other. Include the pod name or another per-instance value.

The simplest way to use the transaction is the template's own callback. If it returns normally the transaction commits, if it throws the transaction aborts. (`NotificationRequested` is the event record of the `notifications` topic.)

```
OrderEventPublisher.java
package com.shopstream.order;

import com.shopstream.events.NotificationRequested;
import com.shopstream.events.OrderCreated;
import org.springframework.kafka.core.KafkaTemplate;
import org.springframework.stereotype.Service;

@Service
public class OrderEventPublisher {

    private final KafkaTemplate<String, Object> template;

    public OrderEventPublisher(KafkaTemplate<String, Object> template) {
        this.template = template;
    }
}
```

OrderEventPublisher.java (continued)

JAVA

```

/** Both records become visible together, or neither does. */
public void publish(OrderCreated created, NotificationRequested notification) {
    template.executeInTransaction(t -> {
        t.send("orders", created.orderId(), created);
        t.send("notifications", created.customerId(), notification);
        return true;
    });
}
}

```

GOTCHA

Atomic publishing only helps consumers that ask for it. The consumer default is `isolation.level=read_uncommitted`, so a consumer using it sees records of a transaction that is later aborted. Downstream consumers of transactional topics must set `isolation.level=read_committed` (`spring.kafka.consumer.isolation-level=read_committed`). They then read only up to the last stable offset, which means that one hung transaction holds them back until it commits, aborts or times out (`transaction.timeout.ms`, 60 seconds by default).

6.4.2 Consume, transform, produce

The classic exactly-once use case is a processor: `payment-service` reads `OrderCreated` from `orders`, authorises the payment and writes `PaymentAuthorized` to `payments`. Without a transaction there are two failure windows: the result is published but the offset is not committed (the order is processed twice), or the offset is committed but the result is lost. A Kafka transaction closes both by committing the output records **and the input offsets** together.

In Spring Kafka the listener container does this for you. When the container factory has a `KafkaTransactionManager` (Boot sets it up from the property above), the container starts a transaction before it calls your listener, any `KafkaTemplate` send inside the listener joins it, and when the listener returns the container adds the consumed offsets to the transaction and commits it. If the listener throws, the transaction aborts, the output records are never visible to `read_committed` readers, and the offsets do not advance.

application.yml

YAML

```

spring:
  kafka:
    bootstrap-servers: localhost:9092
    producer:
      transaction-id-prefix: payment-service-${HOSTNAME:local}-
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.springframework.kafka.support.serializer.JsonSerializer
    consumer:
      group-id: payment-service
      auto-offset-reset: earliest
      enable-auto-commit: false           # offsets are committed through the transaction
      isolation-level: read_committed      # do not read aborted records
      key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
      value-deserializer: org.springframework.kafka.support.serializer.ErrorHandlingDeserializer
    properties:
      spring.deserializer.value.delegate.class: >-
        org.springframework.kafka.support.serializer.JsonDeserializer
      spring.json.value.default.type: com.shopstream.events.OrderCreated
      spring.json.trusted.packages: com.shopstream.events

```

PaymentListener.java

JAVA

```

package com.shopstream.payment;

import com.shopstream.events.OrderCreated;
import com.shopstream.events.PaymentAuthorized;
import org.springframework.kafka.annotation.KafkaListener;
import org.springframework.kafka.core.KafkaTemplate;
import org.springframework.stereotype.Component;

@Component
public class PaymentListener {

    private final PaymentGateway gateway;
    private final KafkaTemplate<String, Object> template;

```

PaymentListener.java (continued)

JAVA

```

public PaymentListener(PaymentGateway gateway, KafkaTemplate<String, Object> template) {
    this.gateway = gateway;
    this.template = template;
}

@KafkaListener(topics = "orders", groupId = "payment-service")
public void onOrderCreated(OrderCreated order) {
    PaymentAuthorized result = gateway.authorize(order);
    // joins the transaction that the listener container has started
    template.send("payments", order.orderId(), result);
}
}

```

What to notice. The listener has no transaction code at all; the container owns the transaction, and the same `KafkaTemplate` is simply used inside it. `isolation-level: read_committed` matters for this service's own input too, because `orders` may itself be written transactionally upstream. Note also what the transaction does **not** include: `gateway.authorize(order)` talks to an external payment provider. If the transaction aborts after that call, the charge has still happened. Pass a stable idempotency key (the `eventId` or `orderId`) to the provider so that the retried attempt is recognised.

When the listener fails and the transaction rolls back, the container hands control to an `AfterRollbackProcessor`. The default, `DefaultAfterRollbackProcessor`, seeks the partition back so that the failed record (and the unprocessed rest) is fetched again, and after ten failures it logs the record and skips it, like `DefaultErrorHandler` does outside transactions. You replace it with a bean of type `AfterRollbackProcessor`, which Boot attaches to the container factory. A recoverer and a limited back-off give you the familiar behaviour:

RollbackConfig.java

JAVA

```

package com.shopstream.payment;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.kafka.listener.AfterRollbackProcessor;
import org.springframework.kafka.listener.DefaultAfterRollbackProcessor;
import org.springframework.util.backoff.FixedBackOff;

@Configuration
public class RollbackConfig {

    @Bean
    public AfterRollbackProcessor<Object, Object> afterRollbackProcessor() {
        // retry twice, one second apart, then hand the record to the recoverer
        return new DefaultAfterRollbackProcessor<>(
            (record, exception) -> {
                // for example: publish to a dead-letter topic or store it for inspection
            },
            new FixedBackOff(1000L, 2L));
    }
}

```

Publishing the failed record to a dead-letter topic from a recoverer is more delicate in a transactional container, because the original transaction has already rolled back. The reference documentation describes running the processor in a new transaction that also commits the recovered record's offset, enabled through the processor's `commitRecovered` and `kafkaTemplate` properties. Read that section before you build it.

A few more rules apply to transactional containers. Non-blocking retries cannot be combined with container transactions. With a batch listener, a rollback redelivers the whole batch, because the container cannot know which record failed. And if another instance takes over a partition, the old producer is fenced with a `ProducerFencedException`; the container cannot tell a rebalance from a timeout, so it does not call the `AfterRollbackProcessor`. Set the container property `stopContainerWhenFenced` to `true` if you prefer to stop and restart instead of continuing.

6.4.3 Where exactly-once ends: Kafka and your database

Everything above is exactly-once **inside Kafka**: read, process, write, with idempotent producer, transactions and `read_committed`. A relational database is a different transaction system, and no standard protocol joins the two. What Spring offers is the *best-effort one-phase commit*: nest a database transaction inside the Kafka transaction (a `@Transactional` listener or service method, with the container owning the Kafka transaction) so that the database commits first and Kafka commits last. The failure window sits between the two commits.

```
container starts Kafka tx
listener runs
  DB tx begin → write rows → DB tx commit      ← durable now
  (crash, fencing or timeout here)
container commits Kafka tx (output records + offsets) ← never happens
→ rows are saved, offsets did not move: the record is redelivered and the
  database work runs a second time
```

The result is a duplicate database effect, not a lost one, and the idempotent-consumer techniques from the previous chapter turn it into a harmless duplicate. Spring Kafka's `ChainedKafkaTransactionManager` was an older attempt to chain the two managers; it is deprecated and you should not use it. The reference documentation recommends the container-level `KafkaTransactionManager` plus `@Transactional` for the other resource, as above.

WATCH OUT

Setting `transaction-id-prefix` adds a second `TransactionManager` bean to the context. In an application that also uses JPA or JDBC, a bare `@Transactional` may then resolve to the wrong manager, or the database manager may not be auto-configured at all. Name the manager you mean, for example `@Transactional("transactionManager")` for the database, and check which managers exist in your context at startup.

So which tool fits which job?

Scenario	Tool
Publish several records atomically	Producer transaction (<code>executeInTransaction</code> or <code>@Transactional</code>), consumers on <code>read_committed</code>
Kafka to Kafka (read, process, write)	Container-managed transaction with <code>transaction-id-prefix</code> and <code>read_committed</code>
Database write plus event publication	Transactional outbox: write the event to a table in the same database transaction, a relay (Debezium or a poller) publishes it
Kafka to database	At-least-once delivery plus an idempotent consumer (processed-event table)
Kafka to an external API	Idempotency key derived from the event id

What to use when.

The outbox is usually the better answer for the third row, and it is also the pattern that production systems use most. It avoids stretching a Kafka transaction around slow database work (an open transaction holds back `read_committed` consumers and must finish within `transaction.timeout.ms`), it keeps working while the broker is briefly unavailable because the event waits in the table, and it makes the database the single source of truth. The price is at-least-once publication, which the idempotent consumers on the other side absorb. Project A in the end-to-end Part builds it with Debezium.

INTERVIEW ANGLE

Is Kafka's exactly-once semantics really exactly-once, and what does it not cover? It is exactly-once for the read-process-write cycle that stays inside Kafka: the idempotent producer removes duplicates caused by retries, transactions commit the output records and the consumed offsets atomically, and `read_committed` consumers ignore aborted data. It does not extend to side effects in other systems such as a database, an email or an HTTP call, because those are not part of the Kafka transaction. For them you combine at-least-once delivery with idempotent handling, or use the outbox pattern.

KEY POINTS

- Enable transactions with `spring.kafka.producer.transaction-id-prefix` (unique per instance); Boot then creates a `KafkaTransactionManager` and wires it into the listener container factory.
- Producer-only atomicity: `KafkaTemplate#executeInTransaction` or `@Transactional`; every consumer of the topic must use `isolation.level=read_committed`, or it will read aborted records.
- Consume-transform-produce is handled by the container: sends inside the listener join its transaction, and the consumed offsets are committed with them. After a rollback an `AfterRollbackProcessor` re-fetches the record.
- Exactly-once ends at the Kafka boundary. A database plus Kafka gives only a best-effort one-phase commit with a failure window; close it with idempotent consumers or, better, an outbox.
- Do not use `ChainedKafkaTransactionManager`; name your transaction managers explicitly when both a database and Kafka are involved.

PART 7

Kafka Streams and ksqlDB

Stateful stream processing as a plain Java library: transformations, aggregations, windows, joins, exactly-once and queryable state, then ksqlDB and the alternatives.

In this part

- 7.1 Stream Processing Concepts
- 7.2 Your First Kafka Streams Application
- 7.3 Stateless Operations: Filter, Map, Branch and Merge
- 7.4 Aggregations and State Stores
- 7.5 Windows and Time

7.1 Stream Processing Concepts

The producer and consumer APIs hand you one record at a time. Most questions a business asks are about many records at once: how many orders were placed in the last minute, which tier does the customer who placed this order belong to, which shopping sessions ended without a checkout, have we seen this payment event before? Answering them with a plain consumer means building a lot of machinery yourself: counts kept in memory, a way to save that memory so a crash does not lose it, a way to route every record of one customer to the same instance, a definition of "the last minute" when events arrive late, and all of it running on several machines at once.

Kafka Streams is the stream processing library that ships with Apache Kafka, and it exists to solve exactly these problems. You describe the computation with a fluent Java DSL (filter, map, group, count, window, join), and Kafka Streams turns it into tasks that consume, process and produce records, keep their state in local stores that are backed up to Kafka, and spread themselves over as many application instances as you start. This chapter gives you the vocabulary; the rest of the Part turns each idea into working code for ShopStream's `analytics-app`.

7.1.1 Streams and tables

A **stream** is an unbounded, ordered sequence of immutable facts. Every record stands on its own: if a shopper views the same product page twice, that is two page views and both count. In Kafka Streams a stream is a `KStream<K, V>`.

A **table** holds the current state for each key. A new record for a key replaces the previous value (an upsert), and a record with a `null` value deletes the key (a tombstone). When order `o-1001` moves from `CREATED` to `CONFIRMED`, the table keeps one row for `o-1001` holding the latest status. In Kafka Streams a table is a `KTable<K, V>`.

The two are views of the same data, an idea called the **stream-table duality**. Replay a stream of changes from the beginning, keep the latest value per key, and you get a table: that is what a database does with its write-ahead log and what a compacted Kafka topic stores. Record every change made to a table and you get a stream, the table's **changelog**.

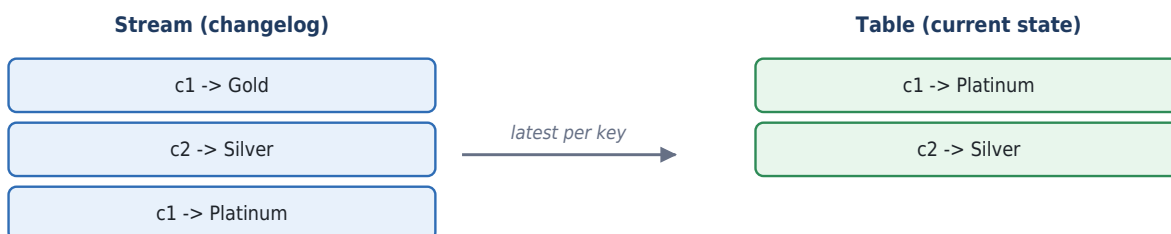


Figure 7.1. Stream-table duality: replaying the changelog stream materialises the table, and every update to the table is a new record in its changelog.

The same three records read both ways:

One topic, two interpretations

offset	key	value	read as a KStream (facts)	read as a KTable (state after it)
0	o-1001	CREATED	"o-1001 was created"	o-1001=CREATED
1	o-1002	CREATED	"o-1002 was created"	o-1001=CREATED, o-1002=CREATED
2	o-1001	CONFIRMED	"o-1001 was confirmed"	o-1001=CONFIRMED, o-1002=CREATED

Which reading is right depends on what the data means, not on the topic. `page-views` is naturally a stream; `customers` (the latest name, tier and city of each customer) is naturally a table and is stored in a compacted topic so Kafka can discard superseded values. Kafka Streams has a third abstraction for small reference data: a `GlobalKTable<K, V>` is a table of which every application instance holds a complete copy, so it can be looked up by any key without moving data around.

INTERVIEW ANGLE

"What is the difference between a KStream and a KTable?" A `KStream` is a record stream: each record is an independent event, like an INSERT into an append-only log, so two records with the same key both count. A `KTable` is a changelog stream: each record is an UPDATE of the value for its key and a `null` value is a DELETE, so only the latest value per key matters. Aggregating a stream produces a table, and `toStream()` turns a table's updates back into a stream. A `GlobalKTable` is a table fully replicated to every application instance.

7.1.2 Event time, processing time and stream time

Every Kafka record carries a timestamp. Depending on the topic setting `message.timestamp.type`, the producer sets it when it creates the record (`CreateTime`, the default) or the broker sets it when it appends the record (`LogAppendTime`). Stream processing distinguishes three kinds of time:

- **Event time:** when the event happened in the real world, for example the moment the shopper clicked. Usually the producer's record timestamp, or a field inside the payload that you extract with a timestamp extractor.
- **Ingestion time:** when the broker stored the record (`LogAppendTime`).
- **Processing time:** when your application gets around to processing the record, which can be milliseconds later or, when you reprocess history, days later.

Kafka Streams works with record timestamps, so by default it works in event time. That is what makes results reproducible: reprocess last week's page views today and "views between 10:15 and 10:16" still means the same events.

Inside a running application, time advances as **stream time**: for each task, the largest record timestamp seen so far. Stream time only moves when records arrive. A window closes, and a "final" windowed result is emitted, when stream time passes the window end plus the grace period, not when the wall clock does. One consequence surprises nearly everybody once: if traffic stops at 10:16:40, the window for 10:16 stays open until a newer record arrives.

Events also arrive **out of order**. A phone that was offline in a tunnel uploads its page views a minute late; a producer retry lands a record after newer ones. Windowed operations accept late records for a configurable **grace period** and drop records that arrive later than that. The chapter on windows and time shows how to choose it.

7.1.3 Stateless and stateful processing

A **stateless** operation looks at one record at a time and needs nothing else: drop bot traffic (`filter`), parse a JSON payload (`mapValues`), send checkout page views to their own topic. Stateless steps are cheap, need no storage and can run on any instance.

A **stateful** operation must remember earlier records: counting, summing, deduplicating, windowing, joining. Kafka Streams keeps that memory in **state stores**, local key-value, window or session stores, held in RocksDB on local disk by default or in memory. Each store is backed up to a compacted **changelog topic** in Kafka, so when an instance dies another instance rebuilds the store by replaying the changelog. You never call a remote database for state: it lives next to the code that uses it, sharded exactly like the input partitions.

7.1.4 A library, not a cluster

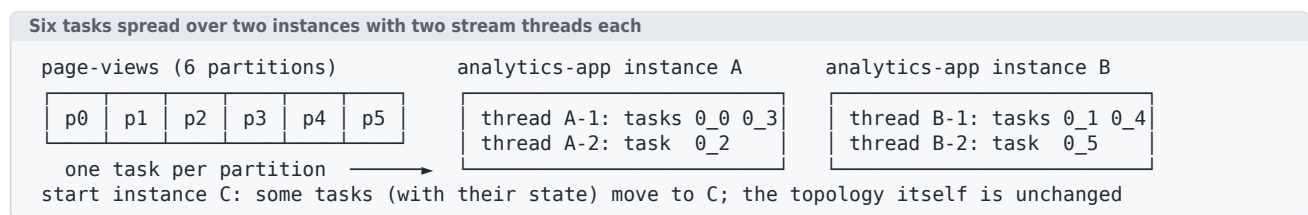
Kafka Streams is a dependency (`org.apache.kafka:kafka-streams`) inside an ordinary Java application. There is no processing cluster to install and no special job format: you package the app like any other service (a jar, a container image) and run as many copies as you need. All instances with the same `application.id` cooperate: they form one group (the `application.id` doubles as the group id), split the input partitions between them, and take over each other's work when one of them fails. Kafka is the only infrastructure.

This design has two consequences. Adoption is easy: a Spring Boot service can host a topology next to its REST controllers. And Kafka Streams reads from Kafka and writes to Kafka only; to bring data in from a database or push results to a search index you pair it with Kafka Connect (see the Part on Kafka Connect and Change Data Capture). Apache Flink and Spark Structured Streaming take the opposite approach, a separate processing cluster that talks to many systems; the last chapter of this Part compares the options.

7.1.5 Topologies, tasks and threads

The processing logic of a Streams application is a **topology**: a directed acyclic graph of nodes. **Source** nodes read topics, **processor** nodes transform records and **sink** nodes write topics. When the logic needs data re-keyed (group by another field, join on another key), Kafka Streams writes the records to an internal **repartition topic** and reads them back, which cuts the graph into **sub-topologies** that are connected only through Kafka topics.

Kafka Streams scales a topology by splitting it into **tasks**. For each sub-topology it creates one task per input partition: a sub-topology reading the 6-partition `page-views` topic gets six tasks named `0_0` to `0_5` (sub-topology 0, partitions 0 to 5). A task is the unit of work and of state: it owns the records of its partitions and its shard of every state store in that sub-topology. Tasks are distributed over **stream threads**, `num.stream.threads` per instance (default 1), across every running instance.



The number of input partitions is therefore the ceiling on parallelism. Six partitions keep at most six threads busy; an eighth thread across the fleet would sit idle as a hot spare. Choose partition counts for input topics with growth in mind, because adding partitions later moves keys to different partitions (and therefore to different state store shards).

KEY POINTS

- A **KStream** is a sequence of independent facts; a **KTable** is the latest value per key. Aggregating a stream yields a table, and a table's updates form a stream.
- Kafka Streams works in event time: windows close when stream time (the largest timestamp seen by a task) passes window end plus grace, not when the wall clock does.
- Stateful operators keep local state stores backed up to compacted changelog topics, so state survives crashes and moves with its task.
- Kafka Streams is a library inside your application; instances with the same `application.id` share the work.
- One task per input partition per sub-topology: partition counts cap the parallelism.

7.2 Your First Kafka Streams Application

Word count is the "hello world" of stream processing, and for good reason: in twenty lines it exercises a stateless step, a re-key, a repartition topic, a stateful aggregation, a state store with a changelog and an output table. Build it once, end to end, and every later chapter is a variation on the same pieces.

7.2.1 Project setup

Kafka Streams needs only one dependency. It pulls in the Kafka client library, a compatible RocksDB JNI build and the SLF4J API. Add a logging backend of your choice (for example `org.slf4j:slf4j-simple`) to see its log output, and the test utilities for the chapter on testing.

pom.xml (dependencies)

XML

```
<dependencies>
  <dependency>
    <groupId>org.apache.kafka</groupId>
    <artifactId>kafka-streams</artifactId>
    <version>4.3.1</version>
  </dependency>
  <dependency>
    <groupId>org.apache.kafka</groupId>
    <artifactId>kafka-streams-test-utils</artifactId>
    <version>4.3.1</version>
    <scope>test</scope>
  </dependency>
</dependencies>
```

7.2.2 The word count topology

The application reads lines of text from `streams-plaintext-input`, splits them into words, counts each word and writes the running count to `streams-wordcount-output`. The topic names match the official Kafka Streams quickstart, so you can compare your results with it.

WordCountApp.java

JAVA

```
package com.shopstream.analytics;

import java.time.Duration;
import java.util.Arrays;
import java.util.Locale;
import java.util.Properties;
import java.util.concurrent.CountDownLatch;

import org.apache.kafka.common.serialization.Serdes;
import org.apache.kafka.streams.KafkaStreams;
import org.apache.kafka.streams.StreamsBuilder;
import org.apache.kafka.streams.StreamsConfig;
import org.apache.kafka.streams.Topology;
import org.apache.kafka.streams.kstream.Consumed;
import org.apache.kafka.streams.kstream.Grouped;
import org.apache.kafka.streams.kstream.Materialized;
import org.apache.kafka.streams.kstream.Produced;

public class WordCountApp {

    static final String INPUT = "streams-plaintext-input";
    static final String OUTPUT = "streams-wordcount-output";

    public static Topology buildTopology() {
        StreamsBuilder builder = new StreamsBuilder();
        builder.stream(INPUT, Consumed.with(Serdes.String(), Serdes.String()))
            .flatMapValues(line -> Arrays.asList(line.toLowerCase(Locale.ROOT).split("\\W+")))
            .filter((key, word) -> !word.isBlank())
            .groupBy((key, word) -> word, Grouped.with(Serdes.String(), Serdes.String()))
            .count(Materialized.as("counts-store"))
            .toStream()
            .to(OUTPUT, Produced.with(Serdes.String(), Serdes.Long()));
        return builder.build();
    }

    public static void main(String[] args) throws InterruptedException {
        Properties props = new Properties();
        props.put(StreamsConfig.APPLICATION_ID_CONFIG, "wordcount-demo");
        props.put(StreamsConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9092");
        props.put(StreamsConfig.COMMIT_INTERVAL_MS_CONFIG, 1000); // demo only: faster output

        Topology topology = buildTopology();
        System.out.println(topology.describe());

        KafkaStreams streams = new KafkaStreams(topology, props);
```

WordCountApp.java (continued)

JAVA

```

    CountDownLatch latch = new CountDownLatch(1);
    Runtime.getRuntime().addShutdownHook(new Thread(() -> {
        streams.close(Duration.ofSeconds(10));
        latch.countDown();
    }, "streams-shutdown"));

    streams.start();
    latch.await();
}
}

```

What to notice:

- **application.id** is the identity of the application. It is the group id that all instances share, the prefix of every internal topic (`wordcount-demo-counts-store-changelog`) and the name of the sub-directory under `state.dir` where local state lives. Change it and Kafka Streams treats the result as a brand-new application that starts from scratch.
- **Serdes** (serializer/deserializer pairs) are given explicitly where data enters (`Consumed`), is re-keyed (`Grouped`) and leaves (`Produced`). The count's store needs no serde here: `count` uses the grouping serde for keys and a `Long` serde for values. Explicit serdes at every boundary beat global defaults (`default.key.serde`, `default.value.serde`), which silently apply to every operator you forget to configure.
- `flatMapValues` turns one line into zero or more words and keeps the key, so it never forces a repartition.
- `groupBy` changes the key from the (empty) line key to the word. All records for one word must end up in the same task, so Kafka Streams inserts a repartition topic before the count.
- `count(Materialized.as("counts-store"))` keeps the counts in a named state store with a changelog topic. Naming it gives stable internal topic names and makes the store queryable (see the chapter on interactive queries).
- `toStream().to(...)` turns the `KTable<String, Long>` of counts into a stream of updates and writes them out.

The `main` method shows the life cycle of a `KafkaStreams` instance. The constructor only validates the topology and configuration; `start()` launches the stream threads, which join the group, receive tasks and start processing in the background. The shutdown hook calls `close(Duration)` when the JVM receives a termination signal, so the threads commit their progress and leave the group cleanly, and the latch keeps `main` alive until then.

GOTCHA

"I typed a line and nothing came out" Aggregation results pass through a record cache that is flushed on commit, every `commit.interval.ms` (30 seconds by default with at-least-once) or when the cache (`statestore.cache.max.bytes`, 10 MB by default) fills up. With default settings the counts appear up to 30 seconds after you type. The demo lowers the commit interval to one second; production applications usually keep the defaults, because the cache collapses many updates of a hot key into one downstream record. The chapter on aggregations explains the trade-off.

LAB 7.1 · Run word count against a local broker

Start a broker as shown in the Part on installation (`localhost:9092`). Create the input topic and a compacted output topic: the output holds the latest count per word, which is table data.

Create the topics

TERMINAL

```

$ bin/kafka-topics.sh --create --topic streams-plaintext-input --partitions 3 \
  --bootstrap-server localhost:9092
Created topic streams-plaintext-input.
$ bin/kafka-topics.sh --create --topic streams-wordcount-output --partitions 3 \
  --config cleanup.policy=compact --bootstrap-server localhost:9092
Created topic streams-wordcount-output.

```

Run `WordCountApp` from your IDE (or with `mvn compile exec:java -Dexec.mainClass=com.shopstream.analytics.WordCountApp`). In a second terminal, type a few lines into the input topic:

Produce some text

TERMINAL

```

$ bin/kafka-console-producer.sh --topic streams-plaintext-input --bootstrap-server localhost:9092
>kafka streams makes stream processing simple
>streams of orders and streams of clicks

```

In a third terminal, read the output. The values are 8-byte longs, so tell the console consumer how to deserialize them:

Read the running counts

TERMINAL

```
$ bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 \
  --topic streams-wordcount-output --from-beginning \
  --formatter-property print.key=true \
  --formatter-property key.deserializer=org.apache.kafka.common.serialization.StringDeserializer \
  --formatter-property value.deserializer=org.apache.kafka.common.serialization.LongDeserializer
```

kafka	1
makes	1
stream	1
streams	1
processing	1
simple	1
of	2
orders	1
streams	3
and	1
clicks	1

Each output record is an update to the table of counts: `streams` first appears with 1 and later with 3. Because the output is spread over three partitions, the order of words in your terminal will differ. The update `streams 2` never appears: both occurrences in the second line were counted within one commit interval, and the cache forwarded only the latest value. Finally, list the topics to see what Kafka Streams created for you:

Internal topics created by the application

TERMINAL

```
$ bin/kafka-topics.sh --list --bootstrap-server localhost:9092
```

```
__consumer_offsets
streams-plaintext-input
streams-wordcount-output
wordcount-demo-counts-store-changelog
wordcount-demo-counts-store-repartition
```

7.2.3 Reading the topology

`topology.describe()` returns a `TopologyDescription` whose `toString()` prints the graph. It is the first thing to look at when an application behaves unexpectedly or creates topics you did not expect. For word count it prints (typical output):

Output of `System.out.println(topology.describe())`

```
Topologies:
Sub-topology: 0
  Source: KSTREAM-SOURCE-0000000000 (topics: [streams-plaintext-input])
    --> KSTREAM-FLATMAPVALUES-0000000001
  Processor: KSTREAM-FLATMAPVALUES-0000000001 (stores: [])
    --> KSTREAM-FILTER-0000000002
    <-- KSTREAM-SOURCE-0000000000
  Processor: KSTREAM-FILTER-0000000002 (stores: [])
    --> KSTREAM-KEY-SELECT-0000000003
    <-- KSTREAM-FLATMAPVALUES-0000000001
  Processor: KSTREAM-KEY-SELECT-0000000003 (stores: [])
    --> counts-store-repartition-filter
    <-- KSTREAM-FILTER-0000000002
  Processor: counts-store-repartition-filter (stores: [])
    --> counts-store-repartition-sink
    <-- KSTREAM-KEY-SELECT-0000000003
  Sink: counts-store-repartition-sink (topic: counts-store-repartition)
    <-- counts-store-repartition-filter

Sub-topology: 1
  Source: counts-store-repartition-source (topics: [counts-store-repartition])
    --> KSTREAM-AGGREGATE-0000000004
  Processor: KSTREAM-AGGREGATE-0000000004 (stores: [counts-store])
    --> KTABLE-TOSTREAM-0000000008
    <-- counts-store-repartition-source
  Processor: KTABLE-TOSTREAM-0000000008 (stores: [])
    --> KSTREAM-SINK-0000000009
    <-- KSTREAM-AGGREGATE-0000000004
  Sink: KSTREAM-SINK-0000000009 (topic: streams-wordcount-output)
    <-- KTABLE-TOSTREAM-0000000008
```

Read it top to bottom. Sub-topology 0 reads the input, splits and filters words, sets the word as the key (**KEY-SELECT**), drops records with a `null` key (the `repartition-filter`, since a null key cannot be partitioned) and writes to the

repartition topic. Sub-topology 1 reads the repartition topic, so every occurrence of a word arrives at the same task, counts in the `counts-store` state store and writes each changed count to the output topic. With three input partitions this application runs six tasks: `0_0` to `0_2` and `1_0` to `1_2`. In the internal topic names, Kafka Streams adds the `application.id` prefix that the description leaves out.

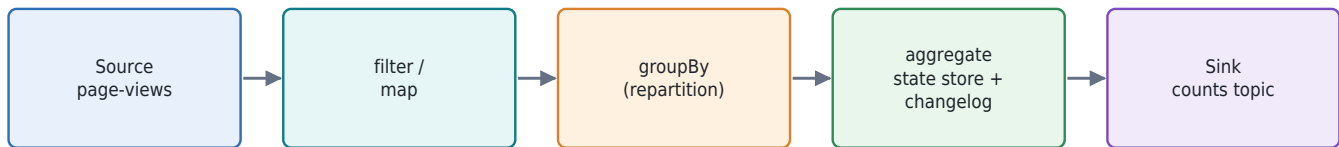


Figure 7.2. The general shape of a stateful topology: stateless steps, a repartition topic created by a re-key, an aggregation backed by a state store and its changelog topic, and a sink.

The generated names (`KSTREAM-AGGREGATE-0000000004`) contain a counter that depends on the position of the operator in the program. Insert a `peek` near the top and the numbers shift. For processor names that only hurts readability, but for stores and internal topics it breaks compatibility (see the chapter on aggregations). Name stateful operators explicitly, as this example does for the store.

WATCH OUT

Defaults that are fine on a laptop and wrong in production

- `state.dir` defaults to `${java.io.tmpdir}`, which many systems clean on reboot. Point it at a persistent volume.
- `replication.factor` for internal topics defaults to `-1` (use the broker's `default.replication.factor`, which is 1 unless changed). Set it to 3 so changelog and repartition topics survive a broker failure.
- Every instance of the application must use the same `application.id` and the same topology; a different topology under the same id corrupts the meaning of the existing internal topics.

INTERVIEW ANGLE

"What does `application.id` do in Kafka Streams?" It identifies the application across all its instances: it is used as the group id for partition assignment and offset commits, as the prefix of every internal topic (`<application.id>-<name>-changelog`, `<application.id>-<name>-repartition`) and as the state directory name. Two deployments with the same id are one application sharing work; a new id means a new application that reprocesses its input according to its offset reset policy.

KEY POINTS

- A Streams app is: build a `Topology` with `StreamsBuilder`, configure `application.id` and `bootstrap.servers`, create `KafkaStreams`, `start()` it, and `close()` it from a shutdown hook.
- Give serdes explicitly at the boundaries: `Consumed`, `Grouped`, `Produced`, `Materialized`.
- Re-keying before a stateful operation creates a repartition topic and a new sub-topology; stateful operators create changelog topics.
- `topology.describe()` shows sources, processors, stores and sinks: use it to understand and debug every topology.
- Results of aggregations are cached and flushed on commit, so output frequency depends on `commit.interval.ms` and the cache size.

7.3 Stateless Operations: Filter, Map, Branch and Merge

Before you count or join anything, most streaming applications spend their first few operators cleaning data: dropping records nobody needs, reshaping the rest and sending different kinds of records to different places. These are **stateless** operations. They look at one record at a time, keep nothing between records, use no state store and (with one exception, re-keying, explained below) need no extra topics. They are cheap and scale exactly with the number of partitions. This chapter builds a small router for ShopStream's `page-views` clickstream and introduces the tools every topology uses: serdes, the stateless operators and branching.

7.3.1 Serdes: turning bytes into objects and back

A Kafka record is a pair of byte arrays, while the DSL works with Java objects. Kafka Streams therefore needs a **serde** (a serializer and deserializer pair, `org.apache.kafka.common.serialization.Serde<T>`) at every point where data crosses a boundary: when records enter the topology (**Consumed**), leave it (**Produced**), travel through a repartition topic (**Grouped**, **Repartitioned**, **Joined**, **StreamJoined**) or are written to a state store (**Materialized**). The class **Serdes** offers ready-made serdes for the basic types: `Serdes.String()`, `Serdes.Long()`, `Serdes.Integer()`, `Serdes.Double()` and `Serdes.ByteArray()`.

If you pass no serde, Kafka Streams falls back to `default.key.serde` and `default.value.serde`. Both are `null` unless you set them, so the application then fails as soon as an operator needs one. Setting defaults looks convenient, but it hides mistakes (see the gotcha below), which is why this book passes serdes explicitly.

For your own types you need a JSON serde. The page views arrive as JSON objects written by the web front end, and the **PageView** record of the running example maps onto them directly. The class below is a complete, small JSON serde on top of Jackson 3 (add the dependency `tools.jackson.core:jackson-databind`). It keeps `null` as `null`, which matters for tombstones and records without a value.

```
JsonSerde.java JAVA

package com.shopstream.analytics;

import org.apache.kafka.common.serialization.Deserializer;
import org.apache.kafka.common.serialization.Serde;
import org.apache.kafka.common.serialization.Serdes;
import org.apache.kafka.common.serialization.Serializer;
import tools.jackson.databind.json.JsonMapper;

/** Minimal JSON serde for records such as PageView; null stays null. */
public final class JsonSerde {

    private static final JsonMapper MAPPER = JsonMapper.builder().build();

    private JsonSerde() {}

    public static <T> Serde<T> of(Class<T> type) {
        Serializer<T> serializer =
            (topic, value) -> value == null ? null : MAPPER.writeValueAsBytes(value);
        Deserializer<T> deserializer =
            (topic, bytes) -> bytes == null ? null : MAPPER.readValue(bytes, type);
        return Serdes.serdeFrom(serializer, deserializer);
    }
}
```

A record that cannot be parsed makes `readValue` throw. Kafka Streams catches the exception and passes it to the **deserialization exception handler**, which by default stops the application; the chapter on fault tolerance shows how to skip such "poison pill" records or send them to a dead-letter topic instead. If your project already uses Spring for Apache Kafka, `new JacksonJsonSerde<>(PageView.class)` is a ready-made equivalent, and the Schema Registry serdes from the Part on serialization plug into the same places.

GOTCHA

Serdes do not follow your generics Java erases generic types at run time, so Kafka Streams cannot work out the serde of a stream after `mapValues` has changed its value type. An operator that needs a serde and does not get one uses the configured default, whatever the type of the data. With a **String** default, writing a **Long** stream with a plain `to("topic")` fails at run time with a `ClassCastException` complaining that the serializer is not compatible with the actual type, not at compile time. Pass `Produced.with(...)`, `Grouped.with(...)` or `Materialized.with(...)` whenever the key or value type differs from what came out of the source.

7.3.2 The stateless toolbox

Every operator in the table returns a new **KStream**, except `foreach`, `print` and `to`, which end a branch of the topology. The original stream is never modified, so one stream can feed several different branches.

Operator	What it does	Key
<code>filter</code> , <code>filterNot</code>	Keep (or drop) the records for which the predicate is true	unchanged
<code>mapValues</code>	Turn each value into a new value; the type may change	unchanged

Operator	What it does	Key
flatMapValues	Turn each value into zero, one or many values	unchanged
map	Turn each record into a new key and value (<code>KeyValue.pair</code>)	may change
flatMap	Turn each record into zero, one or many new key-value pairs	may change
selectKey	Compute a new key and keep the value	may change
peek	Run a side effect and pass the record on unchanged	unchanged
foreach	Run a side effect; ends the branch	n/a
print	Write each record to standard output, for debugging	n/a
merge	Combine two streams of the same type into one	unchanged
repartition	Write the stream to an internal topic and read it back	unchanged
to	Write the stream to a topic; ends the branch	unchanged

The stateless operators of KStream.

The last column matters. Records with the same key must be processed by the same task, which means they must sit in the same partition. An operator that can change the key (`map`, `flatMap`, `selectKey`) breaks that guarantee, so Kafka Streams **marks the stream as needing repartitioning**. Nothing happens yet. The moment a later operator needs records of one key together (`groupByKey`, `groupByKey`, a join, a windowed aggregation), Kafka Streams inserts a repartition topic in front of it, which costs a write, a read and some latency for every record. The rule of thumb: when the key stays the same, use the `Values` variant (`mapValues`, `flatMapValues`), which never marks the stream.

The fragment below shows the common operators. `views` is a `KStream<String, PageView>` keyed by `sessionId`.

Common stateless operators

JAVA

```
KStream<String, PageView> real =
    views.filterNot((sessionId, v) -> v.url().startsWith("/health"));
KStream<String, String> urls =
    views.mapValues(v -> v.url()); // value type changes
KStream<String, String> segments =
    views.flatMapValues(v -> List.of(v.url().split("/"))); // one record becomes many
KStream<String, PageView> byUrl =
    views.selectKey((sessionId, v) -> v.url()); // new key: marks repartitioning
KStream<String, PageView> traced =
    views.peek((sessionId, v) -> System.out.println("view " + v.url()));
views.print(Printed.<String, PageView>toSysOut().withLabel("views")); // debugging only
```

WATCH OUT

Side effects in `peek` and `foreach` run at least once Kafka Streams guarantees at-least-once processing by default (exactly-once is covered later in this Part). After a crash, records since the last commit are processed again, so a `peek` that sends an e-mail or calls a REST API will do it twice. Keep side effects idempotent, or write the data to a topic and let a consumer perform the action.

7.3.3 Branching and merging

`split()` sends each record down the **first** branch whose predicate matches, like a chain of `if` and `else if`. You close the chain with `defaultBranch()` for everything that matched nothing, or with `noDefaultBranch()` to drop it. The result is a `Map<String, KStream<K, V>>` from branch name to stream. A key in the map is the prefix given to `split(Named)` followed by the name given to the branch with `Branched.as(...)`. A branch without a name is numbered by its position starting at 1, and an unnamed default branch gets 0.

You can also process a branch in place with `Branched.withConsumer(stream -> ...)` or `Branched.withFunction(...)`. A branch that has a consumer does not appear in the map. The old `branch(Predicate...)` method, which returned an array, no longer exists in current Kafka releases.

`merge` is the opposite operation: it combines two streams with the same key and value types into one, for example page views from the web shop and from a mobile app. The merged stream interleaves the records of both inputs. Within a task Kafka Streams processes input records in timestamp order as far as the data is available; the setting `max.task.idle.ms` (default 0) controls how long a task waits for a lagging input to avoid processing out of order.

Choosing the output topic per record

`to` also accepts a `TopicNameExtractor`, a function of key, value and record context that returns the topic name. Kafka Streams creates only its own internal topics, so every topic the function can return must exist.

Dynamic output topic

JAVA

```
views.to((sessionId, v, recordContext) ->
    v.url().startsWith("/checkout") ? "checkout-views" : "product-views",
    Produced.with(Serdes.String(), viewSerde));
```

Forcing a repartition

`repartition()` writes the stream to an internal topic and reads it back. Use it to spread the work of an expensive step over more partitions than the input topic has, or after a key change when the next step is a Processor API operator that does not repartition by itself. It is also the replacement for the removed `through(topic)`, with the difference that Kafka Streams creates and manages the topic. The name you give becomes part of the topic name.

Repartition into twelve partitions

JAVA

```
KStream<String, PageView> spread = views.repartition(
    Repartitioned.with(Serdes.String(), viewSerde)
    .withName("views-spread")
    .withNumberOfPartitions(12));
```

7.3.4 A complete example: cleaning and routing page views

The topology below reads `page-views`, drops malformed records and health checks, strips query strings from URLs, and routes checkout and product views to their own topics. Everything else is dropped. Create the two output topics before you run it.

PageViewRouter.java

JAVA

```
package com.shopstream.analytics;

import com.shopstream.events.PageView;
import java.util.Map;
import org.apache.kafka.common.serialization.Serde;
import org.apache.kafka.common.serialization.Serdes;
import org.apache.kafka.streams.StreamsBuilder;
import org.apache.kafka.streams.Topology;
import org.apache.kafka.streams.kstream.Branched;
import org.apache.kafka.streams.kstream.Consumed;
import org.apache.kafka.streams.kstream.KStream;
import org.apache.kafka.streams.kstream.Named;
import org.apache.kafka.streams.kstream.Produced;

public class PageViewRouter {

    static final String INPUT = "page-views";
    static final String CHECKOUT = "checkout-views";
    static final String PRODUCT = "product-views";

    public static Topology buildTopology() {
        Serde<PageView> viewSerde = JsonSerde.of(PageView.class);
        StreamsBuilder builder = new StreamsBuilder();

        Map<String, KStream<String, PageView>> routes = builder
            .stream(INPUT, Consumed.with(Serdes.String(), viewSerde))
            .filter((sessionId, view) -> view != null && view.url() != null)
            .filterNot((sessionId, view) -> view.url().startsWith("/health"))
            .mapValues(PageViewRouter::withoutQueryString)
            .split(Named.as("route-"))
            .branch((sessionId, view) -> view.url().startsWith("/checkout"),
                Branched.as("checkout"))
            .branch((sessionId, view) -> view.url().startsWith("/product/"),
                Branched.as("product"))
            .noDefaultBranch();

        routes.get("route-checkout").to(CHECKOUT, Produced.with(Serdes.String(), viewSerde));
        routes.get("route-product").to(PRODUCT, Produced.with(Serdes.String(), viewSerde));
        return builder.build();
    }
}
```

PageViewRouter.java (continued)

JAVA

```

    static PageView withoutQueryString(PageView view) {
        int q = view.url().indexOf('?');
        String path = q < 0 ? view.url() : view.url().substring(0, q);
        return new PageView(view.userId(), view.sessionId(), path, view.referrer(), view.ts());
    }
}

```

What to notice:

- The first `filter` guards against `null` values and missing URLs. A lambda that dereferences a `null` throws a `NullPointerException`, and an exception thrown in your code stops the stream thread (the chapter on fault tolerance shows the alternatives).
- `mapValues` keeps the key, so this whole topology is one sub-topology without repartition topics, and it runs as one task per `page-views` partition.
- The branch names combine with the prefix of `Named.as("route-")`, so the map keys are `route-checkout` and `route-product`.
- No state store and no internal topics are involved, so there is nothing to clean up when you change the topology.

7.3.5 Testing without a broker

Kafka Streams ships a test driver that runs a topology inside your JVM, synchronously, with no broker. You pipe records into a `TestInputTopic` and read results from a `TestOutputTopic`. Put `org.apache.kafka:kafka-streams-test-utils` and JUnit 5 on the test classpath (the chapter on testing covers the details and the limits of the driver).

PageViewRouterTest.java

JAVA

```

package com.shopstream.analytics;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertTrue;

import com.shopstream.events.PageView;
import java.time.Instant;
import java.util.Properties;
import org.apache.kafka.common.serialization.Serde;
import org.apache.kafka.common.serialization.Serdes;
import org.apache.kafka.streams.StreamsConfig;
import org.apache.kafka.streams.TestInputTopic;
import org.apache.kafka.streams.TestOutputTopic;
import org.apache.kafka.streams.TopologyTestDriver;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

class PageViewRouterTest {

    private TopologyTestDriver driver;
    private TestInputTopic<String, PageView> input;
    private TestOutputTopic<String, PageView> checkout;
    private TestOutputTopic<String, PageView> product;

    @BeforeEach
    void setUp() {
        Properties props = new Properties();
        props.put(StreamsConfig.APPLICATION_ID_CONFIG, "router-test");
        props.put(StreamsConfig.BOOTSTRAP_SERVERS_CONFIG, "dummy:9092");
        driver = new TopologyTestDriver(PageViewRouter.buildTopology(), props);

        Serde<PageView> viewSerde = JsonSerde.of(PageView.class);
        input = driver.createInputTopic(
            "page-views", Serdes.String().serializer(), viewSerde.serializer());
        checkout = driver.createOutputTopic(
            "checkout-views", Serdes.String().deserializer(), viewSerde.deserializer());
        product = driver.createOutputTopic(
            "product-views", Serdes.String().deserializer(), viewSerde.deserializer());
    }

    @AfterEach
    void tearDown() {
        driver.close();
    }
}

```

PageViewRouterTest.java (continued)

JAVA

```

@Test
void routesByUrlAndStripsQueryStrings() {
    input.pipeInput("s-1", view("s-1", "/checkout/pay?step=2"));
    input.pipeInput("s-2", view("s-2", "/product/SKU-RED-SHOE-42?utm=mail"));
    input.pipeInput("s-3", view("s-3", "/health"));
    input.pipeInput("s-4", view("s-4", "/about"));

    assertEquals("/checkout/pay", checkout.readValue().url());
    assertEquals("/product/SKU-RED-SHOE-42", product.readValue().url());
    assertTrue(checkout.isEmpty());
    assertTrue(product.isEmpty());
}

private static PageView view(String sessionId, String url) {
    Instant ts = Instant.parse("2026-10-08T10:15:30Z");
    return new PageView("c-77", sessionId, url, "/home", ts);
}
}

```

The driver needs an `application.id` and a `bootstrap.servers` value to satisfy configuration validation, but it never connects to anything. Remember to call `close()`, here in `@AfterEach`, because the driver holds state directories and threads.

TIP

Clean first, then analyse. A common design is a small "sanitising" topology that reads the raw topic, validates and normalises the records and writes a clean topic. Every other application then reads the clean topic and can trust its shape. Bad data is handled in exactly one place.

INTERVIEW ANGLE

"What is the difference between `map` and `mapValues` in Kafka Streams, and why does it matter?" `mapValues` transforms only the value, so the key and therefore the partitioning stay the same. `map` can change the key, so Kafka Streams marks the stream for repartitioning, and the next operator that needs co-located keys (a grouping, a join, a windowed aggregation) writes the records to an internal repartition topic and reads them back. That is extra network traffic, storage and latency. Use `mapValues` unless you really need a new key.

KEY POINTS

- Stateless operators (`filter`, `mapValues`, `flatMapValues`, `peek`, `split`, `merge`, `to`) process one record at a time and need no state store.
- Provide serdes explicitly at every boundary (`Consumed`, `Produced`, `Grouped`, `Materialized`); defaults hide type mistakes until run time.
- Operators that can change the key (`map`, `flatMap`, `selectKey`) mark the stream for repartitioning; prefer the `Values` variants when the key stays.
- `split().branch(...)` routes each record to the first matching branch; finish with `defaultBranch` or `noDefaultBranch`. `repartition()` replaces `through()`.
- Test topologies without a broker using `TopologyTestDriver`, `TestInputTopic` and `TestOutputTopic`.

7.4 Aggregations and State Stores

Counting, summing and tracking "the latest value" per key all need memory that survives from one record to the next. Kafka Streams provides it as **state**: stores that live next to your code, are sharded like the input partitions and are backed up to Kafka. This chapter answers three questions about ShopStream: how many views does each URL get, what is the latest known state of each shopping session, and how many customers are in each loyalty tier? Along the way it explains where state lives, how it survives a crash and, a point that surprises many people, when an aggregate becomes visible downstream.

7.4.1 Group, then aggregate

An aggregation always has two steps. First you **group** the records by key. That gives a `KGroupedStream` (or `KGroupedTable`), which is only a description of the grouping and cannot be read. Then you apply an aggregation operator, which returns a `KTable` holding the current aggregate for each key.

- `groupByKey()` keeps the existing key. If no earlier operator changed the key, no repartition topic is needed.
- `groupBy(selector)` computes a new key from each record and always repartitions, because records with the new key may currently sit in any partition.

In both cases pass `Grouped.with(keySerde, valueSerde)`, and give the grouping a name (`Grouped.with("views-by-url", ...)`) so that the repartition topic gets a stable name.

Operator	Result type	You supply
<code>count()</code>	Long	Nothing; it counts the records of each key
<code>reduce(reducer)</code>	Same as the value type	A function (aggregate, value) → combined value, for sum, min, max or latest
<code>aggregate(init, aggregator)</code>	Any type you choose	A starting value and a function (key, value, aggregate) → new aggregate

The three aggregation operators of a grouped stream.

The topology below computes three results. `Customer` is the record `Customer(String name, String tier, String city)` of the compacted `customers` topic, and `SessionSummary` is defined inside the listing.

PageViewStats.java

JAVA

```
package com.shopstream.analytics;

import com.shopstream.events.Customer;
import com.shopstream.events.PageView;
import java.time.Instant;
import org.apache.kafka.common.serialization.Serde;
import org.apache.kafka.common.serialization.Serdes;
import org.apache.kafka.common.utils.Bytes;
import org.apache.kafka.streams.KeyValue;
import org.apache.kafka.streams.StreamsBuilder;
import org.apache.kafka.streams.Topology;
import org.apache.kafka.streams.kstream.Consumed;
import org.apache.kafka.streams.kstream.Grouped;
import org.apache.kafka.streams.kstream.KStream;
import org.apache.kafka.streams.kstream.KTable;
import org.apache.kafka.streams.kstream.Materialized;
import org.apache.kafka.streams.kstream.Produced;
import org.apache.kafka.streams.state.KeyValueStore;

public class PageViewStats {

    /** What we know about one shopping session. */
    public record SessionSummary(long views, String lastUrl, Instant lastSeen) {

        public static SessionSummary empty() {
            return new SessionSummary(0, null, null);
        }

        public SessionSummary add(PageView view) {
            boolean newest = lastSeen == null || view.ts().isAfter(lastSeen);
            return new SessionSummary(views + 1,
                newest ? view.url() : lastUrl,
                newest ? view.ts() : lastSeen);
        }
    }

    public static Topology buildTopology() {
        Serde<PageView> viewSerde = JsonSerde.of(PageView.class);
        Serde<SessionSummary> summarySerde = JsonSerde.of(SessionSummary.class);
        Serde<Customer> customerSerde = JsonSerde.of(Customer.class);
        StreamsBuilder builder = new StreamsBuilder();

        KStream<String, PageView> views =
            builder.stream("page-views", Consumed.with(Serdes.String(), viewSerde));

        // 1. count: views per URL. The key changes from sessionId to url: repartition topic.
```

PageViewStats.java (continued)

JAVA

```

views.groupBy((sessionId, view) -> view.url(),
    Grouped.with("views-by-url", Serdes.String(), viewSerde))
    .count(Materialized.as("views-per-url"))
    .toStream()
    .to("page-view-counts", Produced.with(Serdes.String(), Serdes.Long()));

// 2. aggregate: one summary per session. The key is already sessionId: no repartition.
views.groupByKey(Grouped.with(Serdes.String(), viewSerde))
    .aggregate(
        SessionSummary::empty,
        (sessionId, view, summary) -> summary.add(view),
        Materialized.<String, SessionSummary, KeyValueStore<Bytes, byte[]>>as(
            "session-summary-store")
            .withKeySerde(Serdes.String())
            .withValueSerde(summarySerde))
    .toStream()
    .to("session-summaries", Produced.with(Serdes.String(), summarySerde));

// 3. table aggregation: customers per loyalty tier
KTable<String, Customer> customers = builder.table("customers",
    Consumed.with(Serdes.String(), customerSerde), Materialized.as("customers-store"));
customers.groupBy((customerId, customer) -> KeyValue.pair(customer.tier(), customer),
    Grouped.with("customers-by-tier", Serdes.String(), customerSerde))
    .count(Materialized.as("customers-per-tier"))
    .toStream()
    .to("customers-per-tier", Produced.with(Serdes.String(), Serdes.Long()));

return builder.build();
}

```

What to notice:

- **Repartition or not.** `groupBy` on the URL re-keys the stream, so Kafka Streams adds the internal topic `shopstream-analytics-views-by-url-repartition` (the application id is the prefix). `groupByKey` on the session summary is free because `page-views` is already keyed by `sessionId`.
- **The aggregator runs once per record and must be correct for any arrival order.** `add` keeps the newest URL by comparing timestamps instead of assuming the last record to arrive is the latest event.
- **Materialized for aggregate needs explicit type arguments and a value serde.** Kafka Streams can reuse the grouping's key serde, but it cannot know the serde of your aggregate type, and chaining `withValueSerde` onto `Materialized.as(...)` defeats type inference, hence the explicit `<String, SessionSummary, KeyValueStore<Bytes, byte[]>>`.
- **Aggregations skip records whose key or value is null**, log a warning and count them in the dropped-records metric.

Aggregating a table: adder and subtractor

The input of the third aggregation is a table, so a record is an update that replaces the previous value of its key. When customer `c-1` moves from `SILVER` to `GOLD`, the `SILVER` count must go down and the `GOLD` count must go up. `KTable.groupBy` therefore maps each row to a new key-value pair, and the aggregation on the resulting `KGroupedTable` applies an **adder** to the new value and a **subtractor** to the value it replaces (`count()` supplies both for you; `reduce` and `aggregate` take them as arguments).

Customers per tier while the customers table changes

update to customers	updates to customers-per-tier
c-1 = SILVER	SILVER = 1
c-2 = SILVER	SILVER = 2
c-1 = GOLD	SILVER = 1, then GOLD = 1 (subtract from old tier, add to new tier)

7.4.2 State stores and changelog topics

A **state store** is a small embedded database that holds one task's share of an aggregate. Kafka Streams has three shapes of store: key-value stores (aggregations of streams and tables), window stores (windowed aggregations and stream-stream joins) and session stores (session windows). The DSL creates them for you; with the Processor API you create and use them yourself.

By default a store is a **RocksDB** database on local disk, one per store and task, under `state.dir`. A typical layout looks like this:

Typical state directory of one instance

```

/var/lib/analytics/state
├── shopstream-analytics
│   ├── 0_0
│   │   ├── rocksdb
│   │   └── views-per-url
│   └── 1_0
│       ├── rocksdb
│       └── session-summary-store
└── ...

```

`<- state.dir`
`<- application.id`
`<- task: sub-topology 0, partition 0`
`<- one RocksDB database per store and task`

RocksDB keeps its write buffers and block cache in native memory, outside the Java heap, so a container memory limit must leave room beyond `-Xmx`. Tuning goes through a class that implements `RocksDBConfigSetter` and is named in `rocksdb.config.setter`. For small, hot state you can choose an in-memory store, either globally with `default.dsl.store=in_memory` or per operator with `Materialized.as(Stores.inMemoryKeyValueStore("name"))`.

Local disk alone is not durable, so every store has a **changelog topic**, named `<application.id>-<store name>-changelog`, with one partition per task. Each update to the store is also produced to its changelog, and the topic is compacted, so it keeps the latest value of every key (stores of windows also expire old data). If an instance dies, or a task moves to another instance, the new owner **restores** the store by replaying the changelog before it processes new records; a surviving local copy with a checkpoint file needs only the missing tail. Restoration time grows with the size of the state, which is the main reason failover of a large application is slow and why standby replicas exist (see the chapter on fault tolerance).

Treat changelog and repartition topics as private to the application. Other applications should read the topic you write with `to(...)`, which is the application's real output.

WATCH OUT

State is the cost of stateful processing A key-value aggregate never forgets a key: counting per `sessionId` grows with every new session, forever. Plan for growth. Use windows when old keys stop mattering (next chapter), watch the store sizes, and remember that every store also occupies disk and a changelog topic in the cluster. Disabling logging with `Materialized.withLoggingDisabled()` removes the changelog but also the fault tolerance: after a failure the state is gone. Do it only for state you can rebuild cheaply.

GOTCHA

At-least-once can count twice With the default `processing.guarantee=at_least_once`, a crash between processing a record and committing its offset replays that record, and the aggregate counts it again. For counters that must be exact, use `exactly_once_v2` (see the chapter on fault tolerance).

7.4.3 When do results appear? Record caches and commits

Run the counting topology and you will notice that the output topic does not receive one record for every input record. Stateful operators write into a **record cache** before they touch the store and forward results downstream. The cache keeps only the newest aggregate per key, so ten updates of `/cart` that arrive close together collapse into one output record. The cache is flushed, which writes the store, its changelog and the downstream records, when the commit interval passes or when the cache is full.

Setting	Default	Effect
<code>commit.interval.ms</code>	30000 (100 with <code>exactly_once_v2</code>)	How often caches are flushed and offsets committed
<code>statestore.cache.max.bytes</code>	10485760 (10 MiB)	Total cache size for all stream threads of the instance; a full cache evicts and forwards entries early
<code>Materialized.withCachingDisabled()</code>	cache enabled	No cache for this store: every update is forwarded at once

Settings that decide when aggregate results leave a stateful operator.

The consequence is a way of thinking: **the output of an aggregation is a changelog**. Every record is the latest value known when the cache was flushed; intermediate values may be skipped, and the same key appears many times. Write it to a compacted topic or an upsert target, and make consumers idempotent. Setting `statestore.cache.max.bytes` to 0 makes every update visible, at the price of more output records and more writes to the changelog. If you need exactly one final result per window, use `suppress`, covered in the next chapter.

7.4.4 Naming, optimisation and changing a topology

Each stateful operator creates internal resources: a store, a changelog topic and often a repartition topic. Unless you name them, their names are generated from the operator's position in the topology, such as `KSTREAM-AGGREGATE-STATE-STORE-0000000003`. Insert one operator earlier in the code and every later number shifts. After a restart the application looks for stores and topics that do not exist, starts counting from zero and leaves the old internal topics behind. Name every stateful operator: `Materialized.as(...)`, `Grouped.with(name, ...)`, `Repartitioned.as(...)`, `StreamJoined.as(...)` and `Named.as(...)`. To enforce the habit, set `ensure.explicit.internal.resource.naming=true` (default `false`); Kafka Streams then refuses to start when an internal resource would get a generated name.

The setting `topology.optimization` (default `none`) lets Kafka Streams rewrite the topology to save internal topics. `all` enables three optimizations. `reuse.ktable.source.topics` uses the input topic of a source table as its changelog instead of creating a second topic. `merge.repartition.topics` shares one repartition topic between operators that need the same re-keyed stream. `single.store.self.join` uses one store for a stream joined with itself. You can also list them individually.

WATCH OUT

Pass the Properties to `build()`. The optimizer runs only when you give the configuration to the builder: `builder.build(props)`. With a plain `builder.build()` followed by `new KafkaStreams(topology, props)` the optimization is not applied. Switching optimizations on or off for a running application changes its internal topics, so treat it like any other topology change: reset the application or deploy it under a new `application.id`.

7.4.5 Testing aggregations

The test driver flushes caches and commits after every input record, so you see every intermediate update, and it lets you look inside the stores. The first test pipes three page views of one session, the last of them late, and checks the summary in the store. The second moves a customer between tiers.

PageViewStatsTest.java

JAVA

```
package com.shopstream.analytics;

import static org.junit.jupiter.api.Assertions.assertEquals;

import com.shopstream.analytics.PageViewStats.SessionSummary;
import com.shopstream.events.Customer;
import com.shopstream.events.PageView;
import java.time.Instant;
import java.util.Properties;
import org.apache.kafka.common.serialization.Serde;
import org.apache.kafka.common.serialization.Serdes;
import org.apache.kafka.streams.StreamsConfig;
import org.apache.kafka.streams.TestInputTopic;
import org.apache.kafka.streams.TopologyTestDriver;
import org.apache.kafka.streams.state.KeyValueStore;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

class PageViewStatsTest {

    private final Serde<PageView> viewSerde = JsonSerde.of(PageView.class);
    private final Serde<Customer> customerSerde = JsonSerde.of(Customer.class);
    private TopologyTestDriver driver;

    @BeforeEach
    void setUp() {
        Properties props = new Properties();
        props.put(StreamsConfig.APPLICATION_ID_CONFIG, "stats-test");
    }
}
```

PageViewStatsTest.java (continued)

JAVA

```

        props.put(StreamsConfig.BOOTSTRAP_SERVERS_CONFIG, "dummy:9092");
        driver = new TopologyTestDriver(PageViewStats.buildTopology(), props);
    }

    @AfterEach
    void tearDown() {
        driver.close();
    }

    @Test
    void keepsTheNewestUrlWhenEventsArriveOutOfOrder() {
        TestInputTopic<String, PageView> input = driver.createInputTopic(
            "page-views", Serdes.String().serializer(), viewSerde.serializer());
        input.pipeInput("s-1", view("/cart", "2026-10-08T10:00:20Z"));
        input.pipeInput("s-1", view("/checkout", "2026-10-08T10:00:50Z"));
        input.pipeInput("s-1", view("/home", "2026-10-08T10:00:05Z")); // late event

        KeyValueStore<String, SessionSummary> store =
            driver.getKeyValueStore("session-summary-store");
        assertEquals(3, store.get("s-1").views());
        assertEquals("/checkout", store.get("s-1").lastUrl());
    }

    @Test
    void movesACustomerBetweenTiers() {
        TestInputTopic<String, Customer> customers = driver.createInputTopic(
            "customers", Serdes.String().serializer(), customerSerde.serializer());
        customers.pipeInput("c-1", new Customer("Asha", "SILVER", "Pune"));
        customers.pipeInput("c-2", new Customer("Ravi", "SILVER", "Hyderabad"));
        customers.pipeInput("c-1", new Customer("Asha", "GOLD", "Pune"));

        KeyValueStore<String, Long> perTier = driver.getKeyValueStore("customers-per-tier");
        assertEquals(1L, perTier.get("SILVER").longValue());
        assertEquals(1L, perTier.get("GOLD").longValue());
    }

    private static PageView view(String url, String timestamp) {
        return new PageView("c-77", "s-1", url, "/home", Instant.parse(timestamp));
    }
}

```

INTERVIEW ANGLE

"How does Kafka Streams make local state fault tolerant?" Every state store is a local database (RocksDB by default) that is also backed by a compacted changelog topic in Kafka: each update to the store is written to the changelog. If an instance fails, another instance takes over the task and rebuilds the store by replaying the changelog before processing continues. Standby replicas keep warm copies on other instances to shorten that recovery. With `exactly_once_v2`, the store update, the output records and the offset commit become one atomic transaction.

KEY POINTS

- An aggregation is group, then count, reduce or aggregate; the result is a `KTable`. `groupBy` re-keys and always repartitions, `groupByKey` repartitions only after an earlier key change.
- Aggregators must be correct for any arrival order; `aggregate` needs an explicit value serde in `Materialized`.
- Every state store is local (RocksDB by default) and backed by a compacted changelog topic; restoration time grows with state size.
- Aggregation output is a stream of updates, thinned by the record cache: flushed every `commit.interval.ms` or when `statestore.cache.max.bytes` fills up.
- Name stateful operators (or set `ensure.explicit.internal.resource.naming`), and pass the `Properties` to `build()` when you use `topology.optimization`.

7.5 Windows and Time

A plain count per URL grows forever. The question a business asks is usually bounded in time: how many views did `/cart` get **per minute**, how long did this visit last, did this card fail three times **within ten minutes**? Windowing slices the infinite stream into finite pieces by time and aggregates each piece separately. It also gives Kafka Streams a reason to forget: when a window is old enough, its state can be dropped, so memory stays bounded.

7.5.1 Four kinds of windows

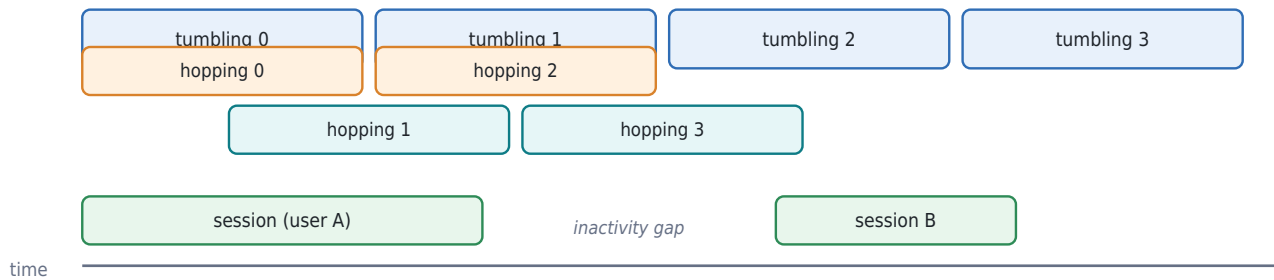


Figure 7.3. The window types on one timeline: tumbling windows tile it without overlap, hopping windows overlap, and session windows grow while a key stays active and end after a gap of inactivity.

Window	Shape	Factory
Tumbling	Fixed size, no overlap; every record falls into exactly one window	<code>TimeWindows.ofSizeAndGrace(size, grace)</code>
Hopping	Fixed size that advances by a smaller step, so windows overlap and a record falls into several	<code>TimeWindows.ofSizeAndGrace(size, grace).advanceBy(step)</code>
Sliding	Fixed maximum time difference between records; a window exists for each distinct set of records	<code>SlidingWindows.ofTimeDifferenceAndGrace(diff, grace)</code>
Session	Dynamic size per key; ends after a gap with no records, and sessions that a late record bridges are merged	<code>SessionWindows.ofInactivityGapAndGrace(gap, grace)</code>

Window types and how to create them.

Each factory has a `WithNoGrace` twin (`ofSizeWithNoGrace`, `ofTimeDifferenceWithNoGrace`, `ofInactivityGapWithNoGrace`). The old factories without a grace argument are gone from current releases, so you must decide about lateness explicitly. Time windows are aligned to the epoch and cover `[start, end)`: with a size of one minute the windows are 10:00:00 to 10:01:00, 10:01:00 to 10:02:00 and so on, whatever time your application started. A window exists only for keys that have records in it; there are no empty windows.

7.5.2 Time, lateness and grace

Which timestamp assigns a record to a window? By default the timestamp of the Kafka record, which is the producer's creation time or the broker's append time, depending on the topic's `message.timestamp.type`. If the real event time is inside the payload, a **timestamp extractor** reads it. It receives the already deserialized record and the highest timestamp seen so far in the partition:

PageViewTimestampExtractor.java

JAVA

```
package com.shopstream.analytics;

import com.shopstream.events.PageView;
import org.apache.kafka.clients.consumer.ConsumerRecord;
import org.apache.kafka.streams.processor.TimestampExtractor;

/** Uses the event time stored in the payload instead of the record timestamp. */
public class PageViewTimestampExtractor implements TimestampExtractor {

    @Override
    public long extract(ConsumerRecord<Object, Object> record, long partitionTime) {
        if (record.value() instanceof PageView view && view.ts() != null) {
            return view.ts().toEpochMilli();
        }
    }
}
```

PageViewTimestampExtractor.java (continued)

JAVA

```

    }
    return record.timestamp();
}
}

```

Attach it to one source with `Consumed.with(...).withTimestampExtractor(...)`, or to all sources with `default.timestamp.extractor`. The built-in extractors in `org.apache.kafka.streams.processor` differ in what they do with an invalid (negative) timestamp: `FailOnInvalidTimestamp` (the default) throws, `LogAndSkipOnInvalidTimestamp` skips the record, `UsePartitionTimeOnInvalidTimestamp` substitutes the partition time. `WallclockTimestampExtractor` ignores the record and returns the current system time, which turns the application into processing-time semantics.

Records arrive out of order, so a window cannot close the moment its end time passes. The **grace period** keeps it open a little longer. A window closes when stream time exceeds **window end plus grace**. Records for an open window are added, even when they are late. A record for a window that has already closed is **dropped**; Kafka Streams logs it and counts it in the `dropped-records` metrics. With `WithNoGrace` the window closes as soon as stream time passes its end.

Grace is a trade-off between completeness and delay: a long grace catches more stragglers but delays every final result by the same amount. Choose it from the lateness you actually observe in your data, for example the delay between event time and arrival time that 99.9 percent of records stay within, and watch the `dropped-records` metric afterwards.

7.5.3 Page views per minute, with one final result

The topology counts page views per URL and minute and writes **one** record per window, after it closed. It uses the extractor above, a grace period of 30 seconds, and turns the windowed key into a readable string.

PageViewWindows.java

JAVA

```

package com.shopstream.analytics;

import com.shopstream.events.PageView;
import java.time.Duration;
import org.apache.kafka.common.serialization.Serde;
import org.apache.kafka.common.serialization.Serdes;
import org.apache.kafka.streams.KeyValue;
import org.apache.kafka.streams.StreamsBuilder;
import org.apache.kafka.streams.Topology;
import org.apache.kafka.streams.kstream.Consumed;
import org.apache.kafka.streams.kstream.Grouped;
import org.apache.kafka.streams.kstream.Materialized;
import org.apache.kafka.streams.kstream.Produced;
import org.apache.kafka.streams.kstream.Suppressed;
import org.apache.kafka.streams.kstream.TimeWindows;

public class PageViewWindows {

    public static Topology buildTopology() {
        Serde<PageView> viewSerde = JsonSerde.of(PageView.class);
        TimeWindows perMinute =
            TimeWindows.ofSizeAndGrace(Duration.ofMinutes(1), Duration.ofSeconds(30));
        StreamsBuilder builder = new StreamsBuilder();

        builder.stream("page-views", Consumed.with(Serdes.String(), viewSerde)
            .withTimestampExtractor(new PageViewTimestampExtractor()))
            .groupBy((sessionId, view) -> view.url(),
                Grouped.with("views-by-url", Serdes.String(), viewSerde))
            .windowedBy(perMinute)
            .count(Materialized.as("views-per-minute"))
            .suppress(Suppressed.untilWindowCloses(Suppressed.BufferConfig.unbounded()))
            .toStream()
            .map((windowedUrl, count) -> KeyValue.pair(
                windowedUrl.key() + "@" + windowedUrl.window().startTime(), count))
            .to("page-views-per-minute", Produced.with(Serdes.String(), Serdes.Long()));

        return builder.build();
    }
}

```

What to notice:

- `windowedBy` turns the grouped stream into a windowed one, and `count` then returns a `KTable<Windowed<String>, Long>`. A `Windowed<K>` pairs the key with its `Window`, which knows `start()`, `end()`, `startTime()` and `endTime()`.
- Without `suppress`, the table would emit an updated count whenever a window changes (thinned by the record cache), so a consumer would see 1, 2, 3 for the same minute. `Suppressed.untilWindowCloses` holds the updates back and emits exactly one record per window when it closes.
- The buffer of `untilWindowCloses` must be **strict**: it may never emit early. `BufferConfig.unbounded()` can exhaust memory if you have many keys. The bounded alternative stops the application instead: `Suppressed.BufferConfig.maxRecords(100_000).shutDownWhenFull()`.
- Windowed keys can be written with `WindowedSerdes.timeWindowedSerdeFrom(String.class, windowSizeMs)`, but a string key such as `/cart@2026-10-08T10:00:00Z` is easier for other consumers.
- Windowed aggregations also accept `emitStrategy(EmitStrategy.onWindowClose())` before the aggregation, which emits only when a window closes without a separate suppression buffer. `suppress` remains the general tool because it also works on any `KTable`; `Suppressed.untilTimeLimit` rate-limits updates instead.

WATCH OUT

"Final" means stream time, not wall-clock time. A window closes only when a **newer record** pushes stream time past `window end + grace`. If a partition receives no more records, its stream time stands still and its last window is never emitted, however late the wall clock says it is. Low-traffic topics and the end of a test need a later record to move time on. Each task has its own stream time, so a quiet partition delays only its own keys.

Sessions and distinct counts

Session windows suit "visits". The fragment counts the page views of each visit of a logged-in user, where a visit ends after 30 minutes of inactivity. When a late record bridges two sessions, Kafka Streams merges them: downstream you see a record with a `null` value (a tombstone) for each old session and a new record for the merged one, so consumers must handle deletes.

Session windows: pages per visit

JAVA

```
KTable<Windowed<String>, Long> pagesPerVisit = views
    .filter((sessionId, v) -> v.userId() != null)
    .groupBy((sessionId, v) -> v.userId(),
        Grouped.with("views-by-user", Serdes.String(), viewSerde))
    .windowedBy(SessionWindows.ofInactivityGapAndGrace(
        Duration.ofMinutes(30), Duration.ofMinutes(5)))
    .count(Materialized.as("pages-per-visit"));
```

"Unique sessions per URL and minute" is an **aggregate** on the same windowed stream whose aggregate holds a set of session ids. That works for modest cardinalities; the set lives in the state store and its changelog, so for millions of distinct values per window an approximate sketch is the better tool.

7.5.4 Testing time

The test driver makes time explicit, so windows are easy to test. The test below proves that nothing is emitted while the window is open and that one final result appears once a later record moves stream time past the window end plus grace.

PageViewWindowsTest.java

JAVA

```
package com.shopstream.analytics;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertTrue;

import com.shopstream.events.PageView;
import java.time.Instant;
import java.util.Properties;
import org.apache.kafka.common.serialization.Serde;
import org.apache.kafka.common.serialization.Serdes;
import org.apache.kafka.streams.KeyValue;
import org.apache.kafka.streams.StreamsConfig;
import org.apache.kafka.streams.TestInputTopic;
import org.apache.kafka.streams.TestOutputTopic;
import org.apache.kafka.streams.TopologyTestDriver;
import org.junit.jupiter.api.Test;

class PageViewWindowsTest {
```

PageViewWindowsTest.java (continued)

JAVA

```

@Test
void emitsOneFinalCountWhenTheWindowCloses() {
    Properties props = new Properties();
    props.put(StreamsConfig.APPLICATION_ID_CONFIG, "windows-test");
    props.put(StreamsConfig.BootstrapServersConfig, "dummy:9092");
    Serde<PageView> viewSerde = JsonSerde.of(PageView.class);

    try (TopologyTestDriver driver =
        new TopologyTestDriver(PageViewWindows.buildTopology(), props)) {
        TestInputTopic<String, PageView> input = driver.createInputTopic(
            "page-views", Serdes.String().serializer(), viewSerde.serializer());
        TestOutputTopic<String, Long> output = driver.createOutputTopic(
            "page-views-per-minute",
            Serdes.String().deserializer(), Serdes.Long().deserializer());

        input.pipeInput("s-1", view("s-1", "/cart", "2026-10-08T10:00:10Z"));
        input.pipeInput("s-2", view("s-2", "/cart", "2026-10-08T10:00:40Z"));
        assertTrue(output.isEmpty()); // the 10:00 window is still open

        // stream time 10:01:45 passes window end plus grace (10:01:30): the window closes
        input.pipeInput("s-3", view("s-3", "/home", "2026-10-08T10:01:45Z"));
        assertEquals(KeyValue.pair("/cart@2026-10-08T10:00:00Z", 2L), output.readKeyValue());
    }

    private static PageView view(String sessionId, String url, String eventTime) {
        return new PageView("c-77", sessionId, url, "/home", Instant.parse(eventTime));
    }
}

```

INTERVIEW ANGLE

"How does Kafka Streams deal with late and out-of-order events in windowed aggregations?" Records are assigned to windows by event time. A window stays open until stream time (the largest timestamp seen by the task) passes window end plus the grace period. Late records inside that period update the window; later ones are dropped and counted in the dropped-records metric. To get one final result per window, add `suppress(Suppressed.untilWindowCloses(...))`, which emits only after the window has closed. Stream time only advances when new records arrive, so the last window of a quiet partition stays open.

KEY POINTS

- Windows slice a stream by time: tumbling (no overlap), hopping (overlap), sliding (time difference) and session (inactivity gap).
- Event time comes from the record timestamp or a custom `TimestampExtractor`; stream time is the highest timestamp a task has seen.
- A window closes when stream time passes window end plus grace; later records are dropped. You must choose the grace period explicitly (`ofSizeAndGrace` or `ofSizeWithNoGrace`).
- Without `suppress` or `EmitStrategy.onWindowClose()`, windowed aggregations emit a stream of updates; with them, exactly one result per window.
- Final results depend on stream time, so a quiet partition never emits its last window until new data arrives.

PART 8

Kafka Connect and Change Data Capture

Move data in and out of Kafka without writing consumers, stream database changes with Debezium, and replicate whole clusters with MirrorMaker 2.

In this part

- 8.1** Why Kafka Connect
- 8.2** Running Connect: Workers and the REST API
- 8.3** Connectors in Practice
- 8.4** Converters and Single Message Transforms

8.1 Why Kafka Connect

Almost every Kafka deployment needs the same plumbing sooner or later. Orders sit in a PostgreSQL database and must reach Kafka. Events in Kafka must end up in a search index, a data warehouse or an object store. Each of these jobs is a small program: read from one side, write to the other, remember how far you got, retry on failure, scale out when the data grows, and tell an operator when something breaks. Writing that program once is easy. Writing it ten times, with correct offset handling and sane restart behaviour, is not.

Kafka Connect is the framework inside Apache Kafka that does the generic part of this work for you. You pick a ready-made **connector**, which is a plugin that knows how to talk to one particular system, give it a configuration and submit it to a Connect cluster. The framework runs it, tracks its progress, spreads the work over several machines and restarts it after a failure. You write no Java at all unless nobody has built a connector for your system yet.

8.1.1 The building blocks

Connect has a small vocabulary. Learn these six words and the rest of this Part reads easily.

Term	What it is	Example
Connector	The logical job, such as "copy this table into Kafka". A plugin class plus a configuration. It decides how to split the job into tasks.	<code>PostgresConnector</code>
Task	The unit of work. A source task reads from an external system, a sink task writes to one. Tasks run as threads inside a worker.	3 tasks copying 3 tables
Worker	A JVM process that hosts connectors and tasks. Several workers with the same <code>group.id</code> form a Connect cluster.	<code>connect-distributed.sh</code>
Converter	Turns Connect's internal data (a schema plus values) into bytes for Kafka, and back. Set with <code>key.converter</code> and <code>value.converter</code> .	<code>JsonConverter</code>
Transform	A Single Message Transform (SMT): a small step that changes each record in flight, for example dropping a field or renaming the topic.	<code>MaskField</code>
Dead letter queue	An optional topic where a sink connector parks records it could not process, instead of failing.	<code>orders-sink-dlq</code>

The vocabulary of Kafka Connect.

A connector is only a manager. When you start it, the framework asks it for a list of task configurations, then hands those to workers. The setting `tasks.max` is an upper bound: the Kafka documentation says the connector "may create fewer tasks if it cannot achieve this level of parallelism". A JDBC source that reads one table cannot use more than one task however large you set `tasks.max`, while a sink reading a topic with 12 partitions can use up to 12.

8.1.2 How a record travels

The order of the steps differs between the two directions. In a **source** connector the data starts outside Kafka and the converter is the last step before the producer. In a **sink** connector the converter is the first step after the consumer.

Data path through a Connect task

SOURCE external system → SourceTask.poll() → transforms → converter → Kafka topic
 SINK Kafka topic → converter → transforms → SinkTask.put() → external system

Behind the scenes a source task owns a normal Kafka producer and a sink task owns a normal Kafka consumer. That is why many producer and consumer settings can be tuned for Connect (a later chapter shows how), and why everything you learned about batching, partitions and consumer groups still applies.



Figure 8.1. A Connect cluster is a set of workers. Connectors are split into tasks, and the workers share the tasks between them; source tasks write to Kafka and sink tasks read from it.

8.1.3 Offsets: how Connect remembers where it is

A sink connector is the easy case. Its tasks are consumers in one consumer group named `connect-` followed by the connector name, and the offsets are committed to `__consumer_offsets` like any other group. You can look at a sink connector's lag with `kafka-consumer-groups.sh --describe --group connect-demo-file-sink`.

A source connector has a harder problem, because "position" means something different for every system: a byte offset in a file, a log sequence number in a database, a page token in a REST API. So each source task attaches a small **source partition** map and a **source offset** map to every record it produces. The framework stores the latest offsets in an internal Kafka topic (the file `offset.storage.file.filename` in standalone mode) and gives them back to the task when it restarts. Offsets are flushed on a timer, `offset.flush.interval.ms`, which defaults to 60000 (one minute).

GOTCHA

By default Connect gives you at-least-once, not exactly-once. A source task writes its records to Kafka first and its offsets later. If the worker dies between the two, the task restarts from the older offset and re-sends some records. Sinks behave the same way: offsets are committed after the data is written, so a crash can replay a batch. Design sinks to be idempotent (an upsert by primary key is the classic example). Kafka does offer exactly-once for source connectors on request; a later chapter in this Part covers it and its limits.

8.1.4 Standalone and distributed mode

A worker can run in two modes. **Standalone** mode is one process that reads its connector configurations from files on the command line and keeps offsets in a local file. It suits development, demos and the occasional single-machine job. **Distributed** mode runs any number of workers that coordinate through Kafka itself: connector configurations, offsets and status live in three internal topics, and you manage connectors through a REST API. Distributed mode is what you run in production, even with a single worker, because its state survives the machine.

Aspect	Standalone	Distributed
Start command	<code>bin/connect-standalone.sh worker.properties connector.properties</code>	<code>bin/connect-distributed.sh worker.properties</code>
Connectors defined by	Property files given at start-up	REST calls with JSON bodies
Offsets stored in	A local file	The offsets topic in Kafka
Worker fails	Work stops until you restart it	The other workers take over its tasks
Scale out	Not possible	Start more workers with the same <code>group.id</code>
Exactly-once for sources	Not available	Available

Standalone versus distributed mode.

8.1.5 When to use Connect, and when not to

Connect shines when the job is "copy data between Kafka and system X, continuously, reliably". It is the wrong tool when the job needs real business logic.

Job	Reach for	Why
Continuously copy the <code>orders</code> table into Kafka	Connect with Debezium	CDC connectors already exist and handle snapshots and restarts
Load the <code>orders</code> topic into a search index or S3	Connect sink	No code, scalable, offsets handled
Send an email when <code>PaymentFailed</code> arrives	A consumer (Spring Kafka)	Business logic and side effects
Join orders with customers, count per minute	Kafka Streams	Stateful processing
One-off import of a small CSV file	A script	Not worth an always-on cluster

Choosing between Connect, your own consumer and Kafka Streams.

Two honest warnings. First, connector quality varies a lot. The connector for a popular database is usually solid, a niche one may be a weekend project, so check the project's activity, its licence and its delivery guarantees before you depend on it. Second, Connect is infrastructure you must operate: a cluster, internal topics, plugins to upgrade, a REST API to secure. If you need one trivial sink and no connector exists, a 60-line consumer is often the cheaper answer.

INTERVIEW ANGLE

What problem does Kafka Connect solve, and how is a connector different from a task? Connect is a framework for streaming data between Kafka and other systems without custom code: it handles offsets, retries, scaling, failover, serialization and a management API. A connector is the logical job and its configuration; it decides how to divide the work. A task is the thread that actually moves data, and the tasks of one connector are spread across the workers. Parallelism is bounded by `tasks.max` and by what the system allows (for example, topic partitions for a sink).

KEY POINTS

Key points

- Connect is part of Apache Kafka and moves data between Kafka and external systems using ready-made connectors.
- Connector = logical job; task = unit of parallel work; worker = JVM process; converter = bytes translation; SMT = per-record change.
- Source offsets are stored by Connect in an internal topic; sink offsets are an ordinary consumer group named `connect-<connector name>`.
- The default guarantee is at-least-once; make sinks idempotent.
- Use distributed mode for anything beyond experiments.
- Connect is for data movement, not for business logic or stream processing.

8.2 Running Connect: Workers and the REST API

A Connect cluster is nothing more than worker processes that share a `group.id`. This chapter starts one, explains the worker settings that matter, shows the three internal topics, and then walks through the REST API that you will use every day. The demo connectors are the two file connectors that ship with Kafka; the next chapter explains them in detail.

8.2.1 Standalone mode in one minute

Standalone mode is the quickest way to see Connect work. The Kafka quickstart does exactly this: it appends the file connector JAR to the worker's `plugin.path` and starts a worker with two connector property files.

Standalone worker with a file source and a file sink (from the Kafka quickstart, version 4.3.1)

TERMINAL

```
$ echo "plugin.path=libs/connect-file-4.3.1.jar" >> config/connect-standalone.properties
$ echo -e "foo\nbar" > test.txt
$ bin/connect-standalone.sh config/connect-standalone.properties \
  config/connect-file-source.properties config/connect-file-sink.properties
# ... log output; stop the worker with Ctrl+C
$ more test.sink.txt
foo
bar
```

The sample worker file keeps offsets in a local file (`offset.storage.file.filename`). Everything else in this chapter is about distributed mode.

8.2.2 A distributed worker

Here is a worker configuration for the ShopStream development cluster. Save it as `config/shopstream-connect.properties` inside the Kafka installation directory.

shopstream-connect.properties	PROPERTIES
<pre># Which Kafka cluster to use, and which Connect cluster this worker joins bootstrap.servers=localhost:9092 group.id=shopstream-connect # Default converters; a connector may override them key.converter=org.apache.kafka.connect.storage.StringConverter value.converter=org.apache.kafka.connect.json.JsonConverter value.converter.schemas.enable=false # Internal topics (replication factor 1 only because the dev cluster has one broker) config.storage.topic=shopstream-connect-configs offset.storage.topic=shopstream-connect-offsets status.storage.topic=shopstream-connect-status config.storage.replication.factor=1 offset.storage.replication.factor=1 status.storage.replication.factor=1 # REST API (0.0.0.0 = all interfaces) and the address other workers use to reach us listeners=http://0.0.0.0:8083 rest.advertised.host.name=localhost # Plugins: the file connectors that ship with Kafka, plus our own plugin directory plugin.path=libs/connect-file-4.3.1.jar,/opt/connect-plugins</pre>	

The table below lists the worker settings you will touch most often. Defaults are from the Kafka 4.2 configuration reference.

Setting	Default	What it does
group.id	none	Names the Connect cluster. Workers with the same value form one cluster.
config.storage.topic	none	Stores connector configurations. One partition, compacted.
offset.storage.topic	none	Stores source connector offsets. Many partitions, compacted.
status.storage.topic	none	Stores connector and task status. Compacted.
config/offset/status.storage.replication.factor	3	Used when Connect creates the topics. Fails on a one-broker cluster.
offset.storage.partitions, status.storage.partitions	25, 5	Partition counts when Connect creates the topics.
plugin.path	null	Comma-separated paths that contain connectors, converters and transformations.
listeners	http://:8083	URLs of the REST API.
rest.advertised.host.name, rest.advertised.port, rest.advertised.listener	null	The address given to other workers; needed in containers.
offset.flush.interval.ms	60000	How often source offsets are committed.
exactly.once.source.support	disabled	Transactional source connectors (see the chapter on exactly-once).
connector.client.config.override.policy	All	Which producer, consumer and admin settings a connector may override.

Important worker settings.

NOTE

Older articles configure the REST port with `rest.port` and `rest.host.name`. The Kafka 4.2 worker configuration reference lists neither. The REST endpoint is set with `listeners`, and the address advertised to other workers with `rest.advertised.host.name`, `rest.advertised.port` and `rest.advertised.listener`. If an old tutorial's `rest.port=8084` seems to have no effect, this is why: use `listeners=http://0.0.0.0:8084` instead.

8.2.3 The three internal topics

A distributed worker keeps all shared state in Kafka. The Kafka documentation describes the three topics like this: the config topic "should have a single partition, be replicated, and be configured for compaction"; the offsets topic should have many partitions; the status topic can have several. If the topics do not exist, Connect creates them with its own defaults, and the documentation recommends that you create them yourself.

Topic setting	Content	Shape
<code>config.storage.topic</code>	One record per connector config and per task config, plus commit markers	1 partition, compacted
<code>offset.storage.topic</code>	The latest source offsets, keyed by connector and source partition	Many partitions, compacted
<code>status.storage.topic</code>	State of every connector and task (running, failed, ...)	A few partitions, compacted

What each internal topic holds.

For production, create them before the first start:

Create the internal topics (development sizes; use replication factor 3 in production)

TERMINAL

```
$ bin/kafka-topics.sh --bootstrap-server localhost:9092 --create \
  --topic shopstream-connect-configs --partitions 1 \
  --replication-factor 1 --config cleanup.policy=compact
Created topic shopstream-connect-configs.
$ bin/kafka-topics.sh --bootstrap-server localhost:9092 --create \
  --topic shopstream-connect-offsets --partitions 25 \
  --replication-factor 1 --config cleanup.policy=compact
Created topic shopstream-connect-offsets.
$ bin/kafka-topics.sh --bootstrap-server localhost:9092 --create \
  --topic shopstream-connect-status --partitions 5 \
  --replication-factor 1 --config cleanup.policy=compact
Created topic shopstream-connect-status.
```

WATCH OUT

The default replication factor of the internal topics is 3. On a single-broker development cluster the worker fails at start-up while creating the topics, with an error saying the replication factor is larger than the number of available brokers. Set the three `*.storage.replication.factor` settings to 1 for development, and keep them at 3 (with `min.insync.replicas=2` on the topics) in production. Never share one set of internal topics between two Connect clusters: the `group.id` and the three topic names must be unique per cluster.

Now start the worker and check that it answers:

Start a distributed worker and ask it who it is

TERMINAL

```
$ bin/connect-distributed.sh config/shopstream-connect.properties
# ... in a second terminal:
$ curl -s http://localhost:8083/ | jq .
{
  "version": "4.3.1",
  "commit": "a1b2c3d4e5f6a7b8",
  "kafka_cluster_id": "MkU30EVbNTcwNTJENDM2Qg"
}
```

The commit hash and cluster ID above are examples. Starting a second worker on another machine with the same `group.id` is all it takes to scale out.

8.2.4 Plugins and secrets

Connect finds connectors, converters and transformations in the paths listed in `plugin.path`. Each entry is either a directory that contains one sub-directory per plugin (with the plugin's JARs and its dependencies), or a single self-contained JAR. Every plugin gets its own class loader, so two connectors can use different versions of the same library without clashing.

A plugin directory

```
/opt/connect-plugins/
├── debezium-connector-postgres/
└── jdbc-sink/
```

one folder per plugin, with its JARs and all dependency JARs (such as the JDBC driver)

Plugins are scanned when the worker starts, so after adding a plugin you must restart the workers (one at a time in a cluster). `GET /connector-plugins` shows what a worker has found. Version 4.x also has a `plugin.discovery` setting (default `hybrid_warn`); leave it alone unless a connector vendor tells you otherwise.

Connector configurations are visible to anyone who can call `GET /connectors/{name}/config`, so database passwords do not belong in them. Use a **config provider** instead: declare it once in the worker file, and reference secrets in the connector JSON with `${provider:path:key}` placeholders. The REST API returns the placeholder, never the secret.

Worker file: enable the file config provider

PROPERTIES

```
config.providers=file
config.providers.file.class=org.apache.kafka.common.config.provider.FileConfigProvider
```

Connector config fragment using the provider

JSON

```
{
  "database.user": "${file:/etc/connect-secrets/shop-db.properties:user}",
  "database.password": "${file:/etc/connect-secrets/shop-db.properties:password}"
}
```

8.2.5 The REST API

Every worker serves the REST API on its `listeners` port (8083 by default), and any worker can answer any call. Reads are answered from the worker's own copy of the cluster state. Calls that change a configuration are forwarded to the cluster's leader worker, which is why `rest.advertised.host.name` must be an address the other workers can reach. The table is the cheat sheet; all paths are from the Kafka 4.2 user guide.

Method	Path	Purpose
GET	<code>/connectors</code>	List connector names; add <code>?expand=status</code> or <code>?expand=info</code> for details
POST	<code>/connectors</code>	Create a connector from a JSON body with <code>name</code> and <code>config</code>
GET	<code>/connectors/{name}</code>	Connector info: config, tasks, type
GET	<code>/connectors/{name}/config</code>	The configuration only
PUT	<code>/connectors/{name}/config</code>	Create or replace a connector from a flat config map (idempotent)
PATCH	<code>/connectors/{name}/config</code>	Change only the keys you send (listed in the 4.2 documentation)
GET	<code>/connectors/{name}/status</code>	State of the connector and each task
PUT	<code>/connectors/{name}/pause</code>	Pause processing, keep tasks allocated
PUT	<code>/connectors/{name}/resume</code>	Resume a paused connector
PUT	<code>/connectors/{name}/stop</code>	Stop and shut down the tasks, freeing resources
POST	<code>/connectors/{name}/restart</code>	Restart; <code>?includeTasks=true&onlyFailed=true</code> restarts failed tasks too
POST	<code>/connectors/{name}/tasks/{id}/restart</code>	Restart one task
DELETE	<code>/connectors/{name}</code>	Delete the connector
GET	<code>/connectors/{name}/tasks</code>	The task configurations
GET	<code>/connectors/{name}/topics</code>	Topics the connector has used

Method	Path	Purpose
GET, PATCH, DELETE	/connectors/{name}/offsets	Read, alter or reset offsets (connector must be stopped to alter or reset)
GET	/connector-plugins	Plugins installed on this worker
PUT	/connector-plugins/{class}/config/validate	Validate a config without creating anything
GET, PUT	/admin/loggers/{logger}	Read or change a log level at run time

REST API cheat sheet. The name is the connector name.

Older Connect versions also had `GET /connectors/{name}/tasks-config`. It was removed in 4.x; use `GET /connectors/{name}/tasks` instead.

Let us create a source connector that reads a text file into a topic. The JSON body goes in a file, which is easier to edit and to keep in Git than a long command line.

demo-file-source.json

JSON

```
{
  "name": "demo-file-source",
  "config": {
    "connector.class": "org.apache.kafka.connect.file.FileStreamSourceConnector",
    "tasks.max": "1",
    "file": "/tmp/demo-in.txt",
    "topic": "connect-demo"
  }
}
```

The connector lifecycle with curl

TERMINAL

```
$ curl -s -X POST -H "Content-Type: application/json" \
  --data @demo-file-source.json http://localhost:8083/connectors | jq '{name, tasks, type}'
{
  "name": "demo-file-source",
  "tasks": [],
  "type": "source"
}
$ curl -s http://localhost:8083/connectors
["demo-file-source"]
$ curl -s http://localhost:8083/connectors/demo-file-source/status | jq .
{
  "name": "demo-file-source",
  "connector": { "state": "RUNNING", "worker_id": "localhost:8083" },
  "tasks": [ { "id": 0, "state": "RUNNING", "worker_id": "localhost:8083" } ],
  "type": "source"
}
$ curl -s -X PUT http://localhost:8083/connectors/demo-file-source/pause
$ curl -s -X PUT http://localhost:8083/connectors/demo-file-source/resume
$ curl -s -X POST \
  "http://localhost:8083/connectors/demo-file-source/restart?includeTasks=true&onlyFailed=true"
$ curl -s -X DELETE http://localhost:8083/connectors/demo-file-source
```

Pause and resume return an empty body with status 202 (accepted): the change is applied asynchronously, so call `/status` to see it take effect. For automation, prefer `PUT /connectors/{name}/config`, because running it twice with the same body gives the same result, which is what a deployment script wants. If the connector does not exist yet, `PUT` creates it.

Before you create a connector, you can ask the worker whether the configuration is valid. The call needs the full class name in the path and in the body; in the response, look at `error_count` and at the `errors` of each entry in `configs`.

Validate a configuration without creating the connector

TERMINAL

```
$ PLUGIN=org.apache.kafka.connect.file.FileStreamSourceConnector
$ curl -s -X PUT -H "Content-Type: application/json" \
  --data '{"connector.class":"$PLUGIN"}' \
  "http://localhost:8083/connector-plugins/$PLUGIN/config/validate" | jq '.error_count'
1
```

Here the single error is the missing `topic`. Finally, `GET /connector-plugins` tells you whether the plugin you copied into `plugin.path` was actually found:

List the connector plugins this worker knows

TERMINAL

```
$ curl -s http://localhost:8083/connector-plugins | jq -r '.[].class'
org.apache.kafka.connect.file.FileStreamSinkConnector
org.apache.kafka.connect.file.FileStreamSourceConnector
org.apache.kafka.connect.mirror.MirrorCheckpointConnector
org.apache.kafka.connect.mirror.MirrorHeartbeatConnector
org.apache.kafka.connect.mirror.MirrorSourceConnector
```

The three MirrorMaker 2 connectors ship with Kafka; a later chapter in this Part uses them.

WATCH OUT

The REST API has no authentication unless you add it. Anyone who can reach port 8083 can create a connector that reads files from the worker, opens database connections with stored credentials or deletes your pipelines. Keep the port on an internal network, put a reverse proxy with authentication in front of it, or use the worker's HTTPS listeners and REST extensions (see the worker configuration reference, `rest.extension.classes`). Never expose it to the internet.

8.2.6 Running a worker in a container

The `apache/kafka` image contains the whole Kafka distribution, so a simple way to get a Connect worker is to build your own image from it and add the worker file and your plugins. Treat this as a sketch to adapt, not a tested recipe.

Dockerfile

DOCKERFILE

```
FROM apache/kafka:4.3.1
COPY connect-worker.properties /opt/kafka/config/connect-worker.properties
COPY plugins/ /opt/connect-plugins/
CMD ["/opt/kafka/bin/connect-distributed.sh", "/opt/kafka/config/connect-worker.properties"]
```

docker-compose.yml (the connect service only)

YAML

```
services:
  connect:
    build: ./connect
    ports:
      - "8083:8083"
    depends_on:
      - kafka
```

In `connect-worker.properties`, set `bootstrap.servers=kafka:9092` (a listener the broker advertises to other containers), `rest.advertised.host.name=connect` (the service name, so other workers can reach this one) and `plugin.path=/opt/connect-plugins`. The CDC lab later in this Part uses the Debezium project's own Connect image instead, which already contains the Debezium plugins.

INTERVIEW ANGLE

Where does a distributed Connect cluster keep its state, and what happens when a worker dies? State lives in Kafka: connector and task configurations in the config topic, source offsets in the offsets topic, status in the status topic, sink offsets in ordinary consumer-group offsets. Workers are stateless. When one dies, the group rebalances and its tasks are assigned to the surviving workers, which resume from the stored offsets. Because of at-least-once delivery, a few records may be processed twice.

KEY POINTS

Key points

- A Connect cluster is workers sharing a `group.id`; add capacity by starting more workers.
- Create the three internal topics yourself: config (1 partition), offsets (many) and status, all compacted; lower their replication factor only on a one-broker dev cluster.
- The REST API is the control plane (`/connectors`, `/status`, `pause`, `resume`, `restart`, `/connector-plugins`); prefer `PUT .../config` in scripts.
- `rest.port` is gone from current docs; use `listeners` and the `rest.advertised.*` settings.
- Plugins are found through `plugin.path` and loaded at start-up; keep secrets out of connector configs with a config provider.
- Secure the REST API; it has no authentication by default.

8.3 Connectors in Practice

You now have a running Connect cluster. This chapter puts real connectors on it: the file connectors that ship with Kafka (for learning), a JDBC source and sink (for relational data), and an object-storage sink (for archives and data lakes). It also gives you a checklist for judging any connector you find on the internet.

8.3.1 The file connectors that ship with Kafka

Apache Kafka includes two small example connectors in the `connect-file` JAR. `FileStreamSourceConnector` reads a text file line by line and keeps reading as new lines are appended. `FileStreamSinkConnector` writes each record it receives as a line in a file. They are meant for learning and testing, not for production. The user guide notes that they are "not included by default to the `CLASSPATH` or the `plugin.path`", which is why the worker file in the previous chapter added `libs/connect-file-4.3.1.jar` to `plugin.path` by hand.

LAB 8.1 · File pipeline: from a text file through Kafka to another text file

The worker from the previous chapter must be running. We use the same two connectors as before, this time with a per-connector converter override: both read and write plain strings, so the worker's JSON default does not apply. `PUT ../config` creates the connector or updates it if it exists.

demo-file-source-config.json

JSON

```
{
  "connector.class": "org.apache.kafka.connect.file.FileStreamSourceConnector",
  "tasks.max": "1",
  "file": "/tmp/demo-in.txt",
  "topic": "connect-demo",
  "value.converter": "org.apache.kafka.connect.storage.StringConverter"
}
```

demo-file-sink-config.json

JSON

```
{
  "connector.class": "org.apache.kafka.connect.file.FileStreamSinkConnector",
  "tasks.max": "1",
  "file": "/tmp/demo-out.txt",
  "topics": "connect-demo",
  "value.converter": "org.apache.kafka.connect.storage.StringConverter"
}
```

Note the difference in the topic setting: a source names the single topic it writes to (`topic`), a sink lists the topics it reads (`topics`). Now create both and feed the source some ShopStream-flavoured lines.

Create the connectors and append three lines

TERMINAL

```
$ curl -s -X PUT -H "Content-Type: application/json" \
  --data @demo-file-source-config.json \
  http://localhost:8083/connectors/demo-file-source/config | jq .name
"demo-file-source"
$ curl -s -X PUT -H "Content-Type: application/json" \
  --data @demo-file-sink-config.json \
  http://localhost:8083/connectors/demo-file-sink/config | jq .name
"demo-file-sink"
$ echo "o-1001,c-77,998.00" >> /tmp/demo-in.txt
$ echo "o-1002,c-12,250.00" >> /tmp/demo-in.txt
$ echo "o-1003,c-77,1499.50" >> /tmp/demo-in.txt
```

Within a few seconds the lines travel through Kafka and appear in the output file. Check both ends:

Look at the topic and at the sink file

TERMINAL

```
$ bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 \
  --topic connect-demo --from-beginning
o-1001,c-77,998.00
o-1002,c-12,250.00
o-1003,c-77,1499.50
^C
$ cat /tmp/demo-out.txt
o-1001,c-77,998.00
o-1002,c-12,250.00
o-1003,c-77,1499.50
```

The sink's progress is a normal consumer group, so `bin/kafka-consumer-groups.sh --bootstrap-server localhost:9092 --describe --group connect-demo-file-sink` shows its lag. The source's progress is a source offset, which the REST API can show you:

Read, then reset, the source offsets

TERMINAL

```
$ curl -s http://localhost:8083/connectors/demo-file-source/offsets | jq -c .
{"offsets":[{"partition":{"filename":"/tmp/demo-in.txt"},"offset":{"position":58}}]}
$ curl -s -X PUT http://localhost:8083/connectors/demo-file-source/stop
$ curl -s -X DELETE http://localhost:8083/connectors/demo-file-source/offsets
$ curl -s -X PUT http://localhost:8083/connectors/demo-file-source/resume
```

The offset is the byte position in the file (the three lines above add up to 58 bytes). After the reset the source forgets it ever read the file, starts at byte 0 and publishes the three lines again, so the topic now holds each line twice. That is a small demonstration of two facts from the first chapter of this Part: offsets are just data that you can inspect and change (the connector must be stopped first), and a rewind produces duplicates unless the sink is idempotent.

8.3.2 A JDBC source and sink

Relational databases are the most common thing people want to connect to Kafka. The generic way is a **JDBC connector**, which talks to the database through a JDBC driver and plain SQL. JDBC connectors are not part of Apache Kafka. The best-known pair is the Confluent JDBC source and sink (`io.confluent.connect.jdbc.JdbcSourceConnector` and `JdbcSinkConnector`); several forks and alternatives exist. The configuration keys below were checked against the Confluent documentation, but connector-specific keys differ between projects, so **check your connector's docs** before copying them.

A JDBC source works by polling. Every `poll.interval.ms` (default 5000) it runs a query and publishes the rows it has not seen yet. How it recognises "not seen yet" is the `mode`:

mode	How it finds new data
bulk	Copies the whole table on every poll. Simple, but every poll re-sends everything.
incrementing	Uses a strictly increasing column (<code>incrementing.column.name</code>). Finds new rows only; updates to old rows are missed.
timestamp	Uses a "last modified" column (<code>timestamp.column.name</code>). Finds inserts and updates, but rows sharing a timestamp at the boundary need care.
timestamp+incrementing	Uses both columns. The most reliable polling mode for inserts and updates.

Modes of the JDBC source connector (key names: check your connector's docs).

Here is a source for the ShopStream `orders` table. The table has a numeric `id` and an `updated_at` timestamp, so `timestamp+incrementing` fits. The connector names the topic by joining `topic.prefix` and the table name, so this one writes to `jdbc-orders`.

orders-jdbc-source.json

JSON

```
{
  "name": "orders-jdbc-source",
  "config": {
    "connector.class": "io.confluent.connect.jdbc.JdbcSourceConnector",
    "tasks.max": "1",
    "connection.url": "jdbc:postgresql://postgres:5432/shop",
    "connection.user": "${file:/etc/connect-secrets/shop-db.properties:user}",
    "connection.password": "${file:/etc/connect-secrets/shop-db.properties:password}",
    "mode": "timestamp+incrementing",
    "incrementing.column.name": "id",
    "timestamp.column.name": "updated_at",
    "table.whitelist": "orders",
    "topic.prefix": "jdbc-",
    "poll.interval.ms": "5000"
  }
}
```

The sink side is the mirror image. It turns each record into an **INSERT** or an upsert. The setting that matters most is `insert.mode`: with `upsert` and a primary key, replays after a crash overwrite the same row instead of creating a duplicate, which is what makes an at-least-once pipeline safe.

orders-jdbc-sink.json

JSON

```
{
  "name": "orders-jdbc-sink",
  "config": {
    "connector.class": "io.confluent.connect.jdbc.JdbcSinkConnector",
    "tasks.max": "3",
    "topics": "orders",
    "connection.url": "jdbc:postgresql://warehouse:5432/analytics",
    "connection.user": "${file:/etc/connect-secrets/wh-db.properties:user}",
    "connection.password": "${file:/etc/connect-secrets/wh-db.properties:password}",
    "insert.mode": "upsert",
    "pk.mode": "record_key",
    "pk.fields": "order_id",
    "auto.create": "true",
    "auto.evolve": "false"
  }
}
```

What to notice: `pk.mode=record_key` takes the primary key from the Kafka record key, and `pk.fields` names the column that receives it. `auto.create=true` lets the sink create the target table; production teams usually prefer `false` and manage tables with migrations. And one requirement is easy to miss: this sink needs to know the **schema** of each record, so the topic must carry Avro, JSON Schema, Protobuf, or JSON with an embedded schema. ShopStream's plain JSON topics would be rejected; the chapter on Schema Registry in Part 6 shows how to produce a schema-aware version of the topic.

GOTCHA

A polling source can never see a deleted row, and it can miss updates. The query only returns rows that exist now. A row deleted between two polls leaves no trace in Kafka, and two updates to the same row between polls arrive as one. The poll also puts load on the database, and latency can be no better than `poll.interval.ms`. If you need deletes, every change in order, or low database load, use log-based change data capture with Debezium, covered later in this Part.

8.3.3 Object storage: archives and data lakes

The most common sink in analytics is an object store such as Amazon S3. The usual reasons: a cheap long-term copy of a topic beyond its retention, input for a data lake or warehouse, and a replayable audit trail. The Confluent S3 sink (`io.confluent.connect.s3.S3SinkConnector`) is the best-known example. The idea is the same for every object-store sink: it buffers records per topic partition, writes a file when a threshold is reached, and commits the Kafka offsets only after the upload succeeded.

Setting	Meaning
<code>s3.bucket.name</code> , <code>s3.region</code>	Where the files go.
<code>format.class</code>	File format: JSON, Avro or Parquet implementations (for example <code>io.confluent.connect.s3.format.json.JsonFormat</code>).
<code>flush.size</code>	Number of records per file. Has no default; you must set it.
<code>rotate.interval.ms</code>	Also close a file after this much time (default -1, disabled).
<code>partitioner.class</code>	How files are laid out in the bucket: by Kafka partition (default), by record field, or by time.

Core settings of the Confluent S3 sink (check your connector's docs for your version).

orders-s3-sink.json

JSON

```
{
  "name": "orders-s3-sink",
  "config": {
    "connector.class": "io.confluent.connect.s3.S3SinkConnector",
    "tasks.max": "3",
    "topics": "orders",
    "s3.bucket.name": "shopstream-archive",
    "s3.region": "ap-south-1",
    "storage.class": "io.confluent.connect.s3.storage.S3Storage",
    "format.class": "io.confluent.connect.s3.format.json.JsonFormat",
    "flush.size": "1000",
    "value.converter": "org.apache.kafka.connect.json.JsonConverter",
    "value.converter.schemas.enable": "false"
  }
}
```

By default the sink writes objects whose names contain the topic, the Kafka partition and the offset of the first record in the file. A replay after a crash then produces an object with the same name and the same content, which overwrites the earlier attempt instead of duplicating it. The exact guarantee depends on the partitioner and on time-based rotation, so read the delivery-guarantee section of the connector's documentation before you rely on it.

8.3.4 Judging a connector before you depend on it

Connect connectors come from the Kafka project, from vendors, and from individuals. Before you adopt one, answer these questions:

- **Maintenance and licence.** Is it released regularly, does it support your Kafka and Java versions, and does its licence allow your use?
- **Source type.** Is a source connector polling (JDBC style) or log-based (Debezium style)? This decides whether deletes and updates are visible.
- **Delivery guarantee.** At-least-once, effectively-once through idempotent writes, or exactly-once? Which settings does that depend on?
- **Data format.** Which converters does it support, does it need schemas, and how does it map types such as decimals and timestamps?
- **Operations.** Does it support `tasks.max` scaling, SMTs, the dead letter queue, and clean restarts after failures? Does it expose metrics?
- **Installation.** Download, unpack into a directory under `plugin.path`, restart the workers, confirm with `GET /connector-plugins`, validate with the `validate` endpoint, then create.

KEY POINTS

Key points

- The Kafka distribution contains only example file connectors (not on the plugin path by default) and the MirrorMaker 2 connectors; everything else is a separate plugin.
- Sources name one `topic` (or a prefix), sinks list `topics` or a `topics.regex`.
- A JDBC source polls: it cannot see deletes and can miss intermediate updates. A JDBC sink should use `insert.mode=upsert` so replays are harmless, and it needs schema-aware records.
- Object-store sinks batch records into files by size or time and commit offsets after the upload.
- Source offsets can be read, altered and reset through the REST API, but only while the connector is stopped.
- Connector-specific keys differ between projects: always read the connector's own documentation.

8.4 Converters and Single Message Transforms

Two small pieces of Connect decide what your data looks like on the way in and out of Kafka. **Converters** choose the byte format on the topic. **Single Message Transforms** (SMTs) make light changes to each record. Both are configured, not coded, and both are the source of a large share of Connect's day-to-day support questions.

8.4.1 Converters: the translation layer

Connectors work with Connect's own data model: a record has a key, a value and headers, and each part carries an optional schema plus a value (a `Struct`, a `Map`, a string, a number and so on). The converter turns that model into bytes before the producer sends it, and back into the model after the consumer receives it. Because the connector never sees the bytes, the same JDBC connector can write JSON today and Avro tomorrow by changing one line.

You set converters on the worker (`key.converter`, `value.converter`, `header.converter`) and may override the key and value converter inside any connector configuration. Settings for a converter are prefixed with the converter's role, for example `value.converter.schemas.enable`.

Class	Format on the topic	Typical use
<code>org.apache.kafka.connect.storage.StringConverter</code>	UTF-8 text	Keys such as <code>orderId</code> , raw text lines
<code>org.apache.kafka.connect.json.JsonConverter</code>	JSON	Human-readable topics; the ShopStream default

Class	Format on the topic	Typical use
<code>org.apache.kafka.connect.converters.ByteArrayConverter</code>	Unchanged bytes	Copying data you do not want to interpret
<code>io.confluent.connect.avro.AvroConverter</code>	Avro plus Schema Registry id	Governed schemas (needs <code>value.converter.schema.registry.url</code>)

Common converters.

JSON and Avro converters for JSON Schema and Protobuf exist too, from the same vendor; the Schema Registry chapter in Part 6 explains the wire format and compatibility rules, so we do not repeat them here.

8.4.2 The schemas.enable trap

`JsonConverter` has a setting that surprises almost everybody: `schemas.enable`. When it is `true`, every message is wrapped in an envelope that carries the schema next to the data. The Kafka quickstart shows the effect for a plain string:

`schemas.enable=true: the schema travels with every message`

JSON

```
{"schema":{"type":"string","optional":false},"payload":"foo"}
```

With `schemas.enable=false` the same record is just `"foo"`, and a structured record is just its JSON object, for example `{"orderId":"o-1001","total":998.00}`. The sample worker configuration in the Kafka distribution enables schemas, because some connectors (the JDBC sink among them) need them. The ShopStream worker from the previous chapter disables them, because our topics hold plain JSON.

The two settings must match on both sides of a topic. The most frequent failures are mirror images of each other:

What you see	Cause	Fix
<code>DataException: JsonConverter with schemas.enable requires "schema" and "payload" fields ...</code>	The topic holds plain JSON but the connector's converter has <code>schemas.enable=true</code>	Set <code>value.converter.schemas.enable=false</code>
<code>SerializationException: Unknown magic byte!</code>	The connector uses the Avro converter on a topic that does not hold Avro with Schema Registry framing	Use the converter that matches the topic
A sink fails with "requires records with a non-null Struct value and schema"	The sink needs schemas but the topic is schemaless JSON	Produce schema-aware data (Avro, or JSON with <code>schemas.enable=true</code>)

Typical converter mismatches.

One more JSON detail matters for money. The Connect data model has a logical type `Decimal`, which database connectors use for `NUMERIC` columns. `JsonConverter` writes it as a Base64 string by default, so `998.00` shows up as something like `"AYag"`. Set `value.converter.decimal.format=NUMERIC` to get a JSON number instead; the CDC chapter returns to this.

8.4.3 Single Message Transforms

An SMT is a small function applied to every record that passes through a connector. You list the transformations by alias in `transforms`, and Connect applies them in exactly that order. Each alias gets a `type` (the class) and its own options. In a source connector they run after the connector produced the record and before the converter; in a sink connector they run after the converter and before the connector's `put`.

Kafka ships with these in the package `org.apache.kafka.connect.transforms` (class names and options from the 4.2 user guide). Those that exist as key and value variants are used as `ClassName$Key` or `ClassName$Value`.

Class	What it does (main options)
<code>InsertField\$Value</code>	Adds a field: a static value, or the topic, partition, offset (sinks only) or timestamp (<code>static.field</code> , <code>static.value</code> , <code>topic.field</code> , <code>timestamp.field</code>)
<code>ReplaceField\$Value</code>	Keeps or drops fields and renames them (<code>include</code> , <code>exclude</code> , <code>renames</code>)
<code>MaskField\$Value</code>	Replaces field values with a null-like value or a fixed one (<code>fields</code> , <code>replacement</code>)
<code>ExtractField\$Key</code> , <code>ExtractField\$Value</code>	Replaces the whole key or value with one of its fields (<code>field</code>)
<code>HoistField\$Value</code>	Wraps the whole value in a one-field structure (<code>field</code>)

Class	What it does (main options)
Flatten\$Value	Flattens nested structures into one level (<code>delimiter</code> , default <code>.</code>)
Cast\$Value	Converts field types (<code>spec</code>)
TimestampConverter\$Value	Converts timestamps between formats (<code>target.type</code> , <code>field</code> , <code>format</code>)
ValueToKey	Builds the key from fields of the value (<code>fields</code>)
RegexRouter	Renames the topic with a regular expression (<code>regex</code> , <code>replacement</code>)
TimestampRouter	Appends a date to the topic name (<code>topic.format</code> , <code>timestamp.format</code>)
HeaderFrom\$Value, InsertHeader, DropHeaders	Create, add or remove record headers
Filter	Drops the record; used together with a predicate

Built-in transformations.

Three recipes cover most real needs. First, ShopStream's JDBC source writes to `jdbc-orders` with a structured key. We want the topic `orders` and a plain string key `order_id`. That takes a router and two key steps (`ValueToKey` builds a key structure, `ExtractField$Key` unwraps the one field):

SMT chain for a source connector (fragment)

JSON

```
{
  "transforms": "route,createKey,extractKey",
  "transforms.route.type": "org.apache.kafka.connect.transforms.RegexRouter",
  "transforms.route.regex": "jdbc-(.*)",
  "transforms.route.replacement": "$1",
  "transforms.createKey.type": "org.apache.kafka.connect.transforms.ValueToKey",
  "transforms.createKey.fields": "order_id",
  "transforms.extractKey.type": "org.apache.kafka.connect.transforms.ExtractField$Key",
  "transforms.extractKey.field": "order_id"
}
```

Second, before data leaves for a third-party system you may need to hide personal data. `MaskField` replaces the listed fields (the field names here are illustrative):

Masking fields in a sink connector (fragment)

JSON

```
{
  "transforms": "mask",
  "transforms.mask.type": "org.apache.kafka.connect.transforms.MaskField$Value",
  "transforms.mask.fields": "email,phone"
}
```

Third, **predicates** make a transformation conditional. A predicate is declared once with `predicates` and attached to a transformation through its implicit `predicate` option; `negate=true` inverts it. Kafka provides `TopicNameMatches` (option `pattern`), `HasHeaderKey` (option `name`) and `RecordIsTombstone`, all in `org.apache.kafka.connect.transforms.predicates`. This example makes a sink ignore tombstones (records with a null value):

Dropping tombstones with Filter and a predicate (fragment)

JSON

```
{
  "transforms": "dropTombstones",
  "transforms.dropTombstones.type": "org.apache.kafka.connect.transforms.Filter",
  "transforms.dropTombstones.predicate": "isTombstone",
  "predicates": "isTombstone",
  "predicates.isTombstone.type": "org.apache.kafka.connect.transforms.predicates.RecordIsTombstone"
}
```

GOTCHA

The dollar sign in `InsertField$Value` is also a shell character. Inside a double-quoted shell string, `$Value` expands to nothing, and Connect then reports an unknown class. Put the JSON in a file and send it with `--data @file.json`, or use single quotes. The same goes for the `$1` in a `RegexRouter` replacement.

8.4.4 What SMTs are not for

SMTs are deliberately limited: they see one record at a time, keep no state, and cannot join with another topic. A chain of five or six transformations is already hard to read and harder to debug, because a failure shows up as a generic error in the connector rather than in your code. Use SMTs for routing, renaming, masking, key selection and simple filtering. For aggregations, joins, enrichment from another system or any real business rule, process the data with Kafka Streams or a consumer first and let Connect move the result.

INTERVIEW ANGLE

What is a Single Message Transform, and when would you not use one? An SMT is a stateless per-record function configured on a connector: it runs between the connector and the converter (before the converter for sources, after it for sinks) and can rename, drop, mask, route or re-key records. Do not use it for stateful or multi-record logic (joins, aggregations, lookups); that belongs in Kafka Streams or an application, where it can be tested and debugged properly.

KEY POINTS

Key points

- Converters set the byte format on Kafka (`StringConverter`, `JsonConverter`, `ByteArrayConverter`, Avro and others); the connector never sees the bytes.
- Set defaults on the worker, override per connector, and prefix converter settings with `key.converter.` or `value.converter.`
- `JsonConverter` with `schemas.enable=true` wraps every message in a schema envelope; both sides of a topic must agree.
- Decimals appear as Base64 in JSON unless you set `decimal.format=NUMERIC`.
- SMTs run in the order listed in `transforms`; use `$Key` or `$Value` classes, and predicates with `Filter` or `negate` for conditional logic.
- Keep SMTs simple: routing, masking, renaming and key selection. Heavy logic belongs elsewhere.

PART 9

Design Patterns with Kafka

Reusable designs for the problems every event-driven system meets: deciding what an event is, publishing it safely, consuming it once in effect, failing gracefully, coordinating several services and keeping read models in step.

In this part

- 9.1 Designing Events and Topics
- 9.2 Event Notification and Event-Carried State Transfer
- 9.3 The Dual-Write Problem and the Transactional Outbox
- 9.4 Idempotent Receivers: Choosing a De-duplication Strategy
- 9.5 Retry and Dead-Letter Patterns
- 9.6 Sagas: Choreography and Orchestration
- 9.7 Event Sourcing
- 9.8 CQRS and Materialised Views
- 9.9 Compacted Topics as Tables

9.1 Designing Events and Topics

Kafka will carry any bytes you give it, which is exactly why design mistakes are so easy to make. A badly named database column can be renamed in a migration. A badly designed event is copied into logs, retained for days or years, read by consumers you have never heard of, and replayed long after the team that wrote it has moved on. This chapter covers the decisions you take before writing any code: what kind of message you are sending, what it is called, what it contains, which topic it goes to and which key it carries.

Every pattern chapter in this Part has the same shape: the problem, the forces that make it hard, the solution, a short implementation sketch using the ShopStream services, the trade-offs, and when not to use it. The sketches are deliberately small. The complete working code is in the project chapters at the end of the book.

9.1.1 Commands, events and queries

Three kinds of message travel between services, and mixing them up is the root of many muddled designs.

Kind	Meaning	Receiver	ShopStream example
Command	"Please do this." An instruction that can be refused or can fail.	One specific handler	<code>ReserveStock</code> , aimed at <code>inventory-service</code>
Event	"This happened." A fact that cannot be refused or changed.	Anyone interested; the publisher does not know who	<code>StockReserved</code> , published by <code>inventory-service</code>
Query	"Tell me this." A question that expects an answer now.	One specific service	"What is the status of order o-1001?"

Three kinds of message and how they differ.

Commands are written in the imperative (`ReserveStock`), events in the past tense (`StockReserved`). A quick test: if nobody ever read this message, would the sender still have done its job? If yes, it is an event. If the sender is waiting for somebody to act, it is a command.

Prefer events for communication between services. A publisher of events does not know its consumers, so `analytics-app` and `search-indexer` could be added to ShopStream without touching `order-service`. A command makes the sender responsible for knowing the receiver. That is acceptable inside an orchestrated saga (a later chapter), but it is more coupling. Queries do not belong in Kafka in most designs: use the service's API or a read model (see the chapters on CQRS and on request-reply).

The names tell readers what they are looking at. `NotificationRequested` on the `notifications` topic is deliberately command-like: `order-service` wants a notification sent and `notification-service` is its single handler. That is a legitimate use of a topic as a work queue, as long as everyone knows that is what it is and nobody treats it as a business fact.

9.1.2 Naming events

Name events with a past-tense verb in the language of the business: `OrderCreated`, `PaymentAuthorized`, `ShipmentDispatched`. Avoid technical or CRUD names such as `OrderUpdated` or `OrdersRowChanged`. A generic `OrderUpdated` forces every consumer to compare the old and new state to work out what happened, while `OrderCancelled` with a `reason` field says it directly.

Two more rules keep event streams clean:

- **One fact per event, published by the service that owns the fact.** `OrderConfirmed` should not secretly mean "stock reserved and payment taken" too. Those are facts owned by `inventory-service` and `payment-service`, and they publish their own events.
- **Do not leak your database.** An event is a public API. Design it from the business point of view, not from the shape of your tables, or every schema refactoring becomes a breaking change for other teams.

9.1.3 The event envelope

Every event carries the same few fields, whatever its content, so that tooling, debugging and consumers can treat events uniformly.

Field	Purpose
eventId	Unique id (a UUID) of this event. The key for de-duplication, tracing and idempotency.
type	Stable name of the event type, for example <code>ORDER_CREATED</code> . Lets a consumer choose a handler without guessing from field names.
version	Version of this event type's schema, an integer starting at 1. Lets consumers cope with old and new shapes during an upgrade.
occurredAt	When the business fact happened (ISO-8601, UTC). Not the same as the Kafka record timestamp, which is set by the producer or the broker depending on <code>message.timestamp.type</code> .
correlationId	Optional. Shared by every event and command caused by the same original request; makes a saga traceable.

Envelope fields. The first four appear in every ShopStream event.

In ShopStream the envelope fields come first and the business data follows in the same flat object:

OrderCreated event (value of a record on the orders topic)

JSON

```
{
  "eventId": "6f1c2d4e-8a47-4d58-9d0e-3b2f6c1a9e10",
  "type": "ORDER_CREATED",
  "version": 1,
  "occurredAt": "2026-10-08T10:15:30Z",
  "orderId": "o-1001",
  "customerId": "c-77",
  "lines": [ { "sku": "SKU-RED-SHOE-42", "qty": 2, "unitPrice": 499.00 } ],
  "total": 998.00,
  "currency": "INR"
}
```

Some teams prefer a nested layout with the business data under a `data` field. The CloudEvents specification, hosted by the CNCF, standardises a set of envelope attributes and defines a Kafka binding in which they travel as record headers. Both are legitimate. What matters is choosing one layout for the whole organisation and writing it down.

Keep `type` in the body so that a consumer in any language can route without special header handling. When you also want to filter or route without deserialising the payload, copy it into a record header. The publisher below does both jobs: it sets the key and the header in one place, so no caller can forget them.

OrderEventPublisher.java

JAVA

```
package com.shopstream.order;

import java.nio.charset.StandardCharsets;
import java.util.concurrent.CompletableFuture;
import org.apache.kafka.clients.producer.ProducerRecord;
import org.springframework.kafka.core.KafkaTemplate;
import org.springframework.kafka.support.SendResult;
import org.springframework.stereotype.Component;

@Component
```

OrderEventPublisher.java (continued)

JAVA

```

public class OrderEventPublisher {

    private final KafkaTemplate<String, Object> kafka;

    public OrderEventPublisher(KafkaTemplate<String, Object> kafka) {
        this.kafka = kafka;
    }

    public CompletableFuture<SendResult<String, Object>> publish(
        String orderId, String type, Object event) {
        var record = new ProducerRecord<String, Object>("orders", orderId, event);
        record.headers().add("event-type", type.getBytes(StandardCharsets.UTF_8));
        return kafka.send(record);
    }
}

```

What to notice: the topic is fixed (`orders`), the key is always the `orderId`, and the method returns the `CompletableFuture` so the caller decides what a failed send means. Publishing from inside a database transaction has a trap of its own, which the chapter on the outbox pattern deals with.

9.1.4 One topic per entity or one per event type?

You have to decide how events map to topics. There are two common answers.

	One topic per event type	One topic per entity (aggregate)
Example	<code>order-created</code> , <code>order-confirmed</code> , <code>order-cancelled</code>	<code>orders</code> holds all three
Ordering	None between types, because they sit in different partitions	Total order per key: an order's lifecycle is replayed in sequence
Schemas	One schema per topic, simple	Several event types in one topic; the schema tooling must support it
Consumers	Subscribe only to what they need	See events they ignore and must skip them
Topic count	Grows with every new event	Grows with entities

Topic-per-event-type versus topic-per-entity.

The deciding question is whether events must be processed in order relative to each other. An order can be created, confirmed and cancelled, and a consumer that sees the cancellation before the creation has a problem. Events that must stay ordered belong in one topic, with the id of the thing they describe as the key. That is why ShopStream uses one topic per entity, owned by one service: `orders`, `payments`, `inventory` and `shipments`.

A consumer of a multi-type topic usually dispatches on the type. Spring for Apache Kafka supports this with a class-level `@KafkaListener` and one `@KafkaHandler` method per payload type, where the payload class is derived from the type information the JSON serializer places in a record header.

OrderEventsListener.java

JAVA

```

package com.shopstream.shipping;

import com.shopstream.events.OrderCancelled;
import com.shopstream.events.OrderConfirmed;
import org.springframework.kafka.annotation.KafkaHandler;
import org.springframework.kafka.annotation.KafkaListener;
import org.springframework.stereotype.Component;

@Component
@KafkaListener(id = "shipping-orders", topics = "orders", groupId = "shipping-service")
public class OrderEventsListener {

    @KafkaHandler
    void on(OrderConfirmed event) {
        // create the shipment for event.orderId()
    }

    @KafkaHandler
    void on(OrderCancelled event) {
        // cancel a pending shipment, if there is one
    }
}

```

OrderEventsListener.java (continued)

JAVA

```

    @KafkaHandler(isDefault = true)
    void ignore(Object other) {
        // OrderCreated and any future event type: not our business
    }
}

```

For the payload type to be found, the producer and consumer agree on short type tokens instead of Java class names, because the two services do not share a package. This is the `spring.json.type.mapping` setting, a comma-separated list of `token:className` pairs:

application.properties (order-service and shipping-service)

PROPERTIES

```

# order-service (producer)
spring.kafka.producer.value-serializer=\
    org.springframework.kafka.support.serializer.JsonSerializer
spring.kafka.producer.properties[spring.json.type.mapping]=\
    OrderConfirmed:com.shopstream.events.OrderConfirmed,\
    OrderCancelled:com.shopstream.events.OrderCancelled

# shipping-service (consumer): same tokens, its own classes
spring.kafka.consumer.value-deserializer=\
    org.springframework.kafka.support.serializer.JsonDeserializer
spring.kafka.consumer.properties[spring.json.type.mapping]=\
    OrderConfirmed:com.shopstream.events.OrderConfirmed,\
    OrderCancelled:com.shopstream.events.OrderCancelled

```

GOTCHA

A new event type can break old consumers. A deserializer that builds a class from a type token cannot build one for a token it has never heard of, so adding `OrderRefunded` to `orders` makes the record fail deserialisation in every consumer that was not updated first. The default handler above never gets a chance to run. Decide in advance how unknown types are handled: wrap the deserializer so the failure is routed to the error handler and skipped, or dispatch on the envelope `type` field after reading the payload into a tolerant structure. The Spring chapters show the error-handling setup.

9.1.5 Keys, ordering and partitions

The key is the id of the thing whose events must stay in order, almost always the aggregate id: `orderId` for the order topics, `customerId` for `notifications` and `customers`. Kafka hashes the key to a partition, a partition is read by one consumer of a group at a time, and a partition is read in offset order. So all events of one order arrive in the order they were published. The idempotent producer, which is on by default, keeps that order even when a send is retried.

Choosing the key badly causes two opposite problems. A key with few distinct values (the country, say) puts most of the traffic on a few partitions and one consumer does all the work. A random key, or none, spreads the load beautifully and destroys any per-entity ordering. Use a null key only for events where order truly does not matter, such as `page-views` that carry their own timestamp.

GOTCHA

A key orders events inside one topic only. `orders` and `payments` are both keyed by `orderId`, but they are different topics with different partitions. A consumer that reads both can see `PaymentAuthorized` before the `OrderCreated` it belongs to. Never build logic that depends on the relative order of events from different topics or from different keys. Use the data in the events (a status, a version) to decide what to do, as the saga chapter shows.

Also remember that the key-to-partition mapping depends on the partition count. Adding partitions to a live topic sends the same key to a different partition, and for a while events of one order can sit in two partitions. Choose the partition count with growth in mind when you create the topic.

9.1.6 Evolving events without breaking consumers

Events outlive the code that wrote them, and producers and consumers are deployed at different times. The practical rules:

- **Only add.** New optional fields with sensible defaults are safe. Removing a field, renaming it, or changing its type or meaning (for example switching `total` from rupees to paise) breaks readers silently.

- **Be a tolerant reader.** A consumer reads the fields it needs and ignores the rest. Configure your deserialiser to ignore unknown properties, because a producer that adds a field must not crash anybody.
- **Use `version` for compatible evolution and a new type for a break.** When a change cannot be made compatible, publish a new event type (or a new topic) next to the old one for a migration period, move the consumers, then retire the old one.
- **Write the contract down.** A schema registry with compatibility checks (see the chapter on schemas and data contracts) enforces the rules by machine; an event catalogue, for example in the AsyncAPI format, tells humans what each event means and who owns it.

INTERVIEW ANGLE

How do you guarantee that all events of one order are processed in order? Use the `orderId` as the record key. The same key always maps to the same partition, a partition is consumed by exactly one consumer of a group, in offset order, and the idempotent producer keeps retries from reordering. The guarantee holds only inside one partition: it does not cover other keys, other topics, or a period during which the partition count was changed.

KEY POINTS

Designing events and topics

- Prefer events (facts, past tense) for communication between services; use commands deliberately and name them as commands.
- Give every event an envelope: `eventId`, `type`, `version`, `occurredAt`, plus a correlation id when requests span services.
- Group events that must stay ordered into one topic and key them by the aggregate id; ordering never crosses keys or topics.
- Treat events as a public API: add fields, never repurpose them, and plan for unknown types and unknown fields.
- Choose the partition count up front, because changing it remaps keys.

9.2 Event Notification and Event-Carried State Transfer

When `order-service` publishes that an order was confirmed, how much should the event say? The shipping service needs the delivery address and the items. The notification service needs the customer's contact details. The search indexer needs almost everything. Put too little in the event and every consumer calls back to the publisher. Put too much in and you copy data all over the company.

There are two classic answers, and a middle path between them.

9.2.1 Event notification (thin events)

An **event notification** says that something happened and carries little more than the identity of the thing:

OrderConfirmed as a thin event

JSON

```
{
  "eventId": "9a0b1c2d-3e4f-4a5b-8c6d-7e8f9a0b1c2d",
  "type": "ORDER_CONFIRMED",
  "version": 1,
  "occurredAt": "2026-10-08T10:15:42Z",
  "orderId": "o-1001"
}
```

A consumer that needs details calls `GET /orders/o-1001` on `order-service`. The publisher stays the single source of truth and the event contract is tiny and stable. The costs show up at run time:

- **Runtime coupling.** If `order-service` is down or slow, every consumer stalls, even though Kafka was supposed to decouple them.
- **Callback load.** Five consumers mean five calls per event, and a replay of a million events means a million calls against a service that is busy with live traffic.
- **Racing the truth.** By the time the consumer calls back, the order may have been cancelled. It handles an `ORDER_CONFIRMED` event but reads a `CANCELLED` order, so the data no longer matches the event.

9.2.2 Event-carried state transfer (fat events)

With **event-carried state transfer** the event contains the data consumers need, so they never have to ask:

OrderConfirmed as a fat event

JSON

```
{
  "eventId": "9a0b1c2d-3e4f-4a5b-8c6d-7e8f9a0b1c2d",
  "type": "ORDER_CONFIRMED",
  "version": 1,
  "occurredAt": "2026-10-08T10:15:42Z",
  "orderId": "o-1001",
  "customerId": "c-77",
  "lines": [ { "sku": "SKU-RED-SHOE-42", "qty": 2, "unitPrice": 499.00 } ],
  "total": 998.00,
  "currency": "INR",
  "shippingAddress": { "name": "Asha Rao", "city": "Hyderabad", "pin": "500081" }
}
```

Consumers are now autonomous: **shipping-service** keeps working while **order-service** is down, a replay needs no callbacks, and the event describes the facts as they were when it was published. The costs are different:

- **Bigger messages and more coupling to the data shape.** Every consumer that reads **shippingAddress** is now affected if its structure changes.
- **Copies of data.** Each consumer holds its own copy, which is eventually consistent with the source and can drift if it mishandles an event.
- **Spreading personal data.** A name and an address in an event are now in every consumer's database, in the topic, in its backups, for as long as retention allows. Data-protection duties (deletion requests, minimisation) apply to all of those copies.

Question	Thin event (notification)	Fat event (state transfer)
Consumer needs details	Calls back to the publisher	Already has them
Publisher unavailable	Consumers stall	Consumers continue
Replaying history	Hammers the publisher	Self-contained, cheap
Data seen by the consumer	Current state at call time	State at event time
Message size, schema surface	Small	Larger, more to maintain
Personal data spread	Low	Higher, plan for it

Thin versus fat events at a glance.

9.2.3 Choosing between them, and the middle path

Choose a thin event when the event is only a trigger, when consumers rarely need details, or when the data is sensitive and must stay in one place. Choose a fat event when many consumers need the same data, when they must keep working without the publisher, or when you want to replay history without load on the source.

The most useful middle path is the **just-enough event**: include the facts that most consumers need to make a decision (ids, status, total, currency) and leave out bulky or sensitive details, which consumers fetch only when they truly need them. Design the event around the decision it enables. If **shipping-service** cannot act without an address, the address must be reachable without a synchronous call to **order-service**, either inside the event or through a replicated copy.

The replicated copy is the second form of state transfer: instead of putting reference data into every event, publish the entity itself to a **compacted topic** and let consumers build a local table. ShopStream does this with **customers**, keyed by **customerId**, so **shipping-service** and **notification-service** keep a local **customer_view**. Events say what happened; state topics say what is true now. The chapter on compacted topics covers the topic side. The consumer side is short:

CustomerViewUpdater.java

JAVA

```
package com.shopstream.shipping;

import com.shopstream.events.Customer;
import org.apache.kafka.clients.consumer.ConsumerRecord;
import org.springframework.jdbc.core.simple.JdbcClient;
import org.springframework.kafka.annotation.KafkaListener;
```

CustomerViewUpdater.java (continued)

JAVA

```
import org.springframework.stereotype.Component;

@Component
public class CustomerViewUpdater {

    private final JdbcClient jdbc;

    public CustomerViewUpdater(JdbcClient jdbc) {
        this.jdbc = jdbc;
    }

    @KafkaListener(topics = "customers", groupId = "shipping-service")
    public void onCustomer(ConsumerRecord<String, Customer> record) {
        String customerId = record.key();
        Customer customer = record.value();
        if (customer == null) {
            // a tombstone: the customer was deleted at the source
            jdbc.sql("DELETE FROM customer_view WHERE customer_id = :id")
                .param("id", customerId)
                .update();
            return;
        }
        jdbc.sql("""
            INSERT INTO customer_view (customer_id, name, tier, city)
            VALUES (:id, :name, :tier, :city)
            ON CONFLICT (customer_id)
            DO UPDATE SET name = :name, tier = :tier, city = :city
            """)
            .param("id", customerId)
            .param("name", customer.name())
            .param("tier", customer.tier())
            .param("city", customer.city())
            .update();
    }
}
```

What to notice: the listener takes the whole `ConsumerRecord` because only then can it see a `null` value, which is how Kafka represents a delete. The write is an upsert of the full state, so it is naturally idempotent: processing the same record twice leaves the same row. And because all records of one customer share a key and therefore a partition, they arrive in order, so the last write wins correctly. If state arrives from several sources or partitions, carry a version number or timestamp and apply an update only when it is newer (the chapter on idempotent consumers shows how).

WATCH OUT

Do not let consumers depend on fields they were never promised. The more data an event carries, the greater the temptation for a consumer to use a field because it happens to be there. When the publisher later removes or changes it, a team it has never met breaks. Document which fields are part of the contract, and use schema compatibility checks to catch accidents.

INTERVIEW ANGLE

What is the difference between event notification and event-carried state transfer, and when do you use each? A notification event announces that something happened and carries only identifiers; consumers call back for details, which keeps the contract small but couples them to the publisher at run time. Event-carried state transfer puts the needed data into the event so consumers can work autonomously and replay without callbacks, at the price of bigger messages, copies of data and a wider contract. Use notifications as triggers and for sensitive data; use state transfer when many consumers need the data or must survive the publisher being down.

KEY POINTS

Event notification and state transfer

- Thin events are small and keep one source of truth, but make consumers call back and expose them to races and outages.
- Fat events make consumers autonomous and replayable, at the cost of bigger contracts and copies of (possibly personal) data.
- Aim for just-enough events, and replicate shared reference data through compacted state topics instead of repeating it in every event.
- State-carrying messages keyed by entity id are naturally idempotent: apply the latest state, delete on a tombstone.

9.3 The Dual-Write Problem and the Transactional Outbox

When a customer places an order, **order-service** has to save the order in its database and publish **OrderCreated** to the **orders** topic. It looks like one step. It is two writes to two systems that do not share a transaction, and in a distributed system anything that can fail between two writes eventually will.

9.3.1 The problem: two writes, no shared transaction

This is the code almost everyone writes first:

The dual write (do not do this)

JAVA

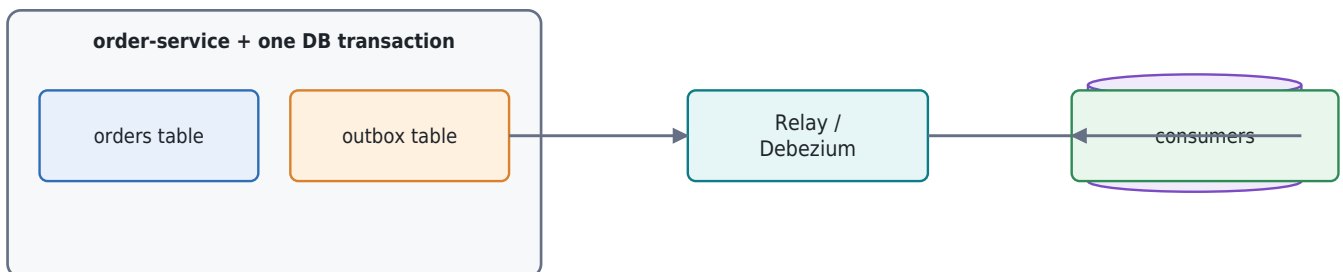
```
@Transactional
public Order placeOrder(PlaceOrderRequest request) {
    Order order = orders.save(Order.create(request));           // write 1: database
    kafka.send("orders", order.getId(), OrderCreated.from(order)); // write 2: Kafka
    return order;
}
```

Two things go wrong. The **@Transactional** proxy commits after the method returns, so Kafka receives the event before the database commits. If the commit then fails, there is an **OrderCreated** event for an order that does not exist, and **payment-service** may charge a customer for nothing. And **send** only hands the record to the producer's buffer: if the process dies before the batch leaves, or the send fails after the commit, the order exists and nobody ever hears about it.

Swapping the two lines only swaps the failure. Kafka transactions do not help, because they cover only Kafka, which does not take part in two-phase commit (XA) with databases. The way out is to make **one** system the source of truth and derive the other from it, atomically.

9.3.2 The solution: write the event into the database first

In the **transactional outbox** pattern the service inserts the event into an **outbox** table in the same database transaction as the business change. Either both rows are committed or neither is. A separate relay then reads the outbox and publishes the events to Kafka, retrying until it succeeds.



business row and event row commit together or not at all

Figure 9.1. The order row and the outbox row are committed in one database transaction; a relay (a polling publisher or Debezium) reads the outbox and publishes to Kafka.

The table uses the column names that Debezium's outbox router expects by default, so the same table works with either relay:

outbox table (PostgreSQL)

SQL

```
CREATE TABLE outbox (
  id          uuid          PRIMARY KEY,      -- the eventId
  aggregate_type varchar(255) NOT NULL,      -- target topic, for example orders
  aggregate_id varchar(255) NOT NULL,      -- record key, for example the orderId
  type        varchar(255) NOT NULL,      -- for example ORDER_CREATED
  payload      jsonb        NOT NULL,      -- the event JSON
  created_at   timestampz   NOT NULL DEFAULT now(),
  published_at timestampz
);
```

The service writes both rows in one transaction. **OutboxWriter** is a few lines of **JdbcClient** code (constructor injection of the client and Spring Boot's Jackson 3 **JsonMapper** omitted); its **MANDATORY** propagation makes it fail fast if someone

calls it outside a transaction. `DomainEvent` is a one-method interface exposing `eventId()`, which the event records already have as a component.

OrderService.java and OutboxWriter.java (essential methods)

JAVA

```
@Transactional
public Order placeOrder(PlaceOrderRequest request) {
    Order order = orders.save(Order.create(request));           // business row
    outbox.add("orders", order.getId(), "ORDER_CREATED",
              OrderCreated.from(order));                         // outbox row, same transaction
    return order;
}

@Transactional(propagation = Propagation.MANDATORY)
public void add(String topic, String key, String type, DomainEvent event) {
    jdbc.sql("""
        INSERT INTO outbox (id, aggregatetype, aggregateid, type, payload)
        VALUES (:id, :topic, :key, :type, CAST(:payload AS jsonb))
        """)
    .param("id", UUID.fromString(event.eventId()))
    .param("topic", topic)
    .param("key", key)
    .param("type", type)
    .param("payload", json.writeValueAsString(event))
    .update();
}
```

9.3.3 Getting the rows to Kafka

Option 1: a polling publisher. A scheduled job selects unpublished rows, sends them and marks them published. It needs no infrastructure beyond the service itself. Project A in the end-to-end part contains the complete class; the essential method is:

OutboxRelay.relay() (the essential method)

JAVA

```
@Scheduled(fixedDelay = 500)
@Transactional
public void relay() {
    List<OutboxRow> batch = jdbc.sql("""
        SELECT id, aggregatetype, aggregateid, type, CAST(payload AS text) AS payload
        FROM outbox
        WHERE published_at IS NULL
        ORDER BY created_at, id
        LIMIT 100
        FOR UPDATE SKIP LOCKED
        """)
    .query((rs, n) -> new OutboxRow(rs.getObject("id", UUID.class),
        rs.getString("aggregatetype"), rs.getString("aggregateid"),
        rs.getString("type"), rs.getString("payload")))
    .list();

    List<CompletableFuture<?>> sends = new ArrayList<>();
    for (OutboxRow row : batch) {
        var record = new ProducerRecord<>(row.topic(), row.key(), row.payload());
        record.headers().add("event-id", row.id().toString().getBytes(UTF_8));
        sends.add(kafka.send(record));
    }
    sends.forEach(CompletableFuture::join);           // a failed send throws and rolls back

    batch.forEach(row -> jdbc.sql("UPDATE outbox SET published_at = now() WHERE id = :id")
        .param("id", row.id()).update());
}
```

What to notice: rows are marked published only after every send was acknowledged, so a crash in between re-sends the batch (at-least-once). `FOR UPDATE SKIP LOCKED` lets several relay instances run without taking the same rows. `OutboxRow` is a small record of the five columns, and `kafka` is a `KafkaTemplate<String, String>`, because the payload is already JSON text. `@EnableScheduling` must be present on a configuration class.

Option 2: log-based change data capture. Debezium reads the database's transaction log and turns each inserted outbox row into a Kafka record: no queries against the table and no `published_at` column. It runs as a connector on Kafka Connect, and the Connect and CDC part shows the complete connector. The heart of it is the `EventRouter` transformation, which maps the columns to the record:

Debezium outbox settings (the essential lines)

PROPERTIES

```
transforms=outbox
transforms.outbox.type=io.debezium.transforms.outbox.EventRouter
transforms.outbox.route.by.field=aggregatetype
transforms.outbox.route.topic.replacement=${routedByValue}
transforms.outbox.table.expand.json.payload=true
value.converter=org.apache.kafka.connect.json.JsonConverter
```

Each row is routed to the topic named in `aggregatetype` (the default replacement would be `outbox.event.<value>`, so the override yields `orders`), the `aggregateid` becomes the record key, and the `payload` the value. Expanding the JSON payload makes the value real JSON instead of an escaped string. The row's `id` is added to the record headers as `id`, which consumers can use to de-duplicate.

Aspect	Polling publisher	Log-based CDC (Debezium)
Infrastructure	A scheduled job in your service	Kafka Connect, a connector, database settings (for PostgreSQL <code>wal_level=logical</code>)
Load on the database	Repeated queries and updates	Reads the log; the table is never queried
Ordering	Needs care with several instances	Commit order of the log
Cleanup	Delete or archive published rows	Rows can be deleted at once
Portability	Any database with row locking	Needs a connector for the database

Polling publisher versus log-based CDC.

9.3.4 Guarantees, ordering and cleanup

The outbox gives you **atomicity between the business change and the decision to publish**, and **at-least-once delivery**. It does not give exactly-once: a relay can crash after Kafka acknowledged a record but before it was marked, and the event goes out twice. Consumers must be idempotent, which is the subject of the next chapter. The same `eventId` on both copies is what lets them recognise the duplicate.

Ordering per aggregate holds only if the relay publishes an aggregate's events in the order they were written. With a polling publisher this breaks when two instances run at once, because one may take a later event of an order while the other still holds an earlier one. Run one active relay (with a lock or leader election), or give each instance a fixed share of the aggregate ids. Do not track "the last id published" as a cursor: transactions commit out of id order, and a row committed late would be skipped forever. The relay above selects on `published_at IS NULL` for that reason. Log-based capture follows commit order and avoids the problem.

Cleanup. A polling relay deletes or archives published rows after a retention period. With Debezium the router ignores `DELETE` events, and the Debezium project's own outbox example inserts each row and removes it again in the same transaction: the log holds both, a connector publishes only the insert, and the table stays empty.

Monitor the age of the oldest unpublished row (or the connector's lag). A growing outbox is the early warning that events are not leaving the building.

Skip the outbox when losing an occasional event is acceptable (metrics, page views), or when a service only transforms Kafka data and Kafka transactions suffice. In event sourcing, the event store plays the part of the outbox.

WATCH OUT

Do not "fix" the dual write with a try/catch. Catching the exception from `send` and retrying later, or publishing in an `afterCommit` hook, narrows the window but does not close it: a crash between the commit and the publish still loses the event, and nothing records that it was owed. Only a durable record written in the same transaction closes the gap.

INTERVIEW ANGLE

What is the dual-write problem, and how does the transactional outbox solve it? Updating a database and publishing to Kafka are two writes without a shared transaction, so a failure between them leaves the two out of step: a lost event or a ghost event. The outbox writes the event as a row in the same database transaction as the business change, and a relay (a polling job or Debezium reading the log) publishes it to Kafka afterwards. Delivery is at-least-once, so consumers must be idempotent.

KEY POINTS

The transactional outbox

- Never update a database and publish to Kafka as two independent steps; one of them will eventually fail alone.
- Write the event to an **outbox** table in the same transaction as the business data; publish it from there.
- A relay is either a polling publisher (simple, any database) or log-based CDC with Debezium (no polling, commit order).
- Delivery is at-least-once with per-aggregate ordering if you take care; consumers de-duplicate on **eventId**.
- Clean up the table and watch the age of the oldest unpublished row.

9.4 Idempotent Receivers: Choosing a De-duplication Strategy

Delivery to a consumer is **at-least-once**. A crash between finishing the work and committing the offset, a rebalance, an outbox relay that resends, a replay of a dead-letter topic: all of them put the same event in front of a handler twice. The consumer cannot prevent that; it must make the second delivery harmless. A handler with that property is **idempotent**. The chapter on idempotent consumers and the inbox pattern in the Spring Kafka production part builds the processed-event table step by step. This chapter is about the decision that comes first: which strategy fits which handler. Kafka's exactly-once features do not remove the question, because they cover only the cycle of reading from Kafka and writing back to Kafka. A database row, an email or a payment call lies outside that transaction.

9.4.1 Five strategies

Strategy	How it works	Covers	Weak spot
Natural idempotency	Set state, upsert the whole entity, delete by key	Writes of complete state	"Add 2", "send", "charge" are not idempotent
Version check	Apply an event only if its revision is newer than the stored one	Duplicates and out-of-order events	Needs a revision in events and rows
Inbox table	Insert the eventId in the same database transaction as the effect	Effects inside one database	Table needs cleanup; blind to external calls
Idempotency key	Pass a stable key (the orderId , the eventId) to the system you call	Calls to external systems	The target must support it, and remembers it only for a limited time
Kafka transactions	Commit output records and consumed offsets atomically; readers use read_committed	Kafka to Kafka	Not databases or HTTP

Five ways to make a handler idempotent, cheapest first.

Look for natural idempotency first, because it needs no extra storage. `UPDATE orders SET status = 'CONFIRMED' WHERE id = :id` can run ten times. Upserting the full state of a customer, as the earlier chapter on state transfer does, is idempotent. Counters, balances and side effects need one of the other strategies.

The version check deserves a closer look because it also repairs **ordering**. Within one partition events arrive in order, but after a replay, a retry topic or a merge of several topics they may not. If every event carries the aggregate's **revision** (a counter incremented with each change; an event-sourced service has it for free as the stream version), a single statement applies an update only when it is newer:

Versioned upsert for a read model

SQL

```
INSERT INTO order_view (order_id, status, revision)
VALUES (:orderId, :status, :revision)
ON CONFLICT (order_id) DO UPDATE
SET status = EXCLUDED.status, revision = EXCLUDED.revision
WHERE order_view.revision < EXCLUDED.revision;
```

A duplicate has the same revision and an old event a lower one: in both cases the **WHERE** clause is false, the statement changes nothing, and the update count of zero tells the handler it can move on. The same statement also creates the row when an update arrives before the creation.

9.4.2 Choosing per handler

The choice is made per handler, not per application. In ShopStream:

Handler	Strategy	Why
<code>search-indexer</code> writing an order document	Natural idempotency	Writing the whole document under the key <code>orderId</code> twice gives the same document
Order summary projection	Version check	Replays and retries may repeat or reorder events; a stale update must lose
<code>inventory-service</code> reserving stock	Inbox table	"Reserve 2 units" is not idempotent; the dedupe row and the reservation share one transaction
<code>payment-service</code> calling the gateway	Inbox plus idempotency key	The gateway call is outside the database; the <code>orderId</code> as key lets the gateway recognise a repeat
<code>notification-service</code> sending an email	Dedupe on <code>eventId</code> , accept a rare duplicate	An email cannot be unsent; decide what a duplicate costs

Strategy per handler.

Three rules apply whatever you choose. **De-duplicate on the event id, never on the offset:** a duplicate published by the outbox relay has a new offset and the same `eventId`. **Record and act together:** marking an event done in a cache and then writing to the database leaves a gap between the two, so the record and the effect must share a transaction, or the effect must be idempotent by itself. **Keep the memory as long as the danger:** an inbox purged after a day cannot recognise an event replayed from last week.

INTERVIEW ANGLE

Kafka offers exactly-once semantics. Why do you still need idempotent consumers? Exactly-once covers reading from Kafka, processing and writing back to Kafka, using the idempotent producer, transactions and `read_committed`. A database update or an HTTP call is outside it, and committing the offset can never be atomic with such a side effect, so after a crash the effect may run again. The handler has to be idempotent: a naturally idempotent operation, a version check, an inbox row in the same transaction, or an idempotency key on the external call.

KEY POINTS

Idempotent receivers

- Assume every message can arrive twice; design the handler so that the second time changes nothing.
- Choose per handler: natural idempotency, version check, inbox table, idempotency key or Kafka transactions.
- A `revision` check handles duplicates and out-of-order events in one statement.
- De-duplicate on `eventId`, write the record and the effect in one transaction, and size the memory to the longest replay.

9.5 Retry and Dead-Letter Patterns

A consumer that processes records for long enough will meet records it cannot process. The cause matters, because each cause needs a different response. A **transient** failure (a database deadlock, a gateway timeout) will probably succeed if tried again. A **permanent** failure (a malformed message, a business rule that rejects the data) will fail every time. A **not-yet** failure (a payment event that arrives before the order row is visible) will succeed later without any change to the record. Treating all three the same way is the most common mistake: either the consumer retries a poison pill forever, or it throws away a message that a ten-second wait would have saved.

Failure	Example	Response
Transient, order matters	Deadlock, brief network glitch	Blocking retry with a short back-off, then the dead-letter topic
Transient, outage of minutes	Email provider down	Non-blocking retry topics with growing delays
Permanent for this record	Validation error, unknown enum value	No retry: straight to the dead-letter topic
Unreadable record	Malformed JSON	<code>ErrorHandlingDeserializer</code> , then the dead-letter topic
Not yet	Event overtakes the data it refers to	Short retry topics, or park it and re-evaluate

Which response fits which failure.

9.5.1 Blocking retry

In a **blocking retry** the consumer retries the failed record in place before it moves on. Order is preserved and nothing else is needed, but the partition makes no progress while the retries last. In Spring for Apache Kafka the `DefaultErrorHandler` does this, with a back-off policy and an optional recoverer that is called when the attempts run out. Without configuration it makes ten delivery attempts with no pause and then logs the record at `ERROR` and skips it, which is rarely what you want.

ConsumerErrorConfig.java

JAVA

```
@Configuration
public class ConsumerErrorConfig {

    @Bean
    DefaultErrorHandler errorHandler(KafkaTemplate<Object, Object> template) {
        var recoverer = new DeadLetterPublishingRecoverer(template);
        var handler = new DefaultErrorHandler(recoverer, new FixedBackOff(1000L, 2L));
        handler.addNotRetryableExceptions(IllegalArgumentException.class);
        return handler;
    }
}
```

This handler makes three delivery attempts one second apart, then publishes the record to `<topic>-dlt`. `IllegalArgumentException` is declared permanent, so it goes to the dead-letter topic on the first failure; Spring already treats deserialisation and conversion errors that way. Spring Boot attaches a `CommonErrorHandler` bean to the auto-configured listener container factory.

Keep the total back-off short. The default handler simply suspends the consumer thread between attempts, so a long wait can exceed `max.poll.interval.ms` and make the broker side consider the consumer dead. If delays must be longer, Spring offers a back-off handler that pauses the container instead of sleeping, but then you are better off with retry topics.

9.5.2 Non-blocking retry topics

When a failure lasts minutes, blocking the partition for minutes is unacceptable: every other record behind it waits. The **retry topic** pattern moves the failed record out of the way. The consumer publishes it to a retry topic together with the time it is due, and carries on with the next record. A second listener reads the retry topic, waits until the record is due, and calls the same handler again. After the last attempt the record goes to the dead-letter topic. The chapter on non-blocking retries in the Spring part has the flow diagram; the annotation is short:

NotificationListener.java (notification-service)

JAVA

```
@Component
public class NotificationListener {

    private static final Logger log = LoggerFactory.getLogger(NotificationListener.class);

    private final EmailGateway gateway;

    public NotificationListener(EmailGateway gateway) {
        this.gateway = gateway;
    }

    @RetryableTopic(attempts = "4", backOff = @BackOff(delay = 5000),
        exclude = IllegalArgumentException.class)
    @KafkaListener(topics = "notifications", groupId = "notification-service")
    public void on(NotificationRequested request) {
        gateway.send(request); // a RuntimeException moves the record to the retry topic
    }

    @DltHandler
    public void onDead(NotificationRequested request) {
        log.error("Notification {} could not be delivered", request.eventId());
    }
}
```

With a fixed delay Spring creates one retry topic, `notifications-retry`, and `notifications-dlt`; an exponential back-off creates one topic per delay, such as `notifications-retry-1000` and `notifications-retry-2000`. The price is **ordering**: while record N waits in the retry topic, record N+1 of the same key is processed. Use retry topics only where handlers tolerate that, for example because they are idempotent and versioned (see the previous chapter), and never

for a sequence of dependent steps on one key. They are not supported for batch listeners or with container transactions.

9.5.3 The dead-letter topic is an inbox, not a bin

The **dead-letter topic** (DLT) holds records that could not be processed. Spring's recoverer publishes the original key and value, the original headers, and extra headers that describe the failure: the exception class and message, the stack trace, and the original topic, partition and offset (the `kafka_dlt-*` headers, with constants such as `KafkaHeaders.DLT_ORIGINAL_TOPIC`). By default the record goes to the same partition number as the original, so give the DLT at least as many partitions as the source topic.

A DLT that nobody watches is a silent data-loss mechanism with extra steps. Treat it as a work queue for humans and automation:

- **Alert** on any record arriving in it, or on its growth, not only on consumer lag.
- **Retain** records long enough to react, usually longer than the source topic (`retention.ms` on the DLT).
- **Write a runbook**: who looks, how to decide between fix-and-replay and discard.
- **Never throw from the handler of the DLT itself**. Persist or alert and move on.

9.5.4 Reprocessing from the DLT

Once the cause is fixed (a bug, a missing reference record, a restored dependency), the parked records are replayed into the original topic. A replay tool must restore the original topic, key, value and headers, drop the failure headers, and be repeatable. This sketch starts stopped and is started by an operator, for example through an admin endpoint that calls `KafkaListenerEndpointRegistry#getListenerContainer("orders-dlt-replayer")` and `start()`:

```
DltReplayer.java JAVA

@Component
public class DltReplayer {

    private final KafkaTemplate<String, String> kafka;

    public DltReplayer(KafkaTemplate<String, String> kafka) {
        this.kafka = kafka;
    }

    @KafkaListener(id = "orders-dlt-replayer", topics = "orders-dlt", autoStartup = "false")
    public void replay(ConsumerRecord<String, String> record) {
        Header origin = record.headers().lastHeader(KafkaHeaders.DLT_ORIGINAL_TOPIC);
        String topic = new String(origin.value(), StandardCharsets.UTF_8);
        var out = new ProducerRecord<>(topic, null, record.key(), record.value());
        for (Header h : record.headers()) {
            if (!h.key().startsWith("kafka_dlt-")) {
                out.headers().add(h); // keeps type headers and the eventId
            }
        }
        kafka.send(out).join();
    }
}
```

The replayer must not interpret the payload, so it needs a listener container factory with string deserialisers. Two properties make a replay safe. The consumer must be idempotent, because some replayed records were partly processed. And its writes should be versioned, because a replayed event is older than events that were processed in the meantime and must not overwrite newer state. If a record fails again it simply returns to the DLT; count the attempts in a header so that a record cannot loop forever.

9.5.5 Poison pills

A **poison pill** is a record that fails every time, and the worst kind cannot even be deserialised. A plain consumer fails inside `poll()` on the same offset again and again, and the partition is stuck. Wrap the real deserialiser in Spring's `ErrorHandlingDeserializer` (set the real class with `spring.deserializer.value.delegate.class`), so a bad record arrives as a `DeserializationException` that the error handler sends to the DLT and skips. Batch listeners do not route such failures automatically.

WATCH OUT

Unbounded retries are an outage with a delay. A handler that retries forever, or a retry topic loop that never ends in a DLT, holds resources and hides the failing record while lag grows. Every retry policy needs a limit and a final destination. If a dependency is down for everyone, retries from many consumers arrive at the same moment when it recovers: use growing delays with jitter, and consider pausing consumption altogether rather than failing record after record.

INTERVIEW ANGLE

Blocking retries or retry topics: how do you choose? Blocking retries keep order but stall the partition, so they suit short transient errors where order matters. Retry topics keep the partition moving by parking the failed record for a delay, which suits longer outages, but the parked record is processed after later records of the same key, so ordering is lost. Both must end in a dead-letter topic, and permanent errors should skip retries entirely.

KEY POINTS

Retry and dead-letter patterns

- Classify failures first: transient, permanent, not yet, unreadable. Only the first and last-but-one deserve retries.
- Blocking retry preserves order and blocks the partition; retry topics unblock the partition and give up ordering.
- Every retry policy is bounded and ends in a dead-letter topic with the original record and failure headers.
- Monitor and own the DLT; replay through a tool that restores the original record and relies on idempotent, versioned consumers.
- Wrap deserialisers in `ErrorHandlingDeserializer` so one bad record cannot stop a partition.

9.6 Sagas: Choreography and Orchestration

Placing an order in ShopStream changes data in three services: `inventory-service` reserves stock, `payment-service` takes the money and `order-service` records the result. Each has its own database, so no single transaction can cover them, and Kafka transactions do not reach into databases. A **saga** gets the same business result without a global transaction. It splits the work into a sequence of local transactions. Each step commits on its own and announces the outcome; if a later step fails, the earlier steps are undone by **compensating transactions**, new transactions that semantically reverse the effect (release the stock, refund the payment).

A saga gives up isolation. Between the steps the world sees intermediate states: an order that is **PENDING** with stock already reserved. The patterns below manage that. There are two ways to coordinate the steps.

9.6.1 The ShopStream order saga

The happy path and the failure paths use only the events defined for ShopStream:

Step	Service reacts to	Local transaction	Publishes
1	HTTP request	Create the order as PENDING	OrderCreated on orders
2	OrderCreated	Reserve the stock	StockReserved or StockRejected on inventory
3	StockReserved	Authorize the payment	PaymentAuthorized or PaymentFailed on payments
4	PaymentAuthorized	Mark the order CONFIRMED	OrderConfirmed on orders
5	OrderConfirmed	Create the shipment	ShipmentCreated on shipments
Fail	StockRejected	Mark the order CANCELLED	OrderCancelled on orders
Fail	PaymentFailed	Release the reservation; cancel the order	StockReleased on inventory , OrderCancelled on orders

The order saga, step by step.

Notice the order of the steps. Reserving stock comes before taking money because releasing a reservation is cheap and invisible, while refunding a customer is slow and visible. Put the steps that are hard to undo as late as possible. The step after which the saga can no longer be rolled back is its **pivot**; the steps behind it must be retried until they succeed rather than compensated.

9.6.2 Choreography: services react to each other's events

In a **choreographed** saga there is no coordinator. Each service listens for the events it cares about, does its local work and publishes its own event. The flow above is exactly that, and the services need nothing but the topics:

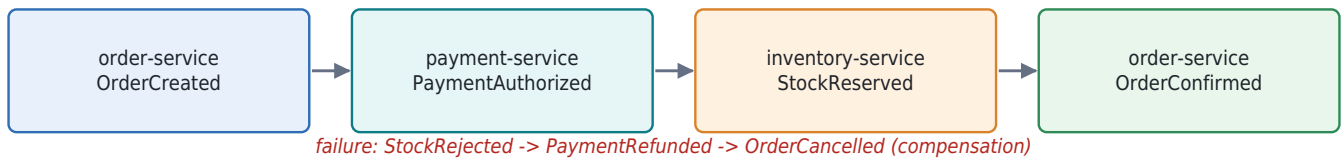


Figure 9.2. Choreographed order saga: each service reacts to events and publishes the next one; failures travel as events (StockRejected, PaymentFailed) and trigger compensations such as StockReleased and OrderCancelled.

Service	Listens to	Publishes
order-service	inventory (StockRejected), payments (PaymentAuthorized, PaymentFailed)	orders
inventory-service	orders (OrderCreated), payments (PaymentFailed)	inventory
payment-service	inventory (StockReserved)	payments
shipping-service	orders (OrderConfirmed)	shipments

Who listens to what in the choreographed saga.

The strengths are simplicity and loose coupling: no central component, and a new participant (say a fraud check) just subscribes. The weakness is that the process exists nowhere as a whole. To learn what happens after **OrderCreated** you must read four services, cycles creep in (order-service listens to payments, inventory-service listens to payments), and finding out where order o-1001 is stuck means correlating events across topics. Carry a **correlationId** in every event, trace it, and keep a saga state view, for example the compacted **order-status** topic, which a small consumer maintains from all the topics.

9.6.3 Orchestration: a coordinator drives the saga

In an **orchestrated** saga one component, the orchestrator (here a part of **order-service**), owns the process. It sends **commands** to the participants and reacts to their replies. The participants still publish their outcomes as events on **inventory** and **payments**; the commands travel on topics of their own (**inventory-commands**, **payment-commands**, keyed by **orderId**).

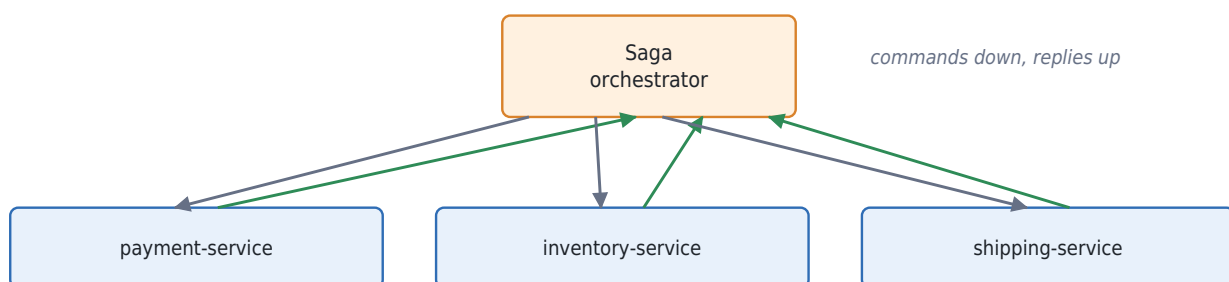


Figure 9.3. Orchestrated order saga: the orchestrator sends commands, receives the replies, stores the saga state and decides about compensation.

The orchestrator is a state machine whose state lives in a database table (one row per order). Every transition and the command it triggers are written in **one** transaction, with the command going through the outbox of the earlier chapter.

OrderSagaListener.java (order-service, one transition)

JAVA

```

@Transactional
@KafkaHandler
public void on(StockReserved event) {
    OrderSaga saga = sagas.require(event.orderId());
    if (saga.state() != SagaState.AWAITING_STOCK) {
        return; // duplicate or late event: ignore
    }
    saga.moveTo(SagaState.AWAITING_PAYMENT, Instant.now().plus(Duration.ofMinutes(5)));
    outbox.add("payment-commands", saga.orderId(), "AUTHORIZE_PAYMENT",
        AuthorizePayment.of(saga));
}

```

What to notice: the state check makes the handler idempotent and robust against events that arrive after the saga has moved on, and the second argument of `moveTo` stores a **deadline** for the next reply. The strengths of orchestration are the explicit flow in one place, one place for timeouts and compensation, and easy monitoring. The costs are an extra moving part, commands that couple the orchestrator to every participant, and the risk that business rules leak into a "god service". Keep the orchestrator free of business logic: it decides *which step is next*, not *how* a step is done.

9.6.4 Concerns every saga must handle

- **Idempotency.** Events are delivered at least once, so every step and every compensation must be idempotent (previous chapters). Releasing a reservation twice must be harmless.
- **Compensation is not rollback.** You cannot un-send an email or erase that a card was authorised. Compensations are new facts (`StockReleased`, `PaymentRefunded`, `OrderCancelled`) and each step needs one designed in advance, or must be declared the pivot.
- **Timeouts.** A participant that never answers leaves the saga hanging. Store a deadline per step; a scheduled job (or a Kafka Streams punctuator) finds expired sagas and starts the compensation.
- **Isolation.** Intermediate states are visible. Use a status as a semantic lock (an order in `PENDING` cannot be modified), re-read data before acting on it, and use version checks.
- **Ordering and lateness.** Events of one order are ordered per topic, not across topics. Decide what happens when `StockReserved` arrives for an order that was already cancelled: usually the participant must immediately compensate.

	Choreography	Orchestration
Good fit	Few steps, one team, linear flow	Many steps, branches, timeouts, several teams
Visibility of the flow	Spread over services	One place
Coupling	Via events only	Orchestrator knows all participants
Risk	Hidden cycles, hard to debug	Orchestrator becomes a monolith

Choosing a coordination style.

Do not reach for a saga when one service and one database can do the whole job in a local transaction; merging two services that always change together is a legitimate fix. And do not expect sagas to give consistency at every instant: they give eventual consistency with explicit failure handling.

INTERVIEW ANGLE

What is a saga, and what is the difference between choreography and orchestration? A saga replaces a distributed transaction by a sequence of local transactions, each publishing an event or command for the next step, with compensating transactions that undo earlier steps when a later one fails. In choreography the services react to each other's events with no central controller; in orchestration a coordinator holds the saga state, sends commands and decides about compensation. Choreography is simpler for short flows but hides the process; orchestration makes it explicit at the cost of a central component.

KEY POINTS**Sagas**

- A saga is a chain of local transactions with compensations; it trades isolation for availability and independence.
- Order the steps so that the hardest-to-undo step comes last; steps after the pivot are retried, not compensated.
- Choreography uses only events and suits short flows; orchestration uses commands and a state machine and suits long or branching flows.
- Make every step and compensation idempotent, give every step a timeout, and carry a correlation id.
- Write the saga state change and the next message in one transaction (the outbox).

9.7 Event Sourcing

Most applications store the **current state**: the `orders` table holds the latest status of each order, and every update overwrites the previous value. The history is gone unless you add audit tables by hand. **Event sourcing** turns this around. The source of truth is the sequence of events that happened to each entity (`OrderCreated`, `OrderConfirmed`, `OrderCancelled`), and the current state is derived by replaying them. You get a complete audit trail, the ability to rebuild state at any point in the past, and the freedom to create new read models later from history that was recorded all along. It fits domains where the history is part of the business: ledgers, order lifecycles, anything audited.

9.7.1 Decide and apply

Event sourcing separates two steps. A command is first checked against the current state and **decides** which events to emit (or rejects). The events are then **applied** to the state. The apply function is pure: no I/O, no validation, and it never fails, because the event is already a fact.

OrderEvent.java and OrderState.java

JAVA

```
public sealed interface OrderEvent extends DomainEvent
    permits OrderCreated, OrderConfirmed, OrderCancelled { }

public record OrderState(String orderId, String status, BigDecimal total) {

    /** Rebuild the state by folding the history, oldest event first. */
    public static OrderState replay(List<OrderEvent> history) {
        OrderState state = null;
        for (OrderEvent event : history) {
            state = apply(state, event);
        }
        return state;
    }

    private static OrderState apply(OrderState state, OrderEvent event) {
        return switch (event) {
            case OrderCreated e -> new OrderState(e.orderId(), "PENDING", e.total());
            case OrderConfirmed e -> new OrderState(state.orderId(), "CONFIRMED", state.total());
            case OrderCancelled e -> new OrderState(state.orderId(), "CANCELLED", state.total());
        };
    }

    /** The command: decide which events follow, or reject. */
    public List<OrderEvent> confirm(Instant now) {
        if (!"PENDING".equals(status)) {
            throw new IllegalStateException("Order " + orderId + " is " + status);
        }
        return List.of(new OrderConfirmed(UUID.randomUUID().toString(), orderId, now));
    }
}
```

What to notice: the event records implement `OrderEvent`, and because it is sealed the `switch` is checked by the compiler for completeness: add a new event type and every place that must handle it stops compiling. `confirm` returns events instead of changing the object.

An **event store** needs only two operations: append events for one aggregate, and load them in order. The append carries the version the caller expects, which gives **optimistic concurrency**. With a relational store the primary key does the work:

event_store table (PostgreSQL)

SQL

```
CREATE TABLE event_store (
  aggregate_id varchar(255) NOT NULL,
  version      int         NOT NULL,           -- 1, 2, 3 ... per aggregate
  event_id     uuid        NOT NULL UNIQUE,
  type         varchar(255) NOT NULL,
  payload      jsonb       NOT NULL,
  occurred_at  timestampz  NOT NULL,
  PRIMARY KEY (aggregate_id, version)
);
```

If two commands load the same order at version 3 and both try to append version 4, one insert violates the primary key and fails. That command reloads the order, decides again on fresh state, and retries or gives up.

9.7.2 Where Kafka fits, and where it does not

Kafka is an excellent **distribution** layer for events: durable, ordered per key, replayable by any number of consumers, and able to keep events forever (`retention.ms=-1`). It is a poor **event store database**, for three reasons. It has no conditional append, so you cannot say "add this event only if the aggregate is still at version 3". It has no index by key, so loading the history of one order means scanning a partition. And it cannot delete or amend a single record in the middle of a log.

Two designs work in practice:

- **Database event store plus Kafka.** Events are appended to a table like `event_store` and published through the outbox (with change data capture, the table itself can serve as the outbox). Consistency comes from the database; Kafka feeds read models, other services and replays.
- **Kafka-native with Kafka Streams.** Commands go to a topic keyed by aggregate id. A Streams application keeps each aggregate's state in a state store, processes the commands of a key one at a time (the partition is the serialisation point, so no version check is needed) and writes the resulting events to an events topic, with exactly-once processing. This removes the second system but makes commands asynchronous and puts the aggregate state in the Streams state stores, which have to be restored after a rebalance.

9.7.3 Snapshots, evolution and erasure

- **Snapshots.** Replaying thousands of events per command is slow. Store a snapshot of the state every N events together with its version, and load the snapshot plus the events after it. In Kafka a compacted topic keyed by aggregate id is a natural home for snapshots.
- **Evolving events.** Stored events are immutable and live forever, so every version ever written must remain readable. Keep the `version` field, and use **upcasters**, small functions that convert an old event shape to the current one when it is loaded. Never edit history.
- **Erasure.** A log of immutable events collides with the right to have personal data erased. Keep personal data out of events where you can (store it elsewhere and reference it by id), or encrypt it with a key per person and destroy the key (**crypto-shredding**). Kafka cannot remove one record from the middle of a partition; compaction removes only older values of a key, and only when the cleaner runs.

Event sourcing adds real cost: a different way of thinking, harder queries (you query projections, not the events), and event versioning for life. Do not use it for plain CRUD data. Do not confuse it with publishing events: a service can publish events to Kafka without ever storing its state as events.

INTERVIEW ANGLE

Is Kafka an event store? Kafka is an excellent durable, replayable log for events, but it lacks what a store of record for aggregates needs: no conditional append for optimistic concurrency, no efficient read of one aggregate's events, and no deletion of single records. Typical designs keep the events in a database with a unique (aggregate, version) key and publish them to Kafka through an outbox, or use Kafka Streams with a state store so that all commands of a key are processed sequentially.

KEY POINTS

Event sourcing

- State is a fold over events; commands decide events, **apply** is pure and never fails.
- Concurrency control needs a conditional append (a unique aggregate-and-version key) or single-writer processing per key.
- Kafka distributes and replays events well; it is not a database for loading aggregates.
- Plan snapshots, upcasting and erasure (crypto-shredding) before the first event is written.

9.8 CQRS and Materialised Views

In a service with its own database, one model has to serve two masters. Updates want a normalised structure that protects consistency, one aggregate at a time. Queries want something else entirely: the "my orders" screen needs orders with product names and delivery status in one row, the search page needs a text index, and the dashboard needs counts per city. Joins across services are impossible, because each service owns its data. **Command Query Responsibility Segregation (CQRS)** gives queries their own model. The write side handles commands and publishes events. **Projections** consume those events and maintain **read models** (also called materialised views), each shaped for the questions it answers.

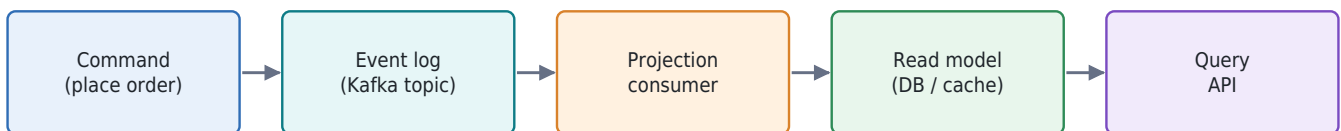


Figure 9.4. Commands change the write side, which records events; projections consume the events from Kafka and keep read models that serve queries.

Read model	Built by	Serves
<code>order_summary</code> table in a relational database	A projection in <code>order-service</code> or a query service	The "my orders" screen
Search index	<code>search-indexer</code>	Product and order search
<code>KTable</code> with interactive queries	<code>analytics-app</code> (Kafka Streams)	Counts and aggregates
Compacted topic <code>order-status</code>	A small consumer or Streams	Latest status for any consumer that needs it

Typical read models in ShopStream.

9.8.1 A projection

A projection is a consumer whose handlers turn events into updates of the read model:

`OrderSummaryProjection.java` (one handler and the helper)

JAVA

```

@KafkaHandler
public void on(OrderCreated event) {
    jdbc.sql("""
        INSERT INTO order_summary (order_id, customer_id, total, status)
        VALUES (:id, :customer, :total, 'PENDING')
        ON CONFLICT (order_id) DO NOTHING
        """)
    .param("id", event.orderId())
    .param("customer", event.customerId())
    .param("total", event.total())
    .update();
}

@KafkaHandler
public void on(OrderConfirmed event) {
    jdbc.sql("UPDATE order_summary SET status = 'CONFIRMED' WHERE order_id = :id")
    .param("id", event.orderId())
    .update();
}
  
```

The class carries a class-level `@KafkaListener` on `orders` with its own group id, like the listeners in the earlier chapters, and the same rules apply: handlers are idempotent, and events of one order arrive in order because they share a key. A projection that is replayed in order from the start always converges to the same state, which is what makes rebuilding possible.

Rebuilding. Because the read model is a function of the event log, a changed or damaged model is rebuilt rather than migrated. Start the projection under a new consumer group with `auto.offset.reset=earliest`, write into a fresh table or index, and switch readers over when it has caught up. This only works if the topic still has the history: decide retention (or a snapshot or compacted topic) when you create it.

Eventual consistency. The read model lags the write side by milliseconds or, during an incident, by minutes. The classic symptom: a customer places an order, is redirected to "my orders" and does not see it. Design for it instead of hoping:

- Return the created resource in the response to the command and show that, instead of querying again.
- Return a revision or event id; the query endpoint can wait briefly until the projection has reached it, or read from the write side for that one id.
- Show an honest state ("processing") and update the page by polling or server-sent events.
- Monitor projection lag as a service-level objective, not only as an operational metric.

WATCH OUT

Projections must not have side effects. A projection that sends an email or calls a payment API will repeat it on every replay and every rebuild. Side effects belong in handlers that are idempotent and reached through events, never in code that you intend to replay at will.

CQRS and event sourcing are separate ideas that fit together. You can use CQRS with an ordinary state-based write model and an outbox, and many teams do. Do not use it where simple reads of the write model are enough and the lag of eventual consistency would only hurt.

KEY POINTS

CQRS and materialised views

- Give queries their own model, built by projections that consume events; one write model can feed many read models.
- A deterministic projection can be rebuilt from the log; keep the history it needs.
- Eventual consistency is a design input: return the new state, use revision tokens, show honest states, watch the lag.
- Keep projections free of side effects and idempotent.

9.9 Compacted Topics as Tables

Some topics do not hold a stream of facts but a **state**: the latest profile of each customer, the current status of each order. Deleting such a topic by age loses customers who happened not to change for a week, while keeping every update forever wastes space on values nobody will read again. **Log compaction** fits: instead of deleting old records by time, Kafka keeps at least the latest record for every key and a background cleaner removes older records of the same key. The topic behaves like a table that can always be rebuilt by reading it from the start. The compaction figure in the chapter on log internals shows the before and after of a cleaning.

9.9.1 What compaction guarantees

The design documentation promises the following:

- A consumer that stays close to the head of the log sees every record that is written.
- A consumer that reads from the start sees at least the final state of every key.
- Order is never changed, and a record keeps its offset; compaction only removes records.
- A record with a key and a `null` value is a **tombstone**: a delete marker for the key. Tombstones are themselves removed after `delete.retention.ms` (default one day), so a consumer that lags by more than that can miss the delete.
- The active segment is never compacted.

Records in a compacted topic must have a key. Compaction is also lazy: the cleaner runs when the share of "dirty" (not yet cleaned) records passes `min.cleanable.dirty.ratio` (default 0.5), so a consumer may see several records per key, and must tolerate it.

Create the compacted customers topic (single local broker)

TERMINAL

```
$ bin/kafka-topics.sh --create --topic customers --partitions 3 --replication-factor 1 \
  --config cleanup.policy=compact --bootstrap-server localhost:9092
Created topic customers.
```

Config	Default	Effect
cleanup.policy	delete	compact enables compaction; compact,delete also deletes by age or size
min.cleanable.dirty.ratio	0.5	Share of uncleaned log that triggers a cleaning
min.compaction.lag.ms	0	A record stays uncompactd at least this long
delete.retention.ms	86400000	How long tombstones are kept
segment.ms	604800000	Segment roll time; only closed segments are compacted

*Settings that shape compaction.***GOTCHA**

A quiet compacted topic keeps its duplicates for a long time. The active segment is never cleaned, and a segment rolls after `segment.bytes` (1 GiB by default) or `segment.ms` (7 days). On a low-traffic topic like `customers`, old values of a key can stay visible for days. If you need prompt cleaning, lower `segment.ms` for that topic (and understand that smaller segments mean more files).

9.9.2 Using a compacted topic as a table

Writers publish the full state under the entity's id, and publish `null` to delete: `kafka.send("customers", "c-77", null)`. Readers fall into two groups. A plain consumer builds its own table by applying every record to a local store, as the `CustomerViewUpdater` of the chapter on state transfer does. Kafka Streams turns the topic into a `KTable` (partitioned across the instances of an application) or a `GlobalKTable` (a full copy in every instance, for reference data of modest size) and supports joins between streams and tables, as the next chapter shows.

The same mechanism appears inside Kafka. `__consumer_offsets` is a compacted topic holding the latest committed offset per group, topic and partition, and Kafka Streams backs its state stores with **changelog topics** that use compaction. In ShopStream, `customers` carries reference data and `order-status` carries the current status of each order for anyone who needs "where is my order" without the history.

Three limits to remember. A compacted topic is not a database: there is no lookup by key, only consumers that build one. Never compact an event stream such as `orders`, where each event matters. And the key-to-partition mapping applies as always: adding partitions to a compacted topic moves keys, so a key can end up with old values in one partition and new ones in another, and "the latest per key" is no longer well defined.

INTERVIEW ANGLE

How does log compaction differ from time-based retention, and what is a tombstone? Time-based retention deletes whole segments by age or size regardless of keys. Compaction keeps at least the latest record per key and removes older records with the same key, so the topic can serve as a table. A tombstone is a record with a key and a `null` value; it marks the key as deleted and is itself cleaned up after `delete.retention.ms`.

KEY POINTS

Compacted topics as tables

- Use compaction for state keyed by entity id (`customers`, `order-status`), never for event streams.
- Guarantee: the latest value per key survives; tombstones delete, and disappear after `delete.retention.ms`.
- Compaction is lazy and skips the active segment: tune `segment.ms` and the dirty ratio when freshness matters.
- Do not change the partition count of a compacted topic that is used as a table.
- Read it as a `KTable` or `GlobalKTable`, or build a local table with a plain consumer.